# Privacy-Preserving Machine Learning for Practitioners

Manuel V. Loureiro

Version 2026-10-26

# Table of contents

**Preface** **1**
How this book is organized . . . 1

**I. Prerequisites** **2**
What each chapter is for . . . 3
Notation . . . 3
How to use the chapters . . . 3

**1. Probability distributions** **4**
1.1. What it is . . . 4
1.2. When you meet it . . . 4
1.3. How to use it . . . 4
1.4. What goes wrong . . . 6

**2. Concentration inequalities** **7**
2.1. What it is . . . 7
2.2. When you meet it . . . 7
2.3. How to use it . . . 7
2.4. What goes wrong . . . 9

**3. Divergences** **10**
3.1. What it is . . . 10
3.2. When you meet it . . . 10
3.3. How to use it . . . 11
3.4. What goes wrong . . . 12

**4. Hypothesis testing** **13**
4.1. Why this matters . . . 13
4.2. Tests and their two errors . . . 13
4.3. The Neyman–Pearson lemma . . . 13
4.4. Trade-off curves . . . 14
4.5. The Gaussian trade-off curve . . . 14
4.6. The Laplace trade-off curve . . . 14
4.7. Best advantage and total variation . . . 15
4.8. ROC summaries: why AUC hides the low-FPR regime . . . 15
4.9. The Bayesian view . . . 16
4.10. Measuring rates from finite trials . . . 16
4.11. Evaluating a detector at a low false positive rate . . . 16
4.12. Cheat sheet . . . 17
4.13. References . . . 18

**5. Information theory** **19**
5.1. What it is . . . 19
5.2. When you meet it . . . 19
5.3. How to use it . . . 19
5.4. What goes wrong . . . 20
5.5. References . . . 21

**6. Optimization** **22**
6.1. What it is . . . 22
6.2. When you meet it . . . 22
6.3. How to use it . . . 22
6.4. What goes wrong . . . 23

**7. Algorithmic stability and generalization** **25**
7.1. What it is . . . 25
7.2. When you meet it . . . 25
7.3. How to use it . . . 25
7.4. What goes wrong . . . 26

**8. Cryptography basics** **27**
8.1. What it is . . . 27
8.2. When you meet it . . . 27
8.3. How to use it . . . 28
8.4. What goes wrong . . . 29

**II. Threat models and attacks** **30**

**9. Threat models and privacy games** **32**
9.1. Why this matters . . . 32
9.2. Writing down the threat model . . . 32
9.3. Scoring an attack against its baseline . . . 34
9.4. Reporting: TPR at a low FPR . . . 34
9.5. Average case and worst case . . . 36
9.6. Cheat sheet . . . 36
9.7. References . . . 36

**10. Membership inference** **37**
10.1. Why this matters . . . 37
10.2. Membership as a test about one record . . . 37
10.3. The quick check: the generalization gap . . . 37
10.4. LiRA: a likelihood-ratio test per record . . . 38
10.5. Running LiRA on your model . . . 40
10.6. Reading the result . . . 40
10.7. Who is exposed, and what helps . . . 41
10.8. Cheat sheet . . . 41
10.9. References . . . 42

**11. Attribute inference and model inversion** **43**
11.1. What it is . . . 43
11.2. When you meet it . . . 43
11.3. How to use it . . . 43
11.4. What goes wrong . . . 45
11.5. References . . . 45

**12. Property inference** **46**
12.1. What it is . . . 46
12.2. When you meet it . . . 46
12.3. How to use it . . . 46
12.4. What goes wrong . . . 47
12.5. References . . . 47

**13. Memorization and training-data extraction** **48**
13.1. Why this matters . . . 48
13.2. Which memorization to measure . . . 48
13.3. What gets memorized, and the first defences . . . 49
13.4. Auditing with canaries and exposure . . . 49
13.5. Extraction without the prefix: generate, rank, verify . . . 51
13.6. Cheat sheet . . . 52
13.7. References . . . 52

**14. Gradient inversion** **53**
14.1. Why this matters . . . 53
14.2. The attacker and what it sees . . . 53
14.3. Exact recovery from a fully connected layer . . . 53
14.4. Label recovery from the last layer . . . 54
14.5. Deep Leakage from Gradients . . . 55
14.6. Batches and averaging . . . 55
14.7. What to do as a deployer . . . 56
14.8. Cheat sheet . . . 57
14.9. References . . . 57

**15. Linkage and re-identification** **58**
15.1. Why this matters . . . 58
15.2. Quasi-identifiers and the linkage attack . . . 58
15.3. Pseudonyms and hashed identifiers . . . 59
15.4. How many people are unique? . . . 60
15.5. Sparse data: the Netflix attack . . . 61
15.6. Does releasing a sample protect? . . . 61
15.7. A pre-release risk check . . . 61
15.8. Cheat sheet . . . 62
15.9. References . . . 62

**16. Syntactic anonymity and how it breaks** **63**
16.1. Why this matters . . . 63
16.2. What the adversary believes about a class . . . 63

16.3. k-anonymity and what it costs . . . 63
16.4. Homogeneity, background knowledge and l-diversity . . . 65
16.5. Measuring what a release teaches . . . 65
16.6. Skewness, similarity and t-closeness . . . 66
16.7. The composition attack . . . 66
16.8. Choosing and running an anonymization . . . 67
16.9. Cheat sheet . . . 67
16.10. References . . . 67

**17. Reconstruction attacks** **69**
17.1. What it is . . . 69
17.2. When you meet it . . . 69
17.3. How to use it . . . 70
17.4. What goes wrong . . . 70
17.5. References . . . 71

**18. Pre-release privacy risk assessment** **72**
18.1. Why this matters . . . 72
18.2. Fix the threat model and the thresholds first . . . 72
18.3. Classifier: the free check . . . 74
18.4. Classifier: LiRA with shadow models . . . 74
18.5. Fine-tuned language model: canaries and extraction . . . 76
18.6. Table: re-identification on quasi-identifiers . . . 77
18.7. Write the decision . . . 78
18.8. Checklist . . . 79
18.9. References . . . 79

**III. Differential privacy foundations** **80**

**19. The definition of differential privacy** **82**
19.1. Why this matters . . . 82
19.2. Neighbouring datasets . . . 82
19.3. The definition . . . 82
19.4. The privacy loss and $\delta$ . . . 83
19.5. Randomized response . . . 83
19.6. What $\varepsilon$ means for an attacker . . . 84
19.7. How small $\delta$ must be . . . 85
19.8. What differential privacy does not protect . . . 86
19.9. Writing the guarantee down . . . 86
19.10. Code . . . 86
19.11. Cheat sheet . . . 86
19.12. References . . . 87

**20. Post-processing, group privacy and composition** **88**
20.1. Why this matters . . . 88
20.2. Post-processing . . . 88
20.3. Group privacy . . . 89

20.4. Sequential composition . . . 90
20.5. Parallel composition . . . 90
20.6. Keeping a budget . . . 91
20.7. Cheat sheet . . . 92
20.8. References . . . 93

**21. The Laplace and Gaussian mechanisms** **94**
21.1. Why this matters . . . 94
21.2. Sensitivity . . . 94
21.3. The Laplace mechanism . . . 95
21.4. The Gaussian mechanism . . . 96
21.5. Laplace or Gaussian? . . . 97
21.6. Calibrating a release . . . 97
21.7. Code . . . 98
21.8. Cheat sheet . . . 98
21.9. References . . . 99

**22. Advanced and optimal composition** **100**
22.1. What it is . . . 100
22.2. When you meet it . . . 100
22.3. How to use it . . . 101
22.4. What goes wrong . . . 102

**23. Selection: exponential mechanism, report-noisy-max, sparse vector** **103**
23.1. Why this matters . . . 103
23.2. Which mechanism to use . . . 103
23.3. The exponential mechanism . . . 104
23.4. Accuracy, and the cost of many candidates . . . 104
23.5. A private quantile or clipping norm . . . 104
23.6. Report-noisy-max and Gumbel noise . . . 106
23.7. The sparse vector technique . . . 106
23.8. When the sparse vector technique goes wrong . . . 107
23.9. Cheat sheet . . . 107
23.10. References . . . 108

**24. Query release and private multiplicative weights** **109**
24.1. Why this matters . . . 109
24.2. Which method to use . . . 109
24.3. Noise on every cell . . . 110
24.4. Releasing a marginal with OpenDP . . . 110
24.5. Synthetic data . . . 111
24.6. Private multiplicative weights . . . 112
24.7. Online release and SmallDB . . . 113
24.8. Cheat sheet . . . 113
24.9. References . . . 113

**25. Beyond global sensitivity** **114**
25.1. What it is . . . 114

25.2. When you meet it . . . 114
25.3. How to use it . . . 115
25.4. What goes wrong . . . 115

**26. Discrete and floating-point-safe mechanisms** **116**
26.1. What it is . . . 116
26.2. When you meet it . . . 116
26.3. How to use it . . . 117
26.4. What goes wrong . . . 117

**27. Local, shuffle and distributed trust models** **119**
27.1. Why this matters . . . 119
27.2. Choosing a trust model . . . 119
27.3. Central and local differential privacy . . . 119
27.4. Frequency estimation: $k$-ary randomized response and unary encoding . . . 121
27.5. Large and unknown domains: hashing and heavy hitters . . . 121
27.6. Numbers in one bit: local mean estimation . . . 121
27.7. Collecting every day: memoization . . . 122
27.8. The shuffle model and amplification by shuffling . . . 122
27.9. Distributed DP: splitting the noise . . . 122
27.10. Cheat sheet . . . 123
27.11. References . . . 124

**28. Lower bounds: what privacy costs** **125**
28.1. What it is . . . 125
28.2. When you meet it . . . 125
28.3. How to use it . . . 125
28.4. What goes wrong . . . 126

**29. Private mean estimation** **127**
29.1. Why this matters . . . 127
29.2. Bounded data: privacy adds a lower-order term . . . 127
29.3. The cost of an unknown range . . . 128
29.4. Many dimensions: clip the norm . . . 129
29.5. Heavy tails: clip into the data . . . 130
29.6. Code . . . 130
29.7. Cheat sheet . . . 131
29.8. References . . . 132

**30. Release statistics with a library** **133**
30.1. Why this matters . . . 133
30.2. Choose the library . . . 133
30.3. Decide the unit and the budget . . . 134
30.4. Bound each customer's contribution . . . 135
30.5. Set up the context . . . 136
30.6. Release overall statistics with error bars . . . 136
30.7. Group by a public key list . . . 137
30.8. Group by unknown keys with partition selection . . . 138

30.9. Check the release before publishing . . . 140
30.10. Write the guarantee down . . . 140
30.11. Checklist . . . 141
30.12. References . . . 141

**IV. Privacy accounting** **142**

**31. Choosing an accountant** **144**
31.1. Why this matters . . . 144
31.2. Which accountant . . . 144
31.3. The inverse problem: noise for a target ε . . . 145
31.4. Choosing δ . . . 146
31.5. The statement . . . 146
31.6. Matching the accountant to the code . . . 147
31.7. Reviewing someone else's claim . . . 148
31.8. Checking an accountant . . . 149
31.9. Cheat sheet . . . 150
31.10. References . . . 150

**32. Calibrate and account a DP-SGD run** **151**
32.1. Why this matters . . . 151
32.2. Fix the privacy unit, n and δ . . . 151
32.3. Choose the sampler . . . 152
32.4. Plan the noise . . . 152
32.5. Check ε with a second accountant . . . 154
32.6. If the loader is not Poisson . . . 155
32.7. Train, stop early and resume . . . 156
32.8. Write the statement . . . 157
32.9. Checklist . . . 158
32.10. References . . . 158

**33. Amplification by subsampling** **160**
33.1. Why this matters . . . 160
33.2. One step under Poisson sampling . . . 160
33.3. Match the sampler to the accountant . . . 161
33.4. Accounting each sampler . . . 162
33.5. Batch size, epochs and noise . . . 163
33.6. Cheat sheet . . . 164
33.7. References . . . 164

**34. Rényi differential privacy** **165**
34.1. Why this matters . . . 165
34.2. The RDP curve . . . 165
34.3. RDP curves of the basic mechanisms . . . 165
34.4. Composition adds curves . . . 166
34.5. Post-processing and group privacy . . . 167
34.6. Compose, then convert once . . . 167

34.7. Check the order grid . . . 168
34.8. Accounting with a library . . . 168
34.9. Cheat sheet . . . 169
34.10. References . . . 169

**35. From Rényi DP to (ε, δ) and to tests** **170**
35.1. What it is . . . 170
35.2. When you meet it . . . 170
35.3. How to use it . . . 171
35.4. What goes wrong . . . 171
35.5. References . . . 172

**36. Numerical accounting with privacy loss distributions** **173**
36.1. What it is . . . 173
36.2. When you meet it . . . 173
36.3. How to use it . . . 174
36.4. What goes wrong . . . 174
36.5. References . . . 175

**37. f-DP and Gaussian differential privacy** **176**
37.1. What it is . . . 176
37.2. When you meet it . . . 176
37.3. How to use it . . . 177
37.4. What goes wrong . . . 177
37.5. References . . . 178

**38. Adaptive composition: filters, odometers and individual accounting** **179**
38.1. What it is . . . 179
38.2. When you meet it . . . 179
38.3. How to use it . . . 179
38.4. What goes wrong . . . 180
38.5. References . . . 181

**39. Concentrated differential privacy** **182**
39.1. What it is . . . 182
39.2. When you meet it . . . 182
39.3. How to use it . . . 182
39.4. What goes wrong . . . 183
39.5. References . . . 184

**40. Amplification by shuffling and by iteration** **185**
40.1. What it is . . . 185
40.2. When you meet it . . . 185
40.3. How to use it . . . 185
40.4. What goes wrong . . . 186
40.5. References . . . 187

**V. Differential privacy in machine learning** **188**

**41. Private empirical risk minimization** **190**
41.1. What it is . . . 190
41.2. When you meet it . . . 190
41.3. How to use it . . . 191
41.4. What goes wrong . . . 192

**42. DP-SGD** **193**
42.1. Why this matters . . . 193
42.2. The algorithm and its guarantee . . . 193
42.3. The implementation checklist . . . 194
42.4. Clipping and the learning rate . . . 195
42.5. Batch size at a fixed budget . . . 195
42.6. Making it fast . . . 196
42.7. What privacy costs in accuracy . . . 196
42.8. Cheat sheet . . . 197
42.9. References . . . 198

**43. Train a model with DP-SGD** **199**
43.1. Why this matters . . . 199
43.2. Fix the privacy unit, n and $\delta$ . . . 199
43.3. Preprocess from public facts . . . 200
43.4. Train the baselines . . . 200
43.5. Check the model . . . 201
43.6. Choose the clipping norm, batch and epochs . . . 201
43.7. Calibrate the noise . . . 202
43.8. Train . . . 202
43.9. Cross-check $\varepsilon$ . . . 203
43.10. Tune honestly . . . 204
43.11. Measure utility and groups . . . 205
43.12. When it goes wrong . . . 206
43.13. Record the run . . . 207
43.14. Checklist . . . 207
43.15. References . . . 208

**44. Hyperparameter tuning and private selection** **209**
44.1. Why this matters . . . 209
44.2. The procedure and the report . . . 209
44.3. Paying for every run . . . 210
44.4. A random number of runs . . . 210
44.5. Selecting by a noisy validation score . . . 211
44.6. Tuning on Adult . . . 211
44.7. Cheat sheet . . . 212
44.8. References . . . 213

**45. User-level differential privacy** **214**
45.1. Why this matters . . . 214

45.2. The privacy unit and δ . . . 214
45.3. What an example-level model gives a user . . . 215
45.4. User-level DP-SGD . . . 215
45.5. Contribution bounding . . . 216
45.6. Heterogeneous users . . . 217
45.7. Which strategy . . . 217
45.8. Code . . . 217
45.9. Cheat sheet . . . 218
45.10. References . . . 219

**46. Choosing ε** **220**
46.1. Why this matters . . . 220
46.2. Justifying a choice . . . 220
46.3. The pair and the whole curve . . . 221
46.4. What an attacker can believe . . . 223
46.5. How small δ must be . . . 223
46.6. Comparing with deployed values . . . 224
46.7. Cheat sheet . . . 224
46.8. References . . . 225

**47. Public data and private fine-tuning** **226**
47.1. Why this matters . . . 226
47.2. How many weights to train . . . 226
47.3. Linear probing first . . . 227
47.4. Low-rank adapters and full fine-tuning . . . 227
47.5. Which method at which budget . . . 228
47.6. What public data does not buy . . . 229
47.7. Cheat sheet . . . 230
47.8. References . . . 230

**48. Fine-tune a language model privately with LoRA** **231**
48.1. Why this matters . . . 231
48.2. Fix the privacy unit, n and δ . . . 231
48.3. Plant canaries . . . 232
48.4. Load a public model and tokenizer . . . 233
48.5. Choose what to train . . . 233
48.6. Make the model Opacus-compatible . . . 234
48.7. Calibrate and train . . . 234
48.8. Evaluate . . . 235
48.9. Check memorization . . . 236
48.10. Variants . . . 237
48.11. Report . . . 238
48.12. Checklist . . . 238
48.13. References . . . 239

**49. Private prediction: PATE and in-context learning** **240**
49.1. What it is . . . 240
49.2. When you meet it . . . 240

49.3. How to use it . . . 241
49.4. What goes wrong . . . 241
49.5. References . . . 242

**50. DP-FTRL and matrix factorization** **243**
50.1. What it is . . . 243
50.2. When you meet it . . . 243
50.3. How to use it . . . 244
50.4. What goes wrong . . . 244
50.5. References . . . 245

**51. Disparate impact** **246**
51.1. Why this matters . . . 246
51.2. Evaluating a private model per group . . . 246
51.3. Disparate impact, measured . . . 247
51.4. Why small groups pay: clipping and noise . . . 249
51.5. Mitigations and what they cost . . . 249
51.6. Cheat sheet . . . 250
51.7. References . . . 251

**52. Auditing private training** **252**
52.1. Why this matters . . . 252
52.2. Known bugs, and which audit catches them . . . 252
52.3. A CI test for the noise and the clipping . . . 253
52.4. From an attack to a lower bound . . . 254
52.5. A one-run audit of the trained model . . . 255
52.6. Cheat sheet . . . 256
52.7. References . . . 256

**53. Adaptive data analysis** **257**
53.1. What it is . . . 257
53.2. When you meet it . . . 257
53.3. How to use it . . . 258
53.4. What goes wrong . . . 258
53.5. References . . . 258

**54. Release and governance** **260**
54.1. Why this matters . . . 260
54.2. Assemble the run record . . . 260
54.3. Recompute the guarantee . . . 261
54.4. Attach the evidence . . . 262
54.5. Write the statement . . . 263
54.6. Map the claim to regulation . . . 264
54.7. Review someone else's claim . . . 265
54.8. After release . . . 266
54.9. Checklist . . . 267
54.10. References . . . 268

**Appendices** **269**

**A. Glossary** **269**

**B. Index** **272**

# Preface

This is the practitioner edition of *Privacy-Preserving Machine Learning*. It is for engineers, data scientists and auditors who build or review machine-learning systems that handle personal data, and it tells them what each result says, when it applies, how to use it and what goes wrong, without proofs.

It keeps the parts and the chapter files of the full edition, so that every page here has a twin there that derives its results step by step, but it is organized around the work rather than the theory. Three kinds of page do that. A **workflow chapter** does one job end to end with real libraries on a real public dataset: assessing a model's privacy risk before release, releasing statistics with OpenDP and diffprivlib, calibrating and reporting a run of differentially private stochastic gradient descent, training and fine-tuning with Opacus, and governing a release. Each part reads in the order of the work, so a workflow comes early in its part and the pages it relies on follow. A **full chapter** covers a topic a practitioner computes with: the result and its formula, why it matters, a recipe with a worked example, what goes wrong, and a cheat sheet. A **reference page** covers a theory-heavy topic a practitioner meets but rarely computes with, in a few hundred words: what it is, when you meet it, how to use it and what goes wrong. Every page ends with a link to the full edition's chapter.

You need some Python and NumPy, basic probability, and enough machine learning to know what a loss and stochastic gradient descent are.

## How this book is organized

1. **Prerequisites** is a mathematical reference to dip into when a later chapter cites it.
2. **Threat models and attacks** shows how models and data leak, and assesses a model's risk before release.
3. **Differential privacy foundations** defines the guarantee, builds its mechanisms, and releases statistics with them.
4. **Privacy accounting** tracks the guarantee through private training, and calibrates and reports a run.
5. **Differential privacy in machine learning** trains, fine-tunes, audits and releases private models.

You do not need to read Part 1 first. With a background in statistics or machine learning, start at Part 2 or Part 3 and follow the links back when you need a fact.

Every piece of code on a page has been run: a snippet runs on its own, and an excerpt is a marked region of a workflow script in the repository's `code/` directory, which a test runs and checks against the numbers the page quotes. To rerun a workflow, run `uv run python code/<part>/practitioner_<topic>.py`.

# Part I.

# Prerequisites

This part is a mathematical toolkit, and it contains no privacy yet. You do not need to read it first. Later chapters link back to the facts they use, so come here when a link sends you.

## What each chapter is for

Two ideas run through the part. The first is the **likelihood ratio**, how many times more likely an observation is under one explanation than under another. The second is the **effect of one input**, how far a single record can move a result.

Hypothesis testing is the one full chapter, because the rest of the book leans on it most. It shows how to read a detector's trade-off curve at a low false positive rate. It also shows how to put an honest interval on a rate measured from a few hundred cases.

The other seven are short reference pages, each with the result, when you meet it, how to use it and what goes wrong. Probability distributions and Concentration inequalities give you the noise families, their scales in NumPy and SciPy, and the error bars and sample sizes you attach to a measured number. Divergences compares two distributions with a single number. Information theory bounds what an output reveals about a hidden value. Optimization gives the constants and learning rates of gradient descent, and Algorithmic stability and generalization explains why a reused holdout set misleads you. Cryptography basics teaches you to read a security claim stated as a game.

## Notation

Table 1.: Notation used throughout the book.

| Symbol | Meaning |
|---|---|
| $\Pr$, $\mathbb{E}$, $\mathrm{Var}$ | Probability, expectation, variance |
| $\mathcal{N}(\mu, \sigma^2)$, $\mathrm{Lap}(b)$ | Normal and Laplace distributions |
| $\ln$, $\log_2$ | Natural logarithm (nats) and base-2 logarithm (bits) |
| $\|x\|$ | Euclidean length of a vector $x$ |
| $D$, $\theta$, $\ell(\theta; z)$ | A dataset, the parameters of a model, and its loss on one record $z$ |

## How to use the chapters

Each page states its results, shows how to use them with a worked number, and links to its derivations in the full edition. Every number comes from a snippet on the page or from the full edition's script for the same chapter, which you can rerun, such as `uv run python code/part01-prerequisites/optimization.py`. The code draws its randomness from seeded NumPy generators, `numpy.random.default_rng`, so reruns give the same numbers.

# 1. Probability distributions

## 1.1. What it is

Most random numbers in a machine-learning system are counts or draws of noise. A count of independent yes-or-no outcomes, each with the same probability, is **binomial**. When the outcomes are many and each is rare, the count becomes **Poisson**, whose variance equals its mean.

Noise usually comes from one of two continuous families. The **Laplace distribution** with scale $b$ is an exponential density mirrored around zero. The normal distribution is the bell curve, set by its mean and its standard deviation $\sigma$.

## 1.2. When you meet it

You meet these families when you size a test set, check which argument of a sampler sets its spread, or put an error bar on a sum.

## 1.3. How to use it

An error rate $p$ measured on a test set of $n$ records is a binomial count divided by $n$, so its standard deviation is

$$\sqrt{\frac{p(1-p)}{n}}. \tag{1.1}$$

With 2,000 records and a true error rate of 5%, this is about half a percentage point. Divided by $p$, it becomes a relative standard deviation, which depends on the expected number of errors $np$ rather than on $n$. Reaching a relative target $r$ needs $n = (1-p)/(p\,r^2)$ records. For a relative standard deviation of 10%, a 5% error rate needs 1,900 records, and a rate fifty times smaller needs 99,900.

A sampled batch is the Poisson case. When each of 60,000 records joins it independently with probability 0.01, its size is close to a Poisson count with mean 600. Its standard deviation is therefore $\sqrt{600}$, about 24.5.

For noise, you set the spread so that a draw stays within $t$ of zero except with a failure probability $\beta$. A Laplace draw with scale $b$ has standard deviation $\sqrt{2}\,b$, not $b$. Its chance of landing at least $t$ from zero is

$$\Pr[|X| \geq t] = e^{-t/b}. \tag{1.2}$$

So the largest Laplace scale and normal standard deviation that meet the target are

$$b = \frac{t}{\ln(1/\beta)}, \qquad \sigma = \frac{t}{z_{1-\beta/2}}. \tag{1.3}$$

Here $z_{1-\beta/2}$ is the standard normal quantile. Both formulas split $\beta$ between the two sides; to bound one side only, replace $\beta$ by $2\beta$. For example, Laplace noise of scale 2 stays within $2\ln 20 = 5.99$ of zero with 95% probability. Table 1.1 shows how differently the two multipliers grow as $\beta$ shrinks.

Table 1.1.: How many standard deviations or scales a draw stays within, except with probability $\beta$.

| Failure probability $\beta$ | 0.05 | $10^{-3}$ | $10^{-6}$ |
|---|---|---|---|
| Normal, $z_{1-\beta/2}$ standard deviations | 1.96 | 3.29 | 4.89 |
| Laplace, $\ln(1/\beta)$ scales | 3.00 | 6.91 | 13.82 |

On a log scale (Figure 1.1), the Laplace tails fall along straight lines, far above the normal density's parabola.

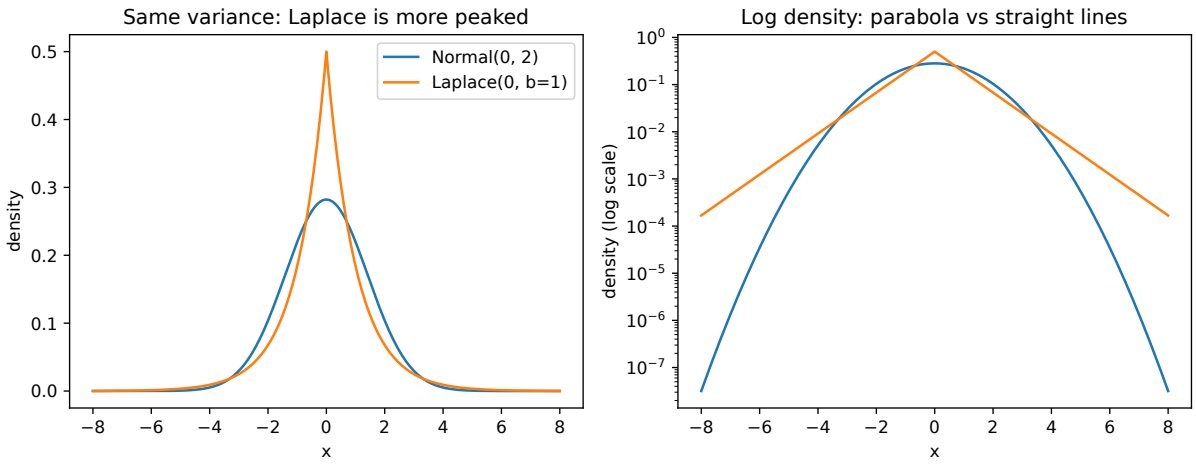



Figure 1.1.: Normal and Laplace densities with variance 2 (Laplace scale 1), on linear and log scales.

Sums of many independent draws look normal by the central limit theorem, but slowly for skewed data. The distribution function of a standardized sum of 30 exponential draws still misses the normal one by up to 0.0243.

A vector of $d$ independent normal coordinates, each with standard deviation $\sigma$, has a length close to $\sigma\sqrt{d}$, about 10 for a million coordinates at $\sigma = 0.01$.

The snippet checks these numbers and the integer case described below.

```
import math

import numpy as np
from scipy import optimize, stats

rng = np.random.default_rng(0)
draws = rng.laplace(loc=0.0, scale=2.0, size=1_000_000)  # scale is b, not the standard devi
print(f"{draws.std():.3f} {stats.laplace(scale=2.0).std():.4f}")  # 2.832 2.8284
print(f"{stats.laplace(scale=2.0).isf(0.025):.2f}")  # 5.99
print(f"{stats.norm.isf(0.05 / 2):.2f} {stats.norm.isf(1e-3 / 2):.2f} {stats.norm.isf(1e-6 /
print(f"{math.log(1 / 0.05):.2f} {math.log(1 / 1e-3):.2f} {math.log(1 / 1e-6):.2f}")  # 3.00
print(math.ceil((1 - 0.05) / (0.05 * 0.1**2)), math.ceil((1 - 0.001) / (0.001 * 0.1**2)))  #
b = 10 / math.log(1 / 0.01)  # t = 10, beta = 0.01
tail = lambda a: 2 * stats.dlaplace(a).sf(9)  # integer version: P[|K| >= 10]
a = optimize.brentq(lambda a: tail(a) - 0.01, 0.01, 5)
print(f"{b:.4f} {tail(1 / b):.4f} {a:.4f} {tail(a):.4f}")  # 2.1715 0.0123 0.4817 0.0100
```

## 1.4. What goes wrong

The commonest bug is a mismatched spread. NumPy's `rng.laplace(0, b)` and SciPy's `stats.laplace(scale=b)` take the scale $b$, but `rng.normal(0, sigma)` and `stats.norm(scale=sigma)` take the standard deviation, never the variance. So compare the standard deviation of many draws with the intended one. For integer data, draw from an integer distribution rather than rounding. In SciPy's `stats.dlaplace(a)`, the probability of an integer falls by a factor $e^{-a}$ per step away from zero, so $a$ plays the part of $1/b$. Its tail is heavier, though: $a = 1/b$ gives 0.0123 where the snippet targets 0.01, so solve for $a$ numerically. NumPy's `rng.exponential(scale)` takes the mean, and `rng.geometric(p)` counts trials up to and including the first success, so it starts at 1. For tiny tail probabilities, call the tail function `sf` or its inverse `isf`, because `1 - cdf` rounds to zero.

These formulas assume independent records, and one user's rows are not independent, so take $n$ to be the number of users. The normal approximation to a binomial count needs $np(1-p)$ of about 10 or more, so it fails for rare events. Finally, the tail bounds of Concentration inequalities need a bounded or light-tailed quantity, so clip an unbounded one, such as a loss, to a range first.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 2. Concentration inequalities

## 2.1. What it is

A concentration inequality bounds the chance that a random quantity lands more than some distance from its mean. Call that event a failure, its chance the **failure probability** $\beta$, and the distance the **width**. Fix $\beta$ and solve for the width to get an error bar, or fix the width and solve for the number of samples.

**Markov's inequality** needs only the mean of a quantity that cannot be negative, and caps the chance of reaching a threshold at the mean divided by the threshold. **Chebyshev's inequality** adds the variance $\sigma^2$. With $\mu$ for the mean, the chance of landing at least a distance $t$ from it is at most

$$\Pr[|X - \mu| \geq t] \leq \frac{\sigma^2}{t^2}. \tag{2.1}$$

Setting this to $\beta$ gives the width $\sigma/\sqrt{\beta}$ for any distribution, at a steep price. At 95% it asks for 4.5 standard deviations, where the one-sided bound $\sigma\sqrt{2\ln(1/\beta)}$ for a normal-shaped tail asks for 2.4. At one in a million the gap grows to 1,000 against 5.3.

## 2.2. When you meet it

You meet these bounds when you put an error bar on a mean or a rate, plan a sample size, or report many numbers at once.

## 2.3. How to use it

Find what you know in Table 2.1.

Table 2.1.: Which bound to use, and the two-sided width it gives.

| What you know | Use | Width at failure probability $\beta$ |
|---|---|---|
| The variance $\sigma^2$ | Chebyshev | $\sigma/\sqrt{\beta}$ |
| A mean, with standard deviation $s$ estimated from the data | Normal approximation | $z_{1-\beta/2}\, s/\sqrt{n}$, smaller but not a guarantee; solve for $n$ to plan |
| The exact noise distribution (Probability distributions) | Its quantile | Laplace $b\ln(1/\beta)$; normal $\sigma z_{1-\beta/2}$ |

| What you know | Use | Width at failure probability $\beta$ |
|---|---|---|
| A count with mean $\mu$ | Chernoff | a miss by a fraction $r$ has chance $\leq e^{-r^2\mu/(2+r)}$ above, $e^{-r^2\mu/2}$ below |
| The mean of $n$ values in a range of length $R$ | Hoeffding | $R\sqrt{\ln(2/\beta)/(2n)}$ |
| A rate: $k$ errors out of $n$ | Clopper–Pearson | exact binomial bound |

**Hoeffding's inequality** is the planning tool, because it needs only the range of the values. Solving its width for $n$ gives the sample size for a width $t$:

$$n \geq \frac{R^2 \ln(2/\beta)}{2t^2}. \tag{2.2}$$

For accuracy, an average of zeros and ones, the range $R$ is 1. So measuring it to within one percentage point at 95% takes 18,445 test records.

For a small rate, Hoeffding is too wide: it assumes the largest variance the range allows. Take one false positive in 1,000 negatives, with one-sided 95% bounds. Hoeffding adds 0.039 to the measured rate, while **Bernstein's inequality** uses the variance $\sigma^2$ and adds only 0.0045. For values in $[0, 1]$ its width is $\sqrt{2\sigma^2 \ln(1/\beta)/n} + 2\ln(1/\beta)/(3n)$, and a variance estimated from the same data makes it a heuristic. The exact **Clopper–Pearson** bound, from the binomial distribution, gives 0.0047. With no errors it gives 0.0030, close to the "rule of three" bound $3/n$.

Noise of a known distribution needs only its quantile: Laplace noise of scale 2 lies within 5.99 of zero with 95% probability. For many draws at once, the **union bound** caps the chance of any failure at the sum of the chances, so give each of $k$ draws $\beta/k$. Because the Laplace width grows only with the logarithm of $k$, covering a thousand draws widens 5.99 only to 19.81.

The snippet prints these numbers, with the one-sided bound for $k$ errors in $n$ as `stats.beta.ppf(0.95, k + 1, n - k)` and a two-sided interval for 10 errors in 10,000.

```
import math

from scipy import stats

print(math.ceil(math.log(2 / 0.05) / (2 * 0.01**2)))  # 18445
print(f"{stats.beta.ppf(0.95, 1, 1000):.4f} {stats.beta.ppf(0.95, 2, 999):.4f}")  # 0.0030 0
ci = stats.binomtest(10, 10_000).proportion_ci(confidence_level=0.95, method="exact")
print(f"{ci.low:.5f} {ci.high:.5f}")  # 0.00048 0.00184
print(f"{stats.laplace(scale=2).isf(0.05 / 2):.2f} {stats.laplace(scale=2).isf(0.05 / 1000 /
```

By the table's two Chernoff tails, a batch whose random size averages 600 strays more than a fifth from it with probability at most $2.4 \times 10^{-5}$.

A sum of $k$ Laplace draws has the far tail of a single draw. With scale $b$, its one-sided width is the larger of $\sqrt{8kb^2 \ln(1/\beta)}$ and $4b\ln(1/\beta)$.

**McDiarmid's inequality** extends Hoeffding's to any function of independent inputs that no single input moves much. If $c_i$ is the most that changing input $i$ can move it, the chance of landing $t$ above its mean, or $t$ below, is at most

$$\exp\Big(-\frac{2t^2}{\sum_i c_i^2}\Big). \tag{2.3}$$

**Azuma's inequality** gives the same tail for a running total of steps that average zero given the past and lie in a range of length $c_i$.

## 2.4. What goes wrong

Hoeffding needs bounded values, so clip unbounded data and use the clipping range as $R$. The bounds for sums, means and rates also need independent values, so with several rows per person, count people. A two-sided error bar must put $\beta/2$ on each side, not $\beta$.

The normal approximation is not a tail bound. For a sum of ten Laplace draws of scale 1, it understates the chance of a total above 20 about ninefold. When you pick the best of $m$ models on one validation set, the union bound asks for $\beta/m$, because the winner is the one most likely to look too good.

**Derivations.** Every result here except Bernstein's inequality is derived step by step in the full edition's chapter.

# 3. Divergences

## 3.1. What it is

A **divergence** puts a number on how different two distributions are. It is zero for equal distributions and grows as they become easier to tell apart. For distributions $P$ and $Q$, write $p(x)$ and $q(x)$ for the probabilities they give an outcome $x$. Every divergence summarizes the **likelihood ratio** $p(x)/q(x)$.

The **Kullback–Leibler (KL) divergence** is the average log ratio when the data come from $P$:

$$D_{\mathrm{KL}}(P\|Q) = \sum_x p(x) \ln \frac{p(x)}{q(x)}. \tag{3.1}$$

It is not symmetric, and it adds up over independent observations.

The **Rényi divergence** is a dial set by an order $\alpha$ above 1:

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \sum_x p(x)^\alpha q(x)^{1-\alpha}. \tag{3.2}$$

It equals the KL divergence as the order approaches 1. Higher orders weigh the outcomes with large ratios more, up to the **max-divergence** $D_\infty$, the largest log ratio that any outcome gives.

The **total variation (TV)** is the mass of $P$ that does not overlap with $Q$:

$$\mathrm{TV}(P, Q) = \frac{1}{2} \sum_x |p(x) - q(x)|. \tag{3.3}$$

The hockey-stick divergence generalizes it by scaling $q$ up by a factor $\gamma$ of at least 1 and measuring the mass of $p$ that still sticks out above it. Writing $(y)_+$ for the positive part of $y$, it is

$$E_\gamma(P\|Q) = \sum_x \big(p(x) - \gamma q(x)\big)_+. \tag{3.4}$$

At $\gamma = 1$ it is the total variation, and it reaches zero once $\gamma$ exceeds the largest likelihood ratio. The KL divergence, the TV and $E_\gamma$ all belong to one family, the $f$-divergences.

## 3.2. When you meet it

You meet divergences when a feature drifts between training and serving, or when a test-data shift may move a classifier's accuracy. Papers state results in one divergence that you may need in another.

## 3.3. How to use it

To measure drift, bin both samples on the same edges and call `scipy.stats.entropy(p, q)`. It normalizes both histograms and returns natural-log units (nats), or bits with `base=2`. Take a feature whose last bin, empty last month, holds a tenth of this month's data. With last month as $P$, the divergence is 0.1116 nats, but the reverse is infinite, because this month has mass where last month had none.

```
import math

from scipy import stats

before, after = [0.5, 0.3, 0.2, 0.0], [0.4, 0.3, 0.2, 0.1]
print(f"{stats.entropy(before, after):.4f} {stats.entropy(before, after, base=2):.4f}")  # 0
print(stats.entropy(after, before))  # inf
print(math.ceil(2 * 0.9**2 / 0.005), math.ceil((2 * stats.norm.ppf(0.95) / 0.1) ** 2))  # 32
```

For a copy shifted by $s$, closed forms replace histograms. For normal noise, write $\sigma$ for the standard deviation and $\mu = s/\sigma$ for the shift measured in it. Then

$$D_{\mathrm{KL}} = \frac{\mu^2}{2}, \qquad D_\alpha = \frac{\alpha\mu^2}{2}, \qquad D_\infty = \infty, \tag{3.5}$$

the last because the ratio of normal densities is unbounded. Because the shift enters squared, a drift of 0.1 standard deviations costs a KL divergence of only 0.005. For Laplace noise, write $b$ for the scale and $t = s/b$ for the shift in scales. Its max-divergence is $t$, its TV is $1 - e^{-t/2}$, and its KL divergence is $t + e^{-t} - 1$, which is 0.3679 at $t = 1$.

The TV says how well any test can tell $P$ from $Q$. If the two are equally likely in advance, the best test from one observation has an **advantage**, its true-positive rate minus its false-positive rate, equal to the TV, and it is right with probability $\frac{1}{2} + \frac{1}{2}\mathrm{TV}$. For two normal curves one standard deviation apart, the TV is 0.3829, so no classifier is right more than 0.6915 of the time. When the test distribution moves, a fixed classifier's accuracy moves by at most the TV.

For the same normal pair, the hockey-stick divergence is

$$E_\gamma = \Phi\Big(\frac{\mu}{2} - \frac{\ln\gamma}{\mu}\Big) - \gamma\,\Phi\Big(-\frac{\mu}{2} - \frac{\ln\gamma}{\mu}\Big), \tag{3.6}$$

where $\Phi$ is the standard normal distribution function. At $\mu = 1$ and $\gamma = 2$ it is 0.1906.

**Pinsker's inequality** converts a KL divergence into a bound on the TV:

$$\mathrm{TV}(P, Q) \le \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(P\|Q)}. \tag{3.7}$$

It is quick but loose. For the drift of 0.1 standard deviations, with KL 0.005 per sample, a test right 95% of the time needs a TV of 0.9, so Pinsker asks for at least 324 samples. The exact answer is 1,083, where the TV of the two sample means, $2\Phi(\sqrt{n}\,\mu/2) - 1$, reaches 0.9 (the snippet's last call).

## 3.4. What goes wrong

The KL divergence has a direction, so say which one you computed, and never mix nats with bits. An empty bin makes it infinite, so add a pseudocount, such as half a count per bin, or merge bins, and report which you did. Estimates change with the bins and the sample size, so compare only estimates made the same way. And `scipy.spatial.distance.jensenshannon` returns the square root of the Jensen–Shannon divergence, a symmetric relative of KL.

The **data processing inequality** says that binning, thresholding or any other processing of an observation alone cannot make two distributions easier to tell apart. For normals one standard deviation apart, keeping only whether an observation lies below the midpoint cuts the KL divergence from 0.5 to 0.3090. But it keeps all of the TV, because that threshold is already the best test.

A small KL divergence is an average, so it can hide rare outputs that are overwhelming evidence. When every output matters, bound the worst case with the max-divergence, a high Rényi order or the hockey-stick divergence.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 4. Hypothesis testing

## 4.1. Why this matters

A spam filter must call an email legitimate or spam, and a fraud detector must call a transaction normal or fraudulent. In each case an observation came from one of two distributions, and you must decide which. So every detector and every binary classifier is a hypothesis test, and evaluating one means measuring its two kinds of error, false alarms and missed cases.

What matters throughout is how a detector does when false alarms must be rare, which accuracy hides.

## 4.2. Tests and their two errors

The default assumption, called the **null hypothesis** $H_0$, is that the observation comes from a distribution $P$, such as a fraud model's scores for normal transactions. The **alternative hypothesis** $H_1$ is that it comes from a distribution $Q$, such as its scores for fraudulent ones. A **test** decides whether to reject the null, which here means flagging the transaction.

A test can make two kinds of mistake. Flagging a normal case is a false positive, and its probability is the **false positive rate (FPR)**, written $\alpha$. Missing a real case is a false negative, with probability $\beta$, the **false negative rate (FNR)**. The share of real cases caught, $1-\beta$, is the **true positive rate (TPR)**.

Lowering a test's threshold catches more real cases but raises more false alarms. So you fix a budget for the FPR first, called the **level**, and then find the test with the highest TPR, or **power**, within it.

## 4.3. The Neyman–Pearson lemma

Write $p(x)$ and $q(x)$ for the probabilities, or densities, of an observation $x$ under the two hypotheses. Their quotient is the **likelihood ratio**

$$\Lambda(x) = \frac{q(x)}{p(x)}, \tag{4.1}$$

and a **likelihood ratio (LR) test** rejects wherever it exceeds a threshold $\tau$, set so that the test's FPR equals the budget. The **Neyman–Pearson lemma** says that no other test at that level catches more real cases.

So the best test looks at an observation only through its likelihood ratio. A threshold on a raw score is that test only when a higher score always means a higher ratio. To estimate the ratio from labelled scores, fit a logistic regression of the label on the score; its log-odds, less the log prior odds, estimate $\ln \Lambda$. If a known group shifts the scores, the ratio depends on the group, so the best test thresholds within each group. Independent observations of one case combine by adding their log ratios, and at a fixed FPR the FNR then falls exponentially with their number.

## 4.4. Trade-off curves

To say how hard it is to tell $P$ from $Q$, record the best FNR at every level. As a function of the budget, this is the **trade-off function** $T(P, Q)$, and you compute it by sweeping the LR threshold. The **receiver operating characteristic (ROC)** curve holds the same information upside down, as TPR against FPR. Any particular detector traces its own ROC curve, which lies on or below the best one.

## 4.5. The Gaussian trade-off curve

A common pair is a score that is normal under both hypotheses, with the same spread, scaled so that normal cases have mean 0 and standard deviation 1. Write $\mu$ for the distance between the two centres in standard deviations, and $\Phi$ for the standard normal cumulative distribution function. The best test thresholds at $\Phi^{-1}(1-\alpha)$, and its FNR at level $\alpha$ is the Gaussian trade-off function

$$G_\mu(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu). \tag{4.2}$$

In SciPy this is `norm.cdf(norm.isf(alpha) - mu)`, with `isf` because `norm.ppf(1 - alpha)` loses the digits that matter at a tiny $\alpha$. So a single number $\mu$ fixes the best TPR at every FPR. Turned around, a measured pair gives $\mu = \Phi^{-1}(1-\alpha) + \Phi^{-1}(\text{TPR})$, `norm.isf(alpha) + norm.ppf(tpr)`. If that estimate changes with $\alpha$, the scores are not of this shape.

For example, take $\mu = 2$. At an FPR of 5%, the best test has a TPR of 0.6388. At an FPR of $10^{-3}$, it drops to 0.1378.

## 4.6. The Laplace trade-off curve

For a Laplace distribution and a copy shifted by $\kappa$ times its scale, the likelihood ratio is capped between $e^{-\kappa}$ and $e^{\kappa}$ (Divergences). No test can buy power faster than the cap per unit of FPR, so every test has

$$\text{TPR} \le e^{\kappa}\,\text{FPR}. \tag{4.3}$$

At a low FPR the best test reaches this cap, so at $\kappa = 1$ and an FPR of 0.05 its TPR is 0.1359.

A Laplace pair at $\kappa = 1$ and a normal pair at $\mu = 1.0301$ have the same best advantage, the largest TPR minus FPR of any test (Section 4.7). But at a level of $10^{-6}$ the normal pair allows 36 times the TPR, because its unbounded ratio lets a test find rare, highly telling observations. So judge a pair at the level you will use.

## 4.7. Best advantage and total variation

The **advantage** of a test is its TPR minus its FPR. The best advantage over all tests is the **total variation (TV)** distance of Divergences:

$$\max(\mathrm{TPR} - \mathrm{FPR}) = \mathrm{TV}(P, Q) = \frac{1}{2} \sum_x |q(x) - p(x)|. \tag{4.4}$$

It is reached by the test that rejects exactly where $q$ exceeds $p$. For the normal pair it is $2\Phi(\mu/2) - 1$, or 0.3829 at a shift of one standard deviation. Accuracy on a balanced set is one half plus half the advantage, so it measures the same point, which says nothing about the low-FPR end of the curve.

## 4.8. ROC summaries: why AUC hides the low-FPR regime

Detectors are often compared by the **area under the curve (AUC)** of their ROC curves. It is the chance that a random positive scores higher than a random negative. But the area weighs every FPR from 0 to 1 equally, so the low FPRs where a strict detector operates add almost nothing to it.

Two detectors with standard normal scores on negatives make the point. Detector A's positives score normal with mean 1 and standard deviation 1. B's are more spread out, with standard deviation 2. Their mean, 1.5811, is chosen so that both detectors have an AUC of 0.7602. Yet at an FPR of $10^{-3}$, A catches 0.0183 of the positives and B about 12 times as many (Figure 4.1). A few of B's positives score far above every negative, and a strict threshold catches exactly those.

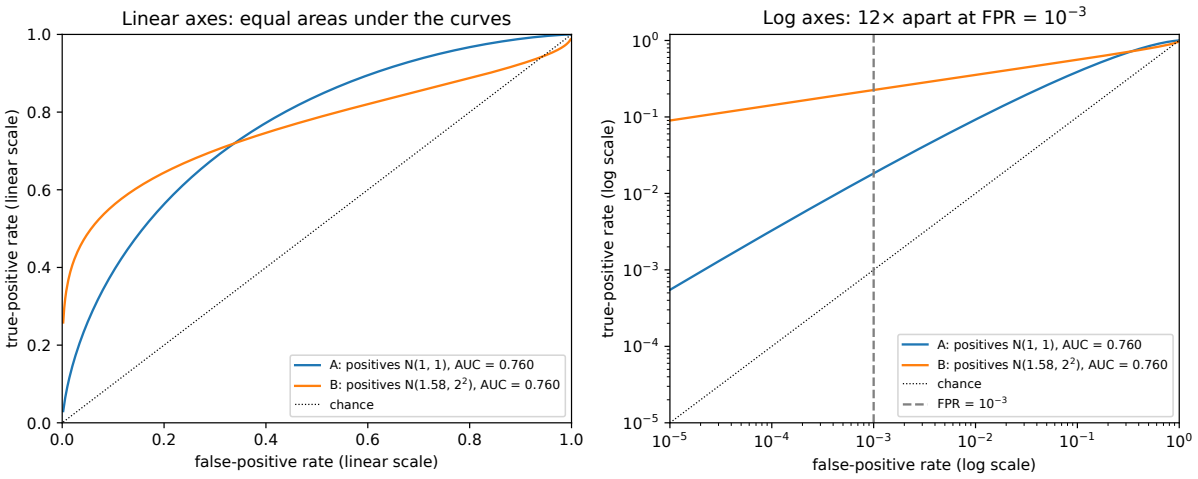



Figure 4.1.: Two detectors with the same AUC of 0.760, on linear axes (left) and log axes (right).

So plot the ROC curve on log–log axes, dropping the first point of `roc_curve`, whose FPR is 0, and report the TPR at fixed low FPRs. In scikit-learn, `roc_curve(y_true, y_score)` needs scores, not the labels from `predict`. At a target FPR, take the highest TPR among the points at or below it, not a value interpolated with `np.interp`, which no single threshold reaches. And don't read a TPR at an FPR below one over the number of negatives, because there the curve rests on one false positive or none.

## 4.9. The Bayesian view

Often you know a base rate $\pi$, such as one transaction in a thousand being fraud. Bayes' rule says that the odds of $H_1$ after seeing $x$ are the odds before, multiplied by the likelihood ratio:

$$\frac{\Pr[H_1 \mid x]}{\Pr[H_0 \mid x]} = \frac{\pi}{1-\pi} \times \Lambda(x). \tag{4.5}$$

So the decision with the fewest errors, which says $H_1$ when these odds exceed 1, is again an LR test.

The same rule applies to an alarm, whose likelihood ratio is the TPR divided by the FPR. Take the test with $\mu = 2$ in a population where 1% are positive. At an FPR of 0.05, a flagged case is positive with probability only 0.114, so most alarms are false. At an FPR of $10^{-3}$, that probability rises to 0.582. So when positives are rare, the FPR decides whether an alarm is worth believing.

## 4.10. Measuring rates from finite trials

You measure a detector's FPR by counting false positives among $n$ test negatives. If each negative is flagged independently with the same small probability, the count $k$ is binomial and fluctuates by about $\sqrt{k}$.

The **Clopper–Pearson interval** gives the range of true rates that make the observed count unsurprising. Write $1-\alpha$ for the confidence level, so that $\alpha$ is the probability that the interval misses the true rate. Its lower end $l$ and upper end $u$ solve

$$\Pr[\mathrm{Bin}(n, l) \geq k] = \frac{\alpha}{2}, \qquad \Pr[\mathrm{Bin}(n, u) \leq k] = \frac{\alpha}{2}. \tag{4.6}$$

It is exact, because it uses the binomial itself, and conservative, because it covers the true rate at least $1-\alpha$ of the time. In SciPy, `binomtest(k, n).proportion_ci(method="exact")` computes it.

To claim that an FPR is below a target, you need only the upper end, and passing `alternative="less"` to `binomtest` gives that one-sided bound. With no events at all, the one-sided 95% bound is about $3/n$, the **rule of three**. Computed exactly, certifying an FPR below $10^{-3}$ takes 2,995 negatives with no false positive.

These intervals are wide. Even 10 false positives in 10,000 negatives give the interval $[0.00048, 0.00184]$, from about half to almost twice the observed rate. About 100 events pin a rate to within about 20%. So measuring an FPR of $p$ takes on the order of $100/p$ negatives. The **Wald interval**, which treats the count as normal, fails here: it is too narrow and collapses to $[0, 0]$ with no events.

## 4.11. Evaluating a detector at a low false positive rate

1. Fix the target FPRs, such as $10^{-2}$ and $10^{-3}$.
2. Plan the trials with about $100/p$ negatives for a target FPR $p$.

3. Choose the threshold on the negatives of one split and measure the rates on another, because a threshold tuned on the evaluation set looks better there than it is. The calibration split needs as many negatives, since below one over their number `np.quantile` only interpolates between the top scores.
4. Report the TPR and FPR measured at that threshold, with Clopper–Pearson intervals on both, and the full ROC curve on log–log axes. A TPR read off the test split's ROC curve is optimistic, because it picks its threshold on the test data.
5. Compare two detectors at the same FPR, never by AUC or accuracy alone, and on a shared evaluation set check the gap with McNemar's test, `binomtest(m1, m1 + m2, 0.5)`, where $m_1$ and $m_2$ count the positives that only one of them catches.

The snippet below runs steps 3 and 4 on the breast cancer dataset bundled with scikit-learn, with a logistic regression as the detector and malignant tumours as positives. It holds out 30% of the data for calibration and 30% for testing. The threshold is the 99th percentile of the calibration split's benign scores (`decision_function`, the model's log-odds), for a target FPR of 1%.

```
import numpy as np
from scipy.stats import binomtest
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y  # make malignant (coded 0) the positive class
X_tr, X_ev, y_tr, y_ev = train_test_split(X, y, test_size=0.6, stratify=y, random_state=0)
X_cal, X_te, y_cal, y_te = train_test_split(X_ev, y_ev, test_size=0.5, stratify=y_ev, random
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X_tr, y_tr)
threshold = np.quantile(model.decision_function(X_cal[y_cal == 0]), 0.99)  # target FPR 1%,
s = model.decision_function(X_te)
pos, neg = s[y_te == 1] > threshold, s[y_te == 0] > threshold  # flagged positives, negative
tpr = binomtest(int(pos.sum()), pos.size).proportion_ci(method="exact")
fpr_upper = binomtest(int(neg.sum()), neg.size, alternative="less").proportion_ci(method="ex
print(pos.sum(), pos.size, f"{pos.mean():.3f} [{tpr.low:.3f}, {tpr.high:.3f}]")  # 56 63 0.8
print(neg.sum(), neg.size, f"{neg.mean():.4f} <= {fpr_upper:.4f}")  # 2 108 0.0185 <= 0.0571
```

The detector catches 56 of the 63 malignant test cases, a TPR of 0.889 with a 95% interval from 0.784 to 0.954. But the threshold aimed at 1% flags 2 of the 108 benign test cases, and the one-sided 95% bound from those counts certifies only an FPR below 0.0571. Step 2 asks for about 10,000 negatives at this target, so on a dataset this small, report the interval and don't claim the target. To use every negative, cross-validate: count on each held-out fold at a threshold set without it, and pool the counts, which gives an approximate interval, since the folds share training data.

## 4.12. Cheat sheet

Table 4.1.: The questions of this chapter and their answers.

| Question | Answer |
|---|---|
| What is the best test? | Threshold the likelihood ratio, per group if groups score differently (Neyman–Pearson) |
| How good can any test be? | The trade-off curve; normal pair $G_\mu(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu)$ |
| What does a capped ratio allow? | $\mathrm{TPR} \le e^\kappa \,\mathrm{FPR}$ |
| What do accuracy and advantage measure? | Best advantage = TV, one point of the curve |
| Why is AUC not enough? | Equal AUC can hide a 12-fold gap in TPR at an FPR of $10^{-3}$ |
| What does a base rate do? | Posterior odds = prior odds × TPR/FPR |
| How sure is a measured rate? | Clopper–Pearson; zero events give about $3/n$ at 95% |

**Rules of thumb**

- Plan for $100/p$ negatives, and use Clopper–Pearson, never Wald.
- Report TPR at fixed low FPRs on log–log axes, never AUC alone.

## 4.13. References


- Fawcett, "An introduction to ROC analysis" (Pattern Recognition Letters, 2006): ROC curves and their pitfalls.
- Brown, Cai & DasGupta, "Interval Estimation for a Binomial Proportion" (Statistical Science, 2001): why Wald fails and what to use.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 5. Information theory

## 5.1. What it is

Information theory measures how much one quantity tells you about another, with two sets of measures for two questions. The first, Shannon's, asks how many bits an observation saves, on average, when you describe a hidden value. The **entropy** of a random variable $X$ is the average number of bits per value that the best compression code needs. Writing $p(x)$ for the probability of the value $x$, with logarithms base 2, it is

$$H(X) = -\sum_{x} p(x) \log_2 p(x). \tag{5.1}$$

The **mutual information** $I(X;Y)$ is the drop in the entropy of $X$ that seeing an observation $Y$ causes on average. From a joint table it is $H(X) + H(Y) - H(X,Y)$, where $H(X,Y)$ is the entropy of the pairs. Unlike a correlation, it is zero only when the two are independent. The most mutual information that a noisy channel can carry, over all input distributions, is its **capacity**.

The second question asks how much more likely a single best guess becomes. A guesser with one try picks the most likely value, so it wins with the largest probability, called the **vulnerability** $V(X)$. Its negative logarithm is the **min-entropy**,

$$H_\infty(X) = -\log_2 \max_x p(x). \tag{5.2}$$

Min-entropy is never larger than entropy.

## 5.2. When you meet it

Decision trees call mutual information the **information gain**, and feature selection often ranks features by it. Min-entropy matters instead whenever a value must be hard to guess, such as a password or an identifier. Use Shannon's for averages and features, and min-entropy for any value that an attacker gets one guess at.

## 5.3. How to use it

`scipy.stats.entropy(p, base=2)` returns the entropy of a distribution in bits. For two discrete columns, `sklearn.metrics.mutual_info_score(a, b)` plugs their observed frequencies into the definition of mutual information. Continuous values rarely repeat, so `mutual_info_classif` in `sklearn.feature_selection` estimates it from nearest-neighbour distances instead. Both return nats (natural-log units), so divide by $\ln 2$ to get bits.

A uniform 4-digit PIN has $\log_2 10{,}000 = 13.2877$ bits of entropy. Now suppose 10% of people choose 1234 and the rest choose evenly among the others. The entropy barely moves, to 12.4278 bits. But the min-entropy falls to 3.3219 bits, because the guess "1234" succeeds one time in ten.

To measure how much an output helps one guess, start from two tables. The prior $\pi(x)$ says how likely each secret is, and the channel $K[x, y]$ says how likely each output $y$ is when the secret is $x$. Multiplying each row of the channel by its prior gives the joint table. The best guess after seeing $y$ is the row with the largest entry in column $y$, so its success probability is the sum of the column maxima:

$$V(X \mid Y) = \sum_y \max_x \pi(x) K[x, y]. \tag{5.3}$$

The logarithm of $V(X \mid Y)/V(X)$ is the **min-entropy leakage**, and 1 bit of it doubles the chance of a correct guess. In the full edition's example with four secrets, the output raises that chance from 0.4 to 0.66, so the leakage is $\log_2(0.66/0.4) = 0.7225$ bits.

If you do not know the prior, use the worst case over priors, the **min-capacity**. It is the logarithm of the sum of the channel's own column maxima, 1.2630 bits in the example.

Mutual information can also show that a task is hard, through the uncertainty $H(X \mid Y) = H(X) - I(X;Y)$ that it leaves. Take a secret with $m$ possible values, and a guess from the output with error probability $P_e$. Writing $h(P_e)$ for the entropy of a coin with that bias, Fano's inequality says

$$H(X \mid Y) \le h(P_e) + P_e \log_2(m - 1). \tag{5.4}$$

For example, a label uniform over 8 classes has 3 bits of entropy. If the features carry at most 1 bit about it, 2 bits remain, and every classifier errs at least 0.3730 of the time. Get such a number by solving Equation 5.4 for $P_e$ numerically, with `scipy.optimize.brentq`. For a binary label, $m - 1 = 1$ and the last term vanishes.

## 5.4. What goes wrong

Plug-in estimates of mutual information run high on small samples, so two independent columns rarely give exactly zero. Compare an estimate with the same estimate after shuffling one column, which shows what independence alone produces.

Shannon entropy is the wrong way to rate guessing difficulty, as the PIN shows, and low mutual information does not mean that little is revealed. Take a uniform 64-bit secret and a channel that outputs all of it when its top 3 bits are zero, one time in eight, and a fixed symbol otherwise. Its mutual information is 8.1686 bits, less than the 9 bits of a channel that always reveals the low 9 bits. Yet a guesser who sees its output wins one time in eight. Min-entropy leakage catches this channel, at 61 bits, but it too averages over secrets, so it can stay small when one rare value is fully exposed.

Processing an output cannot add information about its source. If a variable $Z$ is computed from $Y$ alone, possibly with fresh randomness, the data-processing inequality says

$$I(X; Z) \le I(X; Y). \tag{5.5}$$

The condition matters, because a $Z$ that also looked at $X$ can know more.

## 5.5. References

- Smith, “On the Foundations of Quantitative Information Flow” (FoSSaCS 2009): why one-try guessing needs min-entropy.

**Derivations.** Every result here is derived step by step in the full edition’s chapter.

# 6. Optimization

## 6.1. What it is

Training a model minimizes the average loss over the $n$ records of its training set $D$, usually by stochastic gradient descent (SGD). Each step moves the parameters $\theta$ against the gradient of a random minibatch, scaled by a learning rate $\eta$, also called the step size. A loss is **$L$-Lipschitz** when its gradient never has length more than $L$, and **smooth** when its gradient changes by at most a constant $\beta$ per unit of distance.

A loss is also **strongly convex** when its curvature is at least a constant $\lambda$ in every direction, which pins down a single minimizer. Table 6.1 gives all three for common losses.

Table 6.1.: The constants of common losses, for labels $\pm 1$ and features of length at most 1.

| Loss | Lipschitz | Smooth | Strongly convex |
|---|---|---|---|
| Logistic, $\ln(1 + e^{-y\langle\theta,x\rangle})$ | $L = 1$ | $\beta = \frac{1}{4}$ | No |
| Hinge, $\max(0, 1 - y\langle\theta, x\rangle)$ | $L = 1$ | No, it has a kink | No |
| Squared, $\frac{1}{2}(\langle\theta, x\rangle - y)^2$ | No, the gradient grows with $\theta$ | $\beta = 1$ | Only if the features span every direction |
| Any of these plus $\frac{\lambda}{2}\|\theta\|^2$ | Only on a bounded set: $L + \lambda r$ within radius $r$ | $\beta + \lambda$ | $\lambda$ |

## 6.2. When you meet it

You meet these results when you choose a learning rate, a batch size and a number of steps, and when you read scikit-learn's regularization arguments. They also bound one training record's influence, which the next chapter turns into a bound on the generalization gap.

## 6.3. How to use it

For a convex loss, run SGD for a number $T$ of steps inside a ball of radius $R$ that contains the minimizer. Return the **averaged iterate** $\bar{\theta}_T$, the average of the parameters over all steps. The minibatch gradients have an average squared length of at most $G^2$, which is $L^2$ for an $L$-Lipschitz

loss. The step that minimizes the guarantee, and the error it guarantees on the average training loss $\mathcal{L}(\theta; D)$ with smallest value $\mathcal{L}^*$, are

$$\eta = \frac{R}{G\sqrt{T}}, \qquad \mathbb{E}[\mathcal{L}(\bar{\theta}_T; D)] - \mathcal{L}^* \le \frac{RG}{\sqrt{T}}. \tag{6.1}$$

Logistic regression on normalized features has $G = 1$. In a ball of radius 5, $T = 10{,}000$ steps then get a learning rate of $5/100 = 0.05$. The formula assumes the worst case, so on the full edition's logistic data the best learning rate on a grid is 3.6 times larger. Start there and tune on a grid around it.

A ridge term $\frac{\lambda}{2}\|\theta\|^2$ makes the error fall about like $1/T$ instead of $1/\sqrt{T}$. It needs a learning rate that shrinks like $1/(\lambda t)$ at step $t$, which `SGDClassifier(alpha=λ, learning_rate="optimal", average=True)` uses. `LogisticRegression` instead minimizes `C` times the summed loss plus $\frac{1}{2}\|\theta\|^2$. Dividing by $n$ times `C` gives a ridge of strength $\lambda$ equal to the reciprocal of that product, so a large `C` means a weak ridge.

If the variance of one record's gradient around the full gradient is $\sigma_1^2$, a minibatch of $B$ records drawn without replacement has a gradient variance of

$$\mathbb{E}\|g_B - \nabla\mathcal{L}(\theta; D)\|^2 = \frac{\sigma_1^2}{B} \cdot \frac{n - B}{n - 1}. \tag{6.2}$$

The constant $G^2$ is the squared length of the full gradient plus this variance. So a larger batch lowers $G$, which allows a larger learning rate.

**Gradient-norm clipping** caps the length of each per-example gradient $g_i$ at a chosen clipping norm $C$, replacing it by $g_i \min(1, C/\|g_i\|)$. Suppose a training step subtracts $\eta/B$ times the sum of the clipped gradients, with $B$ fixed in advance. Then adding or removing one record changes the step by at most the first amount below, and replacing one by at most the second, whatever the loss:

$$\frac{\eta C}{B}, \qquad \frac{2\eta C}{B}. \tag{6.3}$$

Dividing by the actual batch size would let a removal rescale every other gradient too. It covers one step from the same starting point; over a run the changes can accumulate.

Regularized **empirical risk minimization (ERM)** minimizes the average loss plus a ridge term of strength $\lambda$. Write $\theta^*(D)$ for the minimizer on a training set $D$, and $D'$ for the same set with one record replaced. If each loss is convex and $L$-Lipschitz,

$$\|\theta^*(D) - \theta^*(D')\| \le \frac{2L}{\lambda n}. \tag{6.4}$$

For logistic regression on 1,000 records, $\lambda = 0.1$ gives a bound of 0.02. On the full edition's data the largest movement observed is $6.66 \times 10^{-3}$, so the bound is loose.

## 6.4. What goes wrong

A single unnormalized feature vector breaks $L = 1$, so normalize every feature vector to length 1 or less. The decaying learning rate needs a value of $\lambda$ no larger than the true curvature, because

an overestimate shrinks the steps too fast and training stalls. And a learning rate above $2/\beta$ can raise the loss, the usual cause of a diverging loss curve.

On a neural network's non-convex loss, SGD guarantees only a point with a small gradient, perhaps a saddle point. The movement bound needs the ridge term's strong convexity: without it, one replaced record can move the median, the absolute loss's minimizer, from 0 to 1. So for a neural network, a statement about one record's influence must be about the training steps, as in Equation 6.3, not about the minimizer.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 7. Algorithmic stability and generalization

## 7.1. What it is

A model's **generalization gap** is its average loss on new data minus its average loss on the training data. A model generalizes when it does not depend much on any single training record. Replace one training record by a fresh draw from the same source, retrain, and see how much higher the new model's loss is than the old one's on the record you took out. This difference is the record's **memorization score**. Averaged over the $n$ training records and over random training sets, the scores give the expected gap exactly:

$$\mathbb{E}[\text{gap}] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}[\text{mem}_i]. \tag{7.1}$$

Here $\text{mem}_i$ is the score of the $i$-th record.

To bound the scores, call a learning algorithm $\beta$**-uniformly stable** when replacing any one training record moves the loss at any test point by at most $\beta$. Every score is then at most $\beta$, so the expected gap is too:

$$\big|\mathbb{E}[\text{gap}]\big| \le \beta. \tag{7.2}$$

Regularized training and stochastic gradient descent (SGD) stopped early are stable in this sense, while a model that stores its training set is not.

## 7.2. When you meet it

You meet stability when you choose a regularization strength and a number of epochs, and when a training error of 0 says nothing about the test error. Memorization scores tell you which records a model has learned by heart.

## 7.3. How to use it

For regularized **empirical risk minimization (ERM)**, which minimizes the training loss plus a ridge penalty of strength $\lambda$, the stability is

$$\beta = \frac{2L^2}{\lambda n}. \tag{7.3}$$

It needs a convex loss whose gradients have length at most $L$ (Optimization). Normalizing the features to length at most 1 gives $L = 1$ for the logistic loss. So with 1,000 records and $\lambda = 0.1$, logistic regression has $\beta = 0.02$. In scikit-learn, `LogisticRegression(C=...)` sets $\lambda$ to the

reciprocal of $n$ times `C`, so a large `C` means a large $\beta$. Its intercept is not penalized, so strictly the bound covers only `fit_intercept=False`.

On the full edition's simulated data, the measured gap falls like $1/n$, as the bound predicts. But it stays between about 3% and 13% of the bound, depending on $\lambda$. Equation 7.1 is an average over training sets, so average the measured gap over several random splits. Use the bound to see which direction helps, and choose $\lambda$ itself on held-out data, since a strong ridge also pulls the weights away from the best fit.

SGD on a convex and smooth loss is also stable if it stops early. Hardt, Recht and Singer (2016) showed that with a constant step $\eta$, its stability after $E$ epochs is at most $2L^2\eta E$. So training longer trades a better fit for less stability, which is what early stopping balances. In the full edition's unregularized example, the gap grows from 0.036 after one epoch to 0.121 after 40. The bound gets the direction right, but after 40 epochs it is 165 times the measured gap.

To measure a memorization score, retrain with the record removed, or replaced by a held-out point in place of a fresh draw, and compare its loss. That costs one retraining per record. So for a large model, score a sample, or train many models on random subsets and compare each record's loss when it is in and out. The scores are uneven, because a typical record is fitted anyway through similar records, while a mislabelled or rare one is fitted only when it is there itself. In the full edition's example at $\lambda = 0.01$, records with the less likely label for their features score three times higher on average than the others.

## 7.4. What goes wrong

A perfect training score is a warning, not a result. The 1-nearest-neighbour rule, `KNeighborsClassifier(n_neighbors=1)`, has training error 0, and under the 0–1 loss its $\beta$ is 1, so the bound is empty. With noisy labels its gap never closes, and in the full edition's example its test error stays above 0.338.

A small average gap does not mean that no record was memorized, because the average leaves room for a few large scores. So look at the largest scores, not only the mean. Some memorization of rare records is needed for accuracy on data with a long tail of rare subpopulations (Feldman, 2020), so a large score is not always a defect.

A bound that holds with 95% confidence for one run is looser still. For logistic regression on 10,000 records with $\lambda = 0.1$, the Bousquet–Elisseeff bound adds 0.614 to the training loss. That is close to the 0.69 loss of a coin flip.

Finally, a held-out set is honest only for questions fixed before you look at it. Once you look at a result, change something and ask again, you have trained on the holdout. Freedman's paradox shows how large the effect is. On 1,000 records whose labels are independent of every feature, each feature's agreement with the label is a question fixed in advance, and their errors stay below the bound, 0.0616. But a majority vote of the features that agree most reports an accuracy of 0.8053 on the same data, against 0.5018 when the features are chosen on a separate half. So split with `train_test_split` before you start, tune with `cross_val_score` on the training part, and use the test set once.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 8. Cryptography basics

## 8.1. What it is

A security claim is a statement about a guessing game. A challenger secretly picks one of two worlds, such as encrypting one of two messages, and an adversary sees the result and guesses. Write $X$ and $Y$ for what it sees in each world. If an adversary, written $\mathcal{A}$, answers 1 for the second, its **advantage** is the gap between how often it says 1 in each:

$$\mathrm{Adv}_{\mathcal{A}}(X, Y) = \big|\Pr[\mathcal{A}(X) = 1] - \Pr[\mathcal{A}(Y) = 1]\big|. \tag{8.1}$$

If each world is equally likely, it wins with probability $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}_{\mathcal{A}}$, and no advantage exceeds the total variation distance between $X$ and $Y$ (Divergences).

A scheme has **$\kappa$-bit security** when no attack beats trying keys. An attacker who tries $t$ keys then gets an advantage of at most about $t/2^{\kappa}$. For example, testing $10^{18}$ keys per second for a year breaks a 128-bit key with probability $9.3 \times 10^{-14}$.

## 8.2. When you meet it

You meet these tools whenever a pipeline pseudonymizes identifiers, generates keys, encrypts data or splits a value among holders. Table 8.1 gives the construction for each job; never implement your own.

Table 8.1.: The construction to use for each job, and the mistake to avoid.

| Job | Use | In Python | What goes wrong |
|---|---|---|---|
| Encrypt data | Authenticated encryption, such as AES in Galois/Counter Mode | `AESGCM` from the `cryptography` package | A reused nonce reveals data |
| A keyed pseudonym or tag | The hash-based message authentication code (HMAC) | `hmac.new(key, msg, hashlib.sha256)` | A plain hash is reversed by hashing every candidate |
| Keys, nonces, identifiers | A cryptographic random generator | `secrets.token_bytes` | `random` and seeded NumPy generators are predictable |

| Job | Use | In Python | What goes wrong |
|---|---|---|---|
| Split a value among holders | Additive secret sharing modulo a prime | `secrets.randbelow(p)` | Shares not reduced modulo $p$ leak |
| Compute on encrypted values | Lattice-based homomorphic encryption | SEAL, TenSEAL or OpenFHE | Every operation spends a noise budget |

## 8.3. How to use it

The **birthday bound** says that when a number $m$ of values are drawn uniformly from a number $N$ of possible values, the chance of a repeat is at most

$$\Pr[\text{collision}] \le \frac{m(m-1)}{2N}. \tag{8.2}$$

A repeat becomes as likely as not after about $1.18\sqrt{N}$ draws, so random $\kappa$-bit values start to collide after about $2^{\kappa/2}$. For example, 1,000 features hashed into $2^{20}$ buckets collide with exact probability 0.379.

To split a secret among several holders, draw all shares but one uniformly modulo a prime $p$, and choose the last so that they add up to the secret. Adding two secrets' shares holder by holder gives shares of their sum. The snippet shares 42 and 100 and adds them. For 10 million identifiers, its birthday bounds show why a pseudonym keeps 128 bits of the HMAC. At 64 bits the bound is $2.7 \times 10^{-6}$.

```
import hashlib
import hmac
import secrets

p = 2**61 - 1  # a prime larger than any secret or sum of secrets
a, b = [secrets.randbelow(p) for _ in range(2)], [secrets.randbelow(p) for _ in range(2)]
a, b = [*a, (42 - sum(a)) % p], [*b, (100 - sum(b)) % p]  # three shares each of 42 and 100
print(sum(a) % p, sum(x + y for x, y in zip(a, b)) % p)  # 42 142

key = secrets.token_bytes(32)  # kept apart from the data
pseudonym = hmac.new(key, b"alice@example.com", hashlib.sha256).hexdigest()[:32]  # 128 bits
q = 10**7  # identifiers
print(f"{q * (q - 1) / (2 * 2**128):.1e} {q * (q - 1) / (2 * 2**64):.1e}")  # 1.5e-25 2.7e-0
q, n = 2**32, 2**96  # messages under one key, 96-bit nonces
print(f"{q * (q - 1) / (2 * n):.1e}")  # 1.2e-10
```

The `cryptography` package is not installed here, so this encryption sketch was not run:

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
key = AESGCM.generate_key(bit_length=256)
nonce = secrets.token_bytes(12)             # 96 bits, fresh for every message
ct = AESGCM(key).encrypt(nonce, data, aad)  # aad: authenticated, not encrypted
```

HMAC is secure if SHA-256's compression function is a pseudorandom function. Encryption with AES assumes that AES is a pseudorandom permutation, and lattice schemes that learning with errors is hard. Secret sharing needs none: a holder's shares are random numbers whatever the secret, which **simulation** makes precise.

Homomorphic encryption hides each value behind a small random error modulo a number $Q$. Decryption works while the total error is below $Q/4$. Adding ciphertexts adds their errors, so if each error is at most a bound $B$ in size, a sum of $k$ ciphertexts is guaranteed to decrypt when $k < Q/(4B)$. With $Q = 4096$ and $B = 32$, that guarantees 32 ciphertexts. A larger modulus buys a larger **noise budget**, at the cost of slower arithmetic.

## 8.4. What goes wrong

A repeated nonce reveals the XOR of the two messages, so rotate keys before the birthday bound bites.

Shares must be reduced modulo $p$, or their range moves with the secret. An unreduced share equal to the secret plus a random number from 0 to 15 tells the secrets 0 and 7 apart with advantage $7/16$.

Finally, "no efficient adversary gets more than a negligible advantage" is an asymptotic claim and says nothing about the key length you use.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# Part II.

# Threat models and attacks

This part teaches you to attack your own models and releases before someone else does. You need the attacks even if you plan to use differential privacy, because they are how you check that an implementation keeps its promise. Every attack here is a game, in which a model or a release depends on a hidden secret and an adversary sees what the threat model allows. Its guess counts only by how much it beats a baseline that never looks at the model.

Start with Threat models and privacy games, and then read Membership inference. After those two, pick the chapter whose secret matches your question. For a trained model, membership inference asks whether one record was used, and Memorization and extraction seeks text the model memorized. In federated learning, Gradient inversion rebuilds a client's input from its updates. Two short reference pages cover attacks you meet less often, attribute inference, which seeks a person's sensitive field, and property inference, which seeks a fact about the whole training population.

For released data, read the last three chapters in order, because together they tell one story about releasing less. Records without names still fall to linkage (Linkage and re-identification). Records hidden in crowds still give away sensitive values (Syntactic anonymity). And accurate aggregates alone can give away the whole dataset, as the reference page on reconstruction attacks shows. This is the Fundamental Law of Information Recovery: answering too many statistics too accurately reveals the data. So "we removed the names" and "we only release aggregates" are not anonymization, and the next part starts from differential privacy instead.

Two lessons drive every audit in this part. First, report the true positive rate at a low false positive rate, such as one in a thousand, not accuracy or the area under the curve. Second, the worst-off records leak first, such as outliers, duplicates and people with a unique combination of attributes, so an average over all records misleads. A failed attack proves little, because an attack only gives a lower bound on the leak.

The part ends with a workflow, Pre-release privacy risk assessment. It fixes the blocking thresholds before any attack runs. Then it audits a classifier with shadow models, a fine-tuned language model with planted secrets and a table with quasi-identifiers, and writes the decision down. Its scripts run on public data, and every number on these pages comes from code you can run. By the end of this part, you will be able to write a threat model and run a membership audit you can defend. You will also be able to test a table for unique records and decide whether a model or a dataset is safe to release.

# 9. Threat models and privacy games

## 9.1. Why this matters

A hospital trains a model on patient records and offers it to doctors through a web service. Whether the model is private has no answer until you say **against whom**. One attacker reads only predicted labels, while another holds the weights, knows the other patients or slips crafted records into the training data. A **threat model** pins these assumptions down. Without one, "the model leaks" is a claim nobody can test or compare.

Once the threat model is fixed, an attack becomes a **game**, in which the adversary guesses a hidden **secret**, such as whether one patient's record was used. Pre-release privacy risk assessment later fills in this chapter's worksheet for three real releases and runs the attacks it names.

## 9.2. Writing down the threat model

Write the threat model before anyone runs an attack, because it decides which attacks to run and what result stops the release. Table 9.1 is the worksheet. Its first six rows are the questions every threat model answers, and the last two turn the answers into a test.

Table 9.1.: The threat-model worksheet.

| Question | What to write down | Default when unsure |
|---|---|---|
| Goal: what is the secret? | Membership, an attribute of a known record, the records themselves, or a property of the whole dataset | Membership, the goal that differential privacy bounds directly |
| Access: what does the adversary see? | Labels only, scores, the weights, or the training transcript | The most anyone outside the training team can get |
| Behaviour: does it follow the rules? | Honest-but-curious, or malicious (poisoning, tampered models) | Malicious, if it runs any part of the system |
| Trust: who sees the raw data? | Central curator, local (data protected on each device before collection), or distributed | Central, only if the data holder is not an adversary |

| Question | What to write down | Default when unsure |
|---|---|---|
| Side information: what does it know? | The population, public datasets, other records, the target's other fields | Every record except the target's |
| Unit: what is one record? | A row, a user, a household | The person, whenever one person has several rows |
| Attack and metric | Membership: the strongest attack's true positive rate (TPR) at a low false positive rate (FPR). A table: unique share and smallest class. A language model: canary exposure and extraction | The row of the goal and artefact in the risk assessment's thresholds |
| Acceptable result | The number that blocks the release | Fixed before any attack is run |

The access row has four levels. The weakest is label-only, the model's decisions on inputs the adversary chooses. Next come black-box scores, and then white-box access to the weights. The strongest is the **transcript**, everything training exposed along the way, such as per-step gradients. Each level can compute the one below it. A label is a function of the scores, the scores are a function of the weights, and the weights are the transcript's last entry. So in this ordering, the **access chain**, more access never weakens the best attack.

To fill the access row, read the serving code and the release plan. An "API only" service that returns full probability vectors gives black-box access, and a checkpoint shared with a partner gives the partner white-box access. A model in several deployments at once carries the risk of the most exposed one. The access chain compares only the *best* attack at each level, though, not the ones someone tried. In the full edition's experiment, the model is the centroid of 100 training points in 50 dimensions, used as an anomaly detector. A black-box attack on its raw loss has an advantage (Section 9.3) of 0.0407. Calibrating each target's score against how typical that target is raises it to 0.2789, as high as the best attack with the weights. So "the attack we tried failed" proves little (Membership inference).

Behaviour and trust decide who the adversary is. An **honest-but-curious** adversary follows the protocol and only analyses what it legitimately sees. A **malicious** one deviates from the protocol. It may insert crafted records into the training data, which is called **poisoning**, or, if it runs part of the system, send tampered models to the other participants. If whoever collects the data is itself a possible adversary, protection must happen before the data leave the user (Local, shuffle and distributed trust models).

Side information is the row you control least. So assume it knows every record except the target's, and list the public datasets that share columns with yours. The unit row matters as much, because a per-row guarantee covers each of a user's 200 rows but not all 200 together.

For example, take the hospital's classifier, served through an API that returns class probabilities. The goal is membership, the access is black-box scores, and a doctor using the service is honest-but-curious but free to query adaptively. The side information is a public sample from the same

population, and the unit is the patient. The release is blocked if the lower end of the TPR's interval at an FPR of $10^{-3}$ exceeds ten times that FPR. Write that threshold down before the attack runs, because a threshold chosen after seeing the results can always be set where they pass.

## 9.3. Scoring an attack against its baseline

In the membership game, the challenger flips a coin to decide whether to train the model with or without the target's record. The adversary then guesses whether the target is a **member**, that is, whether its record was used.

Two rates describe the guesses. The attack's **true positive rate** is the fraction of members it flags, and its **false positive rate** is the fraction of non-members it wrongly flags. Neither rate depends on how often members occur, but the accuracy, the share of games won, does, which is why accuracy misleads. Write $\pi_1$ for the chance that the coin makes the target a member, the prior, and $\pi_0 = 1 - \pi_1$ for the opposite. The attack then wins with probability

$$\Pr[\text{win}] = \pi_1 \,\text{TPR} + \pi_0 \,(1 - \text{FPR}). \tag{9.1}$$

If only one target in ten is a member, always saying "not a member" wins 90% of the games. So score every attack against a **baseline**, the best score of a guess that never looks at the model.

With a fair prior, half members and half non-members, the baseline is one half. The attack's **advantage** is how far its TPR exceeds its FPR, and at a fair prior it sets the win rate as

$$\Pr[\text{win}] = \frac{1}{2} + \frac{1}{2}\,(\text{TPR} - \text{FPR}). \tag{9.2}$$

So an attack with 55% accuracy at a fair prior has an advantage of 0.1. The advantage does not change with the prior. No attack's advantage exceeds the total variation distance between the adversary's views with and without the target (Divergences), which differential privacy later bounds.

## 9.4. Reporting: TPR at a low FPR

An attack's **receiver operating characteristic (ROC)** curve plots its TPR against its FPR as its threshold moves, and the advantage is the curve's largest height above the diagonal. Like the area under the curve, it says nothing about the region where false positives are rare, which is where privacy breaks. An attack that flags 1% of members with no false alarms has a tiny advantage, yet it identifies those members with certainty. So Carlini et al. (2022) report membership attacks by their TPR at a low FPR such as $10^{-3}$.

A flagged person cares about a third number, the chance that the verdict is right. That is the **precision**,

$$\Pr[\text{member} \mid \text{flagged}] = \frac{\pi_1 \,\text{TPR}}{\pi_1 \,\text{TPR} + \pi_0 \,\text{FPR}}. \tag{9.3}$$

When members are rare, even a small FPR applied to the many non-members swamps the true positives (Jayaraman et al., 2021). Take a count of people with a condition, released with Gaussian

noise of standard deviation 1, so that a member adds 1 to the count. An adversary who knows every other record flags the target when the count is high, with advantage 0.3829. Held to an FPR of $10^{-3}$, it catches members with probability 0.0183. Yet when it fires at a fair prior, its precision is $0.0183/(0.0183 + 0.001) = 0.948$. If only one target in a hundred is a member, the precision drops to 0.156, so never quote a precision without its prior.

The rates you report are measured on a finite set of targets, so each needs an interval. Hypothesis testing gives the recipe. It plans about $100/\alpha$ non-members for an FPR of $\alpha$, so that about 100 false positives pin the rate down. It sets the threshold on one split, measures on another, and puts a Clopper–Pearson interval on each rate. The snippet below runs a real attack, the black-box membership attack of the Adversarial Robustness Toolbox (ART). Its target is a random forest trained on half of scikit-learn's breast cancer data, 569 patient records. The attacker trains its attack on records whose membership it knows, then scores the others. The snippet reads the TPR at an FPR of at most 0.01, with both intervals. To stay short it sets the threshold on the scores it measures, but a real evaluation sets it on a separate split.

```
import numpy as np
from art.attacks.inference.membership_inference import MembershipInferenceBlackBox
from art.estimators.classification import SklearnClassifier
from scipy.stats import binomtest
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import auc, roc_curve

np.random.seed(0)  # ART's attack model draws from NumPy's global generator
x, y = load_breast_cancer(return_X_y=True)
member = np.random.default_rng(0).random(len(y)) < 0.5  # the target's training records
target = SklearnClassifier(RandomForestClassifier(random_state=0).fit(x[member], y[member]))
known = np.arange(len(y)) % 2 == 0  # records whose membership the attacker knows, to train
attack = MembershipInferenceBlackBox(target, attack_model_type="rf")
attack.fit(x[known & member], y[known & member], x[known & ~member], y[known & ~member])
fpr, tpr, _ = roc_curve(member[~known], attack.infer(x[~known], y[~known], probabilities=Tru
i = np.flatnonzero(fpr <= 0.01)[-1]  # the best TPR whose FPR is at most 0.01
n_in, n_out = int(member[~known].sum()), int((~member[~known]).sum())
tp, fp = round(tpr[i] * n_in), round(fpr[i] * n_out)
print(f"AUC={auc(fpr, tpr):.3f}  TP={tp} of {n_in}  FP={fp} of {n_out}")  # AUC=0.552  TP=3
t, f = (binomtest(k, n).proportion_ci(method="exact") for k, n in ((tp, n_in), (fp, n_out)))
print(f"TPR 95% CI [{t.low:.4f}, {t.high:.4f}]  FPR 95% CI [{f.low:.4f}, {f.high:.4f}]")  #
```

The attack is scored on only 151 non-members, so the smallest FPR it can show above zero is one in that many. An FPR of $10^{-3}$ cannot be measured at all. Even one false positive leaves the FPR's interval running from 0.0002 to 0.0363. The TPR's interval, from 0.0047 to 0.0645, overlaps it, so this test cannot show that the attack beats chance at a low FPR. Its area under the ROC curve, 0.552, adds little. Measuring an FPR of $10^{-3}$ takes about 100,000 non-members, which is why audits retrain many models on many records (Membership inference).

A report you can act on therefore gives the TPR at a stated low FPR, with its counts, intervals on both rates and the ROC curve on log axes. It also states the threat model, the baseline, the prior behind any precision, and whether the targets were random records or chosen worst cases.

## 9.5. Average case and worst case

An evaluation can draw the data and the target from the population, which gives the **average case**. Or it can let the adversary know or pick the other records and the target, which gives the **worst case**. Audits of real models measure the first, but a guarantee must control the second, and the gap can be large. In the noisy count, the adversary who knows the other records has an advantage of 0.3829. If 100 of them are unknown fair coins, their sum blurs the count like extra noise, and the advantage falls to 0.0779. But an adversary who planted the other records knows them, so a defence that relies on the crowd fails against the adversary it most needs to stop.

Averaging over targets hides the worst-treated person in the same way, because exposing a few outliers completely barely moves an average over everyone. So treat an average-case number as a lower bound on the risk. Audits narrow the gap by planting **canaries**, records built to be as exposed as possible (Memorization and training-data extraction). The definition of differential privacy bounds the worst case directly.

## 9.6. Cheat sheet

Table 9.2.: The results of this chapter.

| Quantity | Formula | Use it for |
|---|---|---|
| Advantage | $\mathrm{TPR} - \mathrm{FPR}$ | Comparing attacks at any prior; 0 means no attack |
| Precision | $\pi_1 \mathrm{TPR} / (\pi_1 \mathrm{TPR} + \pi_0 \mathrm{FPR})$ | What a flagged person faces; state the prior |

**Rules of thumb**

- Fill in the worksheet, including the blocking threshold, before running or reading any attack.
- Assume the adversary knows the other records, and audit with worst-case canaries.

## 9.7. References


- Salem et al., “SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning” (IEEE S&P 2023): every inference attack written as a game.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, “Membership Inference Attacks From First Principles” (IEEE S&P 2022): the case for reporting TPR at a low FPR on log-scaled ROC curves.


**Derivations.** Every result here is derived step by step in the full edition’s chapter.

# 10. Membership inference

## 10.1. Why this matters

A hospital publishes a model that predicts a disease, and you hold a patient's record. **Was that record in the training set?** If it was, the patient was treated there for that disease. This is **membership inference**, and it is the standard measure of leakage, because a model that cannot hide whether a record was used will hide little else.

The one idea of this chapter is that membership is a test about one record, so an attack must compare each record with itself. The attack to use in an audit is the likelihood ratio attack (LiRA). It compares the model's output on a record with that record's outputs under models trained with and without it.

## 10.2. Membership as a test about one record

The game is the one of Threat models and privacy games. The attacker gets a training member or a fresh record and says "member" or "not". It is scored by its **true positive rate (TPR)**, the share of members it flags, and its **false positive rate (FPR)**, the share of non-members it flags.

Fix one target record, written $z$, and look at the model's output on it, such as its loss. Retrain on fresh random draws of the data, and that output has one distribution over the runs that include $z$ and another over the runs without it. Call them the **IN** and **OUT** distributions of $z$. The best test of membership compares their likelihoods at the observed output (the Neyman–Pearson lemma), and it does well only where the two lie far apart. A typical record's distributions almost overlap, because the rest of the data already predict it. An outlier's lie far apart, because the model gets it right only if it trained on it, so the records an attack finds at a low FPR are mostly outliers.

The numbers in this chapter come from practitioner_lira.py, which trains 64 logistic-regression classifiers on random halves of scikit-learn's digits dataset, 1,797 images of handwritten digits. Each model takes a turn as the target, while the other 63 serve as the attacker's **shadow models**, copies trained the same way whose training images the attacker knows.

## 10.3. The quick check: the generalization gap

Before any attack, compare the training and test errors. A model fits its training data, so it gets members right more often than fresh records. Yeom et al. (2018) turned that into the attack

"member if correctly classified". When members and non-members come from the same population, its advantage, the TPR minus the FPR, equals the gap between the two errors:

$$\text{Adv} = \text{test error} - \text{training error}. \tag{10.1}$$

On the digits, averaged over the 64 target models, the gap is 0.0422.

So a large gap shows a leak at no cost. But **a small gap does not mean a private model**, because the gap is an average over records, and an average hides who is exposed. The loss attack, which flags every record whose loss falls below one threshold, shows why. On the digits, at an FPR of $10^{-3}$, it reaches a TPR of 0.0009. That is no higher than its FPR, so it does no better than a coin weighted to say "member" one time in a thousand.

The classic attack of Shokri et al. (2017) uses shadow models to learn one rule for all records. The Adversarial Robustness Toolbox (ART), a Python library of attacks, implements it, and the script runs it with four shadow models.

```
# ART copies the target's recipe and trains each shadow on a random half of the images.
shadows = ShadowModels(models.first, num_shadow_models=N_ART_SHADOWS, random_state=seed)
(x_in, y_in, pred_in), (x_out, y_out, pred_out) = shadows.generate_shadow_dataset(x, onehot)
# One attack model learns "member" from a shadow's probabilities and the true label.
attack = MembershipInferenceBlackBox(models.first, attack_model_type="rf")
attack.fit(x_in, y_in, x_out, y_out, pred=pred_in, test_pred=pred_out)
# The same attack model scores every target's probabilities on every image.
scores = [attack.infer(x, onehot, pred=p, probabilities=True)[:, 0] for p in models.probs]
```

Its one rule fails at a low FPR too, since at an FPR of $10^{-3}$ it finds only 0.0014 of the members.

## 10.4. LiRA: a likelihood-ratio test per record

LiRA, from Carlini et al. (2022), uses the shadow models instead to estimate each record's own IN and OUT distributions. Its statistic is the logit of the model's confidence $p_y$ in the true label $y$,

$$\phi = \ln \frac{p_y}{1 - p_y}. \tag{10.2}$$

LiRA uses the logit because it is close to normal over retrainings, while the confidence itself piles up against 1.

For each record, the attacker fits a normal distribution to the record's logit over the shadow models that trained on it. Its mean is written $\mu_{\text{in}}$ and its standard deviation $\sigma_{\text{in}}$. A second fit, over the shadow models that did not train on the record, has mean $\mu_{\text{out}}$ and standard deviation $\sigma_{\text{out}}$.

**Online LiRA** scores the target model's logit $\phi_z$ on the record by how much more likely it is under the IN fit than under the OUT fit. Writing $\mathcal{N}(\phi; \mu, \sigma^2)$ for the normal density, the score is

$$\ln \Lambda_z = \ln \mathcal{N}(\phi_z;\, \mu_{\text{in}}, \sigma_{\text{in}}^2) - \ln \mathcal{N}(\phi_z;\, \mu_{\text{out}}, \sigma_{\text{out}}^2). \tag{10.3}$$

**Offline LiRA** uses only OUT models. Unlike IN models, which must contain the record being tested, they can be trained once and reused for every new record. It asks how many OUT standard deviations the logit lies above its usual value,

$$\frac{\phi_z - \mu_{\text{out}}}{\sigma_{\text{out}}}. \tag{10.4}$$

Each score already compares a record with its own fits, so one threshold then serves for all records. In the script, ART's `SklearnClassifier` wraps the scikit-learn recipe, trains every model and returns its probabilities.

```
# Each image is in each model's training set with probability 1/2.
member = rng.random((n_models, n)) < 0.5
for k in range(n_models):
    model = recipe()  # a fresh, untrained copy of the target's recipe
    model.fit(x[member[k]], onehot[member[k]])
    probs[k] = model.predict(x)  # the probabilities the attacker sees, for every image
    p = np.clip(probs[k, np.arange(n), y], 1e-15, 1 - 1e-15)  # confidence on the true label
    phi[k] = np.log(p) - np.log(1 - p)  # its logit, close to Gaussian over retrainings
    confidence[k] = p
```

So ART trains the models, while the LiRA scores come from a helper, because no library installed with this book computes them. The helper's `lira_scores`, in practitioner_privacy_tests.py, fits the two normals per image and returns Equation 10.3 and Equation 10.4. It follows TensorFlow Privacy's `compute_score_lira`, which the `tensorflow-privacy` package does not ship.

The command `uv run python code/part02-attacks/practitioner_lira.py` runs it in seconds. It compares the attacks by their TPR at two low FPRs and by their area under the curve (AUC) of the TPR against the FPR.

```
  loss attack: AUC=0.518  TPR@0.001=0.0009  TPR@0.01=0.0105
  ART shadow-model attack: AUC=0.570  TPR@0.001=0.0014  TPR@0.01=0.0146
  LiRA offline: AUC=0.577  TPR@0.001=0.0087  TPR@0.01=0.0345
  LiRA online: AUC=0.623  TPR@0.001=0.0318  TPR@0.01=0.0790
  LiRA online, global variance: AUC=0.614  TPR@0.001=0.0250  TPR@0.01=0.0737
```

By AUC the attacks look close, and ART's attack nearly matches offline LiRA. At an FPR of $10^{-3}$ they are not close, because online LiRA finds 0.0318 of the members and the loss attack only 0.0009. All of them use the same model outputs, so the whole gain comes from comparing each record's output with that record's own fits. Online LiRA beats offline LiRA because its IN fit also tells it how far training on a record moves that record's logit.

Noisy fits are the main practical failure. Each record's variance comes from only about half the shadow models, and an underestimated one makes an ordinary output look like a far-tail event. That output becomes a false positive at exactly the strict thresholds that matter. So Carlini et al. recommend one **global variance**, shared by all records, when shadow models are few. With a global variance 16 shadows give a usable attack, while per-record variances need about 64. With 63 shadows, as here, the per-record fits win.

## 10.5. Running LiRA on your model

The recipe is the same for any model. Train the target and its shadows each on a random half of the data pool, all with exactly the target's recipe. A shadow trained differently, even with another augmentation, estimates the wrong IN and OUT distributions. Then compute each model's logit on each record and fit the IN and OUT normals. Let each model take a turn as the target, and score every pair of target model and record, for enough non-member pairs to measure a low FPR. The shadow count is the budget decision, since each shadow is a training run. Pre-release privacy risk assessment runs the whole job on the digits.

For a confident model, $1-p_y$ rounds to 0 and the logit becomes infinite. So compute it from the class scores $f$ before the softmax, as $f_y - \ln \sum_{y' \neq y} e^{f_{y'}}$, or from the log-probabilities of `log_softmax`, which keep the tail where members and non-members differ. Clipping the probabilities is no substitute. On a nearly separable model most confidences hit the clip, so the fits have no spread, those pairs drop out, and the most exposed records go with them. The script clips only because no digits model comes near the clip. Treat pairs dropped for want of spread as a failed audit.

The most common way a membership audit goes wrong is to draw members and non-members from different distributions. If non-members come from a later period, another site or another preprocessing pipeline, the attack measures that shift rather than the model, so split one pool at random.

When shadows cost too much, the reference-model attack of Zarifzadeh et al. (2024) stays strong with only a few. Returning labels only does not stop the attack, because a label-only attack measures how much an input must change before its label flips (Choquette-Choo et al., 2021). Check which attack a library runs, because TensorFlow Privacy's `run_attacks` and ART's `MembershipInferenceBlackBox` treat all records alike, while ML Privacy Meter implements LiRA.

To catch a training change that makes a model leak, rerun a cheaper LiRA on every release candidate. Train 16 models with the candidate's own recipe, since shadows cached from an older recipe are stale, and score online LiRA with a global variance. On the digits, `practitioner_lira.py --models 16` runs this check in seconds, and it blocks the model, as the 64-model run does. Apply the release rule of the risk assessment unchanged, so the check and the decision agree. To catch a drift below that rule, store the last release's interval and alert when a candidate's lower end exceeds its upper end. Comparing point values would alarm on seed noise alone.

## 10.6. Reading the result

Report the TPR at an FPR of $10^{-3}$ or lower, with the receiver operating characteristic (ROC) curve on log axes, because averages hide breaches. Part 1's recipe for evaluating a detector at a low FPR says how many non-members that takes and how to put Clopper–Pearson intervals on both rates. It also sets the thresholds on a separate split, a step the script skips, as the literature does.

```
  LiRA offline: TP=502/57704  TPR=0.0087 [0.0080, 0.0095]  FP=56/57304  FPR=0.00098 [0.00074
  LiRA online: TP=1833/57704  TPR=0.0318 [0.0303, 0.0332]  FP=57/57304  FPR=0.00099 [0.00075
```

The last word on each line applies the risk assessment's blocking rule. It blocks a release when the lower end of the TPR's 95% interval at an FPR of $10^{-3}$ is above 0.01, ten times that FPR. Online LiRA blocks this model, while offline LiRA alone would have passed it. The logarithm of the ratio of the rates is 3.46 for online LiRA. It is a point estimate of how large the privacy level $\varepsilon$ of differential privacy (Part 3) would have to be to allow this attack, not a certified lower bound. The pairs share models, so they are not independent trials, and a certified bound needs independent ones, such as the canaries of Auditing.

## 10.7. Who is exposed, and what helps

Look at the records an attack scores highest, usually outliers, rare subgroups or mislabelled points. Their IN and OUT distributions lie far apart, because the rest of the data contradict their label or their features are unusual. On the digits, the most exposed image is a 9 that the models classify with confidence 0.812 when they trained on it and 0.003 when they did not. Online LiRA flags it at an FPR of $10^{-3}$ in all 33 target models that trained on it.

Removing the exposed records does not help, because then the next layer of records becomes exposed, which Carlini et al. (2022b) call the onion effect. Closing the generalization gap is not enough either, because the low-FPR end is set by the worst records, not the average.

Differential privacy does help, because private training limits how far any one record can move the model, and so how far apart every record's IN and OUT distributions can lie. Expect a trade-off, because when many subgroups are rare, memorizing them can be what makes a model accurate (Feldman, 2020).

## 10.8. Cheat sheet

Table 10.1.: Which attack or check to use.

| Situation | Use | Notes |
|---|---|---|
| A first look, no attack budget | Test error − training error | A lower bound on the leak; a small gap proves nothing |
| A release decision | Online LiRA, Equation 10.3, 64 shadows | Report TPR at FPR $10^{-3}$ with intervals |
| Few shadow models | Online LiRA with a global variance | 16 shadows are usable this way |

**Rules of thumb**

- Audit with LiRA. A failed weak attack, such as a global loss threshold, is not evidence of safety.

## 10.9. References


- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): LiRA, the global variance, and the audit protocol.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018): the loss attack and its link to the generalization gap.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 11. Attribute inference and model inversion

## 11.1. What it is

An **attribute inference** attack uses a trained model to recover one sensitive field of a person's record, such as smoking status, from the fields the attacker already knows. Take a hospital model that predicts a lung disease from age and smoking. An insurer who knows your age and your diagnosis, the model's label, can query it to guess whether you smoke. But a published study on smoking would allow the same guess, so success alone is not a privacy failure. What matters is whether the model reveals more about you because your record was in its training data.

Yeom et al. (2018) turned this question into a game (Threat models and privacy games). A fair coin decides whether the target is a training record, a **member**, or a fresh draw from the same population. The attacker, who knows the target's public features and label, queries the model and guesses the sensitive value. Its score is the **attribute advantage**,

$$\mathrm{Adv}^{\mathrm{attr}} = \Pr[\text{guess right} \mid \text{member}] - \Pr[\text{guess right} \mid \text{non-member}]. \tag{11.1}$$

Members come from the same population as everyone else, so a guess that rests only on the whole population, called **population inference**, helps both equally and cancels in the difference. What is left is the leak about individuals.

## 11.2. When you meet it

You meet it when someone claims that "the model reveals" a sensitive field, or asks whether dropping a column made a model safe. The Adversarial Robustness Toolbox (ART), a Python library of attacks, offers `AttributeInferenceBlackBox`, and `AttributeInferenceBaselineTrueLabel` for the baseline below. The attribute advantage decides how to classify a finding. Success up to what the population allows would be there without the person's record, so it is a question for policy and fairness. But success on members beyond non-members is a privacy leak.

## 11.3. How to use it

Start with the **imputation baseline**, the success of the best guess of the sensitive value from the public features and label alone, without the model. Jayaraman & Evans (2022) found that many published attacks do no better, so it is the number to beat. To estimate it, fit a classifier that predicts the sensitive field from the public features and label, on records from the same population that the model never saw.

Then run the attack. **Model inversion** (Fredrikson et al. 2014) scores each candidate value $s$ of the sensitive field by two numbers. The first is how likely a person with the target's public features $x$ is to have that value, the probability $p(s \mid x)$, which the attacker estimates from population data. The second is $f_y(x, s)$, the model's confidence in the known label $y$ when it is fed $x$ and the candidate. The maximum a posteriori (MAP) inversion attack then guesses

$$\hat{s} = \arg\max_s \; p(s \mid x)\, f_y(x, s). \tag{11.2}$$

Against a scikit-learn model, each candidate costs one call to `predict_proba`, from which you read the column of the known label.

Give the attacker every column it could hold, because a near-copy of the sensitive field, such as a relationship column next to marital status, moves the baseline most. Run the attack on training members and on an equal number of held-out non-members. Then report the baseline and both success rates with confidence intervals, and give their difference, the advantage, its own interval too. Pre-release privacy risk assessment blocks a release when the advantage's interval lies above zero.

The script practitioner_attribute.py runs ART on Adult, census records of 45,222 people, with `uv run python code/part02-attacks/practitioner_attribute.py`. A random forest predicts whether income exceeds $50K from every column, and sex is the sensitive field. The attacker holds 4,000 records of its own, and everyone's other columns and income.

```
def label(rows: IntArray) -> Array:  # the label the model returns for each record, as a col
    return np.argmax(target.predict(x[rows]), axis=1).reshape(-1, 1).astype(np.float64)

np.random.seed(seed)  # ART builds its random forests unseeded, from NumPy's global generato
attack = AttributeInferenceBlackBox(target, attack_model_type="rf", attack_feature=SENSITIVE
attack.fit(x[attacker])  # the attacker's own records, sensitive field included
baseline = AttributeInferenceBaselineTrueLabel(attack_model_type="rf", attack_feature=SENSIT
baseline.fit(x[attacker], y[attacker])  # the same records and the true label, without the m
hits: dict[str, list[int]] = {}
for name, rows in (("members", members), ("non-members", others)):
    known = np.delete(x[rows], SENSITIVE, axis=1)  # everything but the sensitive field
    guess = attack.infer(known, pred=label(rows), values=[0.0, 1.0])
    imputed = baseline.infer(known, y=y[rows], values=[0.0, 1.0])
    hits[name] = [int(np.count_nonzero(np.equal(g.ravel(), female[rows]))) for g in (guess,
```

```
ART black-box attack: right on members 0.8300 [0.8180, 0.8415]  on non-members 0.8423 [0.830
ART black-box attack: advantage -0.0123 [-0.0285, +0.0040]
imputation baseline: right on members 0.8307 [0.8188, 0.8423]  on non-members 0.8417 [0.8301
```

The attack is right on 0.8300 of the members, and the baseline, which never queries the model, on 0.8307. So the success comes from the population, and the advantage's interval contains zero, which passes the rule. The full edition finds the same on a synthetic population. There, inversion is right on 0.7832 of members but also on 0.7262 of non-members. Its advantage is only 0.0569.

## 11.4. What goes wrong

The commonest error is to report one success rate, such as the 0.7832 on members, without the baseline and the rate on non-members, which overstates the leak. Measuring only on fresh records understates it. A model is too sure of the true label on its own members, which is the generalization gap of Membership inference, so the attack does better on them.

Dropping the sensitive column from the inputs does not stop population inference, because its correlates remain. **Confidence masking**, which rounds the confidences or returns only the top label, puts no bound on the gap between members and non-members, and that gap can grow. In the same experiment, returning labels only raised the advantage from 0.0569 to 0.0650, because success fell more on non-members than on members. Less overfitting shrinks the member gap, and differential privacy bounds it for every attacker and every output format.

## 11.5. References


- Fredrikson, Jha & Ristenpart, "Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures" (CCS 2015): inversion from confidences, and rounding as a countermeasure.
- Jayaraman & Evans, "Are Attribute Inference Attacks Just Imputation?" (CCS 2022): the imputation baseline, and what it takes to beat it.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 12. Property inference

## 12.1. What it is

A **property** is a fact about a training set as a whole, such as the share of a hospital's patients who are over 65. **Property inference** recovers it from a trained model. In the property game (Threat models and privacy games), the data owner flips a fair coin to pick one of two shares for a subgroup. The owner then trains a model on data with that share. The attacker knows both shares and can train models the way the owner does. It guesses which share was used, so a random guess is right half the time.

A property can leak when no single record does. Write $n$ for the number of training records and $g$ for the gap between the two shares. Moving from one share to the other changes about $ng$ records at once, so it moves the model far more than one record can. That is why property leakage grows with $n$, while the risk to one record shrinks like $1/\sqrt{n}$. Take a release of nothing but the exact count of subgroup records. At 10,000 records, the best attacker who asks whether one patient is in the data is right with probability 0.502. But the one who asks whether the share was 0.45 or 0.55 is right with probability 1.0000 to four decimals.

## 12.2. When you meet it

You meet it when you share a model trained on your organization's data, or publish it. The first decision is whether the property is a secret at all. Learning that smokers get more lung cancer is the point of a medical study, not a breach. A property is harmful to reveal only when it describes the *data owner* rather than the world, such as one hospital's patient mix. If it is not a secret, record that in the threat model of Pre-release privacy risk assessment and move on.

If it is a secret, differential privacy (DP) does not protect it. DP protects one record at a time, and its guarantee for a group multiplies the privacy level $\varepsilon$ by the group's size (Post-processing, group privacy and composition). For a hospital of 1,000 patients and shares a tenth apart, the group is the $ng = 100$ patients that differ between the two datasets. So the property gets only $100\varepsilon$, which promises nothing at any useful $\varepsilon$. A site's data volume is a property too, and the group is then the records by which the two candidate sizes differ.

## 12.3. How to use it

To measure the risk, run the shadow-model attack of Ateniese et al. (2015). Like the attacks of Membership inference, it trains copies of the pipeline, but here whole models are the data points. Draw many datasets of your training set's size at each of the two shares, from auxiliary data of

the same population or, for your own release, from your own data. Train a **shadow model** on each, with your architecture and training code, and record its outputs on a fixed set of probe inputs. A linear **meta-classifier**, such as scikit-learn's `LogisticRegression`, then learns to tell the two shares apart from those outputs. Its accuracy on held-out shadow models is the attack's accuracy. Put a confidence interval on it, and if the property is a secret, block the release when the interval lies above one half (Pre-release privacy risk assessment).

Use probe outputs rather than raw weights. Reordering hidden units, or flipping the signs of tanh units, leaves a network's function unchanged. So on the full edition's small tanh networks, every raw weight averages to zero under both shares. There a linear meta-classifier on raw weights scores 0.497, which is chance, and on the outputs at the probe inputs it scores 1.000. Probes need only query access, so a model behind an application programming interface is exposed too.

## 12.4. What goes wrong

The commonest error is to assume that DP, or a passed membership audit, covers the property. A weak measured attack is not evidence of safety either, because with few records and a small gap, the meta-classifier, fitted to noisy shadow models, finds the shift badly.

There is no cheap defence. Removing the group column leaves its correlates, which still move the model, and noise large enough to hide $ng$ records destroys the model's use. If the property is a secret, the strongest option is not to release a model trained on that mix. If you must release, report the attack's accuracy and state in the release notes that DP protects individuals only.

## 12.5. References


- Ganju, Wang, Yang, Gunter & Borisov, "Property Inference Attacks on Fully Connected Neural Networks using Permutation Invariant Representations" (CCS 2018): the practical attack on neural networks, and why raw weights fail.
- Zhang, Tople & Ohrimenko, "Leakage of Dataset Properties in Multi-Party Machine Learning" (USENIX Security 2021): property inference through queries alone.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 13. Memorization and training-data extraction

## 13.1. Why this matters

A language model trained on scraped text or support tickets sees phone numbers, addresses, keys and private messages. Ask it to continue "Anna Smith's phone number is", and it may finish the sentence with the real number. This is **training-data extraction**. Membership inference needs the record in hand to ask whether it was used, while extraction hands the attacker the record itself.

One idea runs through the chapter. Take a secret of a fixed format, such as the five digits after "my pin is". Order all its possible values from the most likely to the least likely under the model. The **rank** of the true secret, its position in that order, is the number of guesses an attacker needs if it tries the values in that order. A model has memorized a string when it ranks it far above the strings that could have stood in its place.

## 13.2. Which memorization to measure

The notions of memorization below give rates that differ by orders of magnitude, so a report must say which one it measures. All use greedy decoding, which writes out the most likely next token at each step.

Table 13.1.: The notions of memorization and when to use each. The first three are verbatim, needing an exact match, and each implies approximate. Discoverable implies extractable.

| Definition | The string counts as memorized if | Use it for |
|---|---|---|
| Discoverable (Carlini et al. 2023) | The text before it in the training data makes greedy decoding produce it | Measuring memorization on a model you own |
| Extractable (Carlini et al. 2021) | Some prompt, any at all, makes greedy decoding produce it | The real risk, which attacks find but no test measures |
| $k$-eidetic | It is extractable and appears in at most $k$ documents | Flagging the dangerous case, $k = 1$ |
| Approximate (Ippolito et al. 2023) | Greedy decoding comes within a set edit distance of it | Near copies, such as a number with one digit changed |

Discoverable memorization is the one you can measure. Carlini et al. (2023) prompt with 50 tokens of a training sequence, decode the next 50 greedily and count exact matches. Treat any verbatim rate as a lower bound on the leak, because near copies leak too. Ippolito et al. (2023) show that a filter blocking verbatim training text is easy to bypass.

Deployed systems sample rather than decode greedily, which turns memorization into a matter of budget. A sample at temperature 1 writes out a string with probability $p$, the model's probability of that string after the prompt. Write $\beta$ for the chance of failure the attacker accepts. Then a number $m$ of independent samples suffices when

$$m \geq \frac{\ln(1/\beta)}{p}. \tag{13.1}$$

For example, a string of probability $10^{-3}$ comes out within about 4,600 samples, except with probability 0.01. So report a sampling budget alongside the greedy rate, and measure with the decoding settings you deploy.

## 13.3. What gets memorized, and the first defences

On open models, the largest reproduced at least 1% of its training data this way (Carlini et al., 2023). Memorization grew log-linearly, by a roughly fixed amount each time the number of copies of a string, the size of the model or the length of the prompt grew tenfold. Copies matter most in practice, because Kandpal, Wallace & Raffel (2022) found that a sequence repeated ten times is generated about a thousand times more often than one seen once.

That gives the order of defences. **Deduplication** is the cheapest, because it lowers the copies of every repeated string. Lee et al. (2022) found that models trained on deduplicated data emit memorized text about ten times less often without losing quality. Deduplicate before you plant the canaries of the next section, so that only they carry known copy counts.

The second defence is to remove personal data before training. A detector of personally identifiable information (PII), such as Microsoft's Presidio, finds names, phone numbers and similar fields so you can mask them. It misses whatever its patterns do not cover, so measure what remains with canaries rather than trusting the filter. Neither defence helps a secret seen once in a large model, where one copy is enough. Only fine-tuning with differential privacy, which limits how much any one training sequence can change the model, gives a guarantee for such singletons.

## 13.4. Auditing with canaries and exposure

Discoverable memorization is all or nothing, while an auditor wants a graded answer. Carlini et al. (2019) proposed planting **canaries**, random secrets of a fixed format such as "my pin is" followed by five random digits. You know each canary's value and the number of values it could have taken, so after training you can see exactly where the model ranks it.

Write $\hat{r}$ for the planted value, $\mathcal{R}$ for the set of values it could have taken and $|\mathcal{R}|$ for their number. The **exposure** is the number of bits of search the model saves an attacker,

$$\text{exposure}(\hat{r}) = \log_2 |\mathcal{R}| - \log_2 \text{rank}(\hat{r}). \tag{13.2}$$

Each bit halves the guesses the attacker needs, and an exposure near the maximum, $\log_2 |\mathcal{R}|$, means the secret can be extracted. For five digits the maximum is 16.61 bits.

Exposure is an audit tool because its behaviour is known when nothing was memorized. Each canary's value is drawn uniformly at random, so a model that never saw it has no reason to favour it, and it lands at a uniformly random rank. So for any threshold of $t$ bits,

$$\Pr\left[\text{exposure}(\hat{r}) \geq t\right] \leq 2^{-t}. \tag{13.3}$$

Then $2^{-\text{exposure}}$ is an upper bound on the p-value, the chance that an uninserted canary would look at least this memorized. The same uniform rank gives an uninserted canary an average exposure of $\log_2 e$, about 1.44 bits, which the controls should match.

To run the audit, draw canaries in the format of the secrets your data could contain, because a format unlike your data measures how the model treats unfamiliar text. Insert each one a chosen number of times, from once to many. Keep some never inserted as controls, then fine-tune as usual. Score each canary's value, and many random values of the same format, by the model's log-probability after the prefix. The share of random values that score at least as high estimates the **rank fraction**, the rank divided by $|\mathcal{R}|$, and minus its base-2 logarithm is the exposure.

The scoring step is where audits most often go wrong. A candidate's log-probability is the sum over its tokens, and the mean loss that Hugging Face models return divides that sum by the number of tokens. Values that tokenize to different lengths then get scores on different scales, and the ranking is wrong. So score every candidate by its summed log-probability, and choose a format in which every candidate has the same number of tokens. The script practitioner_lora.py fine-tunes the small language model Pythia-14m with low-rank adapters (LoRA) from the `peft` library. It separates the digits of each pin by spaces, as in "the pin of Alice is 8 6 5 2 3", so that each digit is one token.

```
@torch.no_grad()
def secret_scores(model: PeftModel, tokenizer: PreTrainedTokenizerBase, prefix: str, secrets
    """Summed log-probability of each secret's five digit tokens after the prefix."""
    prefix_ids = tokenizer(prefix)["input_ids"]
    rows = [prefix_ids + tokenizer(" " + s)["input_ids"] for s in secrets]
    ids = torch.tensor(rows)  # every secret is five tokens, so the rows have equal length
    logits = model(input_ids=ids[:, :-1], logits_to_keep=5).logits  # the predictions of the
    logprobs = logits.log_softmax(dim=-1).gather(-1, ids[:, -5:, None])
    return logprobs.sum(dim=(1, 2)).double().numpy()


def canary_exposure(
    model: PeftModel, tokenizer: PreTrainedTokenizerBase, canary: Canary, candidates: list[s
) -> tuple[float, float]:
    """The exposure of the canary's secret among the candidates, measured and extrapolated p
    candidate_scores = scores_batched(model, tokenizer, canary.prefix, candidates)
    score = float(scores_batched(model, tokenizer, canary.prefix, [canary.secret])[0])
    return exposure(score, candidate_scores), extrapolated_exposure(score, candidate_scores,
```

The model comes from `transformers` and its adapters from `peft`, but no library installed with this book computes exposure. So `exposure` and `extrapolated_exposure` come from

the book's helper module practitioner_privacy_tests.py, which follows TensorFlow Privacy's `compute_exposure_interpolation` and `compute_exposure_extrapolation`.

The model trains on 1,024 sequences of Tiny Shakespeare, a public extract of Shakespeare's plays. The script inserts three canaries at each copy count from 1 to 64, keeps three as controls, and ranks each value among 2,000 random candidates. The command `uv run python code/part05-dp-ml/practitioner_lora.py` runs it in about two and a half minutes on an idle CPU, and several times longer on a busy one. Each line gives a group's mean exposure, then that exposure extrapolated past the cap the candidates set (explained below), and how many of its secrets greedy decoding wrote out.

```
non-private 1 copies: mean exposure 2.0 bits  extrapolated 2.0  extracted 0/3
non-private 4 copies: mean exposure 5.9 bits  extrapolated 5.7  extracted 0/3
non-private 16 copies: mean exposure 10.0 bits  extrapolated 13.0  extracted 0/3
non-private 64 copies: mean exposure 11.0 bits  extrapolated 16.3  extracted 3/3
non-private controls: mean exposure 1.4 bits  extrapolated 1.5  extracted 0/3
```

Exposure rises with the copies, and the controls sit at 1.4 bits, as they should. Greedy decoding extracts only the three canaries seen 64 times. Yet two of the canaries seen 4 times reach 6.9 and 8.2 bits, while no extraction test notices them. That is why an audit measures exposure and not only extraction. The canaries fill a quarter of the training set here, which lets so small a model memorize them in a short run. In the same script, fine-tuning with differential privacy, at the privacy level of 8 that Part 3 defines, extracts nothing (Fine-tune a language model privately).

Two more things go wrong. First, a sample of candidates caps the exposure it can resolve. A canary that beats all 2,000 candidates only shows that its rank fraction is below about 1 in 2,000. So the exposure is capped at 10.97 bits, which the 64-copy canaries reach. Past the cap, Carlini et al. (2019) fit a skew-normal distribution to the candidates' scores and read off its tail. A plain normal fit would overestimate the exposure, because the scores have a heavier right tail than a normal. Second, ties hide memorization, because when every candidate gets the same score, every exposure is 0.

A canary repeated 16 or more times is the positive control. If it does not reach a high exposure without privacy, the run is too short to tell you anything. Pre-release privacy risk assessment sets the exposure that blocks a release and runs this audit as one of its steps.

## 13.5. Extraction without the prefix: generate, rank, verify

A real attacker has only the model. Carlini et al. (2021) extracted data from GPT-2 by generating many samples, ranking them by how likely each was to be memorized, and checking the top against public sources. The ranking is the hard step, because a model finds a string likely either because it is ordinary text or because it was memorized. A **reference model**, trained on similar data but not on the target's training set, stands in for ordinary text. Write $p_\theta(x)$ for the target's probability of a string $x$, and $p_{\text{ref}}(x)$ for the reference's. Rank by the log-likelihood ratio

$$\Lambda(x) = \ln p_\theta(x) - \ln p_{\text{ref}}(x). \tag{13.4}$$

Raw likelihood puts first the strings that any model finds likely, such as software licences and boilerplate, which reveal nothing, and the ratio cancels them. A crude reference works. Carlini et al. (2021) used a smaller model of the same family. They also used the compressed length of the text, `len(zlib.compress(text.encode()))`, which measures how predictable the text is without any model. To audit your own fine-tuned model this way, the base model before fine-tuning is a natural reference.

An attacker without the prefix extracts less than the discoverable rate suggests, but production models are not safe. Nasr et al. (2023) asked ChatGPT to repeat one word forever, until it left the assistant behaviour that alignment training gives it and wrote memorized text. For about 200 US dollars of queries they extracted over 10,000 records, so alignment hides memorization from casual use without removing it.

## 13.6. Cheat sheet

Table 13.2.: The results of this chapter.

| Question | Answer |
|---|---|
| Exposure of a canary | $\log_2\lvert\mathcal{R}\rvert - \log_2$ rank, at most 16.61 bits for five digits; p-value at most $2^{-\text{exposure}}$; controls average 1.44 bits |
| How to score a candidate | Summed log-probability, in a format where all candidates have the same token count |
| Ranking generations | $\ln p_\theta(x) - \ln p_{\text{ref}}(x)$, never raw likelihood |
| Defences, in order | Deduplicate (`deduplicate-text-datasets`, `datasketch`), scrub PII, then differential privacy for singletons |

**Rules of thumb**

- A verbatim rate is a lower bound on the leak, not a measure of it.

## 13.7. References


- Carlini, Liu, Erlingsson, Kos & Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, exposure and how to estimate it.
- Carlini, Ippolito, Jagielski, Lee, Tramèr & Zhang, "Quantifying Memorization Across Neural Language Models" (ICLR 2023): how memorization scales with duplication, model size and prompt length.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 14. Gradient inversion

## 14.1. Why this matters

A phone keyboard learns from what its users type, and the raw text never leaves the phones. Instead, a server sends the current model to each phone, the phone computes the gradient of the loss on its own few records, and it sends back only that gradient. The server averages them and takes a training step. This is **federated learning (FL)**, often called private because "the data stay on the device".

Before you deploy it, ask what a single client's gradient reveals about its data. Often, everything. A gradient holds one derivative for every parameter, and many of them are simple functions of the input, so the input can be read back. Averaging over a batch protects much less than people expect (Section 14.6), so treat a per-client update as being as sensitive as the data it came from.

## 14.2. The attacker and what it sees

The attacker is the server, which sees each client's own update and knows every parameter, since it sent the model out. In the terms of Threat models and privacy games, its access is therefore the **transcript**, everything training exposes along the way. Assume first that it is **honest-but-curious**, so it only analyses what it receives. What it receives is simple, because each client holds a batch of $B$ records and sends the average of their gradients. Write $\ell_i$ for the loss on record $i$, which depends on all the parameters $\theta$. The client then sends

$$g = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell_i. \tag{14.1}$$

## 14.3. Exact recovery from a fully connected layer

The most common first layer, a fully connected layer with a bias, gives its input back by one division. Write $W_1$ and $b_1$ for this first layer's weights and bias. Each unit $j$ multiplies the input $x$ by its row of weights and adds its bias. Its backward signal $u_j$ is how fast the loss responds to its output, and the chain rule turns it into the weight and bias gradients

$$\nabla_{W_1} \ell = u\, x^\top, \qquad \nabla_{b_1} \ell = u. \tag{14.2}$$

So row $j$ of the weight gradient is the input scaled by $u_j$, and entry $j$ of the bias gradient is that same scale. For a single record, dividing one by the other gives the input exactly, for every unit whose bias gradient is not zero:

$$x = \frac{(\nabla_{W_1} \ell)_{j,:}}{(\nabla_{b_1} \ell)_j}. \tag{14.3}$$

Everything above the layer, including the depth of the network, the loss and the label, sits inside the scale $u_j$ and cancels. So the attack needs no optimization and no knowledge of the data. The snippet plays one client with one image of scikit-learn's bundled digits in PyTorch. Its last line reads the label, as Section 14.4 explains.

```
import torch
import torch.nn.functional as F
from sklearn.datasets import load_digits

torch.manual_seed(0)
images, labels = load_digits(return_X_y=True)
x = torch.tensor(images[[7]] / 16, dtype=torch.float32)  # one client's image, pixels in [0,
y = torch.tensor(labels[[7]])
first, last = torch.nn.Linear(64, 32), torch.nn.Linear(32, 10)
F.cross_entropy(last(torch.relu(first(x))), y).backward()  # the gradient the client sends
# Row j of the weight gradient is the input times the bias gradient of unit j: divide them.
j = int(first.bias.grad.abs().argmax())
recovered = first.weight.grad[j] / first.bias.grad[j]
print(f"image recovered: {torch.allclose(recovered, x[0], atol=1e-6)}")  # image recovered:
# Only the true label's logit has a negative gradient (softmax minus one-hot).
print(f"recovered label {int(last.bias.grad.argmin())}, true label {int(y)}")  # recovered l
```

The snippet picks the unit whose bias gradient is largest in size, because a switched-off ReLU unit has a zero gradient, and one active unit is enough. What stops the division is a convolutional first layer. Its filter is reused at every position of the image, so its gradient adds up one such product per image patch, and no division separates the terms of a sum. The attacker then has to search instead (Section 14.5).

## 14.4. Label recovery from the last layer

The label is often the most sensitive part of an example, such as a diagnosis. It can be read off the last layer without any search (Zhao, Mopuri & Bilen 2020), if that layer has a bias and feeds the softmax and cross-entropy loss. Write $p$ for the predicted probabilities and $e_y$ for the vector with a 1 at the position of the label $y$. The gradient of the last layer's bias $b_2$ is then $p - e_y$, which is negative only at the true label. So for a single record

$$y = \arg\min_k (\nabla_{b_2} \ell)_k, \tag{14.4}$$

which is the snippet's `argmin`. The one failure is a very confident model. If its true-label probability $p_y$ rounds to 1 in floating point, the label's entry becomes zero, so no entry is negative.

A batch hides which record has which label, but not how many records there are of each class. Each entry of the bias gradient is the class's average predicted probability minus its share of the batch. So a server that estimates the average prediction on data of its own can read off each class's count. In the full edition's experiment, every batch of 4 gave all its counts exactly. Even among batches of 64, 52.5% of the batches did.

## 14.5. Deep Leakage from Gradients

When the division does not apply, the attacker searches. **Deep Leakage from Gradients (DLG)** (Zhu, Liu & Han 2019) guesses an input and computes the gradient it would produce, then moves the guess until that gradient matches the observed one. Write $x'_i$ for the guessed inputs, $y_i$ for their labels, read first as in Section 14.4, and $\ell(\theta; x', y)$ for the loss of one guess. The **matching loss** $R$ is the squared distance between the guesses' average gradient and the observed one, so the attack looks for the guesses that make it smallest:

$$\hat{x}_1, \dots, \hat{x}_B = \arg\min_{x'_1,\dots,x'_B} R(x'), \qquad R(x') = \Big\| \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x'_i, y_i) - g \Big\|^2. \tag{14.5}$$

In PyTorch the attack is a short loop around one call. With `create_graph=True`, `torch.autograd.grad` returns the guess's gradient in a form that can itself be differentiated. An optimizer such as Adam can then follow the slope of $R$ back to the input.

Two things decide whether the search works. The activation's slope must vary, because the search differentiates a gradient, and the ReLU's slope is constant on each side of zero. So Zhu et al. used the sigmoid. And because $R$ is not convex, the search can get stuck. So run it until $R$ stops falling, and judge the final value against the squared size $\|g\|^2$ of the observed gradient $g$, in the precision you compute in.

The full edition's experiment runs 600 steps of Adam in double precision, with a learning rate starting at 0.05 and decaying linearly to zero. There a successful run ends near $10^{-27}$ and a stuck one near $10^{-5}$, and one image comes back exact to about 13 significant digits. Another input can also give a small $R$, so restart from several random guesses, and trust an image only when the restarts agree on it.

## 14.6. Batches and averaging

Clients usually send the average over a batch, or the change in their parameters after several training steps on the device (**local steps**). Averaging breaks the division and makes the search harder, but much of the structure survives. Each record adds one outer product to the first-layer gradient, so every row of that gradient is a combination of the $B$ inputs. The server therefore learns the **span** of the batch, the set of all such combinations, which contains every record.

The division also still recovers an example exactly when some ReLU unit is switched on for that example only. That unit then **isolates** the record. If units switched on like independent fair coins, a unit would isolate a given record with probability $2^{-B}$, so isolation would become rare as the batch grows. But real images make a unit fire on many records or on few, so far more units fire on just one. Take the full edition's network with 1,024 hidden units and batches of 16 synthetic $8 \times 8$ images. The coin model predicts that 1.6% of the batch is exposed, but 69.1% was.

The share exposed depends on both the batch size and the width of the first layer, so report both. Table 14.1 gives the shares from the same experiment.

Table 14.1.: Isolation attack on synthetic images, from the full edition.

| Hidden units | Batch size | Share of the batch recovered exactly |
|---|---|---|
| 64 | 4 | 0.838 |
| 64 | 8 | 0.550 |
| 64 | 16 | 0.144 |
| 1,024 | 16 | 0.691 |

A larger batch or a narrower layer lowers the share, but a malicious server undoes both. So measure at your own width and batch size, and treat any isolated record as a breach.

DLG suffers more from batching. On the full edition's sigmoid network with 32 hidden units, the median run from a batch of 4 gets stuck. That is a harder search, not less information: the isolation attack recovered all four images of such a batch. Local steps do not protect a single record either. Each step's first-layer gradient is an outer product with the same input, and only the backward signal changes, so the division still works on the total change. And a **malicious** server that chooses the weights can plant units that fire only on rare inputs, each isolating one record. It then recovers most records of even a large batch by design (Boenisch et al. 2023; Fowl et al. 2022).

## 14.7. What to do as a deployer

Table 14.2 reads off the risk of an FL deployment from who can see one client's update.

Table 14.2.: Gradient leakage by FL setting, and the response.

| Setting | What leaks to the server | What to do |
|---|---|---|
| One record per update, fully connected first layer with bias | The input exactly, and the label | Never deploy without protection |
| Batches of any size, or several local steps | Label counts, the span of the batch, isolated records, and the rest by search in small batches | Treat each update as raw data |
| Server not trusted to choose weights | Most records of any batch | Batch size is no defence; treat each update as raw data |
| Updates protected by differential privacy (DP) | Bounded per client by the DP guarantee | The only protection with a guarantee |

**Secure aggregation** is a cryptographic protocol that lets the server see only the sum over many clients, never one client's update. That removes the per-client view that every attack here uses, but a sum over very few clients is close to a single update. So set a minimum cohort size, and make sure that clients dropping out mid-round cannot shrink a sum below it.

Secure aggregation alone is not enough. The sum is still a batch average that the attacks above can read, the final model still leaks like any model, and a malicious server can still choose its weights.

For a guarantee, add DP with the client as the unit of privacy. Each client's update is clipped, that is, scaled down to a fixed maximum norm, and noise is added to the sum. That bounds what any one client's data can change (User-level privacy). Flower and TensorFlow Federated offer both protections. Heuristic defences such as gradient pruning raise the cost of the search but carry no guarantee.

To assess your own deployment, start by finding out what the server sees, single updates or only sums, and with what minimum cohort. Then note the client batch size, the number of local steps, and the type and width of the first layer. Run the attacks on your own architecture at that batch size. Geiping's open-source `breaching` framework implements the published search attacks for PyTorch. If any client's update is visible on its own, block the deployment until secure aggregation or client-level DP is in place. Pre-release privacy risk assessment records this verdict next to the other attacks.

## 14.8. Cheat sheet

Table 14.3.: The results of this chapter.

| Attack | Formula | Needs | Fails when |
|---|---|---|---|
| Exact recovery, one record | $x = (\nabla_{W_1}\ell)_{j,:}/(\nabla_{b_1}\ell)_j$ | Fully connected first layer with bias | Convolutional first layer; batches, in part |
| Label, one record | $y = \arg\min_k(\nabla_{b_2}\ell)_k$ | Softmax and cross-entropy | $p_y$ rounds to 1 |
| Deep Leakage from Gradients | Minimize the matching loss $R$ | An activation whose slope varies; the labels | Search stuck (final $R \gg 0$); larger batches |
| Isolation, batch | Isolated records recovered exactly | Fully connected first layer, ReLU units | Rarely on real data |

**Rules of thumb**

- Secure aggregation needs a minimum cohort, and only client-level DP bounds what any update reveals.

## 14.9. References


- Zhu, Liu & Han, "Deep Leakage from Gradients" (NeurIPS 2019): the gradient-matching attack.
- Geiping, Bauermeister, Dröge & Moeller, "Inverting Gradients — How easy is it to break privacy in federated learning?" (NeurIPS 2020): cosine matching with an image prior, on trained networks, local steps and batches.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 15. Linkage and re-identification

## 15.1. Why this matters

A hospital wants to share its discharge records with researchers. It deletes the names but keeps the postcode, the date of birth, the sex and the diagnosis. Anyone who holds a voter roll, with names next to those same three columns, can then join the two tables and put a name back on most rows. This is a **linkage attack**, and putting a name back on a "de-identified" record is **re-identification**.

The practical question is whether a table you are about to release can be linked like this. The answer comes from counting identifying information in bits. In a population of $n$ people, each bit halves the number of candidates, so singling out one person takes $\log_2 n$ bits. That is 27.9 bits for the 248.7 million people of the United States in 1990. The bits of separate attributes add up as long as the attributes are independent. Once the attributes an adversary knows carry a few bits more than $\log_2 n$, almost everyone is unique, and a unique record can be linked.

## 15.2. Quasi-identifiers and the linkage attack

**Direct identifiers**, like a name or an email address, name a person on their own, while **sensitive attributes** such as a diagnosis are what the release is about. **Quasi-identifiers** sit in between. None of them names anyone alone, but together they often single out one person, and they appear in other datasets next to names.

To score a release you hold, group its $n$ records into **equivalence classes**, the sets of records with identical quasi-identifiers, and write $J$ for the number of classes. Suppose an adversary knows the target's quasi-identifiers and knows that the target is in the release. If the target is alone in its class, the adversary names its record for sure, and if the class holds $k$ look-alikes, a guess is right one time in $k$. A class of $k$ records is drawn $k$ times as often as a single record, so every class adds one record's worth of wins whatever its size. Averaged over all targets, the chance of naming the right record is therefore

$$\Pr[\text{win}] = \frac{J}{n}, \tag{15.1}$$

and no adversary that sees only the quasi-identifiers can do better. This is the **prosecutor model**, the worst case, because it assumes the adversary knows the target is in the data.

On real populations, most people are unique. From the 1990 census, Latanya Sweeney estimated that 87% of Americans were unique on ZIP code, birth date and sex.

To see how fast uniqueness grows on a real table, the script practitioner_table_risk.py takes Adult, the 48,842 people of the 1994 US Current Population Survey in the UCI repository. Adult has no ZIP code or birth date, so the script starts from age, sex and race. It then adds one

quasi-identifier at a time, with native country as the only geography. For each set it prints the share of unique records, the share in classes smaller than 5, the smallest class size and $J/n$. The counts are plain Polars group-bys, because no maintained Python package computes them, while ARX in Java and sdcMicro in R do.

```
n = df.height
for qis in QI_SETS:
    # One row per equivalence class (records that agree on every quasi-identifier), with its
    sizes = df.group_by(qis).len()
    unique = sizes.filter(pl.col("len") == 1).height / n  # share of records alone in their
    small = sizes.filter(pl.col("len") < 5)["len"].sum() / n  # share in classes smaller tha
    min_k = sizes["len"].min()  # the table is k-anonymous for this k
    print(
        f"{' + '.join(qis)}: unique={unique:.2%}  in classes <5={small:.2%}  "
        f"min k={min_k}  J/n={sizes.height / n:.3f}"
    )
```

Run it with `uv run python code/part02-attacks/practitioner_table_risk.py`, which prints:

```
age + sex + race: unique=0.12%  in classes <5=0.75%  min k=1  J/n=0.012
age + sex + race + native-country: unique=3.07%  in classes <5=7.83%  min k=1  J/n=0.060
age + sex + race + native-country + marital-status: unique=5.88%  in classes <5=11.99%  min
age + sex + race + native-country + marital-status + education: unique=14.64%  in classes <5
age + sex + race + native-country + marital-status + education + hours-per-week: unique=36.5
```

Three columns that look harmless leave 0.12% of the records unique, and six leave 14.64%. One more column, the hours worked per week, which nobody would list as an identifier, makes 36.55% unique and puts more than half the records in classes smaller than 5. So add every column an adversary could know to the test, one at a time, because the one that breaks the table may be one nobody suspected. A minimum class size of 5 is a common convention, yet every set here has classes of a single record.

## 15.3. Pseudonyms and hashed identifiers

Many releases replace each identifier by a **pseudonym**, a code that stays the same across a person's records, and such codes fail in two ways. First, a plain hash such as MD5, computed without a secret key, can be reversed whenever the possible identifiers can be listed. The adversary hashes every candidate once and looks each pseudonym up. New York City's release of its 2013 taxi trips hashed the driver licences this way, but there were only about 2 million possible licences. In 2014 Vijay Pandurangan hashed every one in minutes, and public lists of licence holders then named each driver.

The second failure survives any key. Because the pseudonym ties a person's records together, linking any one of them reveals all the others. In the taxi data, a pickup minute and place single out one trip. A passenger who knows who drove that trip can find it, and even a random driver

pseudonym would then reveal every trip that driver made. This link is why the General Data Protection Regulation (GDPR) still counts pseudonymized data as personal data.

If you must keep a join key, derive it with a keyed hash, a hash-based message authentication code (HMAC), and keep the key away from the data.

```
import hashlib
import hmac
import secrets

licence = "482913"  # a six-digit licence number, the identifier being pseudonymized
pseudonym = hashlib.md5(licence.encode()).hexdigest()  # what an unkeyed release prints

# Dictionary attack: hash all 10**6 possible licences once, then look the pseudonym up.
lookup = {hashlib.md5(f"{i:06d}".encode()).hexdigest(): f"{i:06d}" for i in range(10**6)}
print(lookup[pseudonym])  # 482913

# A keyed hash still gives one code per person, but without the key nobody can build the tab
key = secrets.token_bytes(32)  # store it apart from the data; draw a new one for each relea
keyed = hmac.new(key, licence.encode(), hashlib.sha256).hexdigest()
print(keyed in lookup)  # False
```

The key stops the dictionary attack but not the second failure, because each person's records still form an identifying history. Reusing the key would join that history across releases, so draw a fresh key for each release.

## 15.4. How many people are unique?

For a release still being designed, there is no table to count, so estimate uniqueness by adding bits. An attribute with $v$ equally likely values carries $\log_2 v$ bits, and the bits of independent attributes add. If the attributes together carry $b$ bits, they split people into $2^b$ cells. Suppose each person lands in a cell at random, every cell equally likely, as independent uniform attributes give. The expected share of the $n$ people who are alone in their cell is then about

$$\frac{\text{unique people}}{n} \approx \exp\left(-2^{\log_2 n - b}\right). \tag{15.2}$$

The share depends only on the bits beyond $\log_2 n$, and it rises steeply with them. Solving the formula for one half gives 0.53 bits beyond $\log_2 n$, and solving it for 99% gives 6.64 bits.

For example, ZIP code, birth date and sex carry 31.1 bits, a few more than the 27.9 bits that single out one American. The formula then predicts that 0.899 of Americans are unique, close to Sweeney's 87%. Real attributes are skewed and correlated, which usually lowers uniqueness, so the estimate errs on the side of caution. The Gaussian copula model of Rocher, Hendrickx and de Montjoye (2019) does better for correlated attributes, because it models how they vary together.

## 15.5. Sparse data: the Netflix attack

Ratings, purchases and locations have no quasi-identifier columns, yet they are even more identifying. In 2006 Netflix released about 100 million movie ratings, with the subscribers' names replaced by random numbers. Narayanan and Shmatikov (2008) then matched its records to public reviews. Their score adds up the items on which a record agrees with what the adversary knows. It weights each item by its rarity, because agreeing on an obscure film says more than agreeing on a hit. An item rated by a share $s$ of the records gets the weight $\ln(1/s)$. With such a score, eight ratings with dates known to within 14 days identified 99% of the records, even when two of the ratings were wrong.

The target may not be in the release at all. So the **eccentricity test** declares a match only if the best score beats the second best by a margin, counted in standard deviations of all the scores. Calibrate that margin on people known not to be in the data. On the full edition's simulated ratings, the original attack's margin also matched 26.3% of such outsiders. So report the attack's success at a stated false positive rate (Hypothesis testing).

## 15.6. Does releasing a sample protect?

Adult is itself a sample. Agencies release samples arguing that a sample-unique record may share its attributes with someone left out, but the argument is weaker than it sounds. Suppose the adversary knows a person and finds exactly one matching record in a release that holds a share $q$ of the population, the **sampling rate**. Write $\lambda$ for the expected number of other people in the population who share the target's cell. The record found can be a look-alike's only if the target was left out, which happens with probability $1 - q$. So for a rare cell, the probability $\kappa$ that the match is right is about

$$\kappa \approx \frac{1}{1 + \lambda(1 - q)}. \tag{15.3}$$

For example, take a rare cell, one expected to hold $\lambda = 0.1$ other people. Even a 1% sample leaves a match right with probability 0.910, and shrinking the sample further barely helps. Sampling enters only through $1 - q$, which can never exceed 1, so the chance of a right match never falls below $1/(1 + \lambda)$. Sampling therefore protects only people whose cells are crowded anyway.

So the Adult figures above are **sample uniqueness**, exact when the target is known to be in the release. The **population uniqueness** behind them is lower, but not by much for rare cells, so estimate it with a model rather than assume the risk away. The open-source tool ARX offers the Pitman and Zayatz estimators for this. The R package sdcMicro estimates each record's risk from its sampling weight, the number of people the record stands for.

## 15.7. A pre-release risk check

List as quasi-identifiers every column an adversary could find next to a name elsewhere, such as dates, locations and rare events. To lower the risk, coarsen geography and dates first because they carry the most bits, and then suppress the records that remain unique (Syntactic anonymity and

how it breaks). If the analysis needs fine-grained or sparse data, stop and release a differentially private output instead (Release statistics with a library).

Pre-release privacy risk assessment runs this check as one step of a release decision, with its thresholds fixed before the counts are seen. No tool knows what auxiliary data exist, so the choice of quasi-identifiers stays your judgement.

## 15.8. Cheat sheet

Table 15.1.: The results of this chapter.

| Question | Result | Use it for |
|---|---|---|
| Bits to name one person | $\log_2 n$ | 27.9 for the US of 1990 |
| Risk of a release you hold | Success $J/n$; at most $1/k$ if every class has at least $k$ records | Prosecutor model |
| Share unique, uniform cells | $\exp(-2^{\log_2 n - b})$ | 50% at $\log_2 n + 0.53$ bits, 99% at $\log_2 n + 6.64$ |
| Releasing a sample | $\kappa \approx 1/(1 + \lambda(1 - q))$ | 0.910 at $\lambda = 0.1$ in a 1% sample |

**Rules of thumb**

- Add quasi-identifiers one at a time, because one innocent column can more than double the unique share.
- Estimate uniqueness for the population, not just the sample.
- Hashing an identifier is not anonymization. Key the hash, and use a new key for each release.

## 15.9. References


- Sweeney, "Simple Demographics Often Identify People Uniquely" (2000): the 87% estimate and its variants with coarser geography.
- Narayanan & Shmatikov, "Robust De-anonymization of Large Sparse Datasets" (2008): the Netflix attack.
- Rocher, Hendrickx & de Montjoye (2019), in *Nature Communications*: population uniqueness from a sample.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 16. Syntactic anonymity and how it breaks

## 16.1. Why this matters

Linkage and re-identification showed that deleting names does not anonymize a table, because the remaining quasi-identifiers, such as age, postcode and sex, single out many people. The classic answer is to coarsen the table until every person hides in a crowd. Between 1998 and 2008, researchers turned this into definitions that constrain the released table itself. They are called **syntactic** because you can check each one by reading the published table.

Statistics offices, clinical data sharing and open-source anonymization tools such as ARX and sdcMicro still use these definitions, so you will meet them in audits. They are not enough, because each is a property of one table against an adversary whose knowledge is fixed in advance. One unanticipated fact, or a second release of the same people, voids the guarantee.

## 16.2. What the adversary believes about a class

To anonymize a table, the custodian, the organization that holds the data, applies a **generalization**, which turns an age into a range or a postcode into a prefix. Records with the same generalized quasi-identifiers form an **equivalence class**. An adversary who knows the target's quasi-identifiers can find the target's class $E$, but the records in it look alike. So if it knows nothing else, its belief about the target's sensitive value is the share of the class with that value.

Call that belief its **posterior**. Write $|E|$ for the size of the class, and $n_E(v)$ for the number of its records with a sensitive value $v$. Then the posterior that the target's sensitive value, written $S_{\text{target}}$, equals $v$ is

$$\Pr[S_{\text{target}} = v \mid \text{release}] = \frac{n_E(v)}{|E|}. \tag{16.1}$$

Background knowledge works by elimination. Ruling out a value crosses out its records, and the posterior is recomputed over the records that remain, so the other values' shares grow. Every attack in this chapter is this calculation.

## 16.3. k-anonymity and what it costs

Sweeney and Samarati proposed **k-anonymity** in 1998. A release is k-anonymous if every class has at least $k$ records, for a crowd size $k$ the custodian chooses. Generalization gets there, together with **suppression**, which deletes the records that still stand out. The previous chapter's linkage success, for an adversary who knows the target is in the release, is the number of classes $J$ over

the number of records $n$. Classes of at least $k$ records number at most $n/k$, so that success is capped,

$$\Pr[\text{win}] = \frac{J}{n} \leq \frac{1}{k}. \tag{16.2}$$

The guarantee is about names only, and it assumes the adversary knows the quasi-identifiers and nothing more.

The choice of $k$ is a policy. A minimum of 5 is a common convention, and some health-data rules ask for more. The US Centers for Medicare & Medicaid Services, for example, forbid publishing any count from 1 to 10 drawn from their data, in effect a minimum of 11. The HIPAA Privacy Rule also offers two routes to de-identification. Safe Harbor removes a fixed list of identifiers, while expert determination has an expert measure the risk with methods like these.

Anonymizers search for a generalization that loses little. **Mondrian**, for example, splits groups of records at the median of one quasi-identifier while both halves keep $k$ records. The full edition tries it on a synthetic table of 10,000 hospital patients. There, at $k = 5$, a class spans on average only 1.6% of each quasi-identifier's range, so k-anonymity itself costs little precision.

On a real table, you can also generalize by hand and recheck. The script practitioner_table_risk.py takes the Adult census extract of the previous chapter, with its six quasi-identifiers. It puts age into 10-year bands, native country into US or other, and education into four levels. It then counts the different occupations in each class, with occupation standing in for a sensitive attribute. Both checks are Polars counts, since no maintained Python package computes k-anonymity or distinct $\ell$. ARX in Java and sdcMicro in R do.

```
general = df.with_columns(
    (pl.col("age") // 10 * 10).alias("age"),  # 10-year bands: 30 stands for 30-39
    pl.when(pl.col("native-country") == "United-States")
    .then(pl.lit("US"))
    .otherwise(pl.lit("other"))
    .alias("native-country"),
    # education-num runs from 1 (preschool) to 16 (doctorate); 9 is high-school graduate.
    pl.col("education-num")
    .cut([8, 9, 12], labels=["no diploma", "HS-grad", "some college", "bachelor+"])
    .alias("education"),
)
# Recheck, and count the occupations in each class (distinct l).
classes = general.group_by(QIS).agg(pl.len(), pl.col("occupation").n_unique().alias("l"))
unique = classes.filter(pl.col("len") == 1).height / general.height
one_job = classes.filter(pl.col("l") == 1)["len"].sum() / general.height
print(f"after generalizing: unique={unique:.2%}  in a class with one occupation={one_job:.2%
```

Run it with `uv run python code/part02-attacks/practitioner_table_risk.py`. The four lines below come from its output. The first two use the six generalized quasi-identifiers, and the third adds the raw hours worked per week. The last counts records in classes of 5 or more with a single occupation.

```
after generalizing: unique=0.94%  in a class with one occupation=1.20%
age + sex + race + native-country + marital-status + education: unique=0.94%  in classes <5=
age + sex + race + native-country + marital-status + education + hours-per-week: unique=10.0
  records in classes of size >=5 with l=1: 0.03%
```

Generalizing cut the unique share to 0.94%, from 14.64% on the same six columns before generalizing. Yet the table is still not even 2-anonymous, since its smallest class holds one record. So the remaining 3.31% of records in classes smaller than 5 must be suppressed, or the bands made coarser. The third line shows how fragile the result is, because an adversary who also knows the hours worked finds 10.09% of the records unique again.

## 16.4. Homogeneity, background knowledge and l-diversity

Machanavajjhala and colleagues (2006) pointed out that k-anonymity hides who a record belongs to, but not what it says. If all three records of Alice's class say heart disease, her posterior is 1 without anyone finding her record. This is the **homogeneity attack**. If Bob's class says cancer, cancer and flu, a neighbour who knows he did not have flu learns that he has cancer, which is the **background knowledge attack**. On the hospital table at $k = 5$, 2.3% of patients sit in classes with a single diagnosis.

**l-diversity** asks each class to hold enough different sensitive values. The distinct version asks for at least $\ell$ different values. The entropy version asks that the class's values carry at least $\log_2 \ell$ bits of entropy, as much as $\ell$ equally common values. Neither caps the share of the most common value, so a class can be 3-diverse with one value in most of its records. The recursive version adds that cap, with a constant $c$ that the custodian picks. It requires the most common value's count to stay below $c$ times the combined count of the values ranked $\ell$-th and lower. Then even an adversary who rules out $\ell - 2$ values keeps a posterior below $c/(1 + c)$. That cap is already one half at $c = 1$, so even the strongest version protects against certainty, not likelihood.

On Adult, 1.20% of the generalized records sit in a class where everyone has the same occupation. Only 0.03% of records sit in such a class of 5 or more. So almost all of them are in small classes, and suppressing those also removes most homogeneous ones. On the hospital table, distinct 3-diversity removed the certain disclosures at little cost, while the recursive version forced some classes to grow to 641 patients.

## 16.5. Measuring what a release teaches

Compare every posterior with what the adversary would believe without the release. That is the **imputation baseline** of Attribute inference and model inversion, the best guess of a sensitive value from the population alone. Only the success beyond it is about the individual.

On the hospital table, guessing the most likely diagnosis from each patient's age, with no release, is right for 53.6% of patients. A 5-anonymous release raises that to 59.3%. So most of what the release teaches is the link between age and diagnosis, which any useful study would report. Only the excess of 5.7 percentage points is a leak.

## 16.6. Skewness, similarity and t-closeness

Li, Li and Venkatasubramanian (2007) found two attacks that l-diversity misses. In the **skewness attack**, a class that is half HIV is 2-diverse, yet in a table where 1% have HIV it raises its members' posterior fifty-fold. In the **similarity attack**, a class of three different stomach diseases is 3-diverse but still reveals a stomach disease.

**t-closeness** answers both, by asking that every class's distribution be within a distance $t$ of the whole table's. Write $P_E(A)$ and $P(A)$ for the chance that a value drawn from the class, or from the whole table, lands in a set $A$ of values. With the **total variation (TV)** distance, the condition says that no such chance, and so no belief, moves by more than $t$,

$$|P_E(A) - P(A)| \le t. \tag{16.3}$$

The bound is a difference, not a ratio, so it protects rare values badly. At $t = 0.15$, a value held by 1% of the table may rise to 16%.

Against the similarity attack, Li and colleagues used the **earth mover's distance (EMD)**, which counts similar values, such as two stomach diseases, as close. t-closeness costs a lot. At that $t$, guessing diagnoses from the hospital release was right for only 22.2% of patients, below the 53.6% of imputation from age alone. The release then told less about the population than no release.

## 16.7. The composition attack

Ganta, Kasiviswanathan and Smith (2008) asked what happens with two releases. Suppose two hospitals each publish a k-anonymous or l-diverse table. A patient treated at both sits in a class $E_A$ of one release and a class $E_B$ of the other. The adversary then keeps only the diagnoses that appear in both classes,

$$\{v : n_{E_A}(v) > 0\} \cap \{v : n_{E_B}(v) > 0\}. \tag{16.4}$$

When one candidate is left, the diagnosis is **pinned down**.

In the full edition's simulation, two hospitals share 6,649 patients. Their two 5-anonymous releases together pin down the diagnosis of 9.9% of them, more than three times what either pins alone. With distinct 3-diversity neither release pins anyone, yet together they still pin 18 of them.

No single-table definition can stop this, because the intersection of two tables is a table that neither custodian checked. A definition that survives must answer all three failures. It must compose across releases, hold whatever the adversary knows, and bound the ratio of beliefs rather than their difference, so that rare values are protected. The definition of differential privacy starts from these requirements, which is why statistics offices such as the US Census Bureau have moved towards it (Reconstruction attacks).

## 16.8. Choosing and running an anonymization

Start from the quasi-identifiers of Linkage and re-identification, since a list that is too short is the commonest failure. Use k-anonymity against linkage, and add distinct l-diversity when a class with one sensitive value would be a disclosure. Use t-closeness only when the sensitive attribute is skewed or has similar values, because it costs the most.

Run the anonymizer, then audit its output yourself, as the excerpt does in Polars. The Python package pycanon reports $k$, $\ell$ and $t$ for a pandas table, with calls such as `pycanon.anonymity.k_anonymity(df.to_pandas(), qis)`. Pre-release privacy risk assessment turns these checks into a release decision, with the blocking thresholds fixed before the counts are seen.

Move to differential privacy when other releases of the same people exist, when the sensitive attribute is rare, or when the data are sparse. For tables of counts and sums, Release statistics with a library does the job end to end.

## 16.9. Cheat sheet

Table 16.1.: The syntactic definitions and the attacks on them.

| Definition or attack | Condition | What it gives, and where it ends |
|---|---|---|
| k-anonymity | Every class has at least $k$ records | Linkage at most $1/k$; says nothing about attributes |
| Distinct, entropy l-diversity | At least $\ell$ values, or $\log_2 \ell$ bits of entropy | No cap on the top value's share |
| Recursive $(c, \ell)$-diversity | Top value below $c$ times the values ranked $\ell$-th and lower | Posterior below $c/(1+c)$ after ruling out $\ell - 2$ values |
| t-closeness (TV) | Each class within $t$ of the table | A 1% value may reach $1\% + t$ |
| Composition | Intersect the target's classes in two releases | No single-table definition prevents it |

**Rules of thumb**

- k-anonymity protects names, not attributes, so check what each class's sensitive values reveal.
- Assume another release exists, because a guarantee about one table is not a guarantee about a person.

## 16.10. References


- Machanavajjhala and colleagues, "l-Diversity" (2006), and Li, Li & Venkatasubramanian, "t-Closeness" (2007): the attacks and the definitions that answer them.

- Ganta, Kasiviswanathan & Smith, "Composition Attacks and Auxiliary Information in Data Privacy" (2008): the intersection attack.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 17. Reconstruction attacks

## 17.1. What it is

A **reconstruction attack** rebuilds the records behind published aggregates. It works because every count is a linear equation in the secret values of the people counted. Suppose each of $n$ people has one secret bit. A **subset-sum query** names a set of people and asks how many of them have bit 1. Write $d$ for the list of secret bits and $A$ for the matrix of zeros and ones that lists the attacker's $m$ queries. The party that answers, the **curator**, publishes the true answers plus an error vector $e$,

$$a = Ad + e. \tag{17.1}$$

**Least squares** finds the real numbers whose answers come closest to the published ones, then clips and rounds each to 0 or 1. Suppose each query includes every person with probability one half. Suppose too that the curator adds independent normal noise, and write $\sigma$ for its standard deviation and $\Phi$ for the standard normal cumulative distribution function. Least squares then recovers about a fraction

$$\Phi\left(\frac{\sqrt{m-n}}{4\sigma}\right) \tag{17.2}$$

of the bits. Compare it with the **majority guess**, which gives everyone the commoner value and scores its share, one half only for balanced bits. With $m$ a few times $n$, noise well below $\sqrt{n}$ therefore reveals the data. A curator who answers all $2^n$ subset sums needs error of order $n$ to stop reconstruction (Dinur and Nissim 2003).

These results are the **Fundamental Law of Information Recovery**: answering too many questions too accurately destroys privacy. Stopping reconstruction is not privacy, though. A curator that answers from a secret random half of the people hides the other half, yet gives the sampled half away exactly. And learning only whether someone took part needs far fewer statistics, as a genome study's published frequencies of genetic variants showed (Homer et al. 2008, Lower bounds).

## 17.2. When you meet it

You meet it whenever you publish counts, cross-tabulations, dashboards or a query interface, because each cell of a table is a query. The Census Bureau attacked its 2010 publications and rebuilt a record for every person. About 46% of the rebuilt records matched a real person exactly on block, sex, age, race and ethnicity. Linking them to commercial data then confirmed the identity of about 17% of the population, so the Bureau moved the 2020 Census to differential privacy. Query systems fall too. The noise of Diffix, a commercial one, did not grow with the number of queries. So Cohen and Nissim (2020) averaged it away with 3,500 queries per group of loans in a banking dataset, recovering every bit they targeted.

## 17.3. How to use it

To audit an aggregate release, first count the equations. If the published cells or allowed queries outnumber the people, exact answers can pin down every secret, and only the noise protects them. Then plug your noise level and number of queries into Equation 17.2, and run the attack if the prediction is far above the majority guess. The release fails when either decoder beats that guess. The snippet draws the noise from OpenDP's Gaussian mechanism, as a release would, and runs least squares, the textbook attack. OpenDP's noise cannot be seeded, so the snippet checks the share of bits recovered against the prediction instead of printing it.

```
from statistics import NormalDist

import numpy as np
import opendp.prelude as dp

dp.enable_features("contrib")
rng = np.random.default_rng(0)
n, m = 500, 2000  # people with a secret bit each, and random subset-sum queries
sigma = 0.25 * np.sqrt(n)  # noise sd added to each answer
secret = rng.integers(0, 2, n)
queries = rng.integers(0, 2, (m, n))
space = dp.vector_domain(dp.atom_domain(T=float, nan=False)), dp.l2_distance(T=float)
gaussian = dp.m.make_gaussian(*space, scale=sigma)  # OpenDP's Gaussian mechanism on all m a
answers = np.array(gaussian((queries @ secret).astype(float).tolist()))
estimate = np.linalg.lstsq(queries, answers, rcond=None)[0]  # the textbook attack: least sq
recovered = np.mean(np.clip(estimate, 0, 1).round() == secret)
predicted = NormalDist().cdf(np.sqrt(m - n) / (4 * sigma))
print(f"predicted {predicted:.4f}")  # predicted 0.9584
print(f"recovered within 0.04 of it: {abs(recovered - predicted) < 0.04}")  # recovered with
```

The number of queries matters as much as the noise. With noise of twice $\sqrt{n}$, four queries per person recovered only 0.5875 of the bits in the full edition's simulation, so the curator looks safe. But solving Equation 17.2 at that noise shows that 106.1 queries per person recover 90%. So the noise must grow with the number of answers, which is what differential privacy provides.

For queries that are not random, Dinur and Nissim's linear-programming decoder fits unknowns between 0 and 1 to the answers with `scipy.optimize.linprog`. For published tables, write each cell as a constraint on whole-number unknowns, solve with `scipy.optimize.milp`, and count the solutions, since a single one rebuilds the records.

## 17.4. What goes wrong

The usual mistake is to treat rounding, swapping households or small-cell suppression as noise. An attacker can round the answers of each dataset it tries, so rounding hides nothing. Swapping leaves most constraints exact, and a suppressed cell even adds one, since it says the count is

below the threshold. Sticky noise, which repeats the same answer to a repeated query, only stops repeats, which the attack never needs.

A second is to test one table at a time, when the risk lies in the union of all releases. A third is to trust least squares alone, because errors chosen against it hurt it more than random noise does. So reconstruct your own release with both decoders before publishing it, and record the result in Pre-release privacy risk assessment.

## 17.5. References


- Cohen & Nissim, "Linear Program Reconstruction in Practice" (Journal of Privacy and Confidentiality, 2020): the attack on Diffix.
- Garfinkel, Abowd & Martindale, "Understanding Database Reconstruction Attacks on Public Data" (2019): a census block rebuilt with a solver.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 18. Pre-release privacy risk assessment

## 18.1. Why this matters

Before a model or a table leaves your hands, someone has to decide whether it leaks too much about the people in it. This chapter runs the attacks of this part together, as the assessment a privacy reviewer would ask for. It covers three typical releases: a classifier behind a scoring API, a language model whose weights go to a partner, and a record-level census extract.

The order of the work matters as much as the attacks. Before you run anything, you write down which results would block the release, because a threshold chosen after seeing the numbers can be bent to fit them. A failed attack shows only that this attack failed, so the assessment also reports how strong its attacks were.

Three scripts do the work on public data. `practitioner_lira.py` and `practitioner_table_risk.py` run in seconds. The language-model step reuses the output of `practitioner_lora.py`, the script of Fine-tune a language model privately with LoRA, which takes a few minutes on an idle CPU.

```
uv run python code/part02-attacks/practitioner_lira.py
uv run python code/part02-attacks/practitioner_table_risk.py
uv run python code/part05-dp-ml/practitioner_lora.py
```

## 18.2. Fix the threat model and the thresholds first

Start from the worksheet of Threat models and privacy games, which asks what the secret is, what the adversary observes and what it already knows. Answer it once per artefact, because the same data can be exposed differently in each release. The default privacy unit, what the assessment protects, is the person. The default adversary sees scores, the confidences a model returns, not just labels, since many services return them.

Then write the blocking rules into the plan, as in Table 18.1. The membership rule uses two rates. The true positive rate (TPR) is the share of members an attack flags, and the false positive rate (FPR) the share of non-members it wrongly flags. At an FPR of $10^{-3}$, the rule blocks when the lower end of the TPR's 95% confidence interval exceeds ten times the FPR, so a lucky run cannot trigger it. The factor of ten is a policy choice: an attack may beat chance, but not tenfold.

Several remedies in the table use differential privacy (DP), Part 3's guarantee that caps what any attack can learn about one person by a privacy level $\varepsilon$. One is stochastic gradient descent (SGD) with each record's gradient clipped and noise added, called DP-SGD.

Table 18.1.: The blocking thresholds, written before any attack runs.

| Artefact | The adversary sees | Attack | Blocks the release when | Then |
|---|---|---|---|---|
| Classifier behind a scoring API | Confidences for any input | Online LiRA, 64 models | Lower end of the TPR's 95% interval at FPR $10^{-3}$ is above 0.01 | Regularize, or train with DP-SGD |
| Fine-tuned language model shared with a partner | The weights | Canaries, exposure, greedy extraction | Any secret extracted, or a one-copy canary's extrapolated exposure within 3 bits of the maximum | Deduplicate, or fine-tune with DP |
| Record-level table | The table and a register with the quasi-identifiers | Uniqueness on the agreed quasi-identifiers | Any record unique, or in a class smaller than 5 (11 for counts from US Medicare and Medicaid data) | Generalize and suppress, else release DP statistics |
| Model, sensitive field | The target's other fields and label | Model inversion | Attribute advantage's 95% interval above 0 | Regularize, or train with DP |
| Model whose training mix is secret | Probe outputs | Shadow models, meta-classifier | Accuracy's 95% interval above one half | Do not release that mix |
| Published counts or queries | Every answer | Least squares, linear programming | Either beats the majority guess at the planned noise and query count | DP noise scaled to all answers |
| Federated training | Each client's update | Division and search | Any single update visible | Secure aggregation, client-level DP |
| Any model claimed to be DP | Depends on the claim | Audit with canaries | Certified lower bound on $\varepsilon$ above the claimed $\varepsilon$ | Block: the code or the accounting has a bug |

A smallest class of 5 is a common threshold, and US Medicare and Medicaid data in effect require 11, because their cell-size policy forbids publishing counts from 1 to 10. A canary within 3 bits of the maximum exposure ranks among the 8 likeliest of all possible secrets. The rule reads one-copy

canaries because a deduplicated corpus holds each secret once. If yours is not deduplicated, apply it at your data's largest repeat count.

The four middle rows block on any leak beyond the attack's own baseline, which attribute, property and reconstruction attacks and gradient inversion each define. The last row is for a model trained with DP. An audit with independent trials certifies a lower bound on its $\varepsilon$, and a bound above the claim proves a bug (Auditing private training).

## 18.3. Classifier: the free check

Compare the model's accuracy on its training data with its accuracy on held-out data. Then run the loss attack, which says "member" whenever the loss on the true label is low (Membership inference). The model is a logistic regression on scikit-learn's digits, 1,797 small images of handwritten digits. The script trains 64 copies, each on a random half of the images, because the next step uses them as shadow models. Every copy in turn plays the target, the model under assessment.

```
target 0: train accuracy=1.0000  test accuracy=0.9650
train-test gap, mean over 64 targets=0.0422
ART rule-based attack on target 0: TPR=1.0000  FPR=0.9650  advantage=0.0350
loss attack at FPR 0.001: TPR=0.0009
loss attack at FPR 0.01: TPR=0.0105
```

The first target gets every training image right, but only 0.9650 of its test images. So even "member if correctly classified" beats chance, by the mean gap of 0.0422, a free lower bound on the leak that does not say who is exposed. The Adversarial Robustness Toolbox (ART), a Python library of attacks, runs that rule as `MembershipInferenceBlackBoxRuleBased`, and on the first target its advantage of 0.0350 is that model's gap. At an FPR of $10^{-3}$ the loss attack flags no more members than non-members. A weak attack's failure proves little, so this check can never clear a model. A large gap is reason enough to regularize first, while a small one only means you need the next step.

## 18.4. Classifier: LiRA with shadow models

The likelihood ratio attack (LiRA) compares the target's output on a record with other models' outputs on that same record. **Shadow models** are trained like the target on random halves of the data, so some trained on a given image and some did not. LiRA works with $\phi = \ln(p/(1-p))$, the logit of the confidence $p$ in the true label, which is close to normal across retrainings. For each image, it fits a normal to $\phi$ over the shadows that trained on it (IN) and another over those that did not (OUT). Online LiRA scores the target's $\phi$ by its log-likelihood ratio between the two fits. Offline LiRA uses only the OUT fit, and scores $\phi$ by how many OUT standard deviations it lies above the OUT mean (Membership inference shows the code).

ART's `SklearnClassifier` trains every model, but no library installed with this book computes the LiRA scores, so they come from the helper practitioner_privacy_tests.py.

Four decisions set the attack's strength. Use online LiRA for a release decision, because it is the strongest. Train 64 shadows when training is cheap. With 16 or fewer, fit one variance shared by all images, because per-image variances from a handful of models are too noisy. Put each record in each model by a fair coin, so that every record has IN and OUT shadows. Finally, rotate the target role, so that each model serves once as the target with the other 63 as its shadows. Every model then judges every image, which gives 57,704 member and 57,304 non-member pairs of a model and an image.

An FPR of $10^{-3}$ rests on few false positives, so the reading needs care. The script takes the best TPR among thresholds whose FPR stays at or below the target. It counts the true and false positives and puts a Clopper–Pearson interval, the exact binomial interval of Hypothesis testing, on each rate.

```
# Every (target model, image) pair is one trial; members are the positives.
fpr, tpr, thresholds = roc_curve(member.ravel(), scores.ravel())
# The best TPR among thresholds whose FPR does not exceed the target.
i = int(np.flatnonzero(fpr <= target_fpr)[-1])
n_members, n_nonmembers = int(member.sum()), int((~member).sum())
true_pos = round(tpr[i] * n_members)
false_pos = round(fpr[i] * n_nonmembers)
# Exact 95% intervals (Clopper-Pearson) for both rates.
tpr_ci = clopper_pearson(true_pos, n_members)
fpr_ci = clopper_pearson(false_pos, n_nonmembers)
```

```
  loss attack: AUC=0.518  TPR@0.001=0.0009  TPR@0.01=0.0105
  ART shadow-model attack: AUC=0.570  TPR@0.001=0.0014  TPR@0.01=0.0146
  LiRA offline: AUC=0.577  TPR@0.001=0.0087  TPR@0.01=0.0345
  LiRA online: AUC=0.623  TPR@0.001=0.0318  TPR@0.01=0.0790
[At FPR 0.001: counts, exact 95% intervals, ln(TPR/FPR) as a point estimate, not a certified
  loss attack: TP=53/57704  TPR=0.0009 [0.0007, 0.0012]  FP=57/57304  FPR=0.00099 [0.00075,
  ART shadow-model attack: TP=82/57704  TPR=0.0014 [0.0011, 0.0018]  FP=57/57304  FPR=0.0009
  LiRA offline: TP=502/57704  TPR=0.0087 [0.0080, 0.0095]  FP=56/57304  FPR=0.00098 [0.00074
  LiRA online: TP=1833/57704  TPR=0.0318 [0.0303, 0.0332]  FP=57/57304  FPR=0.00099 [0.00075
```

By area under the curve (AUC) the four attacks look alike, but at an FPR of $10^{-3}$ they differ sharply. Online LiRA flags 1,833 members, a TPR of 0.0318. The lower end of its interval, 0.0303, is above 0.01, so the model fails Table 18.1. Offline LiRA would have passed it, and so would ART's attack of Shokri et al. (2017), which learns one rule for all images from four shadow models of its own. That is why a release decision rests on the strongest attack you can afford. A model with privacy level $\varepsilon$ keeps the TPR within about $e^{\varepsilon}$ times the FPR, so $\ln(\text{TPR}/\text{FPR})$ is a point estimate of the $\varepsilon$ the attack implies. It is not a certified lower bound, because the pairs share models and so are not independent trials (Auditing private training).

The script also prints the ten images whose IN and OUT fits lie furthest apart, the records an attacker exposes first. The top one is image 1658, a 9, given a confidence of 0.812 by models trained on it and 0.003 by the others. Online LiRA flags it, at an FPR of $10^{-3}$, in all 33 models that trained on it. Inspect such records, usually rare or mislabelled ones.

Three things go wrong here. First, shadows trained with another recipe describe another model and so weaken the attack. Second, too few non-members leave the FPR uncertain, and even here its interval runs from 0.00075 to 0.00129. Hypothesis testing says how many you need, and advises choosing thresholds on one split and measuring on another. The script does both on the same pairs, which flatters the attack slightly. Third, a confidence of 1 makes $\phi$ infinite. Compute it from the logits, the model's raw outputs. Clipping the confidence, which the script can afford on the digits, hides a confident model's most exposed records (Membership inference).

## 18.5. Fine-tuned language model: canaries and extraction

A language model can repeat a secret word for word, which a membership test does not measure. So the assessment plants **canaries**, random secrets of a fixed format, and checks how well the trained model knows them (Memorization and training-data extraction). The run adapts Pythia-14m with low-rank adaptation (LoRA) to 1,024 sequences of Tiny Shakespeare, once without privacy and once with DP at $\varepsilon = 8$ per sequence.

The canaries should look like the secrets you fear. Each one reads "the pin of", a name such as Alice, and "is", followed by five random digits, separated by spaces so that every possible secret is exactly five tokens. Three canaries are inserted once, and three others each 4, 16 and 64 times. Three controls are drawn the same way and never inserted. The canaries make up a quarter of the training set, far denser than secrets in real data. So even a model this small memorizes them, and the non-private model is a positive control, one known to leak.

Each canary is scored in two ways, with the functions shown in Fine-tune a language model privately with LoRA. Exposure ranks the secret among random secrets of the same format, by the summed log-probability of its five digit tokens, and turns the rank into bits, as $\log_2$ of the number of candidates minus $\log_2$ of the rank. Use the sum, since the mean loss that `transformers` returns ranks secrets of different token counts wrongly. Extraction gives the model the prefix, lets it write the likeliest token five times (greedy decoding), and checks whether it wrote the secret. Run it with the partner's decoding settings.

```
non-private 1 copies: mean exposure 2.0 bits  extrapolated 2.0  extracted 0/3
non-private 4 copies: mean exposure 5.9 bits  extrapolated 5.7  extracted 0/3
non-private 16 copies: mean exposure 10.0 bits  extrapolated 13.0  extracted 0/3
non-private 64 copies: mean exposure 11.0 bits  extrapolated 16.3  extracted 3/3
non-private controls: mean exposure 1.4 bits  extrapolated 1.5  extracted 0/3
```

Ranking among 2,000 random candidates caps the measured exposure at 10.97 bits. The whole space of secrets allows 16.61, so the measured column can never reach the blocking line 3 bits below that maximum. The extrapolated column, an estimate rather than a measurement, fits a skew-normal distribution to the candidates' scores and reads how far into its tail each secret lies. A never-seen secret has a random rank, worth $\log_2 e = 1.44$ bits on average, and the controls' 1.4 bits match it, so the measurement is sound. The one-copy canaries stay far below the blocking line, at 2.0 bits extrapolated on average. But greedy decoding writes out all three 64-copy secrets, so the non-private model fails Table 18.1 and must not be shared.

The 4-copy canaries show why you measure both. Two of them reach 6.9 and 8.2 bits, yet neither is extracted, so exposure finds memorization that extraction misses. The DP run extracts nothing,

but its most repeated canaries still average about 4 bits, against 2.2 for its controls. That is because $\varepsilon$ protects one sequence, and a repeated secret is many sequences. So deduplicate before training, whatever else you do.

The usual failures are a canary format unlike your data, no controls to show what chance looks like, and ranking by a mean loss over secrets of different lengths. The script also skips the discoverable rate, the share of training sequences whose start makes greedy decoding reproduce the rest. Add it for a release, as Memorization and training-data extraction describes.

## 18.6. Table: re-identification on quasi-identifiers

**Quasi-identifiers** are the columns an adversary can find elsewhere, such as age, sex and country in a register (Linkage and re-identification). Records that agree on all of them form an **equivalence class**, and a record alone in its class can be linked to a name.

The table is Adult, a census extract from the UCI Machine Learning Repository, with 48,842 records. The agreed set has six columns, from age to education, and the script adds hours worked per week to show what one more column does. For each set it prints the share of unique records and the share in classes smaller than 5. It also prints the smallest class size $k$, and J/n, the number of classes per record, which nears 1 as records become unique. Linkage and re-identification shows the loop.

```
age + sex + race: unique=0.12%  in classes <5=0.75%  min k=1  J/n=0.012
age + sex + race + native-country: unique=3.07%  in classes <5=7.83%  min k=1  J/n=0.060
age + sex + race + native-country + marital-status + education: unique=14.64%  in classes <5
age + sex + race + native-country + marital-status + education + hours-per-week: unique=36.5
```

On the agreed six, 14.64% of the records are unique, and hours per week, a column that looks harmless, raises that to 36.55%. The list must therefore include every column an adversary might hold, and when in doubt you test with the column added.

The standard remedy is to generalize. Age goes into 10-year bands, country into United States or other, and education into four levels. The script then checks the classes again and counts the distinct occupations in each, with occupation standing in for a sensitive column. That count is the distinct $\ell$ of Syntactic anonymity and how it breaks, which shows the code.

```
after generalizing: unique=0.94%  in a class with one occupation=1.20%
age + sex + race + native-country + marital-status + education: unique=0.94%  in classes <5=
age + sex + race + native-country + marital-status + education + hours-per-week: unique=10.0
```

Generalizing cuts the unique share on the six from 14.64% to 0.94%, but the smallest class still has one record, so the table still fails Table 18.1. Suppressing the classes smaller than 5 would remove 3.31% of the records. The result would hold only for this list of columns, since adding raw hours per week makes 10.09% unique again. Generalizing also leaves 1.20% of the records in a class with a single occupation, which reveals it even without a unique match.

These figures count uniqueness in the sample, which fits the prosecutor model, where the adversary knows the target is in the table. Otherwise estimate population uniqueness (Linkage and re-identification).

Even a table that passes after suppression has two limits. A second release of the same people can be intersected with this one (Syntactic anonymity and how it breaks). And k-anonymity, which promises each class at least $k$ records, says nothing against richer side information. So when a record-level table cannot pass without losing its use, release statistics with DP instead, as Release statistics with a library does. Published aggregates carry their own risk, since many overlapping counts let an attacker rebuild records (Reconstruction attacks).

## 18.7. Write the decision

The last step compares each result with its threshold and writes a release memo. A reviewer who did not run the attacks must be able to check it, so it gives each estimated rate with its interval, while counts on the table are exact. Table 18.2 fills it in for the three artefacts.

Table 18.2.: A release memo for the three artefacts.

| Memo entry | This assessment |
|---|---|
| Threat model | Scores from an API; weights shared with a partner; a table linked with a register. Unit: the person |
| Attacks and their strength | Loss attack, offline and online LiRA with 64 models; 12 canaries and 3 controls with exposure and greedy extraction; uniqueness on six quasi-identifiers plus one |
| Results | Online LiRA TPR 0.0318 [0.0303, 0.0332] at FPR 0.00099; all three 64-copy secrets extracted; 14.64% unique, 0.94% after generalizing |
| Verdict | All three blocked |
| Mitigation | Classifier: regularize or train with DP-SGD, then rerun. Language model: deduplicate and fine-tune with DP. Table: release DP statistics |
| Residual risk | Not assessed: attribute inference, property inference and reconstruction from aggregates. Not applicable, with no federated training: gradient leakage |

The residual-risk entry, most often left out, names the attacks that were not run. Write it even when every attack fails, because a model can pass this assessment and still leak to an attack nobody tried, and only a DP guarantee bounds every attack at once. For a DP model, the memo

also carries the guarantee and its evidence, which Release and governance assembles into the release package.

## 18.8. Checklist

Table 18.3.: The steps of a pre-release risk assessment, their defaults and their checks.

| Step | Default | Check |
|---|---|---|
| Threat model and thresholds | Person as unit; adversary sees scores; rules of Table 18.1 | Written and dated before any attack runs |
| Free check | Train and test accuracy; loss attack | Gap and loss TPR at FPR $10^{-3}$ reported; a pass clears nothing |
| LiRA | Online, 64 models, fair-coin inclusion, target role rotated | TPR's interval at FPR $10^{-3}$ against 0.01; enough non-members; top records inspected |
| Canaries | Secret format, 1 to many copies, uninserted controls | Controls near 1.44 bits; summed log-probability; deployed decoding |
| Table | Every column an adversary may hold; classes of at least 5 | Unique share and smallest class after generalizing; distinct $\ell$ on the sensitive column |
| Decision | Memo with intervals, verdict, mitigation and residual risk | Every attack not run is named |

## 18.9. References


- Carlini et al., "Membership Inference Attacks From First Principles" (IEEE S&P 2022): LiRA, and why attacks are reported by their TPR at a low FPR.
- Carlini et al., "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, exposure and the skew-normal extrapolation.
- Rocher, Hendrickx & de Montjoye, "Estimating the success of re-identifications in incomplete datasets using generative models" (Nature Communications 2019): population uniqueness from a sample.


**Derivations.** This workflow has no counterpart in the full edition. It draws on the full edition's chapters on threat models in the full edition, membership inference in the full edition, memorization and extraction in the full edition, linkage and re-identification in the full edition and syntactic anonymity in the full edition.

# Part III.

# Differential privacy foundations

The previous part showed that anonymization and access control do not stop releases from leaking. This part introduces **differential privacy (DP)**, the defence the rest of the book builds on. It is a property of the release process, not of a dataset. Changing one person's data may change the probability of any output by at most a factor $e^{\varepsilon}$, set by a privacy level $\varepsilon$, plus a small slack $\delta$. That one condition caps every attack of Part 2. So every DP claim must state three things, namely $\varepsilon$, $\delta$ and the unit of privacy, such as a record, a user or a user-day.

Table 18.4.: Notation used throughout Part 3.

| Symbol | Meaning |
|---|---|
| $\varepsilon$, $\delta$ | Privacy level and slack |
| $\Delta_1 f$, $\Delta_2 f$ | Sensitivity, the most one record moves the answer |
| $\mathrm{Lap}(b)$, $\mathcal{N}(0, \sigma^2)$ | Laplace noise of scale $b$, normal noise of standard deviation $\sigma$ |

The part teaches one workflow, and its last page carries it out. First fix the unit of privacy and the neighbouring relation, as the definition of differential privacy explains. Then bound each person's contribution and calibrate a mechanism to it (The Laplace and Gaussian mechanisms). Next, account for every release, as Post-processing, group privacy and composition explains. Post-processing the outputs is free. Release statistics with a library does all of this with OpenDP on a real shop's invoices, from bounding each customer's lines to the written guarantee.

The full chapters between them cover tasks you meet often. Selection picks the best of many options or a private median. Query release publishes many counts or marginals at once. Trust models apply when nobody can be trusted with the raw data, and Private mean estimation says how many records a private mean needs.

Four short reference pages cover the rest. Advanced composition and Beyond global sensitivity matter when the simple rules are too loose. Read Discrete mechanisms before you sample noise yourself, and Lower bounds before you promise an accuracy.

Part 4 then replaces hand composition with accountants, and Part 5 applies all of it to training models. The code here runs OpenDP and diffprivlib on the Adult census data and on the UCI Online Retail invoices. Every number comes from a snippet or a script you can run. By the end, you will be able to release a count, a sum, a median or a table with error bars and a guarantee you can defend.

# 19. The definition of differential privacy

## 19.1. Why this matters

Every attack in Threat models and attacks runs an algorithm on one of two datasets that differ in one person, and guesses from the output which one was used. **Differential privacy (DP)** makes the two outputs look alike against every test, including tests nobody has invented yet. It is a property of the algorithm, not of the data, so you can check it before any data are touched. This chapter teaches you to read a privacy claim, knowing what it protects, what it promises and what it leaves out.

## 19.2. Neighbouring datasets

The first question about any claim is what "differ in one record" means, and the **neighbouring relation** answers it. Under **add/remove**, two datasets are neighbours when one is the other with one record added or taken away. The guarantee then hides whether a person is in the data at all. Under **replace**, the two datasets have the same size and differ in the value of one record, so only what the record says is hidden. A replacement is a removal followed by an addition, two steps that each cost the privacy level $\varepsilon$ of Section 19.3, so a guarantee converts as

$$\varepsilon\text{-DP under add/remove} \implies 2\varepsilon\text{-DP under replace.} \tag{19.1}$$

The reverse fails, because replace never compares datasets of different sizes, so it cannot hide whether someone is in the data.

You also choose the **unit of privacy**, what one record is. At the **example level**, it is one training record or one event, and at the **user level**, all the data of one person. Under example-level DP, a user who wrote 200 messages is a group of 200 records. A group of $k$ records is protected only at $k\varepsilon$ (Post-processing, group privacy and composition), so check that the claim protects the person. The guarantee must also hide every record the data format allows, even a person aged 900. That is why private algorithms clip each input to a fixed range before they add noise.

## 19.3. The definition

Call a randomized algorithm that takes a dataset and returns an output a **mechanism**, written $M$. Write $D$ for a dataset and $D'$ for one of its neighbours. The definition asks that no set of outputs become much more likely when one record changes. It measures "much more likely" by a factor $e^{\varepsilon}$, for a non-negative privacy parameter $\varepsilon$, and allows a small additive slack $\delta$ on top. A

mechanism is $(\varepsilon, \delta)$**-differentially private** if every pair of neighbours and every set $S$ of outputs satisfy

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{19.2}$$

The probabilities come only from the mechanism's own randomness, with the data held fixed.

When $\delta = 0$ the guarantee is called **pure DP**, and otherwise **approximate DP**. Either way, a private mechanism must be random, because a deterministic one that answers differently on two neighbours gives that answer probability 1 on one and 0 on the other. So rounding, aggregating or hiding small table cells gives no guarantee by itself, however coarse the result looks.

## 19.4. The privacy loss and δ

For an output $y$, write $p_{M(D)}(y)$ for its probability, or its density, when the mechanism runs on $D$. The **privacy loss** of $y$ is the logarithm of how much more likely the output is on $D$ than on $D'$,

$$L(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}. \tag{19.3}$$

Independent outputs multiply their likelihood ratios, so their losses add. That is why the accountants of Part 4, the tools that add up the cost of many releases, work with the loss. Pure $\varepsilon$-DP holds exactly when every output has a loss between $-\varepsilon$ and $\varepsilon$. Laplace noise meets it, while the loss of Gaussian noise grows without limit in the far tail (The Laplace and Gaussian mechanisms).

Under approximate DP, $\delta$ pays for the outputs whose loss exceeds $\varepsilon$. It counts only the part of each one's probability on $D$ that goes beyond $e^{\varepsilon}$ times its probability on $D'$. This total is the **hockey-stick divergence** (Divergences), so $\delta$ is not the chance that the loss exceeds $\varepsilon$. Each $\varepsilon$ has its own smallest $\delta$, and the function $\delta(\varepsilon)$ is the mechanism's **privacy curve**. A quoted pair is one point on it. The noise parameters fix the whole curve, so report them with the pair and others can recompute it (Numerical accounting).

## 19.5. Randomized response

The first private mechanism comes from survey design (Warner, 1965). Each person tells the truth with some probability $t$ a little above one half, and lies otherwise, so a single answer proves nothing about them. A person who tosses a coin, answers truthfully on heads and answers by a second toss on tails tells the truth with probability $\frac{3}{4}$. A true answer is then 3 times as likely as a false one, which gives pure DP at $\varepsilon = \ln 3$. In general, **randomized response** at level $\varepsilon$ tells the truth with probability

$$t = \frac{e^{\varepsilon}}{1 + e^{\varepsilon}}. \tag{19.4}$$

The guarantee is $\varepsilon$ per person however many people answer, provided their coins are independent, but only under replace, since under add/remove the number of reports reveals who answered.

Write $\bar{y}$ for the share of "yes" reports among the $n$ people. The estimate of the true fraction $\pi$ and its standard deviation are

$$\hat{\pi} = \frac{\bar{y} - (1 - t)}{2t - 1}, \qquad \mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon/2}}{(e^{\varepsilon} - 1)\sqrt{n}}, \tag{19.5}$$

For 1,000 people, the two-coin scheme has a standard deviation of 0.0274. When the true fraction is 0.3, that is about twice the sampling error of 0.0145 that the survey already has. It is the price of the **local** model, in which nobody sees a true answer (Local, shuffle and distributed trust models).

OpenDP builds the mechanism from its truth probability, and `map` turns one changed answer into the mechanism's $\varepsilon$, here $\ln 3$. Its `contrib` feature switches on mechanisms not yet formally vetted.

```
import opendp.prelude as dp

dp.enable_features("contrib")
rr = dp.m.make_randomized_response_bool(prob=0.75)
print(f"{rr.map(1):.4f}")  # 1.0986
```

## 19.6. What ε means for an attacker

A membership attacker looks at the output and decides whether to accuse the target of being in the data, so it has two error rates (Hypothesis testing). Its **false positive rate (FPR)** is how often it accuses a target who is out, and its **true positive rate (TPR)** how often it accuses one who is in. The outputs on which it accuses form a set, which the definition bounds, so under add/remove every attack at every FPR satisfies

$$\mathrm{TPR} \le e^{\varepsilon}\,\mathrm{FPR} + \delta. \tag{19.6}$$

This is the form to use, because Part 2 judged attacks by their TPR at a low FPR. For example, take $\varepsilon = 1$ and $\delta = 10^{-5}$, the value that the rule of Section 19.7 gives for 10,000 people. Then at an FPR of $10^{-3}$, no attack reaches a TPR above 0.002728. As the FPR falls towards 0, the $\delta$ term takes over, so at a very low FPR the guarantee rests almost entirely on $\delta$.

Read backwards, the cap turns a measured attack into a lower bound on $\varepsilon$,

$$\varepsilon \ge \ln \frac{\mathrm{TPR} - \delta}{\mathrm{FPR}}. \tag{19.7}$$

Suppose a model is claimed to be $(2, 10^{-5})$-DP, and an attack reaches a TPR of 0.05 at an FPR of 0.001. If those rates were exact, $\varepsilon$ would be at least 3.91, and the claim would be false. A real audit puts confidence bounds on the rates and plants a **canary**, a record built to be as easy to detect as possible (Auditing).

Two summaries are easier to explain to a lawyer. The **membership advantage**, the TPR minus the FPR, says how much better than a coin flip an attacker does. Under $(\varepsilon, \delta)$-DP it is at most

$$\mathrm{TPR} - \mathrm{FPR} \le \frac{e^{\varepsilon} - 1 + 2\delta}{e^{\varepsilon} + 1}. \tag{19.8}$$

The second summary, the posterior bound, speaks about beliefs. Under pure DP, one output can multiply an adversary's odds between two neighbours by at most $e^{\varepsilon}$, whatever it believed before. So a prior belief $p$ that the data are $D$ rises at most to

$$\Pr[D \mid y] \leq \frac{e^{\varepsilon} p}{e^{\varepsilon} p + 1 - p}. \tag{19.9}$$

Table 19.1 turns a claimed $\varepsilon$ into both numbers.

Table 19.1.: What a pure $\varepsilon$-DP guarantee promises against the strongest attacker.

| $\varepsilon$ | Advantage bound | Posterior from a prior of 0.01 | Posterior from a prior of 0.5 |
|---|---|---|---|
| 0.1 | 0.0500 | 0.0110 | 0.5250 |
| 1 | 0.4621 | 0.0267 | 0.7311 |
| 3 | 0.9051 | 0.1687 | 0.9526 |
| 8 | 0.9993 | 0.9679 | 0.9997 |

So an $\varepsilon$ of at most 1 is strong for any mechanism. Between 1 and 10 a claim needs the mechanism's own privacy curve as justification, and above 10 an audit as well. Choosing $\varepsilon$ uses the same tiers to set a value for training. For statistics, start from the error you can accept, solve it for $\varepsilon$ (The Laplace and Gaussian mechanisms), and add up the costs of all the releases.

The table is for pure DP. For an $(\varepsilon, \delta)$ claim, quote the advantage bound and the cap on the TPR at a low FPR, which both include $\delta$.

## 19.7. How small δ must be

A small $\delta$ barely moves the advantage bound, but the advantage averages over everyone, so it can hide total harm to a few people. For example, take a mechanism that publishes each record in full, independently with probability $\delta$. Under add/remove it is $(0, \delta)$-DP, because a record changes the output only when its own coin publishes it. Yet over $n$ records it publishes at least one with probability

$$1 - (1 - \delta)^n \approx 1 - e^{-n\delta}. \tag{19.10}$$

With 10,000 records and $\delta = 1/n$, someone's full record is published with probability 0.6321.

So treat $\delta$ as a per-person budget for catastrophe, and unless you know the mechanism, assume it may be spent on total failures like these. This book's default keeps $n\delta$, the expected number of people exposed this way, below 1. Let $n$ count people rather than records, and choose

$$\delta \leq \frac{1}{n^{1.1}}, \tag{19.11}$$

rounded down to a power of ten. For 10,000 people the rule gives $10^{-5}$. Even the publishing mechanism above then exposes someone with probability only 0.0952.

## 19.8. What differential privacy does not protect

DP compares a world with your record and a world without it, so anything that can be learned in both worlds is outside the guarantee. A study that never used Alice's record may show that smokers get cancer at a high rate. That changes what anyone believes about Alice, a smoker, yet the study is 0-DP with respect to her. Correlated records are a second gap, since a household of five that always answers alike is a group of five, unless the household is the unit of privacy. The third gap is the implementation, which needs its own review (Discrete and floating-point-safe mechanisms).

For a reported violation, ask whether the inference would differ materially without the person's record. If not, as when public "likes" predict someone's politics, it is not a violation in the DP sense, even when it causes harm.

## 19.9. Writing the guarantee down

A reviewer can check only what a claim writes down, so a guarantee needs more than its two numbers. It names the unit with its neighbouring relation, and $\varepsilon$ and $\delta$ for the whole release rather than per query. It also names the mechanism, the library version, the trust model and what was treated as public, since public information is not protected. The record uses the **central model**, in which a trusted curator holds the raw data and adds the noise. For example:

> This release satisfies $(1, 10^{-5})$-DP for adding or removing one customer and all their invoice lines, at most 100 per customer, under the central model, with Gaussian noise from OpenDP 0.16.0. The market list was fixed in advance.

Release statistics with a library and Release and governance print such statements from the code that made the release.

## 19.10. Code

This snippet checks two numbers quoted above.

```
from math import exp, log

print(f"{(exp(3) - 1) / (exp(3) + 1):.4f}")  # 0.9051
print(f"{log((0.05 - 1e-5) / 1e-3):.2f}")  # 3.91
```

## 19.11. Cheat sheet

Table 19.2.: The results of this chapter.

| Result | Formula | Notes |
|---|---|---|
| $(\varepsilon, \delta)$-DP | $\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$ | All neighbours, all sets $S$ |
| Attack cap | $\text{TPR} \leq e^{\varepsilon} \text{FPR} + \delta$ | Audit: $\varepsilon \geq \ln((\text{TPR} - \delta)/\text{FPR})$ |
| Choosing $\delta$ | $\delta \leq 1/n^{1.1}$, rounded down to a power of ten | $n$ counts people, not records |

**Rules of thumb**

- When people contribute many rows, the unit of privacy is the person.
- An $\varepsilon$ is never a probability of re-identification. Under pure DP it bounds how far any output can shift an attacker's odds.

## 19.12. References


- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 2: the definition and what DP promises.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): the hypothesis-testing reading behind the attack cap.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 20. Post-processing, group privacy and composition

## 20.1. Why this matters

A private mechanism is never the whole system. Its output gets rounded, plotted or fed into a model, people may own many records each, and an analyst rarely asks only one question. You need to know what guarantee holds for what you actually publish, without a new proof for every pipeline.

Three properties of differential privacy (DP) give that answer. The first, post-processing, says that anything computed from a private output without reading the data again is free. The second, group privacy, says what a guarantee about one record promises a person who owns several. And composition says that running several mechanisms on the same data costs at most the sum of their budgets. Together they turn privacy into bookkeeping, because you prove each building block once and then keep a ledger of what you spend.

## 20.2. Post-processing

Take a mechanism $M$ with privacy level $\varepsilon$ and slack $\delta$, so that it is $(\varepsilon, \delta)$-DP. Apply to its output any function $f$, which may use randomness of its own but never reads the data. Then the processed release is just as private,

$$f(M(D)) \text{ is } (\varepsilon, \delta)\text{-DP.} \tag{20.1}$$

Processing can blur the evidence that an output carries about the data, and so lower the privacy loss, but it can never raise it.

This is what makes DP practical. Rounding a noisy count, clamping it at zero, or adjusting noisy subtotals so that they add up to their noisy total costs no extra budget. So a private system adds noise at one narrow point, such as noisy counts or noisy gradients, and treats everything after it as public. The same rule covers attackers, because a membership-inference attack is a function of the output. So the definition's cap on how often any attack can succeed holds even for attacks invented after the release.

The condition is that $f$ never reads the data. "Take the noisy top category, then report its exact count" breaks it, because the second step is a new and non-private mechanism.

## 20.3. Group privacy

The definition protects one record, but a customer of an online shop has dozens of invoice lines. If the guarantee is per record and a person owns $k$ records, you need a guarantee for datasets that differ in $k$ records. You can walk from one dataset to the other by changing one record at a time, and each of the $k$ steps costs $\varepsilon$. So for pure DP, where $\delta = 0$, $\varepsilon$-DP per record becomes $k\varepsilon$-DP for the group. With a slack, the slack grows much faster than $k$. For a group of $k$ records, an $(\varepsilon, \delta)$-DP mechanism is

$$\Big(k\varepsilon,\ \delta_k\Big)\text{-DP}, \qquad \delta_k = \delta\,\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1} \le k\,e^{(k-1)\varepsilon}\delta. \tag{20.2}$$

The fraction is the sum $\delta\,(1 + e^{\varepsilon} + \cdots + e^{(k-1)\varepsilon})$, and the simpler form on the right is looser. Take $\varepsilon = 0.5$ and $\delta = 10^{-5}$ per record. Then a person with 10 records gets a slack of $2.272 \times 10^{-3}$. The simple form would claim $9.002 \times 10^{-3}$, about four times too much. The growth is far worse at $\varepsilon = 1$ per record, where the fraction multiplies $\delta$ by 12818.3 for a group of ten. That hurts, because $\delta$ was chosen far below one over the number of people in the first place (How small δ must be).

Don't compute group guarantees by hand, because OpenDP computes them from the mechanism itself. A measurement's `map(d_in=k)` is its guarantee when neighbouring datasets differ in $k$ records, here for a count with Laplace noise and one with Gaussian noise.

```
count = dp.t.make_count(dp.vector_domain(dp.atom_domain(T=int)), dp.symmetric_distance())
laplace = count >> dp.m.then_laplace(scale=2.0)  # epsilon = 0.5 per record
print(f"Laplace count: epsilon {laplace.map(d_in=1):.1f} per record, {laplace.map(d_in=10):.

def gaussian(scale: float) -> dp.Measurement:  # a Gaussian count, as a privacy curve
    return dp.c.make_zCDP_to_approxDP(count >> dp.m.then_gaussian(scale=scale))

def at_1e5(scale: float) -> dp.Measurement:
    return dp.c.make_fix_delta(gaussian(scale), delta=1e-5)

scale = dp.binary_search_param(at_1e5, d_in=1, d_out=(0.5, 1e-5))  # (0.5, 1e-5) per record
curve = gaussian(scale).map(d_in=10)  # the privacy curve for a group of 10 records
print(f"Gaussian count, group of 10: epsilon {curve.epsilon(1e-5):.2f} at delta 1e-5")
# delta_k is the slack that the group-privacy formula gives the group
print(f"Gaussian count, group of 10: epsilon {curve.epsilon(delta_k):.2f} at delta {delta_k:
```

```
Laplace count: epsilon 0.5 per record, 5.0 for 10
Gaussian count, group of 10: epsilon 6.43 at delta 1e-5
Gaussian count, group of 10: epsilon 4.57 at delta 2.272e-03
```

For the Laplace count, `map(d_in=10)` gives $k\varepsilon = 5.0$, as the formula does. The Gaussian count, calibrated to $(0.5, 10^{-5})$ per record, does better than the formula, because OpenDP reads the group's guarantee off the mechanism's whole privacy curve. At the formula's slack its group gets $\varepsilon = 4.57$, which is below the formula's 5.0, and keeping the original slack instead costs more. Read

a group guarantee off the library's map, then, and fall back on the formula only for a mechanism the library cannot map.

So if you care about people, don't patch a per-record guarantee with group privacy. Make the person the unit of privacy, so that neighbouring datasets differ in all of one person's records. Then bound how many records each person may contribute before any noise is added, as Release statistics with a library does on real data. Group privacy is the fallback for when you cannot, and for records of different people that reveal each other, such as the members of a household who share an address and an income.

## 20.4. Sequential composition

An analyst asks several queries of the same data, each answered with fresh noise, and an adversary sees all the answers together. Say there are $k$ mechanisms, and write $(\varepsilon_i, \delta_i)$ for the budget of the $i$-th. Releasing all their outputs is then

$$\Big(\sum_{i=1}^{k} \varepsilon_i,\ \sum_{i=1}^{k} \delta_i\Big)\text{-DP}. \tag{20.3}$$

This is **basic composition**. It needs every mechanism to draw fresh noise and to use the same neighbouring relation, add/remove or replace.

The rule also covers **adaptive** composition, where each query is chosen after reading the earlier answers, as long as each budget is fixed in advance. Budgets chosen from the answers are another matter, as in "keep going until the estimate looks good". They need a privacy filter, a rule that decides when to stop as the budget is spent (Adaptive composition).

Basic composition is loose for many releases. Take ten counts with Gaussian noise, each calibrated to $(0.5, 10^{-5})$. Basic composition charges them $(5, 10^{-4})$, but at the same total slack their exact $\varepsilon$ is only 1.504. So once releases number more than a handful, use an accountant, which computes the total from the mechanisms themselves (Advanced and optimal composition, Part 4).

## 20.5. Parallel composition

Per-region statistics use disjoint subsets of the data. When each person has one record, each region can spend the full budget, because that record falls in one region only. To use this, fix in advance a public rule that sends every possible record to exactly one part, and run a mechanism $M_i$ with independent noise on the $i$-th part. If each $M_i$ is $(\varepsilon_i, \delta_i)$-DP under add/remove, the whole release is

$$\big(\max_i \varepsilon_i,\ \max_i \delta_i\big)\text{-DP under add/remove}. \tag{20.4}$$

This is why a histogram costs one $\varepsilon$, not one per bin. With Laplace noise of scale $1/\varepsilon$ on every bin, the histogram is $\varepsilon$-DP however many bins it has. Under replace, a record can move from one bin to another, so two bins change and the cost doubles to $2\varepsilon$. A person with records in several bins breaks the rule too, and then you must either charge each bin as a separate query or first bound the number of bins a person may touch.

The rule fixed in advance is the condition that is easiest to break. A split into "the top 10% of earners and the rest" moves its boundary when one high earner is added, so records cross it and both parts change. The same goes for partitioning on the list of keys that happen to appear in the data, because a key that appears only through one person's record reveals that this person is present. So use fixed bins or a public key list. When the keys are not public, release only the keys whose noisy count clears a threshold, a step called **partition selection** (Release statistics with a library).

## 20.6. Keeping a budget

Fix a total budget for a dataset before the first release, and charge every release against it. Libraries keep this ledger for you. In the Python library diffprivlib, a `BudgetAccountant` holds a total, and each diffprivlib function you hand it charges its own $\varepsilon$ and $\delta$. A call that would exceed the total raises a `BudgetError` instead of answering. The script practitioner_adult_statistics.py runs it on the Adult census data, 48,842 people with one row each, for which the default slack is $10^{-6}$. It spends half the budget on the mean age and half on the number of people over 50, and then asks for one more count.

```
age = adult["age"].to_numpy()
acc = BudgetAccountant(epsilon=1.0, delta=1e-6)  # the total, fixed in advance
mean_age = tools.mean(age, epsilon=0.5, bounds=(17, 90), accountant=acc, random_state=seed)
over_50 = tools.count_nonzero(age > 50, epsilon=0.5, accountant=acc, random_state=seed)
print(f"private mean age: {mean_age:.2f}  private count over 50: {over_50:,}")
print(f"spent: {acc.total()}")
try:
    tools.count_nonzero(age > 60, epsilon=0.1, accountant=acc, random_state=seed)
except BudgetError as err:
    print(f"refused: {err}")
```

Run it with `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`. With `random_state=0` it prints the following.

```
private mean age: 38.64  private count over 50: 9,813
spent: (epsilon=1.0, delta=0.0)
refused: Privacy spend of (0.1,0) not permissible; will exceed remaining privacy budget. Use
```

The true values are 38.64 and 9,808. Both calls add Laplace noise, so no $\delta$ was spent and the $10^{-6}$ stays available for a Gaussian query. The accountant applies basic composition to every call, so it overcharges calls on disjoint parts of the data, and it trusts each call's own guarantee. For example, diffprivlib's mean divides by the exact number of rows, treating it as public. That number is public only when neighbours have the same size, which is the replace relation, so this ledger holds under replace neighbours.

Most real failures break a condition of the three properties, and the damage is usually total rather than a small loss. Table 20.1 lists the common cases.

Table 20.1.: What composes and what doesn't.

| Situation | Composes? | What to do |
|---|---|---|
| A function of the output, with its own coins | Yes, for free | Process as much as you like |
| Another query on the same data, fresh noise | Yes, budgets add | Charge it to the ledger |
| A query chosen from earlier answers | Yes, budgets add | Fix its budget in advance |
| Two releases that share one noise draw | No | Draw fresh noise for every query |
| "Post-processing" that reads the data again | No | Charge it as a new mechanism |
| A DP release beside an exact statistic of the same data | No, the exact one has no guarantee | Release the statistic privately too |
| A split whose boundaries depend on the data | No, not in parallel | Fix the bins in advance |
| Guarantees under add/remove and replace in one ledger | Only after conversion | Convert to replace: $2\varepsilon$ and $(1 + e^{\varepsilon})\delta$ |

Shared noise is the clearest case. Release the number of smokers and the number of smokers aged 40 or over, adding the same Laplace draw to both. With noise of scale 1, each release alone is 1-DP, but their difference cancels the noise and gives the exact number of smokers under 40, so the pair is not DP at all. In code, the bug hides in two mechanisms seeded with the same random seed, in a cached noise value re-added to a related query, and in noise reused across shards. A fixed seed such as the `random_state=0` above reproduces a book's numbers, but in production it makes the noise reproducible and so removable. Returning the same noisy answer to a repeated identical query is fine, because that is post-processing of one release.

## 20.7. Cheat sheet

Table 20.2.: The results of this chapter.

| Property | Rule | Condition |
|---|---|---|
| Post-processing | $f(M(D))$ is $(\varepsilon, \delta)$-DP | $f$ never reads the data |
| Group privacy | $(k\varepsilon,\ \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1})$ for $k$ records | Prefer the person as the unit, with bounded contributions |
| Basic composition | $(\sum \varepsilon_i, \sum \delta_i)$ | Fresh noise, one neighbouring relation, budgets fixed in advance |
| Parallel composition | $(\max \varepsilon_i, \max \delta_i)$ | Add/remove, parts fixed in advance; $2\varepsilon$ under replace |

**Rules of thumb**

- Add noise once, as early as possible, and everything computed from the noisy output after it is free.
- Fix the total budget before the first release, and let one accountant refuse what exceeds it.
- Never share a random seed between mechanisms, and never fix one in production.

## 20.8. References


- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): post-processing, group privacy and basic composition, the standard reference.
- McSherry, "Privacy Integrated Queries" (SIGMOD 2009): parallel composition, and a query system built on post-processing and composition as its only rules.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 21. The Laplace and Gaussian mechanisms

## 21.1. Why this matters

Suppose you run a hospital database and want to publish how many patients have diabetes with a differential privacy (DP) guarantee. Almost every such release is built with one of the two mechanisms of this chapter, and so are the noisy gradients of private model training.

Both compute the exact answer of a query $f$ on the dataset $D$. They then add fresh random noise, written $Z$, so that the mechanism publishes

$$M(D) = f(D) + Z. \tag{21.1}$$

Three decisions fix the whole mechanism. The **sensitivity**, how far one record can move the answer, is what the noise must hide. The noise distribution sets the kind of guarantee, since Laplace noise gives pure DP while Gaussian noise needs a small slack $\delta$. And a calibration formula sets the amount of noise for a target $\varepsilon$, through the privacy loss of the definition of differential privacy.

## 21.2. Sensitivity

The noise must cover the largest change that one record can make to the answer. A query often returns a vector of $d$ numbers, such as the bins of a histogram, so you need a size for a change to a vector. Laplace noise uses the $\ell_1$ norm, the sum of the absolute changes. Gaussian noise uses the $\ell_2$ norm, the square root of the sum of the squared changes. Writing $D \sim D'$ for a pair of neighbours, the $\ell_p$ **sensitivity** of the query is the largest change between their answers,

$$\Delta_p f = \max_{D \sim D'} \left\| f(D) - f(D') \right\|_p, \qquad p \in \{1, 2\}. \tag{21.2}$$

The two are equal when one record moves a single coordinate. When it moves all $d$ of them, the $\ell_1$ sensitivity can be up to $\sqrt{d}$ times the $\ell_2$ one.

The sensitivity also depends on the neighbouring relation. Under add/remove one dataset has one record more than the other, and under replace one record is swapped for another, so the size $n$ is public. Write $B$ for the largest value a record may take after clipping, which the next paragraphs explain. Table 21.1 then gives the standard queries.

Table 21.1.: Sensitivities of the standard queries under the two neighbouring relations.

| Query | Add/remove $\Delta_1$ | Add/remove $\Delta_2$ | Replace $\Delta_1$ | Replace $\Delta_2$ |
|---|---|---|---|---|
| Count $\lvert D \rvert$ | 1 | 1 | 0 | 0 |

| Query | Add/remove $\Delta_1$ | Add/remove $\Delta_2$ | Replace $\Delta_1$ | Replace $\Delta_2$ |
|---|---|---|---|---|
| Sum, values in $[0, B]$ | $B$ | $B$ | $B$ | $B$ |
| Mean, values in $[0, B]$, $n$ public | none (Section 21.6) | none | $B/n$ | $B/n$ |
| Histogram with $k$ bins | 1 | 1 | 2 | $\sqrt{2}$ |
| $d$ threshold counts $\#\{x \geq j\}$, $j = 1..d$ | $d$ | $\sqrt{d}$ | $d$ | $\sqrt{d}$ |

A replaced record leaves one histogram bin and joins another, which is why the relation must be named. The threshold counts, the numbers of records at or above each of $d$ values, are a **dense** query, in which one record can move every coordinate.

Compute the sensitivity from the domain, before you look at the data. A sensitivity read off the data, for example from the largest value present, makes the noise itself leak that value (Beyond global sensitivity). For unbounded data the sensitivity is infinite, so clip every value to a fixed range first. For example, take the mean age of 1,000 patients, with ages clipped to at most 100. Under replace its sensitivity is 0.1, since no patient can move the mean by more than a tenth of a year.

## 21.3. The Laplace mechanism

The Laplace mechanism (Dwork et al., 2006) adds independent noise from the **Laplace distribution** to each coordinate, a two-sided exponential peaked at 0. Its scale, written $b$, sets its typical size, and $\mathrm{Lap}(b)$ denotes it. For an $\varepsilon$-DP guarantee, the mechanism sets the scale to the sensitivity divided by $\varepsilon$,

$$b = \frac{\Delta_1 f}{\varepsilon}. \tag{21.3}$$

The error of the mechanism is the noise itself. Each coordinate is off by $b$ on average, and lands more than a distance $t$ from the truth with probability $e^{-t/b}$. To bound all $d$ coordinates at once, except with a small failure probability $\beta$, give each coordinate a share $\beta/d$ of it, so that the shares add up to the whole. The width that results is

$$\Pr\left[\max_i |Z_i| > b \ln \frac{d}{\beta}\right] \leq \beta. \tag{21.4}$$

Publish this width with the release. You can also read it backwards as an error budget. For a single number, where $d = 1$, fix a target width $t$ and solve for the privacy level, which gives

$$\varepsilon = \frac{\Delta_1 f \, \ln(1/\beta)}{t}. \tag{21.5}$$

For example, a count that must be within 10 of the truth, with probability 0.95, needs an $\varepsilon$ of 0.2996. For $d$ numbers released together, solve each one with the failure probability $\beta/d$. Their $\varepsilon$ then add up by composition.

At the hospital, 1,234 of the 10,000 patients have diabetes. A count has sensitivity 1 under add/remove, so at $\varepsilon = 0.1$ the noise is Lap(10). Its 95% error bar is $10 \ln 20 = 29.96$, a relative error of 0.0243.

The scale does not contain the size of the data, so the relative error of a count of size $n$ is about $1/(n\varepsilon)$. But the sensitivity must include every number one record can move. If one record can change all of 10 released counts, noise calibrated for one count makes the release $10\varepsilon$-DP, not $\varepsilon$-DP.

## 21.4. The Gaussian mechanism

The Gaussian mechanism adds independent normal noise with mean 0 to each coordinate, with a standard deviation written $\sigma$. Its privacy loss is unbounded in the far tail, so no $\varepsilon$ covers every output, and it always carries a $\delta$. In exchange, it is calibrated to the $\ell_2$ sensitivity, which can be much smaller than the $\ell_1$ one. Only the ratio of the sensitivity to the noise matters, so write $\mu = \Delta_2 f/\sigma$ for the number of noise standard deviations between the answers on two neighbours. Write $\Phi$ for the cumulative distribution function (CDF) of the standard normal distribution. The mechanism is then $(\varepsilon, \delta)$-DP exactly when $\delta$ is at least

$$\delta_\mu(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{21.6}$$

The formula is how you calibrate. As $\mu$ grows, $\delta_\mu(\varepsilon)$ rises steadily from 0 to 1, so exactly one value $\mu^*$ hits your target $\delta$. The smallest noise that works is then

$$\sigma^* = \frac{\Delta_2 f}{\mu^*}, \qquad \text{where } \delta_{\mu^*}(\varepsilon) = \delta, \tag{21.7}$$

and libraries solve for $\mu^*$ for you (Section 21.7). Take $\varepsilon = 1$ and $\delta = 10^{-5}$, the book's default for the hospital's 10,000 patients (The definition of differential privacy). Then this **analytic calibration** gives $\sigma^* = 3.7306\,\Delta_2 f$.

Many papers and libraries still use the classic calibration of Dwork and Roth (2014). For $\varepsilon$ and $\delta$ both between 0 and 1, it sets

$$\sigma = \frac{\Delta_2 f \sqrt{2\ln(1.25/\delta)}}{\varepsilon}. \tag{21.8}$$

Table 21.2 compares the two at $\delta = 10^{-5}$.

Table 21.2.: Classic against analytic calibration of the Gaussian mechanism at $\delta = 10^{-5}$.

| $\varepsilon$ | Classic $\sigma/\Delta_2 f$ | Analytic $\sigma^*/\Delta_2 f$ | Ratio |
|---|---|---|---|
| 0.1 | 48.4481 | 30.7496 | 1.576 |
| 1 | 4.8448 | 3.7306 | 1.299 |
| 8 | 0.6056 | 0.6002 | 1.009 |

At $\varepsilon = 1$ the classic formula adds 30% more noise than needed, and used above 1 it is wrong, not just loose. At $\delta = 10^{-5}$ the two calibrations cross at $\varepsilon = 8.42$, and beyond it the classic $\sigma$ is too small to give the claimed guarantee.

A Gaussian mechanism satisfies a whole curve of guarantees, one $\delta$ for each $\varepsilon$, and the accountants of Part 4 compose whole curves. So report $\sigma/\Delta_2 f$ with every Gaussian release, as well as the pair you chose.

Return to the diabetes count, at the Laplace record's $\varepsilon = 0.1$ and with $\delta = 10^{-5}$. Its 95% error bar is 1.96 standard deviations, or 60.27. Laplace noise gave 29.96 with no $\delta$ at all, so for a single count it wins on both error and guarantee.

## 21.5. Laplace or Gaussian?

The choice depends on how one record spreads its change across the coordinates, as the root-mean-square error (RMSE) of each released number shows. Write $\sigma_1$ for the analytic noise level at unit $\ell_2$ sensitivity. A Lap($b$) variable has standard deviation $\sqrt{2}\,b$, so at the same $\varepsilon$ the errors per coordinate are

$$\text{Laplace: } \frac{\sqrt{2}\,\Delta_1 f}{\varepsilon}, \qquad \text{Gaussian: } \Delta_2 f\,\sigma_1. \tag{21.9}$$

In a sparse query, such as a histogram under add/remove, one record moves one coordinate. The two sensitivities are then equal, so at $\varepsilon = 1$ and the default $\delta$ Laplace noise wins at every $d$, since $\sqrt{2} = 1.4142$ is below 3.7306. In a dense query, by contrast, one record moves every coordinate. The Laplace error then grows like $d$ while the Gaussian error grows only like $\sqrt{d}$, so Laplace noise is more accurate only while

$$d < \frac{(\varepsilon\sigma_1)^2}{2}. \tag{21.10}$$

At $\varepsilon = 1$ and the default $\delta$, the unit noise level is 3.7306, so for threshold counts Gaussian noise wins from 7 counts on.

Gaussian noise also wins when releases are composed many times, because several Gaussian releases compose exactly into one Gaussian mechanism, whose $\ell_2$ sensitivity is the square root of the sum of the squared sensitivities. Private training needs both, since one record's gradient moves every coordinate of a model at every step. So it clips each gradient's $\ell_2$ norm and calibrates Gaussian noise to that clip.

## 21.6. Calibrating a release

Every noisy release takes five steps.

1. Fix the neighbouring relation and the unit of privacy, usually one person under add/remove.
2. Clip every value to a range fixed in advance, and read the sensitivity off Table 21.1 or work it out from the worst record.
3. Choose Laplace noise for one number or a sparse vector, and Gaussian noise for a dense vector.
4. Set the scale with Equation 21.3 or Equation 21.6, and charge it to your budget (Post-processing, group privacy and composition).
5. Draw the noise with a vetted library, and publish the error bar and the noise parameters with the result.

A mean under add/remove has no simple sensitivity, because its denominator moves too. Release instead a noisy sum and a noisy count, each with half of the budget, and divide one by the other. Dividing is post-processing, so it costs nothing more. The mean's error bar comes from theirs. Take each bar at 97.5%, so that both hold together 95% of the time. The mean then lies above the sum's lower end divided by the count's upper end. It lies below the sum's upper end divided by the count's lower end, provided both lower ends are positive. OpenDP's `.dp.mean` makes the same split into a sum and a count, and `summarize(alpha=0.025)` reports the two bars at 97.5%. When you do not know the range to clip to, Private mean estimation shows how to find it privately.

Plain `numpy.random.laplace` is for prototypes only, because textbook floating-point samplers can leak through the low-order bits of their output (Discrete and floating-point-safe mechanisms). For a whole release with clipping, grouping and one budget, follow Release statistics with a library.

## 21.7. Code

The libraries do the calibration for you. In OpenDP, a mechanism's `map` takes the sensitivity and returns the $\varepsilon$ it costs. The accuracy helpers turn a scale into a 95% error bar or back, and since a count has sensitivity 1, one over the scale is the $\varepsilon$ of the error budget. IBM's diffprivlib calibrates the analytic Gaussian from $\varepsilon$, $\delta$ and the sensitivity. The snippet checks the hospital's numbers and a row of Table 21.2.

```
import math

import opendp.prelude as dp
from diffprivlib.mechanisms import GaussianAnalytic

dp.enable_features("contrib")
laplace = dp.m.make_laplace(dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float
print(laplace.map(1.0))  # 0.1
print(f"{dp.laplacian_scale_to_accuracy(10.0, alpha=0.05):.2f}")  # 29.96
print(f"{1 / dp.accuracy_to_laplacian_scale(10.0, alpha=0.05):.4f}")  # 0.2996
gauss = GaussianAnalytic(epsilon=1.0, delta=1e-5, sensitivity=1.0)
print(f"{math.sqrt(gauss.variance(0.0)):.4f}")  # 3.7306
```

To add the noise, call the mechanism on the value, as in `laplace(1234.0)`.

## 21.8. Cheat sheet

Table 21.3.: The results of this chapter.

| Result | Formula | Notes |
|---|---|---|
| Laplace mechanism | $\mathrm{Lap}(\Delta_1 f/\varepsilon)$ per coordinate | Pure DP; error does not depend on $n$ |
| Laplace error bar | $b\ln(d/\beta)$ for all $d$ coordinates, scale $b$ | Solve it for $\varepsilon$ to budget error |
| Exact Gaussian $\delta$ | $\Phi(\frac{\mu}{2}-\frac{\varepsilon}{\mu})-e^{\varepsilon}\Phi(-\frac{\mu}{2}-\frac{\varepsilon}{\mu})$, $\mu=\Delta_2 f/\sigma$ | Solve for $\sigma$ with `GaussianAnalytic` |
| Classic Gaussian | $\sigma=\Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$ | Too much noise below 1, unsafe above 8.42 |
| Dense query | Laplace wins only for $d<(\varepsilon\sigma_1)^2/2$ | Otherwise use Gaussian noise |

**Rules of thumb**

- Calibrate Laplace noise to $\Delta_1 f$ and Gaussian noise to $\Delta_2 f$, never the other way round.
- When $n$ is not public, never use $B/n$ as the sensitivity of a mean.

## 21.9. References


- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 and Appendix A: the Laplace mechanism and the classic Gaussian calibration.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy" (ICML 2018): the exact privacy curve and the bisection calibration.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 22. Advanced and optimal composition

## 22.1. What it is

A dashboard refreshes a hundred counts, and a training run takes thousands of noisy steps, yet you report one differential privacy (DP) guarantee for the whole run. Each step has its own privacy level $\varepsilon$ and slack $\delta$ (The definition of differential privacy). **Basic composition** simply adds them, so a run of $k$ steps costs $k\varepsilon$ and $k\delta$ (Post-processing, group privacy and composition). That is correct but pessimistic. The privacy losses of the steps add up, but each depends on the noise drawn and is often negative. So they partly cancel, and their sum almost never reaches its maximum $k\varepsilon$.

**Advanced composition** (Dwork, Rothblum & Vadhan 2010) uses this. You pick an extra slack $\delta'$ for the whole run. The steps are then together $(\varepsilon_{\text{total}}, k\delta + \delta')$-DP, with

$$\varepsilon_{\text{total}} = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2). \tag{22.1}$$

The first term is the spread of the total loss, which grows only like $\sqrt{k}$. The second is its average, the drift.

**Optimal composition** (Kairouz, Oh & Viswanath 2015) is the exact best bound for steps that are known only by their $(\varepsilon, \delta)$. For example, take a hundred counts at $\varepsilon = 0.1$ each, with no slack, on a dataset of $n = 10{,}000$ people. Basic composition gives a total of 10, still with no slack. The other two rules spend a slack $\delta' = 10^{-5}$, the book's default of $1/n^{1.1}$ rounded down to a power of ten. At that slack, advanced composition gives 5.30 and optimal composition 4.31.

## 22.2. When you meet it

You meet these rules in papers, in privacy reviews and in any library that quotes a total $\varepsilon$ for many steps. They change which number you plan with and which one you report (Table 22.1).

Table 22.1.: Which composition rule to use, at a slack of $10^{-5}$ (none for basic composition).

| What you know about the steps | Rule | Total $\varepsilon$ for 100 steps at 0.1 |
|---|---|---|
| Each step's $\varepsilon$, and $\delta$ must stay 0 | Basic composition | 10 |
| Each step's $(\varepsilon, \delta)$, to plan by hand | Advanced composition | 5.30 |
| Each step's $(\varepsilon, \delta)$, for the number you report | Optimal composition | 4.31 |

| What you know about the steps | Rule | Total $\varepsilon$ for 100 steps at 0.1 |
|---|---|---|
| The mechanism itself, such as Gaussian noise | An accountant (Part 4), which composes whole privacy curves | Often far smaller |
| Budgets chosen during the run | A privacy filter, which enforces a budget fixed in advance | — |

"Optimal" means optimal for steps described by one pair each. A real mechanism has a whole privacy curve, one $\delta$ for every $\varepsilon$, and a single pair forgets the rest of it. Take a thousand Gaussian steps with noise standard deviation 20 on a query of sensitivity 1. Composed as curves, they are exactly $(7.51, 10^{-5})$-DP. Summarizing each step by its best single pair and composing optimally gives 62.7, about eight times worse. So whenever you know the mechanism, compose its curve with an accountant (Privacy accounting).

## 22.3. How to use it

Composition does two jobs. The first is planning, where you split a total budget $\varepsilon_{\text{total}}$ over $k$ steps. Run advanced composition backwards. When $\varepsilon\sqrt{k}$ is small the drift is negligible, so each step can have about

$$\varepsilon \approx \frac{\varepsilon_{\text{total}}}{\sqrt{2k \ln(1/\delta')}}. \tag{22.2}$$

Once $\varepsilon\sqrt{k}$ nears 1, the drift takes over and this shortcut gives each step too much.

The second job is reporting. When you know the mechanism, report the accountant's number at the run's total $\delta$, and otherwise report optimal composition. This snippet plans the hundred counts within a total of $(1, 10^{-5})$, giving the whole slack to $\delta'$. It then reports the thousand Gaussian steps with the privacy loss distribution (PLD) and Rényi differential privacy (RDP) accountants of Google's `dp_accounting`.

```
import math

import dp_accounting

print(f"{1.0 / math.sqrt(2 * 100 * math.log(1e5)):.4f}")  # 0.0208
step = dp_accounting.GaussianDpEvent(noise_multiplier=20.0)
run = dp_accounting.SelfComposedDpEvent(step, 1000)
pld = dp_accounting.pld.PLDAccountant()
pld.compose(run)
print(f"{pld.get_epsilon(target_delta=1e-5):.2f}")  # 7.51
rdp = dp_accounting.rdp.RdpAccountant()
rdp.compose(run)
print(f"{rdp.get_epsilon(target_delta=1e-5):.2f}")  # 8.08
```

Each count gets 0.0208, against basic composition's 0.01. The PLD accountant matches the exact 7.51 quoted above, and the RDP accountant gives a valid but looser 8.08 (Choosing an accountant).

## 22.4. What goes wrong

For a few steps, advanced composition can lose to basic composition, so always take the smaller of the two. Every rule also adds up the steps' slacks, so count each $\delta$. When the steps have different privacy levels, optimal composition is computationally hard (Murtagh & Vadhan 2016), which is one more reason to use an accountant.

All these rules still hold when each query depends on earlier answers, provided each step's budget is fixed in advance. Choosing the budgets from the answers needs a privacy filter (Adaptive composition). Stopping early is different. A stopping rule that reads only the noisy answers is post-processing, so the run costs no more than its planned length. But a run that stops when the true count of some group crosses a threshold reveals that count through its length. Every answer can then be $\varepsilon$-DP while the whole run is not DP for any finite $\varepsilon$.

For a reviewer, a total below optimal composition is a red flag unless an accountant computed it from the mechanism's curve.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 23. Selection: exponential mechanism, report-noisy-max, sparse vector

## 23.1. Why this matters

Many questions ask you to pick something rather than to compute a number. You might want the model that scores best on a validation set, a median, or the first alarm among a thousand statistics.

Adding Laplace noise to every score and publishing them all would charge the budget once per candidate, because the budgets of separate releases add. The mechanisms of this chapter follow one rule instead, which is to **release the position, not the values**. One record can change which candidate wins only a little, however many candidates there are. So under differential privacy (DP), a selection costs as much as a single query, and the number of candidates enters only the accuracy, through its logarithm.

## 23.2. Which mechanism to use

Table 23.1 matches common tasks to the mechanisms explained below. Throughout, the **sensitivity** $\Delta u$ is the most that one person's record can change any candidate's score.

Table 23.1.: Which selection mechanism to use.

| Task | Mechanism | Privacy cost |
|---|---|---|
| Pick the best of a fixed list | Exponential mechanism or report-noisy-max | $\varepsilon$ |
| Pick the top $c$, such as the most frequent items | Gumbel noise, keep the $c$ largest, or noisy max $c$ times | $c\varepsilon$ |
| Pick a number from a range, such as a quantile or a clipping norm | Exponential mechanism over the range | $\varepsilon$ |
| Find the first query in a stream above a threshold | AboveThreshold | $\varepsilon$, however long the stream |

## 23.3. The exponential mechanism

The candidates form a fixed set, written $\mathcal{R}$. Each candidate $r$ has a **score** on the dataset, written $u(D, r)$, such as the number of validation records a model gets right. The **exponential mechanism** of McSherry and Talwar (2007) picks each candidate with probability proportional to an exponential of its score. With a privacy budget $\varepsilon$, it outputs candidate $r$ with probability

$$\Pr[M_E(D) = r] = \frac{\exp\left(\varepsilon\, u(D, r)/(2\Delta u)\right)}{\sum_s \exp\left(\varepsilon\, u(D, s)/(2\Delta u)\right)}. \tag{23.1}$$

It is $\varepsilon$-DP because one record changes each weight, and also their sum, by at most a factor $e^{\varepsilon/2}$, and the two factors together make $e^{\varepsilon}$.

Two conditions decide whether a use is correct. First, fix the list of candidates before you look at the data, because a list built from the data reveals records by itself. A word in a corpus's vocabulary does that when only one person used it. Second, drop the 2 in the exponent only when the scores are **monotone**, meaning that one record pushes all of them the same way. Counts under add/remove neighbours are monotone. A histogram under replace neighbours is not, because moving one record lowers one count and raises another, so dropping the 2 there doubles the real cost.

## 23.4. Accuracy, and the cost of many candidates

The number of candidates $|\mathcal{R}|$ is free for privacy but not for accuracy, because more candidates means more bad ones to pick. Call the best score minus the chosen candidate's score the **gap**. On average it is at most

$$\frac{2\Delta u}{\varepsilon}(\ln|\mathcal{R}| + 1), \tag{23.2}$$

so doubling the candidates adds only $\ln 2$ to $\ln|\mathcal{R}| + 1$.

For example, suppose you trained 50 models and want to release the best one, judged on a private validation set of 1,000 records. Model $r$, numbered from 1, gets $900 - r$ records right, so model 1 is the best. The score is that count, so $\Delta u = 1$. At $\varepsilon = 1$ the mechanism picks the best model only 39% of the time. Yet it loses just 1.54 correct predictions on average, or 0.15% accuracy, because its mistakes are mostly near-ties. Hyperparameter tuning picks models this way.

## 23.5. A private quantile or clipping norm

Reading a median or a clipping norm off the data leaks it, so pick it with the exponential mechanism. First fix a range before looking at the data, and clip every record into it. The mechanism then scores each point of the range by how close the share of records below it is to the quantile you want, such as one half for the median. One added or removed record moves any score by at most 1. The score is the same between consecutive sorted records. So the mechanism picks one of these gaps, called **pieces**, favouring long pieces near the quantile, and returns a uniform point inside it.

Libraries do this. diffprivlib's `tools.quantile` and `tools.median` take an `epsilon` and the range as `bounds`. OpenDP's `.dp.quantile` and `.dp.median`, which work on Polars data frames, pick one value from a finite list of candidates instead. The script practitioner_adult_statistics.py takes the median age of the Adult census data at $\varepsilon = 1$. It holds 48,842 people with one row each, and the range of 17 to 90 comes from the survey's documentation.

```
age = adult["age"].to_numpy()
private = tools.median(age, epsilon=1.0, bounds=(17, 90), random_state=seed)
print(f"true median age: {np.median(age):.1f}  diffprivlib median: {private:.2f}")
```

With `random_state=0` it prints a true median of 37.0 and a private one of 37.72. No age lies strictly between 37 and 38, so every point there has the same score, and the mechanism picked that piece and a point in it at random. OpenDP refuses to run the median without a public upper bound on the number of rows, declared as a `Margin`. Choose it generously from public facts: it must be at least the true number of rows, and a larger one leaves counts and quantiles unchanged and adds only a small rounding allowance to floating-point sums.

```
context = dp.Context.compositor(
    data=adult.lazy(),
    privacy_unit=dp.unit_of(contributions=1),  # one row per person
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(max_length=100_000)],  # a public bound on the number of rows
)
age_col: Any = pl.col("age")  # OpenDP adds .dp to Polars expressions when it is imported
query = context.query().select(age_col.dp.median(candidates=list(range(17, 91))))
print(query.summarize(alpha=0.05))
released = query.release().collect()  # changes on every run: OpenDP's noise has no seed
```

Its summary reports `ExponentialMin` with a scale of 2.0. Under pure DP, OpenDP 0.16 adds exponential noise to every candidate's score and reports the best, which samples **permute-and-flip** (McKenna & Sheldon 2020). That selection mechanism has the exponential mechanism's $\varepsilon$ guarantee, and its expected error is never worse. The accuracy is `null`, because OpenDP gives no error bar for a quantile. Its noise has no seed, so the released value changes from run to run, and `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py` prints one at the end.

The error bar you can state is about rank, not value, because the guarantee bounds how unevenly the output splits the records. With probability $1 - \beta$, the exponential mechanism's gap is at most $(2\Delta u/\varepsilon)(\ln|\mathcal{R}| + \ln(1/\beta))$, the tail form of Equation 23.2.

For a median over $|\mathcal{R}|$ candidates, with $\Delta u = 1$, the gap is the number of records by which the output misses the middle rank, so $\beta = 0.05$ gives its 95% bar. A bar in value would need the number of records near the median, which the release does not reveal. So state the bar in ranks, or spend budget on a few noisy counts below candidate values if a dashboard needs values. The width of the range enters that bound only through its logarithm, so a generous range is cheap. Private training often clips gradients at the median of their norms. For 1,000 norms whose median

is 0.9804, the private median at $\varepsilon = 1$ lands on average 0.0013 from it. Widening the range from 10 to 1,000 leaves that unchanged.

Two things go wrong in practice. Where the data near the quantile are sparse, the best piece is long and the output can land anywhere in it. A small error in rank is then a large error in value. The second is that the range or the candidates must not come from the data. Setting the upper end to the observed maximum reintroduces the leak, and diffprivlib warns with a `PrivacyLeakWarning` when you leave out `bounds` (Sensitivity).

## 23.6. Report-noisy-max and Gumbel noise

**Report-noisy-max** adds independent Laplace noise to each candidate's score, and outputs the candidate with the largest noisy score. It is $\varepsilon$-DP for any number of candidates when the noise has scale $2\Delta u/\varepsilon$, and with monotone scores a scale of $\Delta u/\varepsilon$ is enough. Release only the winner's index, and pay for a fresh noisy answer if you need its score.

On the 50 models at $\varepsilon = 1$, it picks the best with probability 0.4241, against 0.3935 for the exponential mechanism. Neither is always better, since with two candidates the exponential mechanism wins.

Noise from the Gumbel distribution, of scale $2\Delta u/\varepsilon$, in place of the Laplace noise gives exactly the exponential mechanism. This **Gumbel-max trick** is how libraries sample it, because the exponential of a score in the thousands overflows, while noisy scores do not. Keeping the $c$ largest Gumbel-noised scores picks the top $c$ at once (Durfee & Rogers 2019).

## 23.7. The sparse vector technique

Sometimes queries arrive one at a time, and you want the first one that is high enough. The **sparse vector technique** (Dwork et al. 2009) compares each with one noisy threshold and pays only for the one that fires. Its basic form, **AboveThreshold**, takes a threshold $\tau$ and a stream of queries. Every query $f$ in the stream has the same sensitivity $\Delta f$. It splits the budget into $\varepsilon_1$ for the threshold and $\varepsilon_2$ for the queries.

1. Draw one threshold noise, written $\nu$, from $\mathrm{Lap}(\Delta f/\varepsilon_1)$ for the whole run.
2. For each query, draw a fresh noise from $\mathrm{Lap}(2\Delta f/\varepsilon_2)$ and add it to the query's answer. If the result is at least $\tau + \nu$, output "above" and halt. Otherwise output "below" and go on.

AboveThreshold is $(\varepsilon_1 + \varepsilon_2)$-DP for any number of "below" answers, even when each query is chosen after the earlier answers. All the "below" answers together pay $\varepsilon_1$, through the one threshold noise, and the single "above" pays $\varepsilon_2$.

The accuracy does depend on the stream length $k$, which must be fixed before the run for the accuracy guarantee to hold. Write $\beta$ for the chance that this guarantee fails. With an equal split, $\varepsilon_1 = \varepsilon_2 = \varepsilon/2$, every query is classified correctly unless its true value is within a margin

$$\alpha = \frac{8\Delta f(\ln k + \ln(2/\beta))}{\varepsilon} \tag{23.3}$$

of the threshold. For example, monitor 1,000 counts, each with $\Delta f = 1$, against a threshold of 500 at $\varepsilon = 1$. With $\beta = 0.05$ the margin is 84.77. So with probability 0.95, only a count within that margin of the threshold can be misclassified.

To find the first $c$ queries above the threshold, restart AboveThreshold with fresh threshold noise after each "above", at $\varepsilon/c$ per run, and halt after the $c$-th. The margin then grows with $c$, so ask for as few alarms as you can. Lyu, Su and Li instead keep one threshold for all $c$ alarms and multiply the query noise by $c$.

## 23.8. When the sparse vector technique goes wrong

Several published variants do not have the privacy they claim (Table 23.2).

Table 23.2.: Variants of the sparse vector technique and their actual privacy, from Lyu, Su & Li (2017).

| Variant (as numbered by Lyu, Su & Li) | What it changes | Actual privacy |
|---|---|---|
| Algorithm 3 | Outputs the noisy value of each "above" | $\infty$ |
| Algorithm 4 | Query noise does not grow with $c$ | $\frac{1+6c}{4}\varepsilon$ |
| Algorithm 5 | No noise on the queries | $\infty$ |
| Algorithm 6 | Never halts | $\infty$ |

The no-halt variant is the one to remember. It adds exactly as much noise per query as AboveThreshold, so a reviewer who checks noise scales will miss it. Yet at $\varepsilon = 1$ its privacy loss reaches 18.1246 on 80 queries.

So review the accounting, not just the noise. An implementation must draw fresh noise for every query and one threshold noise per run. A threshold redrawn for each query makes every "below" answer cost budget that nobody counted. It must also halt after the allowed number of "above" answers and never publish the noisy values it compared. Reusing one query-noise draw for all queries quietly turns it into Algorithm 5.

## 23.9. Cheat sheet

Table 23.3.: The selection mechanisms of this chapter, with $|\mathcal{R}|$ candidates or $k$ queries.

| Mechanism | Privacy | Accuracy |
|---|---|---|
| Exponential mechanism, Equation 23.1 | $\varepsilon$; drop the 2 only for monotone scores | Mean gap $\leq \frac{2\Delta u}{\varepsilon}(\ln \lvert\mathcal{R}\rvert + 1)$ |

| Mechanism | Privacy | Accuracy |
|---|---|---|
| Report-noisy-max, noise $\mathrm{Lap}(2\Delta u/\varepsilon)$ | $\varepsilon$, index only | Similar to the exponential mechanism |
| AboveThreshold | $\varepsilon_1 + \varepsilon_2$ | Margin $8\Delta f(\ln k + \ln \frac{2}{\beta})/\varepsilon$, failure probability $\beta$ |

**Rules of thumb**

- Release positions, not values.
- Fix the candidate list, and the range of any number you pick, before you look at the data.

## 23.10. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3, §3.4 and §3.6: the three mechanisms in their standard form.
- Lyu, Su & Li, "Understanding the Sparse Vector Technique for Differential Privacy" (VLDB 2017): the budget split and the survey of broken variants.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 24. Query release and private multiplicative weights

## 24.1. Why this matters

Analysts rarely want one statistic. They ask for every two-way table over a handful of columns, or for a synthetic file they can query as often as they like. Both must satisfy differential privacy (DP). Such tables are **marginal tables**, and they are the most common batch. Each one counts the records for every combination of values of a few attributes, and each count is a **cell**.

The idea that makes these releases work is to add noise to whole tables rather than to single cells, scaled to how much one person can change the table. One person sits in one cell of a table, so a whole table costs about as much as one count. Everything computed from the noisy tables afterwards is free, including repairs that make the cells of a table add up to its total. When the queries are too many for noise on every cell, synthetic data fitted to a few noisy tables takes over.

## 24.2. Which method to use

The choice depends on how many queries you have, compared with the number of records, and on how many attributes a record has (Table 24.1).

Table 24.1.: Which query-release method to use.

| Situation | Use |
|---|---|
| A few marginal tables, with a public list of cells | Noisy counts on every cell, one budget share per table |
| Hundreds of tables or thousands of cells | Gaussian noise on every cell, scaled to the whole set of tables |
| A table whose list of values is not public | Noisy counts, keeping only cells that clear a noisy threshold (Release statistics with a library) |
| Far more queries than records, or queries not known in advance | Synthetic data from AIM or MST |
| Queries that arrive one at a time, perhaps adaptively | Online private multiplicative weights |

Where "a few" tables end is the dense-query rule of the Laplace and Gaussian mechanisms. One person sits in every table, so at a privacy level $\varepsilon$ of 1 and a slack $\delta$ of $10^{-5}$, Gaussian noise wins from 7 tables on.

## 24.3. Noise on every cell

Under add/remove neighbours, a person with one record changes one cell of a marginal table by 1, so the whole table has sensitivity 1, whatever its number of cells. Laplace noise of scale $1/\varepsilon$ on every cell then makes the table $\varepsilon$-DP, by the parallel composition of Post-processing, group privacy and composition. Under replace neighbours a person can move from one cell to another, so two cells change and the sensitivity doubles.

Several tables of the same people compose, because one person sits in one cell of every table. With Laplace noise the budget is split among the tables, so the error per cell grows in proportion to their number. Gaussian noise is calibrated to the square root of the sum of squared changes. For one person in a number $m$ of tables that is $\sqrt{m}$, so its error grows only like the square root of the number of tables.

An example from the full edition shows the gap. A table of 10,000 records has eight yes/no attributes. The release is every cell of the marginal table of every set of attributes, 6,560 cells in all, each as a fraction of the records, under replace neighbours. A uniform guess, which ignores the data, has a largest error of 0.232 over these cells. At $\varepsilon = 1$, Laplace noise on every cell has a largest error of 0.497, which is worse than ignoring the data. Gaussian noise does much better, with 0.037, at a slack $\delta$ of $10^{-6}$, stricter than the book's default of $10^{-5}$ for 10,000 records.

Two mistakes are common. Scaling each cell's noise to its own sensitivity, as if it were the only query, silently spends $\varepsilon$ once for every table, since each person sits in one cell of each. And splitting $\varepsilon$ evenly over thousands of separately noised cells with basic composition wastes most of the budget. Calibrate to the whole set of tables instead.

## 24.4. Releasing a marginal with OpenDP

The script practitioner_adult_statistics.py releases the education-by-income table of the Adult census data, 48,842 people with one row each, at $\varepsilon = 1$. The cells come from the survey's documentation, which lists 16 levels of education and two income classes. So the keys are public, and the table has 32 cells, including any that happen to be empty, since omitting one would reveal that nobody has those values. A missing value needs a cell too: map it to a listed key such as "Unknown", because a row whose key is not in the list is dropped. The `Margin` (an OpenDP setting, not a marginal table) with `invariant="keys"` tells OpenDP that the list of cells is public, and `with_keys` pins the release to exactly those cells.

```
keys = pl.LazyFrame({"education": EDUCATION}).join(pl.LazyFrame({"income": INCOME}), how="cr
context = dp.Context.compositor(
    data=adult.lazy(),
    privacy_unit=dp.unit_of(contributions=1),
    privacy_loss=dp.loss_of(epsilon=1.0),
    split_evenly_over=1,
    margins=[dp.polars.Margin(by=["education", "income"], invariant="keys")],  # the cells a
)
query = context.query().group_by("education", "income").agg(dp.len(signed=True)).with_keys(k
```

```
print(query.summarize(alpha=0.05))
released = query.release().collect()  # all 32 cells, zeros included; changes on every run
```

Run it with `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`. The summary is deterministic.

```
shape: (1, 5)
┌────────┬──────────────┬──────────────────┬───────┬──────────┐
│ column ┆ aggregate    ┆ distribution     ┆ scale ┆ accuracy │
│ ---    ┆ ---          ┆ ---              ┆ ---   ┆ ---      │
│ str    ┆ str          ┆ str              ┆ f64   ┆ f64      │
╞════════╪══════════════╪══════════════════╪═══════╪══════════╡
│ len    ┆ Frame Length ┆ Integer Laplace  ┆ 1.0   ┆ 3.375618 │
└────────┴──────────────┴──────────────────┴───────┴──────────┘
```

Each cell gets integer Laplace noise of scale 1 (Discrete and floating-point-safe mechanisms). The 95% accuracy of 3.38 means each noisy count lands that close to the truth 95% of the time. Print that error bar beside every cell, and remember that it covers the noise only, not sampling error or mistakes in the data. OpenDP's noise has no seed, so the released counts change on every run, and these four cells come from one run, beside the true counts.

```
  Doctorate >50K: 432  true 431  accuracy 3.38
  HS-grad <=50K: 13281  true 13281  accuracy 3.38
  Preschool <=50K: 82  true 82  accuracy 3.38
  Preschool >50K: 0  true 1  accuracy 3.38
```

The large cells barely move, but the smallest true cell, one person with a preschool education and a high income, is smaller than the noise. If small cells matter, coarsen the table by merging rare categories in advance, because noise of a fixed size is large only next to small counts. With `signed=True` a count may come out negative, which keeps it unbiased. Clamping it at zero, rounding, and scaling the cells so that each table adds up to a total already released are all post-processing, so they cost nothing.

Three things go wrong. Taking the list of cells from the data, as a `unique()` on the private column does, reveals rare values by itself. When no codebook exists, use partition selection instead (Release statistics with a library). A second table of the same people needs its own share of the budget, so set `split_evenly_over` to the number of tables. And for many tables, give the budget as `dp.loss_of(rho=...)`, with the $\rho$ that converts to your $(\varepsilon, \delta)$, which Release statistics with a library finds with OpenDP's own conversion. OpenDP then accounts in zero-concentrated differential privacy (zCDP) and uses Gaussian noise, which composes far better there than Laplace noise (Concentrated differential privacy).

## 24.5. Synthetic data

When the queries far outnumber what noise on every cell can bear, or nobody knows the queries in advance, release synthetic data instead. The practical tools follow four steps.

1. **Select** a set of marginal tables to measure, from domain knowledge or privately with the exponential mechanism, scoring each table by how badly the current model fits it.
2. **Measure** each whole table with Gaussian noise, scaled to the whole set.
3. **Generate** a compact graphical model that fits the noisy tables.
4. **Sample** synthetic rows from the model.

Everything after the measurements is post-processing, so the synthetic rows can be queried and shared at no further privacy cost. MST and AIM, both by McKenna and coauthors, are the standard methods. AIM, an adaptive and iterative mechanism, chooses which tables to measure as it goes. SmartNoise Synth, the `snsynth` package, wraps both behind one interface. It is not installed in this book's environment, so nothing on this page produces synthetic data, and the call below was not run. To use it, install it in an environment of its own.

```
# smartnoise-synth 1.0.8: not installed in this book's environment, not run
from snsynth import Synthesizer

synth = Synthesizer.create("aim", epsilon=1.0, delta=1e-6)
synth.fit(adult_pandas, categorical_columns=["education", "sex", "race", "occupation", "inco
rows = synth.sample(48_842)
```

A synthetic table is accurate on the tables it measured, by construction, so evaluate it on other queries, but not freely against the private data. Keeping the best of several runs, judged against the private data, is a selection on that data, and each extra run of the generator spends the generator's budget again. So score candidate settings on a public dataset of the same shape, or charge the comparisons to the budget. A generator trained without DP is not anonymised, however synthetic its rows look.

## 24.6. Private multiplicative weights

The **multiplicative weights exponential mechanism (MWEM)** of Hardt, Ligett and McSherry (2012) is the practical form of private multiplicative weights, and these tools keep its select, measure and update loop. It keeps a guessed histogram with one weight for every possible record, 256 in the example above, starting from uniform, and runs a fixed number of rounds. Each round selects a query that the guess answers badly, with the exponential mechanism. It then measures that query with fresh Laplace noise, and moves the guess toward the measurement with one multiplicative-weights step. An online-learning bound caps the number of corrections that can ever be needed, whatever the number of queries. So the budget pays for corrections rather than for queries, and the error grows only with the logarithm of the number of queries.

That pays off only for very large sets of queries. On the same example at $\varepsilon = 1$, with 38 rounds chosen as the next paragraph says, its largest error is 0.078. That beats Laplace noise but loses to Gaussian noise by a factor of about two. For arbitrary queries, unlike marginals, over $2^{20}$ possible records, its bound on the records needed beats Gaussian noise only near $10^9$ queries. Its running time also grows with the number of possible records, which limits it to a few attributes.

Three rules make an implementation correct. Fix the number of rounds before seeing the data, by minimizing the error bound, which uses only public quantities. Measure with fresh noise, because

the selection step's scores are exact and never released, so reusing one would use the data without paying for it. And release the average of the guesses, not the last one, because the accuracy guarantee covers only the average.

## 24.7. Online release and SmallDB

A query server that receives queries one at a time, perhaps chosen after earlier answers, can run the online form (Hardt & Rothblum 2010). It answers from the guess while the sparse vector technique finds the guess's error below a threshold, and pays only for updates. It is mainly a theoretical guarantee, used in Adaptive data analysis. SmallDB (Blum, Ligett & Roth 2008) picks a whole small dataset with one exponential mechanism. On the same example it would weigh about $10^{4568}$ candidate datasets, so it shows what is possible rather than what you can run.

## 24.8. Cheat sheet

Table 24.2.: The query-release methods of this chapter.

| Method | Privacy | Error grows with |
|---|---|---|
| Laplace noise on every cell | $\varepsilon$; one table costs $\varepsilon$ under add/remove | The number of tables |
| Gaussian noise on every cell | $(\varepsilon, \delta)$ or zCDP | The square root of the number of tables |
| Private multiplicative weights | $\varepsilon$ or $(\varepsilon, \delta)$ | The logarithms of the numbers of queries and possible records |
| Select, measure, generate (MST, AIM) | $(\varepsilon, \delta)$ | No closed form; the practical choice for many attributes |

**Rules of thumb**

- Take the list of cells from a codebook, never from the private data.
- Add noise to whole tables, scaled to the set of tables, and post-process freely afterwards.
- Fit synthetic data once, and never pick the best of several runs on the private data.

## 24.9. References


- McKenna, Mullins, Sheldon & Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (VLDB 2022): the select, measure and generate loop as used in practice.
- Hardt, Ligett & McSherry, "A Simple and Practical Algorithm for Differentially Private Data Release" (NeurIPS 2012): MWEM, with experiments on real tables.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 25. Beyond global sensitivity

## 25.1. What it is

The Laplace and Gaussian mechanisms set their noise by the **global sensitivity**, the most one record can move the answer over every pair of neighbouring datasets that could exist. The median of ages between 0 and 100 has global sensitivity 100. So under differential privacy (DP) at a privacy level of $\varepsilon = 1$, the Laplace mechanism's median error is 69.31 years. Yet on a simulated sample of 1,001 ages, one person moves the median by only 0.1386 years. That smaller number is the **local sensitivity**, the most one record can move the answer starting from the dataset $D$ you hold:

$$\mathrm{LS}_f(D) = \max_{D' \text{ a neighbour of } D} |f(D) - f(D')|, \tag{25.1}$$

where $f$ is the query.

The techniques of this page add noise on the scale of the data you hold, under one rule. **A noise scale, bound or threshold computed from the data is itself a statistic of the data, so it must be private too.** Throughout, datasets have a fixed size $n$, and neighbours replace one record.

## 25.2. When you meet it

You meet them whenever a statistic has no useful worst case (Table 25.1).

Table 25.1.: Which technique releases what, and where you meet it.

| You release | Technique | Where you meet it |
|---|---|---|
| A sum, mean or gradient of unbounded values | Clipping to a bound fixed in advance | Every library's `bounds`, and the clipping norm of private training |
| A median or quantile | Inverse sensitivity | `diffprivlib.tools.quantile`, OpenDP's `dp.quantile` |
| A discrete answer with a clear winner | Stability-based release | The threshold of private partition selection |
| A black-box model or estimator | Subsample-and-aggregate | Teacher ensembles for private prediction |

A **stability-based release** publishes a discrete answer, such as the most common value, exactly. It does so only when a noisy test shows that the answer leads the runner-up by about $2\ln(1/\delta)/\varepsilon$

records. Otherwise it refuses, so plan for refusals downstream. Libraries use the same test to decide which keys found in the data a group-by may release (Release statistics with a library). **Subsample-and-aggregate** runs any estimator on disjoint blocks fixed in advance, clips each block's answer and averages them with noise. Private aggregation of teacher ensembles (PATE) is its voting form (Private prediction).

**Propose-test-release** proposes a bound on the local sensitivity, checks with noise that the data are many records away from breaking it, and refuses otherwise. **Smooth sensitivity** scales heavy-tailed noise to a bound that cannot jump between neighbours. PATE uses it to publish the $\varepsilon$ it computes from its votes. On the 1,001 ages it is about twelve times less accurate than inverse sensitivity.

## 25.3. How to use it

Most of the time you bound the inputs. **Clipping** replaces each value $x$ by $\min(x, C)$, for a clip bound $C$ that you fix before seeing the data. For values that cannot be negative, one record then moves the sum by at most $C$ and the mean by at most $C/n$. So Laplace noise of scale $C/(n\varepsilon)$ on the clipped mean gives $\varepsilon$-DP. Raising $C$ shrinks the bias but grows the noise.

Choose $C$ in this order. Take it from domain knowledge, such as a legal maximum or a sensor range. Otherwise, assume a distribution for the data in advance and take the $C$ that minimizes bias plus noise under it. Otherwise, spend a small slice of the budget on a private high quantile and clip there, charging that slice by composition. Libraries take the bound as an argument. If you omit it, diffprivlib computes bounds from the data and issues a `PrivacyLeakWarning`, so turn that warning into an error with `warnings.simplefilter("error", PrivacyLeakWarning)`, as practitioner_adult_statistics.py does. Bounding the rows each person contributes is the same move for a whole person (Release statistics with a library).

For a median or quantile, use **inverse sensitivity**. It favours each candidate output by how few records must change before the median equals it. That is the exponential mechanism, and it gives pure $\varepsilon$-DP. Pass it bounds or a candidate grid fixed in advance. On the 1,001 ages at $\varepsilon = 1$, its median error is 0.0783 years. That is below even the local sensitivity.

## 25.4. What goes wrong

Noise scaled to the local sensitivity is not private. The five ages $(0, 0, 0, 0, 100)$ have median 0, and no single change moves it, so they get no noise. In their neighbour $(0, 0, 0, 100, 100)$, one change moves the median to 100, so it gets continuous noise and its output is never exactly 0. An output of exactly 0 therefore names the first dataset. The same leak hides in a clip bound set to the observed maximum, a candidate grid built from the data's values, or keys read off the data. In review, look for a `max()`, `quantile()` or `unique()` on the private data that feeds a parameter.

Clipping from above always biases a mean downward, and a two-sided clip cancels the bias only when the range is centred on the data (Private mean estimation). If neighbours add or remove a record, $n$ is not public, so release a noisy clipped sum and a noisy count and divide them.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 26. Discrete and floating-point-safe mechanisms

## 26.1. What it is

The proof of the Laplace mechanism is about densities on the real line, but a computer samples floating-point numbers, a finite set with uneven gaps, which the proof says nothing about. The textbook sampler adds $\pm b \ln(1/U)$ to the true answer, for a noise scale $b$ and a uniform draw $U$. Mironov (2012) showed that in double precision it leaks the true answer through the last bits of its output. Floats are dense near 0 and sparse far from it, so many outputs can come from a true answer of 0 or of 1, but not from both.

Such an output names its input with certainty. Differential privacy (DP) caps how much one record can change the chance of any output, through a privacy level $\varepsilon$ and a slack $\delta$ (The definition of differential privacy). Now take a set $S$ of outputs that a dataset $D$ can reach but its neighbour never can. No factor $e^{\varepsilon}$ can lift the neighbour's probability of $S$ above 0, so for a mechanism $M$ the slack must cover

$$\delta \geq \Pr[M(D) \in S]. \tag{26.1}$$

In a toy sampler with 12-bit floats, such outputs carry probability 0.4791, and more bits do not shrink it.

So **DP is a property of the distribution the code actually produces.** There are two repairs. The **snapping mechanism** keeps floats, but clamps the answer, adds noise, rounds to a coarse power-of-two grid and clamps again. The other makes the query integer-valued and adds exact discrete Laplace or discrete Gaussian noise.

## 26.2. When you meet it

You meet this when you review code that adds noise itself, such as `x + np.random.laplace(0, b)`, unsafe to ship, and when you choose a library. OpenDP's samplers are exact, and on integer data they give discrete Laplace and discrete Gaussian noise. diffprivlib offers `mechanisms.Geometric`, its name for the discrete Laplace, `mechanisms.GaussianDiscrete` and `mechanisms.Snapping`. Its continuous `Laplace`, which its `tools` functions use, draws from four uniform variates to resist this attack (Holohan & Braghin 2021), a mitigation rather than an exact sampler. Secure aggregation needs integer noise too (Local, shuffle and distributed trust models).

In private training, Opacus adds continuous Gaussian noise in floating point, so the proof of Private stochastic gradient descent covers idealized code. Its `secure_mode=True` draws the noise from a cryptographic generator to blunt these attacks, so check that a released model used it and states its guarantee for the idealized mechanism.

## 26.3. How to use it

Use a library on integer data. Its **discrete Laplace** noise gives each integer $z$ a probability proportional to $e^{-|z|/b}$, and you calibrate it like continuous noise. For an integer query that moves by at most $\Delta f$ between neighbours, it gives pure $\varepsilon$-DP with $\varepsilon = \Delta f/b$. For example, at $b = 1$ a count comes out exact with probability 0.4621.

The **discrete Gaussian** gives each integer $x$ a probability proportional to $e^{-x^2/(2\sigma^2)}$, for a width $\sigma$. Its exact privacy curve differs slightly: for a sensitivity of 1 at $\sigma = 1$, its $\delta$ at $\varepsilon = 1$ is 0.1414, where the continuous formula gives 0.1269. So either report the discrete curve, or account in zero-concentrated DP. There the discrete Gaussian has the continuous Gaussian's privacy parameter $\rho = (\Delta f)^2/(2\sigma^2)$, and OpenDP accounts this way. Here `map(1)` gives the guarantee at sensitivity 1.

```
import opendp.prelude as dp

dp.enable_features("contrib")
integers, distance = dp.atom_domain(T=int), dp.absolute_distance(T=int)
laplace = dp.m.make_laplace(integers, distance, scale=1.0)
print(laplace.map(1))  # 1.0
gauss = dp.m.make_gaussian(integers, distance, scale=2.0)
print(gauss.map(1))  # 0.125
print(gauss.output_measure)  # ZeroConcentratedDivergence
```

A real-valued query, such as a sum of clipped values, becomes integer in three steps. Divide by a step much smaller than the noise and round randomly, up with probability equal to the fractional part, so the result stays unbiased. Recompute the sensitivity in those units, since rounding can push neighbours slightly apart. Then add discrete noise and multiply back.

## 26.4. What goes wrong

Ask these questions of any implementation.

- Is every sensitivity computed from bounds fixed in advance? A `max()`, `quantile()` or `unique()` on the private data that feeds a bound, a scale or a key list leaks (Beyond global sensitivity).
- Under add/remove, is a mean a noisy sum over a noisy count, not over the true $n$ (The Laplace and Gaussian mechanisms)?
- Do all releases about the same people draw on one budget, so that five queries at $\varepsilon = 1$ are not claimed as $\varepsilon = 1$ (Post-processing, group privacy and composition)?
- Is an exact statistic of the same data, such as a maximum, published beside the private ones? It has no guarantee.
- Is the noise drawn by a vetted library, or by an exact sampler with no floating-point `exp`, `log` or `random()` inside? Search for `np.random`, `random.`, `scipy.stats`, `default_rng`, `.laplace(` and `.normal(`.
- Does integer data get integer noise, and is every floating-point output snapped and clamped to a fixed range?

- Does each release draw fresh noise from a cryptographic source, such as `secrets.SystemRandom()`? A fixed seed makes the noise reproducible, and so removable.
- Is the running time fixed or padded? Rejection samplers' rounds depend on the value drawn, which Jin et al. (2022) exploit.
- Does the stated guarantee match the mechanism, including the discrete Gaussian's own curve and the sensitivity after rounding?

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 27. Local, shuffle and distributed trust models

## 27.1. Why this matters

Every mechanism so far in this part assumed a **trusted curator**, which holds the raw data and releases only the mechanism's output. In this **central model**, the differential privacy (DP) guarantee protects what the curator publishes, but not against a breach of its database, a curious engineer or a court order. A browser vendor that collects usage statistics from its users is often not trusted that far.

The other three models trust less. In the **local model** nobody is trusted, and each person randomizes their own record before it leaves their device. The **shuffle model** trusts a shuffler only to cut the link between each report and its sender. The **distributed model** trusts cryptography that reveals only a sum, run by enough honest participants. The closer to each person the noise is added, the fewer parties you trust, and the more noise reaches the answer.

## 27.2. Choosing a trust model

Each model has a price. As the population grows, the noise on a private count stays fixed in the central model but grows with its square root in the local model. On the full edition's eight-value histogram over a hundred thousand people, the local model's squared error is about 42,000 times the central one at the same guarantee. The shuffle and distributed models close most of that gap.

So use the central model whenever a curator can be trusted with the raw data, and Release statistics with a library does that job end to end. Choose local only when nobody may see a raw record and the population is large enough, which you check by solving the standard deviation of Section 27.3 for the number of people. Either way, state the trust model next to the privacy parameters, because the same $\varepsilon$ protects against a different adversary in each model.

## 27.3. Central and local differential privacy

In the local model, each person sends only a **report**, the output of a **local randomizer** $R$ run on their own record. Write $\varepsilon_0$ for the privacy level of one report. The randomizer is $\varepsilon_0$-DP when no set of possible reports becomes more than $e^{\varepsilon_0}$ times as likely when the record changes. In symbols, every set of reports $S$ and every two records $x$ and $x'$ satisfy

$$\Pr[R(x) \in S] \le e^{\varepsilon_0} \Pr[R(x') \in S]. \tag{27.1}$$

Changing one person's record changes only their own report, so the whole collection is then $\varepsilon_0$-DP in the central sense, with neighbours that differ in one replaced record. The guarantee covers only the randomized value, so a network address stored with a report links it back to the person.

The basic local randomizer is randomized response on one bit. Each person reports their true bit with a probability, written $p$, of $e^{\varepsilon_0}/(1+e^{\varepsilon_0})$, and the flipped bit otherwise. The analyst undoes the expected flips by subtracting $1-p$ from the share of reported ones and dividing by $2p-1$. Every report has the same variance whatever its true bit, so the estimate's standard deviation depends only on the level and the number of people $n$:

$$\mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon_0/2}}{(e^{\varepsilon_0}-1)\sqrt{n}}. \tag{27.2}$$

The script practitioner_adult_statistics.py runs both models at $\varepsilon_0 = \ln 3$ on the 48,842 people of the Adult census data, where each bit says whether a person's income is over $50K. In the local model, each bit goes through diffprivlib's `Binary` mechanism, and the analyst sees only the reports.

```
answers = (adult["income"] == ">50K").cast(pl.Int8).cast(pl.String).to_list()  # "1" or "0"
n, eps = len(answers), log(3)
coin = Binary(epsilon=eps, value0="0", value1="1", random_state=seed)
reports = [coin.randomise(a) for a in answers]  # each person randomizes their own answer
p = exp(eps) / (1 + exp(eps))  # chance a report is truthful: 0.75
reported = reports.count("1") / n
local = (reported - (1 - p)) / (2 * p - 1)  # undo the expected flips
local_sd = math.sqrt(p * (1 - p) / n) / (2 * p - 1)  # the randomization noise alone
print(f"true share over 50K: {answers.count('1') / n:.4f}")
print(f"local estimate: {local:.4f}  sd {local_sd:.4f}")
```

In the central model, a trusted curator counts the true bits with diffprivlib's `tools.count_nonzero` at the same level, which adds Laplace noise with standard deviation $\sqrt{2}/\varepsilon$ to the count. The standard deviation comes from the variance that diffprivlib's own `Laplace` mechanism reports.

```
rich = (adult["income"] == ">50K").to_numpy()
central = tools.count_nonzero(rich, epsilon=log(3), random_state=seed) / len(rich)
central_sd = math.sqrt(Laplace(epsilon=log(3), sensitivity=1).variance(0)) / len(rich)  # th
print(f"central estimate: {central:.4f}  sd {central_sd:.6f}")
```

Its seed-0 run, `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`, prints this.

```
true share over 50K: 0.2393
local estimate: 0.2359  sd 0.0039
central estimate: 0.2393  sd 0.000026
local sd / central sd: 149
```

So at the same guarantee, the local estimate is 149 times noisier, and the factor grows like the square root of the population. No local protocol can estimate a fraction with an error below order $1/((e^{\varepsilon_0}-1)\sqrt{n})$, and randomized response reaches that order, so the factor belongs to the model. That is why Apple, Google and Microsoft use local DP only for the most frequent items in very large populations.

## 27.4. Frequency estimation: $k$-ary randomized response and unary encoding

The workhorse of deployed local DP is **frequency estimation**. Each person holds one value from a known list, such as an emoji, and the analyst wants the share holding each one. A report votes for a value with some probability $p_1$ if the person holds it, and with a smaller probability $p_0$ otherwise. Write $c_v$ for the number of the $n$ reports that vote for a value $v$. Removing the false votes on average gives the unbiased estimate

$$\hat{f}_v = \frac{c_v/n - p_0}{p_1 - p_0}. \tag{27.3}$$

Two designs set $p_1$ and $p_0$ for a list of $k$ values. In **$k$-ary randomized response**, each person reports their true value with probability $e^{\varepsilon_0}/(e^{\varepsilon_0}+k-1)$, and another value otherwise. **Optimized unary encoding** sends one bit per value, and it keeps the 1 of the value held only half the time but almost never turns a 0 into a 1. Most values of a long list are rare, and a rare value's variance sets the smallest share that can be seen. Comparing those variances, use $k$-ary randomized response when $k < 3e^{\varepsilon_0} + 2$, and unary encoding otherwise, at the price of a $k$-bit report. The pure-ldp Python package implements both.

For example, a keyboard app wants the share of each of 128 emoji from a million users at $\varepsilon_0 = 1$. A rare emoji's estimate then has a standard deviation of 0.0019 under unary encoding, against 0.0066 under $k$-ary randomized response. By convention, an estimate must stand three standard deviations above zero to be told from noise. So even the better design loses every emoji used by fewer than about 0.6% of users, and you should size the population from that floor before collecting.

## 27.5. Large and unknown domains: hashing and heavy hitters

When $k$ bits per report are too many, **local hashing** hashes each value into one of about $e^{\varepsilon_0} + 1$ buckets and randomizes the bucket. That matches unary encoding's variance with a few bits. When the analyst cannot even list the candidates, **heavy hitters** protocols build them up letter by letter, keeping only frequent prefixes (Bassily et al. 2017).

## 27.6. Numbers in one bit: local mean estimation

Telemetry also collects numbers, such as minutes in an app, each between 0 and a known bound. One bit per person is enough (Ding, Kulkarni & Yekhanin 2017). The chance of sending a 1 rises in a straight line with the value, so the analyst can invert that line to get an unbiased mean. Take

a million users and a bound of 1,440 minutes. At $\varepsilon_0 = 1$, the mean has a standard deviation of 1.420 minutes. Fix the bound before collection and clip to it on the device, because a bound computed from the reports would let one person's value change everyone's randomizer.

## 27.7. Collecting every day: memoization

Telemetry asks the same question every day. Averaging the daily reports of an unchanged value removes fresh noise, and composition counts that leak by adding each report's $\varepsilon_0$. The fix is **memoization**. The device draws one permanent randomized response per value and reuses it, so an unchanged value costs only that response's level, written $\varepsilon_\infty$. RAPPOR, Google's collection of Chrome users' homepages, has $\varepsilon_\infty = 2.043$.

Memoization fails when the value changes, since each new value costs another $\varepsilon_\infty$, so a user who changes their homepage every month reaches $\varepsilon = 24.5$ in a year. For drifting numbers, Microsoft's Windows telemetry compares each value with a secret random threshold per device, so a report changes only when the value crosses it. Its guarantee is about behaviour, not rounds. A user who stays within a band reveals nothing new, and one who changes reveals roughly when.

## 27.8. The shuffle model and amplification by shuffling

In the **shuffle model** (Cheu et al. 2019), each person sends a local report to a **shuffler**, which strips identifiers and timing and forwards the reports in random order. Each report then hides among the others, so the collection meets central $(\varepsilon, \delta)$-DP, with a small slack $\delta$, at an $\varepsilon$ far below $\varepsilon_0$ (Amplification by shuffling and by iteration). The closed-form bound of Feldman, McMillan & Talwar (2021) gives that $\varepsilon$ only when there are enough people to hide among:

$$\varepsilon_0 \le \ln \frac{n}{16 \ln(2/\delta)}. \tag{27.4}$$

In practice you fix the central target and solve the bound for the largest $\varepsilon_0$ it allows. For a central $(1, 10^{-6})$, ten thousand people may each randomize at $\varepsilon_0 = 3.504$. At a thousand people the condition above binds first and caps it at 1.460. Both use a $\delta$ below the book's default $1/n^{1.1}$. The closed form overstates $\varepsilon$ by a factor of 2.6 to 4.6 at ten thousand people. So plan with it, and deploy with the paper's published numerical bound.

Three things undo the amplification. A shuffler that colludes with the analyst leaves each report at $\varepsilon_0$, so choose an $\varepsilon_0$ you could defend on its own, within the tiers of the definition. Fake senders give no cover, so count only honest ones in $n$. And real shufflers batch and pad their traffic, so that timing and sizes do not leak.

## 27.9. Distributed DP: splitting the noise

In the **distributed model**, each client adds a small share of the central noise, and **secure aggregation** reveals only the sum to the server (Cryptography basics). Gaussian noise splits cleanly, because variances of independent shares add. If each of $n$ clients adds normal noise of

variance $\sigma^2/n$, the shares sum to central noise with standard deviation $\sigma$, as private as the central Gaussian mechanism.

The small shares are also the weakness. A client that colludes with the server can reveal its share, and one that drops out adds none. With a number $t$ of such clients, only the honest clients' noise is left, with standard deviation

$$\sigma\sqrt{\frac{n-t}{n}}. \tag{27.5}$$

So fix in advance the largest fraction $\rho$ of clients that may collude or drop out, and have each client add variance $\sigma^2/(n(1-\rho))$. The honest clients alone then supply the full noise.

For example, a server sums a histogram of many users through 100 clients and targets $(1, 10^{-6})$, the default $\delta$ for about a hundred thousand users. A user who changes value moves one count down and another up, so the $\ell_2$ sensitivity is $\sqrt{2}$. The analytic Gaussian calibration of the Laplace and Gaussian mechanisms gives a total $\sigma$ of 5.975, or a standard deviation of 0.597 per client. Without over-provisioning, a fifth of the clients colluding weakens the guarantee to $\varepsilon = 1.128$ at the same $\delta$. Half colluding gives 1.455, and nine-tenths gives 3.525.

Secure aggregation works on integers, so clients add integer noise, Skellam or the distributed discrete Gaussian of Discrete mechanisms, as TensorFlow Federated's `tff.aggregators.DistributedDpSumFactory` does. Secure aggregation does not check inputs, so a malicious client can still send any vector.

## 27.10. Cheat sheet

Table 27.1.: Which trust model and protocol to use, by situation.

| Situation | Use | Rule |
|---|---|---|
| A curator can be trusted | Central Laplace or Gaussian | The default |
| Nobody may see raw records | Local randomized response | Frequent items, very large populations |
| Frequencies over $k$ values | $k$-ary randomized response or unary encoding | $k$-ary if $k < 3e^{\varepsilon_0} + 2$ |
| The same question daily | Memoization | Each new value costs $\varepsilon_\infty$ |
| An anonymizing relay | Shuffle model | Deploy with a numerical bound |
| Federated sums | Distributed Gaussian or Skellam | Over-provision for colluders |

**Rules of thumb**

- Local noise exceeds central noise by a factor that grows like $\sqrt{n}$, 149 on Adult at $\ln 3$.

## 27.11. References

- Wang, Blocki, Li & Jha, "Locally Differentially Private Protocols for Frequency Estimation" (USENIX Security 2017): frequency oracles and local hashing, compared.
- Feldman, McMillan & Talwar, "Hiding Among the Clones" (FOCS 2021): the shuffling bounds, closed form and numerical.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 28. Lower bounds: what privacy costs

## 28.1. What it is

A **lower bound** says that no mechanism satisfying differential privacy (DP) can be more accurate than some floor. It holds for every mechanism at once, so it works as a feasibility check. Accuracy has two numbers here. The **error** $\alpha$ is how far the output may land from the true answer, and the **failure probability** $\beta$ is the chance that it lands further away.

The floor you meet most is for a histogram over $N$ bins under pure DP, with a privacy level $\varepsilon$ and no slack. If such a histogram is within $\alpha$ records of every count with probability at least $1 - \beta$, then

$$\alpha \geq \frac{1}{2}\left(\frac{\ln\left(N(1-\beta)\right)}{\varepsilon} - 1\right) \tag{28.1}$$

It grows only with the logarithm of the number of bins, but under pure DP it never goes away. Under approximate DP, which allows a small slack $\delta$, the floor stops growing at about $\ln(1/\delta)/\varepsilon$ records, however many bins there are.

## 28.2. When you meet it

You meet a lower bound when a design looks too good. Someone asks, for example, for a histogram over a million locations at $\varepsilon = 1$ with no slack, accurate to five records in every bin with 95% probability. First ask whether one person moves one of the statistics, as in the disjoint cells of a histogram, or all of them, as in the means of many attributes. The histogram floor governs the first case, and the fingerprinting and reconstruction floors below the second. Either way, a floor decides whether to keep tuning. Below it, the only ways out are more data, a coarser answer or a weaker guarantee, but within a small factor of it you should stop, because no algorithm gains more than that factor.

## 28.3. How to use it

For the million locations, every pure-DP histogram has an error of at least 6.38 records in some bin, so the five-record promise is impossible. The exact error of the Laplace histogram of the Laplace and Gaussian mechanisms is 16.79, about 2.6 times the floor, so it is already near the best. With $\delta = 10^{-6}$, the floor can never pass about 13.82 records, whatever $N$ is. A stability-based histogram releases only non-empty bins, so its error does not grow with $N$ at all (Beyond global sensitivity). So when a huge, mostly empty universe puts the floor too high, switch to approximate DP.

The same floor sizes a dataset, since an error in records becomes an error in frequencies once you divide by the number of records $n$. A pure-DP histogram whose frequencies must all be within $\alpha$ needs

$$n \geq \frac{\ln\left(N(1-\beta)\right) - \varepsilon}{2\alpha\varepsilon}. \tag{28.2}$$

If the data are a sample from a population, sampling costs records too, about $\ln(2N/\beta)/(2\alpha^2)$ by Hoeffding's inequality, and you need the larger of the two. At $\varepsilon = 1$ and 95% probability, a million cells at an error of 0.1 need at least 64 records for privacy and at most 875 for sampling. At an error of 0.01, privacy needs 638 and sampling needs 87,522. So at $\varepsilon = 1$, fine answers are short of records because of sampling, which no mechanism fixes.

Many attributes change the rate. Releasing the means of $d$ yes/no attributes costs records in proportion to $d$ under pure DP. Under approximate DP, the **fingerprinting** bound puts the cost at about $\sqrt{d\ln(1/\delta)}/(\alpha\varepsilon)$, and the Gaussian mechanism matches it to within $\sqrt{\ln d}$. So use approximate DP for outputs with many coordinates, such as a gradient, and once the Gaussian noise is calibrated, look for more records, not a better mechanism. For about $n$ counting queries on $n$ records, the reconstruction attack of Dinur and Nissim sets the floor at an error of order $\sqrt{n}$ per answer (Reconstruction attacks). With far more queries than records, answer them all from one private synthetic dataset, whose error grows only slowly with the number of queries (Query release).

## 28.4. What goes wrong

Aggregates are not a defence. In 2008, Homer and colleagues showed that published frequencies of genetic variants from a genome study reveal whether a given person took part. The attack scores how far the released means lean, attribute by attribute, towards the target's record and away from the population means. Members score higher, because each pulls the means toward their own record, and the fingerprinting bound rests on this attack. On the means of 50 people over 2,000 attributes, it reaches a true positive rate (TPR) of 0.242 at a false positive rate (FPR) of 1%. The noise of a $(1, 10^{-5})$-DP Gaussian mechanism cuts that TPR to 0.014.

Use the score to audit any release of many averages, with population means from a separate sample. Compare its TPR at a low FPR with the cap $\text{TPR} \leq e^{\varepsilon}\,\text{FPR} + \delta$ from the definition. An audit that beats the cap has found a bug in the mechanism or its accounting. Passing proves nothing, because an attack only bounds $\varepsilon$ from below (Auditing private training).

Every approximate-DP floor here assumes a meaningful $\delta$, such as the book's default $\delta \leq 1/n^{1.1}$ (How small δ must be). A mechanism that publishes each record outright with probability $\delta$ is $(0, \delta)$-DP, so a claim with $\delta$ near $1/n$ allows publishing about one record.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 29. Private mean estimation

## 29.1. Why this matters

The mean is the statistic that private machine learning computes most often. Each step of private stochastic gradient descent averages per-example gradients, each clipped to a maximum length first, and even a survey share is a mean of 0/1 answers. The planning questions are how many records a private estimate needs, and how to choose a clipping range without looking at the data.

Differential privacy (DP) must hold for every dataset, including datasets your assumptions say cannot happen (The definition of differential privacy). So an assumption about the data, such as "the records are normal", may make a mechanism more accurate, but it must never enter the privacy argument. Every mechanism below clips each record to a range, which caps one record's influence whatever the data are, and its noise grows with the width of that range. A narrow range is cheap, but it must not be read off the data. So spend a small part of the budget finding where the data are, then clip there and spend the rest on the mean.

## 29.2. Bounded data: privacy adds a lower-order term

Throughout the chapter, datasets have a fixed size $n$, and neighbours differ in one replaced record. An estimate is judged by its **root-mean-square error** (RMSE), the typical distance between the estimate and the mean of the population the records came from.

Start with records in a known interval, such as yes/no answers coded 0 and 1. Write $w$ for the width of the interval and $\sigma^2$ for the variance of a record. Release the average plus Laplace noise with scale $w/(n\varepsilon)$, which is $\varepsilon$-DP because one record moves the average by at most $w/n$ (The Laplace and Gaussian mechanisms). Laplace noise has variance twice its scale squared, and it is independent of the sampling error, so the two variances add:

$$\mathrm{RMSE} = \sqrt{\frac{\sigma^2}{n} + \frac{2w^2}{n^2\varepsilon^2}}. \tag{29.1}$$

To plan a study, set the RMSE equal to a target error $\alpha$ and solve for $n$. The snippet in the Code section does this exactly, and the number of records needed is at most

$$\frac{\sigma^2}{\alpha^2} + \frac{\sqrt{2}\,w}{\alpha\varepsilon}. \tag{29.2}$$

So sampling needs about $\sigma^2/\alpha^2$ records whatever you do, and privacy adds at most about $w/(\alpha\varepsilon)$ more. For a yes/no survey of a population that is half yes, the worst case for sampling, an RMSE of 0.05 needs 100 people. At $\varepsilon = 1$, privacy raises that to 108. Its term shrinks only like $1/\alpha$,

against $1/\alpha^2$ for sampling, so it matters most at a coarse target and a small $\varepsilon$. It also grows with the width $w$, so the rest of the chapter is about narrowing the range.

Libraries make you state the range. In diffprivlib, `tools.mean(x, epsilon=..., bounds=(lower, upper))` clips to the bounds and adds noise. Without `bounds`, it computes them from the data and raises a `PrivacyLeakWarning`, because a range read off the observed minimum and maximum leaks them. OpenDP likewise needs clamping bounds, its word for clipping bounds, before it builds a mean, and Release statistics with a library does the whole job with it.

Bounds that are too tight fail in another way, because they clip real records. The script practitioner_adult_statistics.py takes the hours worked per week by the 48,842 people of the Adult census data. It releases the mean at $\varepsilon = 1$ twice, once with loose bounds and once with tight ones.

```
hours = adult["hours-per-week"].to_numpy()
loose = tools.mean(hours, epsilon=1.0, bounds=(1, 99), random_state=seed)
tight = tools.mean(hours, epsilon=1.0, bounds=(20, 60), random_state=seed)
clipped = np.clip(hours, 20, 60).mean()  # what the tight bounds can at best recover
print(f"true mean hours: {hours.mean():.3f}")
print(f"bounds (1, 99): {loose:.3f}  bounds (20, 60): {tight:.3f}")
print(f"bias of (20, 60): {clipped - hours.mean():+.3f}")
```

Its seed-0 run, `uv run python code/part03-dp-foundations/practitioner_adult_statistics.py`, prints these lines, the last two from code next to the excerpt.

```
true mean hours: 40.422
bounds (1, 99): 40.421  bounds (20, 60): 40.361
bias of (20, 60): -0.061
people outside (20, 60): 8.7%
noise scale (1, 99): 0.0020  (20, 60): 0.0008
```

The tight bounds shrink the noise scale from 0.0020 to 0.0008 hours. But they clip the 8.7% of people outside them, which moves the mean by 0.061 hours, far more than the noise they save. No budget removes that bias, so report the error in three parts, which are sampling, clipping bias and noise. Choose the bounds from the schema or public data, and check the share they clip on data you may look at.

The formulas also treat $n$ as public, which holds when neighbours replace a record, since they then have the same size. If the size is itself private, divide a noisy sum by a noisy count, with the budget split between the two.

## 29.3. The cost of an unknown range

Now let the records be normal with variance 1, and suppose you know only that their mean lies within some distance $R$ of zero. Normal records rarely stray far from their mean, so write $\tau$ for a

distance from the mean that holds them all, except with a small failure probability $\beta$. For 1,000 records at $\beta = 0.05$, it is 4.604.

The **naive estimator** clips every record to the widest place the data could be, $R + \tau$ on either side of zero, and adds Laplace noise for that width. It is private, but its error grows in step with $R$. With 1,000 records at $\varepsilon = 1$, its RMSE reaches 2828.44 when $R$ is a million. Yet Lower bounds shows that an unknown range need cost only about $\ln R/\varepsilon$ extra records.

**Coarse then fine** (Karwa & Vadhan 2018) comes close to that floor in two steps. The coarse step cuts the range into bins of width $2\tau$ and picks the fullest bin with the exponential mechanism, spending a part $\varepsilon_1$ of the budget. It succeeds with a number of records that grows only with the logarithm of the number of bins. The fine step clips every record to the chosen bin and its two neighbours, because the data can straddle two bins. It releases their average with Laplace noise for that window, spending the rest of the budget. By composition, the two steps together are $\varepsilon$-DP.

With a quarter of the budget on the coarse step, the full edition's 1,000 records give an RMSE of 0.0609 for every $R$ from 10 to a million. But the fine step gets only three quarters of the budget and a window three bins wide, so for small ranges the naive estimator wins, here up to $R = 13.81$. Beyond it, use coarse then fine, or locate the data with a private quantile such as diffprivlib's `tools.quantile`. Under approximate DP, with its small slack $\delta$, a stability-based histogram does the coarse step with about $\ln(1/\delta)/\varepsilon$ records whatever the number of bins, so $R$ drops out (Beyond global sensitivity). Still, a budget split, bin width or $\beta$ chosen after looking at the data leaks it. And a coarse step with too few records can pick an empty bin and clip everything to the wrong place.

## 29.4. Many dimensions: clip the norm

Vectors in $d$ dimensions, such as embeddings or gradients, follow the same recipe with a different measure of size. Clip each record's $\ell_2$ norm, its length, to a radius $C$, so that one replaced record moves the average by at most $2C/n$. Then add Gaussian noise calibrated to that sensitivity (The Laplace and Gaussian mechanisms). This is exactly the step of private gradient descent, and private training gives the rules for choosing the clipping norm there. With unit variance per coordinate, records lie about $\sqrt{d}$ from their mean, so the radius must grow like $\sqrt{d}$. Noise at that scale in every coordinate then grows like $d$ in total, against $\sqrt{d}$ for the sampling error. So in high dimension privacy is the larger cost, and the fingerprinting lower bound shows that no mechanism avoids it.

When you do not know where the mean is, the range problem returns, and a grid of bins would pay for the range once per dimension. The **shrinking balls** of CoinPress (Biswas, Dong, Kamath & Ullman 2020) avoid the grid. Over a few rounds, they clip to a ball around the current guess, release a noisy average and shrink the ball. The full edition runs it on 100,000 records in a thousand dimensions at $\varepsilon = 1$, with the default $\delta = 10^{-6}$. The whole vector's RMSE is 0.1736, against 0.1000 without privacy. Fix the number of rounds and the budget split in advance, because the rounds compose and too many of them eat the budget.

## 29.5. Heavy tails: clip into the data

Incomes, session lengths and per-example gradient norms have heavy tails. When all you know is that the variance is at most $\sigma^2$, a range wide enough to touch no record is enormous. So clip into the data on purpose, and accept a bias.

Reuse $\tau$ for the clipping half-width, and clip every record to within $\tau$ of a centre near the mean, found by a coarse step. Only the rare records beyond $\tau$ move, so with the centre at the mean the bias is at most $\sigma^2/\tau$, while the noise grows in proportion to $\tau$. The bound on the total error is smallest at

$$\tau_b = \sigma\sqrt{\frac{n\varepsilon}{2\sqrt{2}}}, \tag{29.3}$$

which uses only $n$, $\varepsilon$ and the variance bound. There, an error $\alpha$ needs about $\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$ records. So privacy is no longer a lower-order cost, since both terms grow like $1/\alpha^2$. No $\varepsilon$-DP mechanism does better for every distribution with that variance, though a bound on a higher moment lowers the cost.

Table 29.1 shows the trade-off between bias and noise on Pareto records, a heavy-tailed distribution scaled to variance 1. Its first width clips no record except with probability 0.05. The worst-case width $\tau_b$ cuts the error to 0.0607, and the best width for this distribution does better still.

Table 29.1.: Clipped private means of Pareto records with variance 1, 1,000 records, $\varepsilon = 1$, for three clipping half-widths.

| Clipping half-width $\tau$ | Bias | Sampling sd | Noise sd | RMSE |
|---|---|---|---|---|
| 141.42 (no record clipped) | 0.0000 | 0.0313 | 0.4000 | 0.4012 |
| 18.80 (worst-case best, $\tau_b$) | −0.0018 | 0.0293 | 0.0532 | 0.0607 |
| 5.36 (best for this distribution) | −0.0153 | 0.0249 | 0.0152 | 0.0329 |

So treat $\tau_b$ as a safe default, not as the best width. A narrower range usually does better, but choose it without looking at the records, from domain knowledge or with a private quantile (Beyond global sensitivity).

## 29.6. Code

The snippet computes the records a bounded mean needs, checks its noise with OpenDP, and computes the default clipping width for heavy tails. Setting the RMSE formula equal to $\alpha$ and multiplying by $n^2$ gives a quadratic in $n$, whose positive root the first function rounds up.

```python
from math import ceil, sqrt

import opendp.prelude as dp


def records_for_bounded_mean(alpha: float, sd: float, width: float, eps: float) -> int:
    """Smallest n with sd^2/n + 2 (width/(n eps))^2 <= alpha^2."""
    a, c = sd**2, 2 * (width / eps) ** 2
    return ceil((a + sqrt(a * a + 4 * c * alpha**2)) / (2 * alpha**2))


def heavy_tail_width(n: int, eps: float, sd: float) -> float:
    """The clipping half-width that balances the bias bound sd^2/tau against the noise."""
    return sd * sqrt(n * eps / (2 * sqrt(2)))


n = records_for_bounded_mean(0.05, 0.5, 1.0, 1.0)
print(n)  # 108
print(f"{dp.laplacian_scale_to_accuracy(1.0 / (n * 1.0), alpha=0.05):.4f}")  # 0.0277
print(f"{heavy_tail_width(1000, 1.0, 1.0):.2f}")  # 18.80
```

OpenDP states accuracy differently, as a half-width that the noise stays within with a chosen probability. Its `laplacian_scale_to_accuracy` turns the scale $w/(n\varepsilon)$ at the planned 108 records into the noise's half-width at 95% confidence, which the snippet prints. When your target is such a half-width rather than an RMSE, `accuracy_to_laplacian_scale` gives the scale instead. Setting $w/(n\varepsilon)$ equal to it then gives the records the noise needs, before sampling adds its share.

## 29.7. Cheat sheet

Table 29.2.: Private mean estimation at a glance, with $\tau_b = \sigma\sqrt{n\varepsilon/(2\sqrt{2})}$.

| What you know | Mechanism | Records for error $\alpha$ (up to log factors) |
|---|---|---|
| Bounded, width $w$, variance $\sigma^2$ | Mean plus $\mathrm{Lap}(w/(n\varepsilon))$ | $\sigma^2/\alpha^2 + \sqrt{2}\,w/(\alpha\varepsilon)$ |
| Normal, mean within $R$ of zero | Coarse histogram, then clip to three bins | $1/\alpha^2 + 1/(\alpha\varepsilon) + \ln R/\varepsilon$ |
| Same, approximate DP | Stability-based coarse step | $1/\alpha^2 + 1/(\alpha\varepsilon) + \ln(1/\delta)/\varepsilon$, no $R$ |
| $d$ dimensions, mean length at most $R$ | Clip the norm, Gaussian noise; shrinking balls to find the mean | $d/\alpha^2 + d\sqrt{\ln(1/\delta)}/(\alpha\varepsilon) + \sqrt{d}\ln R/\varepsilon$ |
| Variance at most $\sigma^2$ only | Clip to within $\tau_b$ of the centre | $\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$ |

**Rules of thumb**

- Never clip to the observed minimum and maximum. Take bounds from the schema or public data, or spend a quarter of the budget or less, a simple default, finding the data.
- Bounds that clip real records bias the mean whatever $\varepsilon$ is, so put the clipping bias into every reported interval.

## 29.8. References


- Karwa & Vadhan, "Finite Sample Differentially Private Confidence Intervals" (ITCS 2018): coarse then fine for normal means and variances, under pure and approximate DP.
- Biswas, Dong, Kamath & Ullman, "CoinPress: Practical Private Mean and Covariance Estimation" (NeurIPS 2020): shrinking balls in practice, with code.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 30. Release statistics with a library

## 30.1. Why this matters

Suppose you are the analytics engineer of an online shop, and the business wants a dashboard of its sales: purchase lines, revenue and the typical sale, overall and per country, and how many customers bought each product. The figures describe customers, so they should come with a differential privacy (DP) guarantee. The difficulty is that one customer appears in many rows, and a guarantee for one row would protect a single purchase, not the person behind a hundred.

This chapter does the whole job with OpenDP, from the unit and the budget to the written guarantee.

The data are the Online Retail set from the University of California, Irvine (UCI) Machine Learning Repository. It holds 541,909 invoice lines of a British online shop. Each line records one product on one invoice, with its quantity, its unit price, the customer and the country. The script practitioner_release_statistics.py runs every step in under a minute, with OpenDP 0.16.0 and Polars 1.36.1.

```
uv run python code/part03-dp-foundations/practitioner_release_statistics.py
```

OpenDP draws its noise from a cryptographic generator, which cannot be seeded, so the released values change on every run. This page quotes the scales, error bars and thresholds, which involve no noise, and shows released values only in a block marked as one run.

## 30.2. Choose the library

The main question is whether a library can bound the contribution of a person who owns many rows (Table 30.1).

Table 30.1.: Libraries for releasing statistics, and what each can express.

| Library | Runs on | Person with many rows | Groups not known in advance | Use it for |
|---|---|---|---|---|
| OpenDP | Polars, one machine | Yes, with an identifier column | Yes, by a noisy threshold | Statistics on one machine; the default here |

| Library | Runs on | Person with many rows | Groups not known in advance | Use it for |
|---|---|---|---|---|
| diffprivlib | NumPy, scikit-learn | No, one row per person | No | Quick experiments and simple models |
| PipelineDP | Apache Beam, Spark | Yes, built in | Yes, built in | Pipelines that already run on Beam or Spark |
| Tumult Analytics | Spark | Yes, with an identifier column | Yes, in a separate query | Large releases on Spark |

Google's differential privacy library serves the same pipelines in C++, Go and Java, and PipelineDP, by OpenMined and Google, is its Python counterpart. PipelineDP and Tumult were not run here, so their rows follow their documentation. Whatever you pick, write no mechanism yourself, and keep one accountant, which totals the budget spent, for the whole release.

## 30.3. Decide the unit and the budget

The **unit of privacy** is what the guarantee protects, and for this dashboard it is a customer. The neighbouring datasets therefore differ by adding or removing one customer with all their lines (The definition of differential privacy). The script drops lines with no customer identifier, which belong to nobody, as well as returns and lines with a zero price. Each of these filters judges a line by that line alone, so adding or removing a customer still changes only that customer's lines.

```
raw = practitioner_datasets.online_retail()
lines, stats = clean(raw)
n_customers = stats.customers
delta = delta_for(n_customers)  # 1/n^1.1, rounded down to a power of ten
per_line = per_customer(lines)
print(f"customers: {n_customers:,}  purchase lines: {lines.height:,}")
print(f"delta: {delta:.0e}")
p50, p90, p99, top = quantiles(per_line, [0.5, 0.9, 0.99, 1.0])
print(f"lines per customer: p50={p50} p90={p90} p99={p99} max={top}")
```

```
customers: 4,372  purchase lines: 397,884
delta: 1e-05
lines per customer: p50=41 p90=208 p99=662 max=7847
```

The 4,372 customers are counted before returns are dropped, which leaves 4,338, and both counts give the same slack. The slack $\delta$ follows the book's default for $n$ people, $\delta \le 1/n^{1.1}$ rounded down to a power of ten, which gives $\delta = 10^{-5}$ here. OpenDP logs a warning that $\delta$ should be at most $10^{-6}$, which is general advice; this book ties $\delta$ to the number of people instead. The privacy level is $\varepsilon = 1$ for the whole release, because all its tables describe the same customers and their costs add up (Post-processing, group privacy and composition).

The median customer has 41 lines, but under a guarantee per line the largest would be a group of 7,847 records, and group privacy multiplies $\varepsilon$ by the size of the group. Printing the maximum is fine only because this set is public. On real data, a bound read off the private data leaks.

## 30.4. Bound each customer's contribution

Write $k$ for the largest number of lines a customer may contribute, so that every query keeps at most $k$ lines of each customer. A small $k$ means little noise but throws data away, while a large $k$ keeps the data but needs more noise. The script measures the trade-off on this public set.

```
k=10: lines kept 10.2%  customers truncated 84.8%
k=25: lines kept 22.4%  customers truncated 64.6%
k=50: lines kept 36.9%  customers truncated 44.0%
k=100: lines kept 55.1%  customers truncated 24.9%
k=200: lines kept 72.7%  customers truncated 10.4%
k=500: lines kept 88.2%  customers truncated 1.8%
```

The default is a high percentile of lines per customer, read from public or historical data, never from the data you release. With neither, spend a small share of the budget on a private high quantile of the lines per customer, one number per customer, over candidates fixed in advance (Selection). This chapter takes $k = 100$ from this public set, lower than the default because the noise grows with $k$. It keeps 55.1% of the lines and truncates a quarter of the customers.

Keeping the first $k$ lines by date would bias every statistic towards each customer's early purchases, so the script keeps a random sample of $k$ lines per customer. The filter below is OpenDP's `truncate_per_group(k, keep="sample")` with a seed, so that the true values of the check stay fixed.

```
def sample_per_customer(k: int, seed: int, *by: str) -> pl.Expr:
    """True for at most k randomly chosen lines of each customer (within each value of `by`)."""
    return pl.int_range(pl.len()).shuffle(seed=seed).over("CustomerID", *by) < k
```

Values need bounds as well. A line's value is its quantity times its unit price, which the script clips to between £0 and £200. The lower end holds because returns were dropped, and the upper end is a bound the business sets, which clips 1.02% of the lines. With both bounds, one customer can move the revenue by at most 100 lines of £200 each. Take value bounds from the domain, never from the data's minimum or maximum (Beyond global sensitivity).

A group-by needs two more bounds. Write $L_0$ for the number of groups one customer may touch, and $L_\infty$ for the lines a customer may put in any one group. Countries need neither, because however a customer spreads their $k$ lines over countries, the country counts change by at most $k$ in all, which the noise already covers. When customers spread over many groups, as with country by month, bounding $L_0$ and $L_\infty$ can beat a total $k$, because the noise then scales with $L_0 L_\infty$ for Laplace and $\sqrt{L_0}\, L_\infty$ for Gaussian. Products need both bounds for partition selection, and Section 30.8 sets them.

## 30.5. Set up the context

OpenDP keeps the whole release in a **context**, which holds the data, the unit, the total budget and its split between queries. Contexts need `dp.enable_features("contrib")`, which switches on components not yet formally vetted, so a reviewer who needs vetted code checks each call in OpenDP's documentation.

```
context = dp.Context.compositor(
    data=lines.lazy(),
    privacy_unit=dp.unit_of(contributions=1, identifier="CustomerID"),
    privacy_loss=dp.loss_of(rho=rho, delta=delta / 2),  # Gaussian noise; delta/2 for thresh
    split_by_weights=WEIGHTS,  # scalars, countries, products
    margins=[dp.polars.Margin(by=[], max_length=MAX_LINES)],
)
```

The identifier column makes the unit a customer, so `contributions=1` means adding or removing one customer with all their lines, which the truncation filters then bound. For replace, OpenDP takes `changes=` instead. The budget is in **zero-concentrated differential privacy (zCDP)** (Concentrated differential privacy). It states privacy as one number, written $\rho$, that adds up across queries and converts to $(\varepsilon, \delta)$ at the end.

With a zCDP budget, the context adds Gaussian noise to every count and sum, which suits the product counts, where one customer touches many groups (The Laplace and Gaussian mechanisms). The script's `rho_for` finds $\rho = 0.0284$, the largest value that OpenDP's own conversion turns into $\varepsilon \leq 1$ with half of $\delta$, and works for any other target. The other half pays for the threshold of Section 30.8. Finally, the margin states a public upper bound on the number of lines, which OpenDP needs before it sums floating-point values.

The weights give a quarter of the budget to the overall statistics, a quarter to the countries and half to the products, which need it most. The script prints the split and converts the whole release back to $(\varepsilon, \delta)$, the claim the statement will make.

```
budget scalars: rho=0.0071 delta=1.25e-06
budget countries: rho=0.0071 delta=1.25e-06
budget products: rho=0.0142 delta=2.50e-06
whole release as (epsilon, delta): (1.0000, 1e-05)
```

A query takes its share of the budget when it is built, but spends it only when it is released. Each new query is built with the next unspent share, so if you build all the queries first, every one gets the first share, and the product query gets a quarter's noise but is charged half. Build, check and release each query in turn, as the script does.

## 30.6. Release overall statistics with error bars

The first query releases the count of lines, the revenue and the median line value. The context holds the raw lines, so each query truncates them itself.

```
value = clipped_value()  # the line value, clipped to [low, high]
scalars = (
    context.query()
    .filter(sample_per_customer(K, seed))  # at most K lines per customer
    .select(
        dp.len(signed=True),
        value.dp.sum((low, high)).alias("revenue"),
        value.dp.median(candidates=MEDIAN_CANDIDATES).alias("median"),
    )
)
scalar_summary = scalars.summarize(alpha=0.05)
scalars_hat = scalars.release().collect()
```

`signed=True` keeps negative noisy counts, and `summarize` reports, before anything is released, each statistic's noise and 95% error bar.

```
scalars len: Frame Length, Integer Gaussian, scale=1,453.38 95% accuracy=2,850.0
scalars revenue: Sum, Float Gaussian, scale=290,804.33 95% accuracy=569,966.0
scalars median: 0.5-Quantile, GumbelMin, scale=1,453.38 95% accuracy=none
```

With probability 0.95, the released count of lines is within 2,850 of the truth, and the revenue within £569,966. They depend only on the bounds and the budget, so you can read them before the release and decide whether the budget is enough.

The table also shows how OpenDP splits a query's share among its statistics: each gets the same noise scale relative to its sensitivity. The count and the median both get 1,453.38, and the revenue, whose sensitivity is 200 times the count's, about 200 times that. A mean, `.dp.mean`, is a sum and a count, so it takes two shares. To give one statistic more budget, put it in a query of its own with its own weight. The error bar of a mean follows from those of its sum and count (The Laplace and Gaussian mechanisms). The median is chosen by the exponential mechanism, whose sampler is `GumbelMin`, among candidates in steps of £0.50 across the value bounds. So its error bar is in ranks, not in `summarize` (Selection).

The error bar covers the noise only, not the bias of truncation and clipping, which together keep only 48.5% of the revenue. So the private revenue estimates a truncated total, not the shop's takings. Say so, and choose a larger $k$ if the full total matters more than the noise.

## 30.7. Group by a public key list

Lines and revenue per country come next, and the key list must not come from the data. The distinct values of the Country column would publish which countries appear in the private data, which no noise covers. An earlier version of the script did that, and its list included Hong Kong, which occurs only on lines without a customer identifier. So the script fixes its keys in advance: the United Kingdom, the 27 EU member states, the 4 of the European Free Trade Association, the United States, Canada, Australia, Japan and China, and "Other". A rule renames three of the source's spellings, such as `EIRE` to Ireland, and maps every other value to Other.

```
by_country = (
    context.query()
    .filter(sample_per_customer(K, seed))
    .group_by("Market")
    .agg(dp.len(signed=True), value.dp.sum((low, high)).alias("revenue"))
    .with_keys(pl.LazyFrame({"Market": MARKETS}))  # every key is released, zeros included
)
country_summary = by_country.summarize(alpha=0.05)
countries_hat = by_country.release().collect()
```

```
per country len: Frame Length, Integer Gaussian, scale=1,185.25 95% accuracy=2,324.0
per country revenue: Sum, Float Gaussian, scale=237,698.48 95% accuracy=465,880.5
```

Every key is released, including the 11 with no purchase lines, whose counts are pure noise, so the release says nothing about which markets have customers. Other holds 1,750 lines from 11 source values.

Each market's count has a 95% error bar of 2,324, and its revenue one of about £466,000. Germany's true count of 5,379 is still useful, but its revenue of £131,902 is smaller than its error bar. Publish the error bar with every cell, or merge small markets into Other before the release.

## 30.8. Group by unknown keys with partition selection

The list of products is not public, and a product bought by one customer would reveal that purchase simply by appearing in the table. So the release decides privately which products to show, by **partition selection**. It releases only the products whose noisy count clears a threshold, so a product with few customers almost never appears. The threshold is what costs $\delta$ (Beyond global sensitivity).

To count customers rather than lines, the query first keeps one line per customer and product, so $L_\infty = 1$. It then keeps a number $m$ of products per customer at most, so $L_0 = m$. The script measures the trade-off that $m$ sets on this public set.

```
m=1: pairs kept 1.6%  customers truncated 97.9%  scale=5.93  threshold=29  products above it
m=5: pairs kept 7.9%  customers truncated 92.4%  scale=13.27  threshold=66  products above i
m=10: pairs kept 15.1%  customers truncated 83.6%  scale=18.77  threshold=96  products above
m=20: pairs kept 27.5%  customers truncated 68.3%  scale=26.54  threshold=138  products abov
m=50: pairs kept 52.7%  customers truncated 38.6%  scale=41.96  threshold=225  products abov
```

The threshold grows with $m$, from 29 to 225, because each customer can push more groups over it, but the counts grow faster. So the number of products whose true count clears the threshold rises from 1 to 50, and the script takes $m = 50$ (`M` in the code), again choosing on public data.

```
by_product = (
    context.query()
    .filter(sample_per_customer(1, seed, "StockCode"))  # one line per customer and product
    .filter(sample_per_customer(M, seed))  # at most M products per customer
    .group_by("StockCode")
    .agg(dp.len(signed=True).alias("customers"))
)
product_summary = by_product.summarize(alpha=0.05)
products_hat = by_product.release().collect()
```

```
per product customers: Frame Length, Integer Gaussian, scale=41.96 95% accuracy=83.0 thresho
```

A product is released only if its noisy count reaches 225, and a released count has a 95% error bar of 83. Never publish how many products were dropped, or which, since both come from the private data. Products just below the threshold get in when their noise is positive, so treat a count near 225 as a sign of popularity, not as a measurement.

OpenDP's `truncate_num_groups` should bound the groups per customer, but in version 0.16.0 it does not work. With the default `keep="sample"`, `summarize` accepts the query, and `release` fails after spending the query's share, so run each query end to end on public data first. With `keep="first"` it keeps one group fewer than asked, the first in sort order, which biases the counts. The filters above avoid both problems only because $L_\infty = 1$ turns groups into lines, and for $L_\infty > 1$ this book knows of no unbiased group bound in this version.

PipelineDP, not run here, selects partitions when you pass no public list.

```
# PipelineDP 0.3.1 (not run)
import pipeline_dp
accountant = pipeline_dp.NaiveBudgetAccountant(total_epsilon=1.0, total_delta=1e-5)
engine = pipeline_dp.DPEngine(accountant, pipeline_dp.LocalBackend())
params = pipeline_dp.AggregateParams(
    metrics=[pipeline_dp.Metrics.PRIVACY_ID_COUNT],
    noise_kind=pipeline_dp.NoiseKind.GAUSSIAN,
    max_partitions_contributed=50,  # L0: products per customer
    max_contributions_per_partition=1,  # L-infinity: lines per customer and product
)
extractors = pipeline_dp.DataExtractors(
    privacy_id_extractor=lambda row: row["CustomerID"],
    partition_extractor=lambda row: row["StockCode"],
    value_extractor=lambda row: 0,
)
result = engine.aggregate(rows, params, extractors)  # no public_partitions: selected privat
accountant.compute_budgets()
```

Tumult Analytics 0.21.0 needs Spark and was not run either. It protects `AddRowsWithID("CustomerID")`, names the two bounds `MaxGroupsPerID` and `MaxRowsPerGroupPerID`, and selects the keys with `get_groups` in one query before it counts them in a second.

## 30.9. Check the release before publishing

Check that the code does what the plan says. The script compares the private values with the true ones, after the same truncation and clipping.

```
true lines after truncation: 219,291 (all purchase lines: 397,884)
true revenue after truncation and clipping: 4,318,947 (all lines, unclipped: 8,911,408)
true median line value after truncation: 15.00 (all lines: 11.80)
```

The release itself is random. One run printed the following.

```
released (one run):
  lines 218,972  revenue 4,397,410  median 14.50
  United Kingdom: lines 196,311  revenue 3,927,959
  Germany: lines 7,151  revenue 643,094
  Spain: lines -546  revenue -37,048
  country line counts within their 95% accuracy: 34 of 38
  products released: 63
```

Read it against the error bars: about 95% of the market counts should fall inside theirs. Negative values are noise on small markets, and rounding them up to zero is post-processing, which costs no budget. Expect more than the 50 products whose true count clears the threshold, and compare the private median with the truncated £15.00, not the £11.80 of all lines.

A check fails when a value lies far outside its error bar, which points to a wrong sensitivity, when far more or far fewer than 95% of the cells fall inside, or when a released product's true count lies far below the threshold. In production you cannot compare against the truth, because a decision to publish that depends on the true values is itself a release of them. Run this check on historical or public data while you develop, and release the real data once, with the code you checked.

## 30.10. Write the guarantee down

The script prints the claim from the variables that set the bounds and the budget, so it cannot drift from the code.

```
This release satisfies (epsilon = 1, delta = 1e-05)-differential privacy for adding or
removing one customer and all their invoice lines, with at most 100 lines per customer
(sampled at random), at most 50 random products per customer in the product counts, and
line values clipped to [0, 200] pounds, under the central model, using OpenDP 0.16.0 with
Gaussian noise (accounted as 0.0284-zCDP plus 5e-06 for thresholds, converted to epsilon
and delta). The market list was fixed in advance; the product list is not public.
```

The statement names everything a reviewer needs to recompute the claim (Writing the guarantee down). Next to it, say what it does not cover. Another release about the same customers adds its own budget. If that one is pure $\varepsilon$-DP, either add the $(\varepsilon, \delta)$ pairs, or convert it to $\rho = \varepsilon^2/2$ and

add it to this release's $\rho$ before converting, whichever total is smaller (Concentrated differential privacy). The figures also describe truncated data, which the dashboard should say. Release and governance maps such a claim to data-protection law.

## 30.11. Checklist

Table 30.2.: The release checklist: each step, its default and its check.

| Step | Default | Check |
|---|---|---|
| Unit and budget | One person, add/remove; one $\varepsilon$; $\delta \leq 1/n^{1.1}$ | $n$ counts people, not rows |
| Contribution bound | $k$ from a high percentile of public or historical data; random sample | No bound read from the release data |
| Value bounds | From the domain or the business | Share clipped and bias stated |
| Context | zCDP with Gaussian noise; weights by need | $\rho$ converted to $(\varepsilon, \delta)$ equals the claim |
| Queries | Build, summarize and release one at a time | Scales match the plan; each query run end to end on public data |
| Public keys | Fixed in advance, values mapped by a rule; every key released | No key list built from the data |
| Unknown keys | Partition selection with $L_0$ and $L_\infty$ | Number dropped never published |
| Check | On historical or public data | About 95% inside their error bars |
| Statement | Printed by the releasing code | Unit, bounds, version and public information named |

## 30.12. References

- The OpenDP documentation, "Polars" and "Grouping" in its user guide: contexts, margins and partition selection.
- Wilson et al., "Differentially Private SQL with Bounded User Contribution" (PETS 2020): contribution bounding in a production SQL engine.
- Desfontaines, Voss, Gipson & Mandayam, "Differentially private partition selection" (PETS 2022): the threshold and why it needs $\delta$.

**Derivations.** This workflow has no counterpart in the full edition. The results it uses are derived in the full edition's chapters on the definition in the full edition, its properties in the full edition, the Laplace and Gaussian mechanisms in the full edition, selection in the full edition and sensitivity in the full edition.

# Part IV.

# Privacy accounting

A model trained with stochastic gradient descent (SGD) under differential privacy (DP), the method called DP-SGD, takes thousands of noisy steps, each on a random batch. The guarantee you report is a privacy level $\varepsilon$ with a small slack $\delta$. The composition theorems of Part 3 are far too loose for thousands of steps. An **accountant** does better, because it tracks a richer summary of each step's privacy cost that composes tightly, and converts the total to $(\varepsilon, \delta)$ once, at the end.

One DP-SGD run is the running example. Each step samples each record with probability 0.01 and adds noise with multiplier 1. After 10,000 steps, advanced composition certifies $\varepsilon = 24{,}379.55$ at $\delta = 10^{-5}$. For the same run, the privacy loss distribution accountant of Google's `dp_accounting` certifies 6.19. So an $\varepsilon$ means nothing until you name the accountant that produced it, and the sampler and the neighbouring relation it assumed.

The part starts with the two jobs. Report and review an ε claim says what a privacy statement must contain, which accountant to use, how to choose $\delta$, and how to check a number someone else hands you. Calibrate and account a DP-SGD run is a workflow. It turns a target $\varepsilon$ into a noise multiplier for the Adult run that Part 5 trains, cross-checks it with a second accountant, and handles early stopping and restarts.

Two full chapters explain what those jobs rely on. Amplification by subsampling shows how much a random batch helps, provided the data loader samples the way the accountant assumes, which is the costliest mistake in practice. Rényi differential privacy is the curve that most libraries track and report.

The rest are short reference pages, in the order you are likely to meet them. The first three finish the DP-SGD story: the conversion from Rényi DP, numerical accounting with privacy loss distributions, and Gaussian DP, which reads a guarantee as a limit on attacks. Then come adaptive composition for budgets decided while training runs, concentrated DP for analytics ledgers, and shuffling and iteration.

The code uses the accountants of Opacus and `dp_accounting`, and every number comes from a snippet or a script you can run. By the end, you will be able to calibrate a DP-SGD run, account for it with two libraries, and report $\varepsilon$ with everything someone else needs to reproduce it.

# 31. Choosing an accountant

## 31.1. Why this matters

You have trained a model with differential privacy (DP), and a paper or a model card needs one sentence. It says that the model is DP with a privacy level $\varepsilon$ and a small slack $\delta$. The same run can be reported as 6.24 or as 24,379.55, depending on the procedure that computed it, and nobody can check either value without knowing what that procedure assumed.

The run in Table 31.1 is private stochastic gradient descent (SGD), called DP-SGD. Each record joins each batch with probability $q = 0.01$, the sampling rate, and the noise multiplier $\sigma$, the noise as a multiple of the clipping norm, is 1. The run has one true $\varepsilon$ that nobody can compute exactly, and a procedure that bounds or estimates it is an **accountant**. The table compares advanced composition, Rényi differential privacy (RDP) and the privacy loss distribution (PLD).

Table 31.1.: One DP-SGD run, $q = 0.01$ and $\sigma = 1$, under three accountants: $\varepsilon$ at $\delta = 10^{-5}$. The PLD columns round the privacy loss up and down to a grid of width $10^{-5}$, so they bracket the true value.

| Steps | Advanced composition | RDP, improved | PLD upper | PLD lower |
|---|---|---|---|---|
| 1,000 | 1258.64 | 2.1014 | 1.8332 | 1.8232 |
| 10,000 | 24379.55 | 6.7127 | 6.2377 | 6.1377 |

The PLD's lower column is a proved lower bound, and the other columns are proved upper bounds. Adding up each step's guarantee is useless after a few steps. RDP is far closer, but it still overstates $\varepsilon$, by 14.6% on the shorter run and 7.6% on the longer one. The $\delta$ of $10^{-5}$ is only for comparison (Section 31.4). So the rule of this chapter is short. **Report the tightest upper bound whose assumptions hold for the code you ran, and give enough detail that someone else can recompute it.**

## 31.2. Which accountant

An accountant is tight, close to the true $\varepsilon$, only when what it keeps of each step fits the mechanism, so the mechanism decides (Table 31.2).

Table 31.2.: Which accountant to report for which mechanism.

| Mechanism | Report with | Avoid |
|---|---|---|
| Gaussian noise, no subsampling | Gaussian differential privacy (GDP), converted exactly (f-DP and Gaussian DP) | Per-step composition |
| DP-SGD with Poisson sampling | PLD, or the privacy loss random variable (PRV) accountant (Numerical accounting), with the PLD's lower value as the error bar (its code) | Zero-concentrated DP (zCDP); the GDP central limit theorem (CLT) |
| Pure-DP steps (Laplace, randomized response) | PLD, exact when the privacy loss takes few values; Laplace with scale $b$ is `LaplaceDpEvent(b/Δ)`, $\Delta$ its $\ell_1$ sensitivity | Advanced composition |
| Many statistical releases | A zCDP ledger (Concentrated DP) or PLD | Adding up each release's $\varepsilon$ |
| Noise or budget chosen during the run | A Rényi filter (Adaptive composition) | Plugging realized values into a fixed theorem |
| Correlated noise across steps: matrix factorization, or the follow the regularized leader (FTRL) optimizer on tree sums, DP-FTRL | The mechanism's own accountant (Matrix mechanisms) | The Poisson DP-SGD accountant |
| A shuffler of locally randomized reports, or only the last model released | Shuffling and iteration, with its conditions; a loader that shuffles is a sampler (Table 31.4) | The Poisson accountant for a shuffler; a last-model claim for non-convex training |

The CLT is missing from Table 31.1 because it is an approximation with no proof. After 1,000 steps of the same run it gives 1.6177. That is below the PLD's certified lower value of 1.8232, so a paper that quotes it claims a guarantee the run does not have. Opacus's default PRV accountant is safe, but its `"gdp"` accountant is this CLT, so never report it.

## 31.3. The inverse problem: noise for a target ε

Usually you fix the budget, batch size and epochs, and ask for the smallest $\sigma$ that meets them. A looser accountant overstates $\varepsilon$, so it asks for more noise. For a target of 3 on the longer run, RDP needs a noise multiplier of 1.662, while the PLD needs only 1.584. So calibrate with the

accountant you will report, and pass `accountant="prv"` to Opacus's `get_noise_multiplier`, whose default is `"rdp"`. Calibrate and account a DP-SGD run does this for a real run.

## 31.4. Choosing δ

Roughly, treat $\delta$ as the chance that the guarantee fails for someone, possibly badly. With $n$ people, a mechanism that publishes one randomly chosen record in full meets a $\delta$ of about $1/n$ with an $\varepsilon$ of zero, yet it exposes someone completely. So $\delta$ must sit well below $1/n$.

Following the convention of Ponomareva et al. (2023), this book chooses

$$\delta \leq \frac{1}{n^{1.1}}, \quad \text{rounded down to a power of ten.} \tag{31.1}$$

Count $n$ in the unit the claim protects, which at the user level means users, not records.

For example, put the longer run of Table 31.1 on 50,000 records, which makes it 100 epochs. Then $1/n^{1.1}$ is $6.8 \times 10^{-6}$, which rounds down to $10^{-6}$. The comparison point $10^{-5}$ is half of $1/n$, so it fails the rule. The US National Institute of Standards and Technology (NIST) recommends the stricter $1/n^2$, so report $\varepsilon$ at both values of $\delta$.

## 31.5. The statement

Ponomareva et al. (2023) and NIST ask for the items of Table 31.3. The table fills them in for the run on 50,000 records, with the full edition's PLD, whose upper value is the claim.

Table 31.3.: A filled privacy statement.

| Item | The run on 50,000 records |
|---|---|
| Setting | DP-SGD in the central model, covering every checkpoint and the final model |
| Unit and neighbours | One record per person; neighbours add or remove one record |
| Training | Poisson sampling at $q = 0.01$, 10,000 steps (100 epochs), clipping norm 1, noise multiplier 1 |
| Accountant | PLD, upper value, on the grid $10^{-5}$, with the library's version |
| Guarantee | $(6.96, 10^{-6})$ and, at $1/n^2$, $(9.06, 4.0 \times 10^{-10})$; the PLD's lower values, 6.8574 and 8.9378, show how far each could still fall |
| Other data | No pretrained weights; no preprocessing fitted on private rows; validation rows held out and not protected |

| Item | The run on 50,000 records |
|---|---|
| Not covered | Nothing: hyperparameters were tuned on a public proxy dataset, so no tuning run touched the private data |

Round an upper bound up, never to the nearest value. The PLD's upper value at $10^{-6}$ is 6.9574, so the claim is 6.96, because a lower value is one the accountant has not certified. Publish the value of the accountant you name. `dp_accounting`'s PLD gives 6.9074 on its own grid (Section 31.8), so a report made with it claims 6.91. Calibrate and account a DP-SGD run prints such a statement from code.

## 31.6. Matching the accountant to the code

An accountant trusts that the code does what its theorem assumes. Yet many papers have reported the Poisson-sampling number while training with shuffled fixed-size batches, a statement about a different algorithm (Ponomareva et al. 2023). So check each row of Table 31.4 against the code before you report.

Table 31.4.: A checklist for matching the accountant to the training code.

| Item | The accountant assumes | Check in the code | If it does not match |
|---|---|---|---|
| Sampler | Poisson sampling at rate $q$ | Do batch sizes vary from step to step? | The sampler's row of Subsampling: fixed-size batches at $\sigma/2$; shuffle and partition unamplified, for zero-out neighbours |
| Neighbours | Add or remove one record | Which relation does the claim state? | Replace-one costs more per step; for Poisson sampling, compose the same steps in `PLDAccountant(NeighboringRelati` |
| Unit | One record | Can one person give several records? | Clip and sample per user, or pay for the group (User-level DP) |

| Item | The accountant assumes | Check in the code | If it does not match |
|---|---|---|---|
| Tuning, adaptivity and steps | Settings and steps fixed in advance | Tuning on private data, early stopping, restarts? | Account for the tuning runs (Hyperparameter tuning); report the planned maximum; keep the accountant's state across restarts |
| Other data | Only the training rows are private | Pretraining, fitted preprocessing, validation rows? | Name them; preprocessing fitted on private rows needs its own budget |

The sampler costs the most. On the run above, Google's `dp_accounting` library gives 6.19 at $10^{-5}$ with its PLD. Fixed-size batches give 43.37, accounted at $\sigma/2$. Shuffle and partition gives 91.82 over the 100 epochs, and only for zero-out neighbours (Subsampling).

## 31.7. Reviewing someone else's claim

A reviewer works in this order.

1. Ask for the inputs of Table 31.3, and reject a claim that cannot be recomputed.
2. Check $\delta$ against Equation 31.1, with $n$ counted in the claim's unit.
3. Recompute $\varepsilon$ with a different library, and expect agreement within a few percent. A PLD accountant's `get_delta` at the claimed $\varepsilon$ gives the $\delta$ at which it holds. A claim well above your tight value is valid but loose, and was usually calibrated with Opacus's default RDP accountant.
4. Read the training code against Table 31.4, starting with the sampler.
5. Ask what the claim leaves out, such as tuning runs, pretraining data and preprocessing fitted on the private rows.

Release and governance runs these steps on a made-up claim and lists what fails.

For example, a team claims $(2, 10^{-5})$-DP for a model trained on one million users. Its batches hold 1,000 users, so $q = 0.001$ and 10 epochs are 10,000 steps, with a noise multiplier of 0.65 and a loader that shuffles and takes fixed batches. This snippet does steps 2 and 3, then accounts for the loader as it ran.

```
from math import floor, log10

from dp_accounting import GaussianDpEvent, PoissonSampledDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant

n, q, sigma, steps, epochs = 1_000_000, 0.001, 0.65, 10_000, 10  # the claim's parameters
delta = 10.0 ** floor(log10(n**-1.1))  # the book's rule for one million users
```

```
print(f"{delta:.0e}")  # 1e-07
poisson = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
pld = PLDAccountant(value_discretization_interval=1e-4).compose(poisson)
print(f"{pld.get_epsilon(1e-5):.2f}")  # 1.72
print(f"{pld.get_delta(2.0):.1e}")  # 3.0e-06
print(f"{pld.get_epsilon(delta):.2f}")  # 2.84
# What the loader ran: shuffle and partition, zero-out neighbours, no amplification
shuffled = SelfComposedDpEvent(GaussianDpEvent(sigma), epochs)
print(f"{PLDAccountant(value_discretization_interval=1e-4).compose(shuffled).get_epsilon(del
```

The arithmetic holds up, because the claim's own parameters give 1.72 at its $\delta$. Its $\delta$ is the problem, since it is a hundred times the rule's $10^{-7}$, at which the same run gives 2.84. The loader then voids both numbers, because the Poisson accountant assumed a sampler that the code never ran. What the code did run is certified only at 36.45, and only for zero-out neighbours, so the review asks for a Poisson loader and a $\delta$ of $10^{-7}$.

## 31.8. Checking an accountant

An accountant is code, so test it on known answers. Setting $q = 1$ turns each step into a plain Gaussian mechanism whose composition is exact (f-DP and Gaussian DP). Ten steps at $\sigma = 1$ are then exactly $(17.8566, 10^{-5})$-DP. Any upper bound must lie above that, and a PLD bracket on the same steps must contain it. Also, $\varepsilon$ must never fall as the steps grow or rise as the noise grows, and no upper bound may fall below the PLD's lower end.

Then compare two maintained libraries, and pin their versions. This snippet recomputes the run of Table 31.1 with Google's `dp_accounting`.

```
import dp_accounting
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

step = dp_accounting.PoissonSampledDpEvent(0.01, dp_accounting.GaussianDpEvent(1.0))
run = dp_accounting.SelfComposedDpEvent(step, 10_000)  # q = 0.01, sigma = 1, 10,000 steps
pld = PLDAccountant(value_discretization_interval=1e-4).compose(run)
rdp = RdpAccountant(orders=list(range(2, 65))).compose(run)  # integer orders only
print(f"{pld.get_epsilon(1e-5):.4f} {rdp.get_epsilon(1e-5):.4f}")  # 6.1877 6.7194
print(f"{pld.get_epsilon(1e-6):.4f} {rdp.get_epsilon(1e-6):.4f}")  # 6.9074 7.4869
```

The libraries agree. Each PLD value lies inside the full edition's bracket, and each RDP value sits just above the full edition's, which also tries fractional orders. When libraries do disagree, there are three usual causes. Two RDP accountants that differ by an amount depending only on the order, 0.3612 at order 10, use different conversion formulas from RDP to $(\varepsilon, \delta)$ (RDP conversions). Brackets that do not overlap mean a bug in one library, and values about a factor of two apart often mean different neighbouring relations.

## 31.9. Cheat sheet

Table 31.5.: Reporting and reviewing an $\varepsilon$ claim.

| Question | Default | Check |
|---|---|---|
| Accountant for DP-SGD | PLD or PRV, upper value | RDP lies above it; never report the CLT |
| $\delta$ | $1/n^{1.1}$ rounded down, $n$ counted in the claim's unit | Also report $\varepsilon$ at $1/n^2$ |
| Fixed-size batches | Poisson accountant at $\sigma/2$ (add/remove); replace-one: see Subsampling | Poisson batch sizes vary from step to step |
| Shuffle and partition | One Gaussian step per epoch, zero-out neighbours | Better, switch to Opacus's Poisson loader |

**Rules of thumb**

- Never publish $\varepsilon$ without $\delta$, the unit, the relation, the accountant and its version, and the inputs to recompute it.

## 31.10. References


- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): matching the accounting to the pipeline, and what to report.
- Near, Darais, Lefkovitz & Howarth, "Guidelines for Evaluating Differential Privacy Guarantees" (NIST SP 800-226, 2025): what an evaluator will ask of your claim.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 32. Calibrate and account a DP-SGD run

## 32.1. Why this matters

An **accountant** is the code that turns a training run's settings into its privacy level $\varepsilon$ under differential privacy (DP). Before the run, it finds the noise that meets your target, and after it, the $\varepsilon$ the run spent. Both can go wrong without any visible sign. The noise may be calibrated with one accountant and reported with another, or the data loader may not sample the way the accountant assumes. A restart may also reset the accountant's count of steps. None of these changes the loss curve, but each makes the privacy claim false.

This chapter does that accounting and trains nothing. Its run is stochastic gradient descent (SGD) with DP, called DP-SGD, on the Adult census data from the UCI Machine Learning Repository. Train a model with DP-SGD trains it. It trains on 30,000 people in batches of 1,000 for ten epochs, aiming at $\varepsilon = 4$. Each section below is one step, and the rules they apply are explained in Report and review an ε claim.

The script practitioner_calibrate.py runs all seven steps in one to two minutes, with Opacus 1.6.0 and `dp-accounting` 0.6.0. Its defaults are Part 5's run. Options set your own run: `--n`, `--batch`, `--epochs`, `--epsilon`, `--dataset`, `--unit` and `--delta`, which defaults to the rule of the next section. Only the planning table stays fixed. The second command plans ten epochs on MNIST's 60,000 images, in batches of 256, at a target of 3.

```
uv run python code/part04-privacy-accounting/practitioner_calibrate.py
uv run python code/part04-privacy-accounting/practitioner_calibrate.py --n 60000 --batch 256
```

## 32.2. Fix the privacy unit, n and δ

The first decision is what one record is, because the guarantee protects exactly that. In Adult each row is one person, so protecting a row protects a person. If one person could own many rows, the number of records $n$ would have to count people, not rows, and training would need the user-level methods of User-level differential privacy.

The slack $\delta$ is roughly the small chance that the guarantee does not hold. The book's rule sets it from $n$ as $\delta \leq 1/n^{1.1}$, rounded down to a power of ten, which keeps it well below $1/n$ (Choosing δ).

```
def delta_for(n: int) -> float:
    """The book's default delta: at most 1/n^1.1, rounded down to a power of ten."""
    return 10.0 ** math.floor(math.log10(n**-1.1))
```

```
Adult rows: 48,842; Part 5's training split: n=30,000
delta_for(30,000) = 1e-05
delta_for(60,000) = 1e-06
delta_for(569) = 1e-04
```

Here $n$ counts the people of the training split, not the whole dataset, and the rule gives $10^{-5}$. Set the validation rows aside first: they are not trained on, so they are neither in $n$ nor protected. The other two lines are for MNIST and scikit-learn's breast cancer data. Adult's $\delta$ copied to MNIST would break the rule.

## 32.3. Choose the sampler

The accountants on this page assume **Poisson sampling**, in which each record joins each batch independently with probability $q$, the sampling rate, so batches vary in size. It is the default because the accountants certify how much it hides each record, while the common alternatives cost far more (Section 32.6). With an expected batch size $B$, the rate is $q = B/n$. A run of $E$ epochs then takes $E/q$ steps, rounded down, which the output calls $T$.

```
Poisson: B=1000, E=10: q=0.033333, T=300
Opacus DPDataLoader.from_data_loader: q=1/30=0.033333, 30 steps an epoch, 300 in 10 epochs
chosen run uses the loader's q and steps; get_noise_multiplier at that q counts int(E/q)=300
```

The second line is what training will do. Opacus replaces your loader with a Poisson `DPDataLoader`. Its rate is one over the number of batches in the original loader, $\lceil n/B \rceil$, and it runs that many steps per epoch. Here the batch size divides $n$, so the first two lines agree. When it does not, they differ slightly, and the loader's count is what training runs. So the script takes $q$ and $T$ from the loader, and the third line checks that Opacus's calibration counts the same steps. This step has gone wrong if the training code builds its own loader with `shuffle=True` and a fixed `batch_size`, which the Poisson accountant does not describe. Batches that all have the same size in the logs are the sign (Amplification by subsampling).

## 32.4. Plan the noise

Next come the batch size, the epochs and the noise multiplier $\sigma$. Training clips each record's gradient to a length of at most $C$, the clipping norm, and $\sigma$ is the noise's standard deviation as a multiple of $C$. With the target and the sampler fixed, an accountant finds the smallest $\sigma$ that meets them, so calibrate with the accountant you will report (The inverse problem). The script uses two libraries that check each other, both with tight accountants (Numerical accounting). Opacus's `get_noise_multiplier`, from `opacus.accountants.utils`, searches with its privacy loss random variable (PRV) accountant and stops within 0.01 of the target $\varepsilon$. Google's `dp_accounting` searches with its privacy loss distribution (PLD) accountant and stops within 0.01 in $\sigma$.

```
def sigma_opacus(epsilon: float, delta: float, q: float, epochs: int) -> float:
    """Opacus: the sigma whose PRV epsilon after `epochs` is at most the target (to 0.01 in
    return get_noise_multiplier(
        target_epsilon=epsilon, target_delta=delta, sample_rate=q, epochs=epochs, accountant
    )


def sigma_dp_accounting(epsilon: float, delta: float, q: float, steps: int) -> float:
    """dp_accounting: the same search with a PLD accountant (to 0.01 in sigma)."""

    def training_run(sigma: float) -> DpEvent:
        return SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)

    def accountant() -> PLDAccountant:
        return PLDAccountant(value_discretization_interval=PLD_GRID)

    return calibrate_dp_mechanism(accountant, training_run, epsilon, delta, LowerEndpointAnd
```

Pass `accountant="prv"` explicitly, because `get_noise_multiplier` defaults to the Rényi differential privacy (RDP) accountant, a valid but looser bound that asks for more noise. In `dp_accounting`, `LowerEndpointAndGuess(0.3, 1.0)` starts the search at a lower end of 0.3, so lower that number if the call raises an error.

The table uses a PLD grid of $10^{-3}$, the step to which the accountant rounds the privacy loss, which keeps each search near a second. Its last column is $\sigma/B$ times 1,000. Averaging over a batch of $B$ divides the noise on the gradient sum by $B$, so that column is the noise on the averaged gradient per unit of clipping norm.

| dataset | n | delta | B | E | eps | T | sigma PRV | sigma PLD | sigma/B x1000 |
|---|---|---|---|---|---|---|---|---|---|
| Adult | 30000 | 1e-05 | 100 | 10 | 4 | 3000 | 0.62 | 0.62 | 6.16 |
| Adult | 30000 | 1e-05 | 300 | 10 | 4 | 1000 | 0.74 | 0.74 | 2.45 |
| Adult | 30000 | 1e-05 | 1000 | 10 | 4 | 300 | 0.97 | 0.98 | 0.97 |
| Adult | 30000 | 1e-05 | 1000 | 10 | 8 | 300 | 0.72 | 0.73 | 0.72 |
| Adult | 30000 | 1e-05 | 1000 | 20 | 4 | 600 | 1.16 | 1.16 | 1.16 |

The block shows 5 of the script's 18 Adult cells. Read it along the batch size first. At $\varepsilon = 4$ and ten epochs, $\sigma$ grows from 0.62 at a batch of 100 to 0.97 at 1,000. Yet the noise on the averaged gradient falls from 6.16 to 0.97, because a larger batch divides the noise among more records. At that batch, twenty epochs raise $\sigma$ from 0.97 to 1.16.

The table cannot choose the batch, because the noise on the average keeps falling as the batch grows, and so does the number of steps. So learn the epochs and steps your model needs from a non-private run or public data. Then take the largest batch that leaves those steps and fits in memory (Batch size at a fixed budget). Compare cells by $\sigma/B$, never by $\sigma$ alone, since a smaller batch has the smaller $\sigma$ but the noisier average. The run takes the largest batch with ten epochs at $\varepsilon = 4$, which is the cell Part 5 trains.

The size of the data changes the picture, as a row from the end of the table shows.

```
breast cancer     569  1e-04    64  20    1    177       4.96       4.91          77.51
largest gap between PRV and PLD sigma: 1.6% (Adult, B=300, E=10, eps=8)
```

With 569 records, a batch of 64 is a large share of the data, so sampling amplifies little. At $\varepsilon = 1$ the noise on the averaged gradient is 77.51, so small datasets need a large $\varepsilon$, public pretraining, or another method.

The two libraries also check each other. Across all 24 cells their $\sigma$ differ by at most 1.6%. A gap above about 3% points to a call given different inputs. Look for a batch size passed as a rate, epochs passed as steps, or a different $\delta$.

## 32.5. Check ε with a second accountant

Train with the $\sigma$ the calibration returned, here Opacus's 0.9729, unrounded, as Part 5 does. If you round it, round up at the precision the statement prints, three decimals, and never down, since less noise means a larger $\varepsilon$. Before training, compute the planned run's $\varepsilon$ with every accountant at hand, three from Opacus and two from `dp_accounting`.

```
def crosscheck_epsilon(sigma: float, q: float, steps: int, delta: float) -> dict[str, float]
    """Epsilon of one planned run by five accountants (GDP is the trap: it is not an upper b
    epsilons: dict[str, float] = {}
    for name, opacus_accountant in [("Opacus RDP", RDPAccountant()), ("Opacus PRV", PRVAccou
        opacus_accountant.history = [(sigma, q, steps)]  # what `steps` calls of .step() wou
        epsilons[name] = opacus_accountant.get_epsilon(delta)
    run = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
    epsilons["dp_accounting RDP"] = RdpAccountant().compose(run).get_epsilon(delta)
    epsilons["dp_accounting PLD"] = PLDAccountant().compose(run).get_epsilon(delta)
    gdp = GaussianAccountant()
    gdp.history = [(sigma, q, steps)]
    epsilons["Opacus GDP"] = gdp.get_epsilon(delta)
    return epsilons
```

```
chosen cell: Adult, B=1000, E=10, eps=4: sigma=0.972900 (Opacus PRV, as Part 5)
Opacus RDP         eps=4.50
Opacus PRV         eps=3.99
dp_accounting RDP  eps=4.50
dp_accounting PLD  eps=3.98
Opacus GDP         eps=3.34
PLD <= target: True; RDP >= PLD: True; |PRV - PLD|/PLD < 3%: True
crosscheck: pass
PLD bracket (grid 1e-4): lower 3.9656 (optimistic), upper 3.9806 (pessimistic); width 0.0150
```

PRV and the PLD, the two tight accountants, agree at 3.99 and 3.98, and both meet the target. The two RDP implementations agree with each other at 4.50, above the target. That is expected of RDP, a valid but looser bound. Opacus may warn that an RDP order failed to converge, or

that the best order is the largest it tried. Both concern only the RDP value, which you do not report. The Gaussian differential privacy (GDP) value of 3.34 is the trap. It comes from a central limit theorem (CLT), which approximates the run rather than bounding it, and here it lands below the certified values, so never report it.

The pass line runs three checks. The PLD value must be at most the target, which fails if $\sigma$ was rounded down or the steps differ from the plan. RDP should lie at or above the PLD, give or take the PLD's small grid error. And PRV and PLD must agree within 3%. A larger gap in either of the last two means the accountants were given different runs.

The last line is the PLD's error bar. The PLD rounds each step's privacy loss up to its grid for the value it reports, 3.9806. Rounding down gives an optimistic 3.9656, and the true $\varepsilon$ lies between them. The claim is the upper value, and the lower one is reported next to it as the error bar (Which accountant). The lower value comes from `privacy_loss_distribution.from_gaussian_mechanism` with `pessimistic_estimate=False` and `use_connect_dots=False`, composed over the steps. The bracket widens as the steps grow, so a long run needs a finer grid to keep it narrow.

## 32.6. If the loader is not Poisson

Not every training loop samples by the Poisson scheme, so price the alternatives before you train. The script builds the same four `dp_accounting` events as the snippet of Amplification by subsampling, for the chosen run and with the steps its loader runs.

```
Poisson (PLD)                                eps=3.98
fixed size, add/remove, sigma/2 (PLD)        eps=23.03
fixed size, replace-one (RDP)                eps=52.60
shuffle, zero-out, no amplification (PLD)    eps=18.51
PLDAccountant supports sampling without replacement: add/remove False, replace-one False
```

Under add/remove neighbours, fixed-size random batches are covered by the Poisson accountant at half the noise multiplier. A record added to a full batch pushes another out, so the sum can move by up to $2C$. On this run that turns 3.98 into 23.03. Under replace-one neighbours the sum also moves by up to $2C$, so these batches are accounted at $\sigma/2$ too. Only the RDP accountant handles them, as the last line shows. They give 52.60, but that number protects a record's value rather than its presence, so it cannot be compared with the add/remove values.

Shuffling once per epoch and cutting the data into batches gets no amplification, so all that is certain is one Gaussian step per epoch, 18.51 here. It holds for zero-out neighbours, which differ by one record's gradient being replaced by zero. Under zero-out a step's clipped sum moves by at most $C$, as under add/remove, so this value does compare with the add/remove ones. Name the relation in the statement. It is below 23.03 here, but on the 100-epoch run of Report and review an $\varepsilon$ claim the order reverses. So the fix for a non-Poisson loader is the loader, not the accountant: keep the Poisson loader that Opacus's `make_private` installs by default.

## 32.7. Train, stop early and resume

During training, Opacus's optimizer calls `accountant.step` once for every noisy update. That is one call per logical batch, even when `BatchMemoryManager` splits it into smaller physical ones. The script simulates the run's 300 calls and asks for $\varepsilon$ at the end of each epoch.

```
def train_epochs(
    accountant: PRVAccountant, sigma: float, q: float, steps_per_epoch: int, epochs: int, de
) -> list[float]:
    """Step the accountant once per logical batch for `epochs` epochs; epsilon at the end of
    epsilons: list[float] = []
    for _ in range(epochs):
        for _ in range(steps_per_epoch):
            accountant.step(noise_multiplier=sigma, sample_rate=q)  # what the optimizer's s
        epsilons.append(float(accountant.get_epsilon(delta)))
    return epsilons
```

Suppose the validation loss stalls and the run stops after six of its ten planned epochs. The $\varepsilon$ to report is still the planned maximum. Stopping early within a maximum fixed in advance is safe, because the stopped run releases a prefix of the planned run, which the planned guarantee covers. The $\varepsilon$ at the stopping point is not a valid claim, because the decision to stop depended on what the run released. Only a privacy filter that ran from the first step could certify a smaller number (Adaptive composition), and this book runs none for subsampled steps. Log $\varepsilon$ every epoch anyway, because it must never decrease, and a drop means the accountant was reset.

Training past the maximum is the opposite case. If you decide after seeing the model to add epochs, the number of steps depends on what the run released. The longer run's $\varepsilon$, computed afterwards, is then the same plug-in mistake. An extension needs a new plan, accounted from the first step. If you might want more epochs, plan the longest run you could train before training starts, and treat stopping sooner as an early stop.

A job that stops and restarts must carry the accountant with it. Save its `state_dict` with the model and load it before the first new step, as Opacus's `PrivacyEngine.save_checkpoint` and `load_checkpoint` do.

```
state = accountant.state_dict()  # saved with the model; PrivacyEngine.save_checkpoint store
# ... the job stops, and later restarts ...
restarted = PRVAccountant()
restarted.load_state_dict(state)  # without this, epsilon would restart from zero
resumed = train_epochs(restarted, sigma, q, steps_per_epoch, epochs - stop, delta)
steps_recorded = sum(steps for _, _, steps in restarted.history)
```

```
steps per epoch: 30; planned: 10 epochs = 300 steps
epoch  1: eps=1.70
epoch  2: eps=2.09
epoch  4: eps=2.69
epoch  6: eps=3.18
```

```
epsilon never decreases over 10 epochs: True
stopped early at epoch 6: eps=3.18; report the planned maximum: eps=3.99
after resuming at epoch 6: eps at epoch 10=3.99; uninterrupted run: eps=3.99; equal: True
steps recorded: 300; planned (the loader's 10 x 30): 300; equal: True
```

The resumed run ends at 3.99, the same as a run that never stopped. Its history holds all 300 steps, the loader's 30 an epoch for ten epochs. Had the restart created a fresh accountant, it would have counted only the last four epochs and reported 2.69, the epoch-4 value above.

## 32.8. Write the statement

The last step writes the claim, with every input a reviewer needs to recompute it. The script fills the template from the run's own parameters, reads $\varepsilon$ from the PLD at the rule's $\delta$ and at $1/n^2$, and rounds both up.

```
def privacy_statement(
    unit: str, dataset: str, n: int, batch: int, epochs: int, sigma: float, clip: float, del
) -> str:
    """The filled template of the reporting page: every input a reviewer needs to recompute
    q, _, steps = opacus_loader_plan(n, batch, epochs)  # the steps the Poisson loader runs
    run = SelfComposedDpEvent(PoissonSampledDpEvent(q, GaussianDpEvent(sigma)), steps)
    epsilon = round_up(PLDAccountant().compose(run).get_epsilon(delta))
    epsilon_tiny_delta = round_up(PLDAccountant().compose(run).get_epsilon(1 / n**2))
    return f"""\
Unit: one {unit} (a row of {dataset}); neighbouring datasets differ by adding or removing on
Data: n = {n:,}; Poisson sampling with q = {q:.6f} (B = {batch}); T = {steps:,} steps ({epoc
Mechanism: DP-SGD, clipping norm C = {clip}, noise multiplier sigma = {sigma:.3f}
Accountant: PLD, dp-accounting {version("dp-accounting")}; {calibration}
Guarantee: (epsilon = {epsilon:.2f}, delta = {delta:.0e}); epsilon = {epsilon_tiny_delta:.2f
Not covered: hyperparameter tuning runs (see Part 5), and any model released before this one
```

```
Unit: one person (a row of Adult); neighbouring datasets differ by adding or removing one pe
Data: n = 30,000; Poisson sampling with q = 0.033333 (B = 1000); T = 300 steps (10 epochs)
Mechanism: DP-SGD, clipping norm C = 1.0, noise multiplier sigma = 0.973
Accountant: PLD, dp-accounting 0.6.0; calibrated with Opacus 1.6.0 (PRV)
Guarantee: (epsilon = 3.99, delta = 1e-05); epsilon = 6.30 at delta = 1/n^2 = 1.1e-09
Not covered: hyperparameter tuning runs (see Part 5), and any model released before this one
```

The guarantee reads 3.99 because the PLD value of 3.98 was rounded up, never to the nearest value. At the stricter $\delta = 1/n^2$, the same run is 6.30. The accountant never sees the unit, the dataset or the clipping norm, so they come from the options `--unit`, `--dataset` and `--clip`. The last line matters most to a reviewer, because it names privacy spent outside the $\varepsilon$ above. Tuning on the private data adds its own cost (Hyperparameter tuning), and so does every earlier model trained on the same people.

A published statement adds what the script cannot know (The statement). It gives the setting, here the central model covering every checkpoint, and the PLD's grid, `dp_accounting`'s default of $10^{-4}$, with the bracket of step 4. It also names the data outside the run. That means any pretrained weights and their training data, any preprocessing fitted on the private rows, and the validation rows, which are not protected. Release and governance builds the release package around it.

## 32.9. Checklist

Table 32.1.: The steps of calibrating and accounting a DP-SGD run.

| Step | Default | Check |
|---|---|---|
| Unit, $n$ and $\delta$ | One person; $n$ from the training split, validation rows excluded; $\delta \leq 1/n^{1.1}$, rounded down | $n$ counts people, not rows; $\delta$ recomputed for each dataset |
| Sampler | Poisson; $q$ and $T$ from Opacus's loader, $\lceil n/B \rceil$ steps an epoch | Calibration counts the same steps; batch sizes vary |
| Plan the noise | Calibrate with PRV or PLD, the accountant you report | Two libraries agree within about 3% |
| Cross-check | Train at the calibrated $\sigma$, or round it up | PLD at most the target; RDP above PLD; GDP never reported; PLD bracket reported |
| Other samplers | Keep Opacus's Poisson loader | No `shuffle=True` loader with fixed batches in the training code |
| Train and resume | One accountant step per logical batch; save the accountant's state; any extension inside the plan | $\varepsilon$ never decreases; steps recorded equal the loader's steps |
| Statement | $\varepsilon$ at the planned maximum, rounded up, at $\delta$ and $1/n^2$ | Every input to recompute it; tuning, earlier models, pretraining, preprocessing and validation named |

## 32.10. References


- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): planning a run's privacy budget, and what to report.

- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): why fixed-size and shuffled batches need their own accounting.

**Derivations.** The results this workflow relies on are derived in the full edition's chapters Amplification by subsampling in the full edition, Numerical accounting in the full edition and Choosing an accountant in the full edition.

# 33. Amplification by subsampling

## 33.1. Why this matters

Most private models are trained with stochastic gradient descent (SGD) made to satisfy differential privacy (DP), a method called DP-SGD. Each step clips every record's gradient to a maximum length, the **clipping norm**, sums the clipped gradients and adds Gaussian noise. The noise has a standard deviation equal to the clipping norm times the **noise multiplier** $\sigma$. Suppose every record paid the full cost of a Gaussian mechanism at every step. Then ten thousand steps with $\sigma = 1$ would have a privacy level $\varepsilon$ of 5425.5 at $\delta = 10^{-5}$ (Rényi DP), which is no guarantee at all.

DP-SGD escapes this cost because each step looks only at a small random batch. Write $q$ for the **sampling rate**, the probability that a given record lands in the batch. An attacker must then first guess whether the record was used at all, and that uncertainty is worth a lot. This effect is called **amplification by subsampling**, and it depends on exactly how the batch was drawn. So the accountant, the code that adds up a run's privacy cost, must assume the sampler that the training code actually runs. Getting that wrong is the largest single error in DP-SGD accounting, far larger than the difference between one accountant and another (Section 33.4).

## 33.2. One step under Poisson sampling

The simplest sampler is **Poisson sampling**, in which each record enters the batch by its own coin flip, with probability $q$, independently of every other record. It goes with **add/remove** neighbours, two datasets of which one has an extra record. For that pairing, Balle, Barthe and Gaboardi (2018) give the best possible result for one step. As in Part 3, $\delta$ is the guarantee's small additive slack. If the step is $(\varepsilon, \delta)$-DP when it sees every record, then the same step run on a Poisson sample is $(\varepsilon', \delta')$-DP with

$$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right), \qquad \delta' = q\delta. \tag{33.1}$$

The formula behaves very differently for small and large $\varepsilon$. For small $\varepsilon$ the amplified cost is about $q\varepsilon$, so a 1% sample makes a step about a hundred times cheaper. With that sample, for example, a 1-DP step costs 0.0170. For large $\varepsilon$, though, the amplified cost is about

$$\varepsilon' \approx \varepsilon - \ln(1/q), \tag{33.2}$$

so sampling only subtracts a fixed amount. That is why a 10-DP step on the same sample still costs 5.3993. The switch happens where $\varepsilon$ is near $\ln(1/q)$. So amplification helps most when each step is already fairly private, which is why DP-SGD takes many cheap steps.

The result has two conditions. Each record's coin must be independent of the others, so that every record is drawn with probability exactly $q$ whatever else is in the batch. And the batch must stay secret, because an attacker who sees the list of records drawn, for example in a readable log, knows whether the target was used.

## 33.3. Match the sampler to the accountant

Most training code does not draw Poisson batches. Some code draws a fresh random batch of a fixed size $B$ from the $n$ records at each step. Much more code shuffles the data once per epoch and cuts the list into consecutive batches, which is called **shuffle and partition**.

Each sampler needs its own accounting under each neighbouring relation. Under **replace-one**, the datasets have the same size and differ in one record's value, and under **zero-out**, the target's gradient is set to zero. Table 33.1 gives the accounting for DP-SGD. It writes each step as an event of Google's `dp_accounting` library, which a Rényi differential privacy (RDP) or privacy loss distribution (PLD) accountant then composes.

Table 33.1.: What to account for under each sampler and neighbouring relation in DP-SGD.

| Sampler | Neighbours | What to account for | `dp_accounting` step |
|---|---|---|---|
| Poisson, rate $q$ | add/remove | A Poisson accountant with $q$ and $\sigma$ | `PoissonSampledDpEvent(q, GaussianDpEvent(σ))` |
| Poisson, rate $q$ | replace-one | The same event, at $\sigma$, in a PLD accountant built with `NeighboringRelation.REPLACE_ONE`, which models the swap itself | `PoissonSampledDpEvent(q, GaussianDpEvent(σ))` |
| Fixed size $B$ of $n$ | add/remove | A Poisson accountant with rate $B/n$ and noise multiplier $\sigma/2$ | `PoissonSampledDpEvent(B/n, GaussianDpEvent(σ/2))` |
| Fixed size $B$ of $n$ | replace-one | Sampling without replacement at noise multiplier $\sigma/2$, in the RDP accountant only | `SampledWithoutReplacementDpEver B, GaussianDpEvent(σ/2))` |
| Shuffle and partition | zero-out | No amplification: each epoch is one full Gaussian release | `GaussianDpEvent(σ)` once per epoch |

The factor of two for fixed-size batches comes from the gradient sum. Under Poisson sampling, adding or removing a record moves the clipped sum by at most the clipping norm. A fixed-size batch has no room for an extra record, so a drawn record takes another's place, and the sum can move by twice as much (Lebeda et al. 2025). Twice the shift with the same noise is the same as

the original shift with half the noise multiplier. Replace-one swaps a record by definition, so its shift doubles too, but the Poisson accountant does not cover that relation. The RDP accountant of `dp_accounting` does, given the replace-one relation and the halved noise multiplier.

Shuffle and partition is worse, because the source of amplification is missing. Within an epoch the record is in exactly one batch and is never absent, so the attacker only has to find which batch held it. Chua et al. (2024) build a dataset in which the target's gradient points against all the others, so its batch's sum stands out. Take this part's running example, with a 1% sampling rate and $\sigma = 1$, under zero-out at $\delta = 10^{-5}$. On that dataset, one shuffled epoch provably has an $\varepsilon$ of at least 4.06, while a Poisson accountant reports at most 1.21. So a Poisson number reported for shuffled training can be provably false. The honest bound for shuffled training is the last row of the table, one full Gaussian release per epoch, since each epoch uses the record exactly once.

So read the data loader before you trust a reported $\varepsilon$. A PyTorch `DataLoader` with `shuffle=True` and a `batch_size` is shuffle and partition. Opacus replaces the loader with a Poisson one by default, since `PrivacyEngine.make_private` takes `poisson_sampling=True` unless you turn it off. You can also convert a loader by hand with `opacus.data_loader.DPDataLoader.from_data_loader`. Either way, the loader's `batch_sampler` is then a `UniformWithReplacementSampler`, which despite its name flips one coin per record. So its batches vary in size from step to step, a quick check in the training log. When the largest batches do not fit in memory, Opacus's `BatchMemoryManager` splits each one into pieces that fit. The optimizer still takes one noisy step, and the accountant counts one step, per sampled batch (Train a model with DP-SGD). If you cannot use Poisson sampling, report under the matching row of Table 33.1, and name the sampler next to $\varepsilon$.

## 33.4. Accounting each sampler

DP-SGD is never accounted step by step with Equation 33.1. That route splits $\delta$ into a tiny share per step, and a tiny $\delta$ forces a large per-step $\varepsilon$, which sampling barely reduces. On the running example's 10,000 steps it gives 24379.55 with advanced composition, worse even than 5425.5 without sampling. An RDP accountant instead adds the steps' Rényi curves order by order and converts the total to $(\varepsilon, \delta)$ once, which gives 6.71 (Rényi DP). A PLD accountant is tighter still (Numerical accounting).

The snippet below accounts the running example under the four rows of Table 33.1, with a training set of $n = 50{,}000$ records and batches of 500. It uses $\delta = 10^{-5}$, the value at which this part compares accountants. A report for 50,000 records would use the book's rule, $\delta \le 1/n^{1.1}$ rounded down to a power of ten, which gives $10^{-6}$ (Report and review an ε claim).

```
from dp_accounting import GaussianDpEvent, NeighboringRelation, PoissonSampledDpEvent
from dp_accounting import SampledWithoutReplacementDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

n, batch, sigma, steps, delta = 50_000, 500, 1.0, 10_000, 1e-5  # q = 0.01, 100 epochs
poisson = SelfComposedDpEvent(PoissonSampledDpEvent(batch / n, GaussianDpEvent(sigma)), step
fixed_size = SelfComposedDpEvent(PoissonSampledDpEvent(batch / n, GaussianDpEvent(sigma / 2)
shuffled = SelfComposedDpEvent(GaussianDpEvent(sigma), 100)  # one Gaussian release per epoc
```

```
pld = [PLDAccountant(value_discretization_interval=1e-3).compose(e) for e in (poisson, fixed
print(" ".join(f"{accountant.get_epsilon(delta):.2f}" for accountant in pld))  # 6.19 43.37
replace_one = RdpAccountant(neighboring_relation=NeighboringRelation.REPLACE_ONE)
swap = SampledWithoutReplacementDpEvent(n, batch, GaussianDpEvent(sigma / 2))  # replace-one
replace_one.compose(SelfComposedDpEvent(swap, steps))
print(f"{replace_one.get_epsilon(delta):.2f}")  # 118.73
```

Poisson sampling costs 6.19. The same noise with fixed-size batches costs 43.37 under add/remove, about seven times more, and shuffling with no amplification costs 91.82 under zero-out. The replace-one value, 118.73, protects a different thing, the value of a record rather than its presence, so compare it only with other replace-one numbers. The sampler moves $\varepsilon$ by a factor of seven or more. On this run the accountant moves it by less than 10%, from 6.71 by RDP to 6.19 by PLD, but the gap is wider on shorter runs, 14.6% at 1,000 steps (Report and review an ε claim).

Two more checks belong here. `PLDAccountant().supports(event)` returns `False` for a `SampledWithoutReplacementDpEvent` under either relation, so call `supports` before you compose an unusual event. And accountants use an exact curve for the subsampled Gaussian but looser general bounds for other mechanisms (Zhu & Wang 2019), so a subsampled step of another mechanism carries more slack.

## 33.5. Batch size, epochs and noise

Amplification also shapes how you plan a run. Write $E$ for the number of epochs, so that a run takes $E/q$ steps. For a small sampling rate and a large noise multiplier, the run's Rényi cost at an order $\alpha$ is roughly

$$\varepsilon(\alpha) \approx \frac{E\, q\, \alpha}{2\sigma^2}. \tag{33.3}$$

Use this only for planning, never as the reported number, because at high orders the true curve rises far above it (from about order 9 at $\sigma = 1$ and $q = 0.01$). It says that at a fixed number of epochs, a smaller sampling rate costs less privacy. Each step costs about $q^2$, and the number of steps grows only as $1/q$.

Privacy is not the only cost, though, because the noise lands on the averaged gradient. With $C$ for the clipping norm, the noise on the sum of a batch of $B$ records has standard deviation $\sigma C$, so the noise on their average has $\sigma C/B$. So compare plans by $\sigma/B$, not by $\sigma$. The planning table of Calibrate and account a DP-SGD run shows the trade-off on the 30,000 training rows of the Adult census data. For $\varepsilon = 4$ over 10 epochs, a batch of 100 needs $\sigma = 0.62$ and a batch of 1,000 needs $\sigma = 0.97$. Yet the larger batch trains with far less noise, since $\sigma/B$, which the table lists times 1,000, falls from 6.16 to 0.97.

Plan in this order, then. First fix the batch size $B$ and the number of epochs $E$ that your accuracy needs. They set the sampling rate $q$, which is $B/n$, and the number of steps $E/q$. Here $n$ is the size of the dataset you actually train on, after any filtering, because a wrong $n$ gives a wrong $\varepsilon$. Then let the accountant solve for $\sigma$. Calibrate and account a DP-SGD run does this with Opacus's `get_noise_multiplier`, naming the numerical accountant because the default is RDP, and checks the result with a second accountant.

## 33.6. Cheat sheet

Table 33.2.: Which accounting to use for each sampler.

| Situation | What to use | Notes |
|---|---|---|
| One step, Poisson, add/remove | $\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$, $\delta' = q\delta$ | About $q\varepsilon$ for small $\varepsilon$ |
| DP-SGD, Poisson loader | PLD or RDP accountant with $q$ and $\sigma$ | 6.19 by PLD for the running example |
| DP-SGD, fixed-size batches, add/remove | Poisson accountant with $B/n$ and $\sigma/2$ | 43.37, about seven times the Poisson value |
| DP-SGD, fixed-size batches, replace-one | `SampledWithoutReplacementDpEvent` at $\sigma/2$ in the RDP accountant | Not comparable with add/remove numbers |
| DP-SGD, shuffle and partition | One Gaussian release per epoch, under zero-out | A Poisson number does not apply; better, switch the loader to Poisson sampling |

**Rules of thumb**

- Account for the sampler you run, and state the neighbouring relation next to $\varepsilon$.
- Check that the loader's batches vary in size. Constant batch sizes mean it is not Poisson sampling.

## 33.7. References


- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018): amplification for each sampler and neighbouring relation.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): fixed-size batches and other accounting pitfalls.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 34. Rényi differential privacy

## 34.1. Why this matters

Training a model runs thousands of private steps, and you report one differential privacy (DP) guarantee for all of them. It is a privacy level $\varepsilon$ with a small additive slack $\delta$. The tools of Part 3 turn each step into one such pair and then add the pairs up with advanced composition (Composition). That route is far too pessimistic. As an illustration, take 1000 steps of full-batch noisy gradient descent. Each step adds Gaussian noise to a gradient, with a standard deviation 20 times the most one record can move it. At $\delta = 10^{-5}$, advanced composition gives this run $\varepsilon = 68.01$, while its exact value is 7.51 (Section 34.4 shows how). So the bound is nine times too large, and a run calibrated to it adds more noise than it needs.

**Rényi differential privacy (RDP)** fixes this by keeping a whole curve of privacy costs for each step instead of one point. The curves of the steps simply add, with no $\delta$ spent along the way, and you convert the total into $(\varepsilon, \delta)$ once, at the end. Even with Mironov's original conversion (Section 34.6), the same run then costs 8.84, only 18% above the exact value. RDP is what the RDP accountants of Opacus, for PyTorch, and of Google's `dp_accounting` track.

## 34.2. The RDP curve

A DP guarantee compares the outputs of a mechanism on two neighbouring datasets, which differ in one person's record. RDP measures how far apart those two output distributions are with the **Rényi divergence** of Divergences. That divergence has a dial, called the **order** and written $\alpha$, which is any number above 1. A small order looks at the typical output, and a large order weighs more heavily the rare outputs that are much likelier on one dataset than on the other.

A mechanism is $(\alpha, \varepsilon)$**-RDP** if, at order $\alpha$, the divergence is at most $\varepsilon$ for every pair of neighbours. The smallest such $\varepsilon$ at each order traces out the mechanism's **RDP curve** $\varepsilon(\alpha)$, and a higher curve means less privacy. The curve never decreases with the order. At order infinity it is the largest privacy loss of any output, which is what pure DP, the case $\delta = 0$, bounds. So an $\varepsilon$-DP mechanism is $(\alpha, \varepsilon)$-RDP at every order. You never compute the divergence yourself, because accountants store the curve's values on a grid of orders.

## 34.3. RDP curves of the basic mechanisms

To account for a mechanism you need its curve, and the basic mechanisms have one in closed form (Mironov 2017). The one you meet most is the **Gaussian mechanism**. Write $\sigma$ for its

noise multiplier, the noise standard deviation in units of the $\ell_2$ sensitivity, which is the most one record can move the answer. Its curve is the straight line

$$\varepsilon(\alpha) = \frac{\alpha}{2\sigma^2}. \tag{34.1}$$

Table 34.1 gives the closed-form curves of the Gaussian, Laplace and randomized-response mechanisms at a few orders, each at sensitivity 1.

Table 34.1.: RDP curves of the three basic mechanisms with sensitivity 1.

| Mechanism | $\varepsilon(1.5)$ | $\varepsilon(2)$ | $\varepsilon(10)$ | $\varepsilon(64)$ | $\varepsilon(\infty)$ |
|---|---|---|---|---|---|
| Gaussian, $\sigma = 1$ | 0.7500 | 1.0000 | 5.0000 | 32.0000 | $\infty$ |
| Laplace, scale $b = 1$ | 0.5129 | 0.6191 | 0.9287 | 0.9891 | 1 |
| Randomized response, which reports a bit truthfully with probability 0.75 | 0.7340 | 0.8473 | 1.0666 | 1.0940 | 1.0986 |

The Laplace and randomized-response curves level off at their pure DP levels. The Gaussian curve grows without limit, so the Gaussian mechanism is not $\varepsilon$-DP for any $\varepsilon$. But it is RDP at every finite order, and an accountant only ever converts at finite orders. These curves are for steps that use the whole dataset. The private form of stochastic gradient descent (SGD), called DP-SGD, uses a random batch at each step. Libraries compute the much smaller curves of its steps for you (Amplification by subsampling).

## 34.4. Composition adds curves

Training runs thousands of mechanisms in sequence, and each may depend on what the earlier ones released, as a gradient step depends on the current model. RDP composes by adding curves order by order. Write $\varepsilon_i$ for the curve of the $i$-th step. Then a whole run of $k$ steps has a curve $\varepsilon_M$ with

$$\varepsilon_M(\alpha) \le \sum_{i=1}^{k} \varepsilon_i(\alpha) \qquad \text{for every order } \alpha. \tag{34.2}$$

Different mechanisms mix freely, because you only add numbers at the same order.

Two conditions come with the sum. First, every curve must describe the same neighbouring relation, such as add/remove or replace-one, and the same unit of privacy, such as one record or one user. For example, the RDP accountant of `dp_accounting` 0.6.0 composes a `PoissonSampledDpEvent` under its default add/remove relation. But it refuses a `RandomizedResponseDpEvent` there, since

it computes that curve only for replaced records. Second, each step's curve must be fixed in advance. Steps may adapt to earlier outputs, but the noise level and the number of steps must not be chosen from the data during the run. Otherwise the curves themselves would depend on the data. If you stop when a validation metric looks good, you need Adaptive composition instead.

For Gaussian steps the sum has a useful shortcut. Adding $k$ copies of the straight line gives the curve of a single Gaussian step with noise multiplier $\sigma/\sqrt{k}$, whose exact $\varepsilon$ you can compute. The shortcut also shows why DP-SGD needs subsampling. This part's running example is a DP-SGD run of 10,000 steps with noise multiplier 1, each on a random 1% of the data. If every step used the full dataset, the run would act like one step with noise multiplier 0.01. Its exact $\varepsilon$ at $\delta = 10^{-5}$ would be 5425.5, which is no guarantee at all.

## 34.5. Post-processing and group privacy

Post-processing is free in RDP. Any computation on the output that does not look at the data again leaves the curve no larger at any order. So releasing the trained model or its predictions costs nothing beyond the training run.

Groups are harder, because when one user contributes several records, you need a guarantee for all of them at once. RDP has a generic group rule, but it is loose, so for Gaussian noise use the group's sensitivity instead. A group of $m$ records moves the answer by at most $m$ times as much as one record, so the curve becomes

$$\varepsilon_{\text{group}}(\alpha) = \frac{m^2 \alpha}{2\sigma^2}, \tag{34.3}$$

which is $m^2$ times the single-record curve. That grows fast. So when users have many records, clip each user's whole contribution and account per user (User-level privacy), which keeps the sensitivity at one clipping norm.

## 34.6. Compose, then convert once

After composing, you hold a curve, but you report an $(\varepsilon, \delta)$ pair. A conversion turns each point of the curve into a pair, and you keep the order that gives the smallest $\varepsilon$, which libraries report as the best order. Use the **improved conversion**, which both widely used libraries apply (Balle et al. 2020, and independently Canonne, Kamath and Steinke 2020). From the curve's value $\varepsilon(\alpha)$ at one order and a target $\delta$, it gives the privacy level

$$\varepsilon' = \varepsilon(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1}. \tag{34.4}$$

You report the smallest $\varepsilon'$ over the grid, or 0 if that comes out negative.

Older papers and code use Mironov's original conversion, which gives larger numbers from the same curve, such as the 8.84 above. On the DP-SGD running example, Mironov's conversion gives 7.43 and the improved one 6.71. The exact value is lower still, as From Rényi DP to (ε, δ) and to tests shows. The **moments accountant** of the original DP-SGD paper (Abadi et al. 2016) is the same RDP curve in a rescaled form, converted with Mironov's formula. So its $\varepsilon$ is loose but not wrong.

## 34.7. Check the order grid

The best order moves with the run. Long runs with little noise want orders close to 1, while short runs with much noise want large orders. In the illustration above, it falls from 128 for one step to 4 for 1000 steps. A missing order can only raise $\varepsilon$, so the grid must cover the order the run needs.

The Opacus default list `RDPAccountant.DEFAULT_ALPHAS` stops at order 63, while the `DEFAULT_RDP_ORDERS` of `dp_accounting` go on to 1024. A best order at either end of the grid means that the true optimum lies outside it and the reported $\varepsilon$ is too large. Then pass a longer list and convert again, with `alphas=[...]` in Opacus or `orders=[...]` in `dp_accounting`. Opacus warns when the best order is at an end of its list. `dp_accounting` does not, so there you compare the returned order with the ends of the list yourself.

## 34.8. Accounting with a library

In practice you describe each step to a library and let it compose and convert. The `dp_accounting` library describes each mechanism as an event. A DP-SGD step is then a `GaussianDpEvent` inside a `PoissonSampledDpEvent`, which samples each record independently, with probability 0.01 in the running example. This snippet composes the 10,000 steps and prints $\varepsilon$ with the best order.

```
import dp_accounting

step = dp_accounting.PoissonSampledDpEvent(0.01, dp_accounting.GaussianDpEvent(1.0))
accountant = dp_accounting.rdp.RdpAccountant()
accountant.compose(step, count=10_000)
eps, order = accountant.get_epsilon_and_optimal_order(1e-5)
print(f"{eps:.4f} {order:g}")  # 6.7128 4.1
```

Opacus works differently, because its accountant is attached to the optimizer, which calls `step` after every noisy update. The next snippet does the same by hand, and it gives the same $\varepsilon$ up to the fourth decimal at the same order, 4.1. Both grids step by tenths below order 11, so that order is well inside them.

```
from opacus.accountants import RDPAccountant

accountant = RDPAccountant()
for _ in range(10_000):  # what the optimizer does at each step of training
    accountant.step(noise_multiplier=1.0, sample_rate=0.01)
eps, order = accountant.get_privacy_spent(delta=1e-5)
print(f"{eps:.4f} {order:g}")  # 6.7127 4.1
```

In training, `PrivacyEngine(accountant="rdp")` selects this class. Ask for it explicitly when you want RDP numbers, since the default accountant of Opacus 1.6.0 is `"prv"`, a numerical accountant of the privacy loss distribution. The function `get_noise_multiplier`, which solves for $\sigma$, defaults to RDP instead, so name the accountant there too and use the same one you report

with. An RDP number is a valid upper bound, but it sits above the numerical accountant's value for the same run, which is 6.19 for the running example.

Two more checks catch most mistakes. Make sure the event matches the sampler your data loader runs, because shuffled fixed-size batches get none of a Poisson event's amplification (Amplification by subsampling). And count each step once, since a manual call to `step` on top of the optimizer's own charges each update twice.

## 34.9. Cheat sheet

Table 34.2.: The results of this chapter.

| Result | Formula | Notes |
|---|---|---|
| RDP curve | $\varepsilon(\alpha)$ over orders $\alpha > 1$ | Never decreases; $\varepsilon(\infty)$ is pure DP |
| Gaussian, noise multiplier $\sigma$ | $\varepsilon(\alpha) = \alpha/(2\sigma^2)$ | $\sigma$ in units of the $\ell_2$ sensitivity |
| Composition | $\varepsilon_M(\alpha) \le \sum_i \varepsilon_i(\alpha)$ | Same relation and unit; curves fixed in advance |
| Group of $m$, Gaussian | $m^2\alpha/(2\sigma^2)$ | The generic group rule is loose |
| Improved conversion | $\varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta+\ln\alpha}{\alpha-1}$, best $\alpha$ | What Opacus and `dp_accounting` use |

**Rules of thumb**

- Compose in RDP and convert once at the end. Never convert each step to a pair first.
- Check that the best order is not at either end of the grid, and extend the grid if it is.
- In Opacus, name the accountant in both `PrivacyEngine` and `get_noise_multiplier`.

## 34.10. References

- Mironov, "Rényi Differential Privacy" (IEEE CSF 2017): the definition, composition, the curves of the basic mechanisms and group privacy.
- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 35. From Rényi DP to (ε, δ) and to tests

## 35.1. What it is

An accountant that tracks Rényi differential privacy (RDP) ends a run holding a curve, which gives a privacy cost $\varepsilon(\alpha)$ at each order $\alpha$ above 1 (Rényi differential privacy). A report needs a differential privacy (DP) guarantee instead, a privacy level $\varepsilon'$ with a small additive slack $\delta$. A conversion turns each point of the curve into such a pair, and you keep the order that gives the smallest $\varepsilon'$.

There are two conversions in use. The original one, **Mironov's conversion** (Mironov 2017), gives at each order

$$\varepsilon' = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{35.1}$$

The **improved conversion**, found independently by three groups in 2020, gives

$$\varepsilon' = \varepsilon(\alpha) + \ln\frac{\alpha - 1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha - 1}. \tag{35.2}$$

Both are valid, but at the same order the improved one is always lower. The saving depends only on the order, 0.3612 at order 10 and more at smaller orders. If the improved value comes out negative, report 0.

For example, take one Gaussian release with noise multiplier 1, whose curve is $\alpha/2$, at $\delta = 10^{-5}$. Mironov's conversion gives 5.299 and the improved one 4.729. The exact value is 4.377.

## 35.2. When you meet it

You meet the conversion whenever you compare an RDP number with someone else's. Both widely used libraries, Opacus 1.6.0 for PyTorch and `dp_accounting` 0.6.0, use the improved conversion. Older papers and code use Mironov's, including the moments accountant of the original paper on the private form of stochastic gradient descent (SGD), called DP-SGD. This part's running example is a DP-SGD run of 10,000 steps, each on a random 1% of the data with noise multiplier 1. Its curve gives 7.43 by Mironov's conversion and 6.71 by the improved one.

The fixed saving gives a reviewer a quick diagnosis. One of two RDP accountants uses Mironov's conversion if they report values about 0.36 apart for the same run, with a best order near 10. Before comparing such numbers, recompute both with one library, so that the conversion and the grid of orders are the same.

## 35.3. How to use it

In `dp_accounting`, `get_epsilon_and_optimal_order` converts at the best order of the grid and returns both. One curve answers every $\delta$, so `get_epsilon` can report several pairs at no extra privacy cost. This snippet converts the single Gaussian release above, so you never type the formula in.

```
from dp_accounting import GaussianDpEvent
from dp_accounting.rdp import RdpAccountant

accountant = RdpAccountant()
accountant.compose(GaussianDpEvent(1.0))  # one release, noise multiplier 1
eps, order = accountant.get_epsilon_and_optimal_order(1e-5)
print(f"{eps:.4f} {order:g}")  # 4.7285 5.4
print(f"{accountant.get_epsilon(1e-8):.4f}")  # 6.0864
```

A $\delta$ a thousand times smaller raises $\varepsilon$ only from 4.7285 to 6.0864, because $\delta$ enters the conversion through a logarithm. The best order depends on the run, and an accountant can only pick it from its fixed list, or grid, of orders. Long runs with little noise want orders close to 1, while short runs with much noise can want orders above 63, where the Opacus default grid stops. So whenever the returned order sits at an end of the grid, extend the grid and convert again (Rényi differential privacy).

## 35.4. What goes wrong

No conversion closes the whole gap to the exact value, because the curve is only a summary, and other mechanisms that satisfy the same curve have a larger exact $\varepsilon$. The improved conversion sits 5 to 10% above the exact $\varepsilon$ for the plain Gaussian, and the gap can be larger for the subsampled steps of DP-SGD. A privacy loss distribution (PLD) accountant keeps the mechanism's whole distribution instead of a summary, and it reaches the exact 4.377 for the single release (Numerical accounting).

The conversion also works in one direction only. A guarantee with $\delta$ above 0 gives no RDP curve, because $\delta$ allows a rare output that reveals the record outright. A mechanism that publishes your record with probability $\delta$ is $(0, \delta)$-DP, yet its Rényi divergence is infinite. So never feed an $(\varepsilon, \delta)$ pair into an RDP accountant, and take the curve from the mechanism behind it instead.

Converting also throws away what the curve says about attacks. Take a membership attack on the single release above, run at a false-positive rate of $10^{-3}$. The curve caps its true-positive rate at $2.4910 \times 10^{-2}$, while the pair at $\varepsilon = 4.729$ only caps it at 0.1131 (f-DP and Gaussian differential privacy).

## 35.5. References


- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020): the improved conversion, and what an RDP curve says about tests.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 36. Numerical accounting with privacy loss distributions

## 36.1. What it is

A privacy loss distribution (PLD) accountant computes the privacy level $\varepsilon$ of a whole differential privacy (DP) run numerically. To define it, take a dataset and a neighbour that differs in one record, and run the mechanism on the first. The **privacy loss** $L$ of that run is the logarithm of how much likelier the output is on the first dataset than on the second. Because the output is random, $L$ is too, and its distribution is the PLD. Only outputs whose loss exceeds $\varepsilon$ add to the slack $\delta$ that goes with $\varepsilon$. With $(\cdot)_+$ turning a negative value into 0, the smallest slack is

$$\delta(\varepsilon) = \mathbb{E}\big[(1 - e^{\varepsilon - L})_+\big]. \tag{36.1}$$

When steps draw their noise independently, their losses add up. So the PLD of a run is the convolution of the steps' PLDs, the exact distribution of their sum. All the error comes from storing each PLD on a grid of losses with spacing $h$, which rounds every loss to a grid point. Rounding up can only raise $\delta$, so the $\varepsilon$ it gives is an upper bound, the **pessimistic** side. Rounding down gives a lower bound, the **optimistic** side. Each rounding moves a step's loss by at most $h$. After a number $T$ of steps, the two values of $\varepsilon$ are therefore at most $Th$ apart, a width known before running anything.

## 36.2. When you meet it

You meet a PLD accountant every time you train with Opacus, whose default accountant `"prv"` is the privacy loss random variable (PRV) accountant of Gopi, Lee and Wutschitz (2021). It rounds each loss to the nearest grid point, so the errors partly cancel. That alone gives an estimate, so it adds a proven error bound and reports the upper value. Google's `dp_accounting` has `PLDAccountant`, which by default rounds up with **connect the dots**, the tightest pessimistic rounding a given grid allows.

Table 36.1 compares a PLD with Rényi differential privacy (RDP) and with the estimate of the central limit theorem (CLT) of Gaussian differential privacy (GDP). The run is the part's example of stochastic gradient descent (SGD) made private, DP-SGD, with sampling rate 0.01 and noise multiplier 1.

Table 36.1.: $\varepsilon$ at $\delta = 10^{-5}$, for the running example. The PLD columns round up and down on the grid $h = 10^{-5}$, without connect the dots.

| Steps | PLD upper | PLD lower | RDP | GDP CLT |
|---|---|---|---|---|
| 1 | 0.1995 | 0.1994 | 0.9555 | 0.0368 |
| 1000 | 1.8332 | 1.8232 | 2.1014 | 1.6177 |
| 10000 | 6.2377 | 6.1377 | 6.7127 | 6.0071 |

RDP gives a valid but loose bound, 6.7127 against the PLD's 6.2377 (Rényi DP). The CLT estimate is worse, because it falls below the PLD's lower bound at every length, so it claims a guarantee the run lacks (f-DP and Gaussian DP).

## 36.3. How to use it

In `dp_accounting` you describe the run as an event, compose it, and read off $\varepsilon$ at your $\delta$. The method `get_delta` runs the other way, from a claimed $\varepsilon$ to a bound on the $\delta$ it carries.

```
from dp_accounting import GaussianDpEvent, PoissonSampledDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from opacus.accountants import PRVAccountant

run = SelfComposedDpEvent(PoissonSampledDpEvent(0.01, GaussianDpEvent(1.0)), 10_000)
pld = PLDAccountant(value_discretization_interval=1e-4).compose(run)  # h, 10x the table's
print(f"{pld.get_epsilon(1e-5):.4f}")  # 6.1877
print(f"{pld.get_delta(6.24):.1e}")  # 8.5e-06

prv = PRVAccountant()
prv.history = [(1.0, 0.01, 10_000)]  # (noise multiplier, sample rate, steps)
print(f"{prv.get_epsilon(delta=1e-5):.4f}")  # 6.1980
```

Both answers are upper bounds inside the bracket of Table 36.1, so you can report either. Connect the dots gives 6.1877 on a grid ten times coarser than the table's. A reviewer handed the claim $(6.24, 10^{-5})$ can confirm it with `get_delta`, because the run's slack at that $\varepsilon$ is at most $8.5 \times 10^{-6}$.

Each library has one accuracy setting. In `dp_accounting` it is the grid spacing `value_discretization_interval` where a tenfold finer grid narrows the bracket tenfold and costs about ten times the memory. In Opacus it is `eps_error`, the error allowed in $\varepsilon$. The error grows with the steps, so choose either setting for your longest run.

## 36.4. What goes wrong

The commonest mistake is reporting the wrong side of the bracket. Only the pessimistic value, or the PRV accountant's upper value, is a guarantee. The optimistic value and the midpoint of the bracket are estimates, never numbers to report.

Under add-or-remove neighbours, a record can be removed or added, and a subsampled step has a different PLD in each direction. Neither is the larger in general, so an accountant must compose both and report the larger, as `dp_accounting` does by default. Opacus's PRV accountant models only removal, the larger direction in the running example. Both assume Poisson sampling, which shuffled or fixed-size batches break (Amplification by subsampling).

A run whose number of steps or noise is chosen from the outputs needs a filter, which enforces a budget fixed in advance (Adaptive composition). And if two libraries' answers for the same run fall outside each other's error bars, one has a bug or a different input (checking an accountant).

## 36.5. References


- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): the PRV accountant, with explicit error bounds.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 37. f-DP and Gaussian differential privacy

## 37.1. What it is

Every differential privacy (DP) guarantee limits how well an attacker can tell whether the data held a target's record. The attacker runs a hypothesis test, so the fullest guarantee is a **trade-off curve**. It gives the lowest false negative rate that any test can reach at each false positive rate. A mechanism is $f$**-DP** when its curve never falls below a given curve $f$ (Dong, Roth and Su 2022). A privacy level $\varepsilon$ with its slack $\delta$ only summarizes that curve.

For Gaussian noise the curve has one parameter. Write $\sigma$ for the noise multiplier, the noise's standard deviation in units of the query's $\ell_2$ sensitivity. In those units the attacker must tell a standard normal draw from one shifted by a distance $\mu$ equal to $1/\sigma$. So the mechanism satisfies Gaussian differential privacy (GDP) with parameter $\mu$, written $\mu$**-GDP**. Write $\Phi$ for the standard normal distribution function, and $\Phi^{-1}$ for its inverse. At a false positive rate $\alpha$, no membership attack then reaches a true positive rate, one minus its false negative rate, above

$$1 - \Phi(\Phi^{-1}(1-\alpha) - \mu). \tag{37.1}$$

## 37.2. When you meet it

You meet GDP when deciding whether a guarantee is enough, and as a trap in Opacus (Section 37.4). Table 37.1 reads $\mu$ as the best true positive rate (TPR) of an attack at a given false positive rate (FPR), whatever the attacker knows about the rest of the data.

Table 37.1.: Reading $\mu$-GDP as an attack and as $(\varepsilon, \delta)$.

| $\mu$ | Best TPR at 0.1% FPR | At 1% | At 5% | $\varepsilon$ at $\delta = 10^{-5}$ |
|---|---|---|---|---|
| 0.5 | 0.0048 | 0.0339 | 0.1261 | 1.9931 |
| 1 | 0.0183 | 0.0924 | 0.2595 | 4.3772 |
| 2 | 0.1378 | 0.3721 | 0.6388 | 9.9973 |

For example, a 1-GDP release lets no attacker detect the record in more than 9.24% of the trials where it is present, at a false positive rate of 1%. That is the ceiling for any membership inference attack, and the one a privacy audit tests. Dong et al. suggest that $\mu \leq 0.5$ is a reasonable guarantee and that $\mu \geq 6$ promises almost nothing. Report $\mu$ next to the $(\varepsilon, \delta)$ of a Gaussian pipeline, since it reads directly as attack rates.

## 37.3. How to use it

Gaussian releases compose exactly within GDP, even when each is chosen after seeing the earlier ones. Write $\mu_i$ for the parameter of the $i$-th release. The run is then $\mu$-GDP with

$$\mu = \sqrt{\mu_1^2 + \mu_2^2 + \cdots + \mu_k^2}, \tag{37.2}$$

so the quantity that adds up is $\mu^2$. No accountant does better for releases on the whole dataset, such as analytics queries and full-batch noisy gradient descent. Keep a ledger of $\mu^2$ and convert once, at the end. For example, four 1-GDP releases compose to 2-GDP, which is $(9.9973, 10^{-5})$-DP. Converting each release first, with a quarter of the slack, and adding the four values of $\varepsilon$ by basic composition gives 18.7607, almost twice as much.

The snippet computes the attack reading and converts both ways. The `dp_accounting` functions take a noise multiplier, so for $\mu$-GDP you pass $1/\mu$, which is 1.0 here.

```
from statistics import NormalDist

import dp_accounting

N = NormalDist()


def attack_power(mu: float, fpr: float) -> float:
    """Best true positive rate of any membership attack on mu-GDP at a false positive rate."
    return 1 - N.cdf(N.inv_cdf(1 - fpr) - mu)


print(" ".join(f"{attack_power(mu, 0.001):.4f}" for mu in (0.5, 1.0, 2.0)))  # 0.0048 0.0183
print(f"{dp_accounting.get_epsilon_gaussian(1.0, 1e-5):.4f}")  # 4.3772
print(f"{dp_accounting.get_sigma_gaussian(1.0, 1e-5):.4f}")  # 3.7306
```

The second print converts 1-GDP to $(4.3772, 10^{-5})$-DP. The third finds the noise multiplier, 3.7306, of a single $(1, 10^{-5})$-DP release. Both use the exact curve, not the classical formula of the Gaussian mechanism, which asks for more noise.

## 37.4. What goes wrong

Opacus offers `accountant="gdp"`, which estimates $\mu$ for stochastic gradient descent (SGD) made private, DP-SGD. It rests on a central limit theorem (CLT), by which many steps that each leak little compose to nearly a Gaussian curve. So for a number $T$ of steps that each sample a record with probability $q$, Bu et al. (2020) estimate

$$\mu \approx q\sqrt{T(e^{1/\sigma^2} - 1)}. \tag{37.3}$$

It is an approximation, not a bound, and it errs on the unsafe side. A small $\delta$ is decided by rare, large losses, and a CLT fits those tails poorly. On the part's running example of 10,000 steps,

the CLT gives 6.0071. But a numerical accountant proves that the true $\varepsilon$ is at least 6.1377, so the CLT number is false. Opacus warns that its `GaussianAccountant` is experimental, so report with its default numerical accountant, `"prv"`.

A mix of Laplace and Gaussian releases needs a numerical accountant, because a curve that is not Gaussian takes the composition out of GDP. And the Rényi DP curve keeps less than the trade-off curve, even for the Gaussian. For 1-GDP, the exact $\delta$ at $\varepsilon = 1$ is 0.1269. The best conversion from its Rényi DP curve certifies only 0.2149 there, so account Gaussian pipelines without subsampling in GDP.

## 37.5. References


- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022).


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 38. Adaptive composition: filters, odometers and individual accounting

## 38.1. What it is

The composition theorems of differential privacy (DP) assume that each step's privacy parameter, and the number of steps, are fixed before the run (Composition). Real runs often choose them from the outputs, as when training stops on the validation loss. Plugging the realized values into those theorems afterwards is then invalid (Rogers et al. 2016). This **plug-in mistake** can make the bound fail about three times as often as its $\delta$ claims.

A **privacy filter** fixes a total budget in advance and refuses any step that would exceed it, so the run keeps the budget's guarantee whatever the stopping rule. A **privacy odometer**, for open-ended analyses, instead reports the spend so far, valid at every step but at a higher $\varepsilon$.

## 38.2. When you meet it

Early stopping with a planned maximum needs neither. You fix the noise, sampling rate and maximum number of steps in advance, then stop when the validation loss stalls. **Report the $\varepsilon$ of the planned maximum, not of the steps you ran.** A stopped run releases a prefix of the planned run, so the planned guarantee covers it. For example, 100 planned full-batch Gaussian steps with noise multiplier 10 are $(5.299, 10^{-5})$-DP, and a run that stops at step 60 reports that.

The rule needs three conditions. The stopping signal must come from the released models or from data outside the protected training set, such as a held-out validation set, so that stopping is post-processing. A decision that reads the training data leaks through the stopping time. Deciding after seeing the model to train past the maximum is the plug-in mistake again, so it needs a new plan, accounted from the first step. And a resumed run must restore its accountant, which Opacus's `PrivacyEngine.save_checkpoint` stores, since a fresh one restarts at zero (the calibration workflow). Calling `get_epsilon` at the stopping point is the plug-in mistake.

You meet filters when the noise changes during training, since a change chosen from the outputs makes the run adaptive. Analytics services enforce them too, as OpenDP's `make_privacy_filter` or Tumult Analytics' `Session`, which refuses queries past its budget.

## 38.3. How to use it

The Rényi filter of Feldman and Zrnic (2021) works in Rényi differential privacy (RDP) at one order, written $\alpha$ and fixed in advance. It charges each step its spend at that order (Rényi DP),

which may depend on earlier outputs but must be known before the step runs. It runs a step only if the spends so far, its own included, stay within a budget $B$. To aim at a target privacy level $\varepsilon_g$ with slack $\delta$, run the conversion from RDP backwards, which sets

$$B = \varepsilon_g - \frac{\ln(1/\delta)}{\alpha - 1}. \tag{38.1}$$

A Gaussian step with a noise multiplier, written $\sigma$, spends $\alpha/(2\sigma^2)$ at order $\alpha$. At order 5.8, the best fit for the early-stopping target, the budget is $B = 2.900$. That lasts 100 full-batch steps at $\sigma = 10$. Suppose training plateaus at step 60 and you halve the noise. Each later step then costs four times as much, so the budget left for 40 steps buys 10 more, and the run still meets the target.

OpenDP's filter works in zero-concentrated DP (zCDP). Write $\rho$ for its single budget number, which bounds the RDP spend at every order at once (Concentrated DP).

```
import opendp.prelude as dp

dp.enable_features("contrib")
domain, metric = dp.atom_domain(T=float, nan=False), dp.absolute_distance(T=float)
odometer = dp.c.make_fully_adaptive_composition(domain, metric, dp.zero_concentrated_diverge
queryable = dp.c.make_privacy_filter(odometer, d_in=1.0, d_out=0.5)(0.0)  # rho = 0.5, i.e.

answered = 0
try:
    for _ in range(1000):  # sigma = 10: rho = 1 / (2 sigma^2) = 0.005 each
        queryable(dp.m.make_gaussian(domain, metric, scale=10.0))
        answered += 1
except dp.OpenDPException:  # the release that would exceed the budget
    pass
print(answered)  # 99
```

OpenDP lets through 99 releases, not 100, because it rounds each charge up for floating-point safety.

For pure-DP queries, the **Whitehouse filter** (Whitehouse et al. 2023) sums the squares of their parameters, as advanced composition does. So it fits many more small queries than a plain sum.

## 38.4. What goes wrong

An odometer costs more than a filter. The **Ville odometer** converts the RDP spend at the best of a grid of orders fixed in advance. It pays for that late choice by dividing $\delta$ by the grid's size, which on full-batch Gaussian steps adds 15 to 19% to $\varepsilon$. The 100-step run above would report 6.253.

**Individual accounting** charges each training record by its own clipped gradient norm, since a small gradient moves the model less. It drops a record once its own spend would pass the budget. In the full edition's simulation, the median typical record ends at an individual $\varepsilon$ of 3.17, against

5.299 for the worst case. But that number comes from realized norms, so it is an estimate, not a guarantee. It is also sensitive, since a small individual $\varepsilon$ says the model fits that record well. And the records dropped first, those with large gradients, are often those the model serves worst (Disparate impact).

Choosing hyperparameters from several runs is a separate cost (Hyperparameter tuning).

## 38.5. References


- Whitehouse, Ramdas, Rogers & Wu, "Fully-Adaptive Composition in Differential Privacy" (ICML 2023).
- Feldman & Zrnic, "Individual Privacy Accounting via a Rényi Filter" (NeurIPS 2021).


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 39. Concentrated differential privacy

## 39.1. What it is

Zero-concentrated differential privacy (zCDP) keeps a differential privacy (DP) budget as one number that adds up. It builds on the curve of Rényi differential privacy (RDP), which bounds how far apart a mechanism's outputs on neighbouring datasets can be, one value for each order $\alpha$. A mechanism is **$\rho$-zCDP** when that curve lies under the straight line $\rho\alpha$ through the origin, so $\rho$ is a slope. Composition adds the slopes, even when each step is chosen after seeing the earlier outputs.

The Gaussian mechanism's curve is exactly such a line. Write $\sigma$ for the standard deviation of its noise. The query $f$ has an $\ell_2$ sensitivity $\Delta_2 f$, the most one record can move it in Euclidean distance. One release then costs

$$\rho = \frac{(\Delta_2 f)^2}{2\sigma^2}. \tag{39.1}$$

So a release with noise multiplier 1 ($\sigma = \Delta_2 f$) costs $\rho = 0.5$, and doubling the noise cuts $\rho$ to a quarter.

## 39.2. When you meet it

You meet zCDP in releases that add Gaussian noise to the whole dataset, such as noisy counts or the noisy marginals behind a synthetic dataset. The 2020 United States Census used it with the discrete Gaussian mechanism (Discrete and floating-point-safe mechanisms). Tumult Analytics and OpenDP both keep budgets in $\rho$.

## 39.3. How to use it

Keep a ledger, a running list of releases, and charge each its $\rho$ from Table 39.1.

Table 39.1.: What each kind of step costs in a zCDP ledger.

| Step | Charge in $\rho$ | Notes |
|---|---|---|
| Gaussian, sensitivity $\Delta_2 f$, noise $\sigma$ | $(\Delta_2 f)^2/(2\sigma^2)$ | Exact |
| Any pure $\varepsilon$-DP step | $\varepsilon^2/2$, or $\varepsilon^2/8$ for an exponential mechanism | Above the plain $\varepsilon$ once $\varepsilon \geq 2$: keep such a step in RDP |

| Step | Charge in $\rho$ | Notes |
|---|---|---|
| An $(\varepsilon, \delta)$-DP step | $\varepsilon^2/2$, plus $\delta$ in a second column that also adds | **Approximate zCDP** |
| Poisson-subsampled Gaussian | $(\Delta_2 f)^2/(2\sigma^2)$, as without sampling | No amplification |

Add the charges and convert to $(\varepsilon, \delta)$ once, at the end, so that you pay the conversion's slack only once. When every release is Gaussian, the total $\rho$ costs the same as one Gaussian release with noise multiplier $1/\sqrt{2\rho}$, so `get_epsilon_gaussian` converts it exactly. The snippet does this, then calibrates a $(1, 10^{-5})$-DP release.

```
from math import sqrt

import dp_accounting

rho = 1 / (2 * 1.0**2)  # one Gaussian release, noise multiplier 1
print(f"{rho:.1f} {dp_accounting.get_epsilon_gaussian(1 / sqrt(2 * rho), 1e-5):.4f}")  # 0.5
sigma = dp_accounting.get_sigma_gaussian(1.0, 1e-5)  # one release that must be (1, 1e-5)-DP
budget = 1 / (2 * sigma**2)
quarter = budget / 4  # four releases share it
print(f"{budget:.5f} {sigma:.3f} {1 / sqrt(2 * quarter):.3f}")  # 0.03593 3.731 7.461
```

So the budget for $(1, 10^{-5})$ is $\rho = 0.03593$. A single query meets it with noise multiplier 3.731, while four queries that share it each get a quarter and need twice the noise. Split the budget in $\rho$, which adds exactly, and never in $\varepsilon$, which adds only loosely. Give each release the share its accuracy needs. A histogram in which each person sits in one cell is charged once, not once per cell (Parallel composition).

A ledger with steps that are not Gaussian has no exact conversion. The simple conversion of Bun and Steinke (2016) gives

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{39.2}$$

It is safe but loose, because at $\rho = 0.5$ it overshoots the exact Gaussian value by 21%. The improved conversion of Canonne, Kamath and Steinke (2020) overshoots by only 8%, and the RDP accountant in `dp_accounting` applies it. So use the improved conversion, or the exact one for an all-Gaussian ledger, and name it in the report. Laplace releases cost less as `LaplaceDpEvent` steps in a numerical accountant than at $\varepsilon^2/2$.

## 39.4. What goes wrong

The commonest error is reading $\rho$ as $\varepsilon$. A Gaussian release at $\rho = 0.5$ is $(4.3772, 10^{-5})$-DP, so reading it as $\varepsilon = 0.5$ badly overrates the protection. And $\rho$ covers only what passed through the noise. The Census Bureau released some counts exactly, and such **invariants** sit outside the guarantee, so list them next to the budget.

Subsampled steps belong outside zCDP, which gives them no amplification. On this part's running example, a run of differentially private stochastic gradient descent (SGD), or DP-SGD, zCDP reports $\varepsilon = 5475.8$ at $\delta = 10^{-5}$ where RDP gives 6.719. So keep a pipeline that includes DP-SGD in RDP or a numerical accountant. A $\rho$-zCDP release joins an RDP accountant without loss, as a `GaussianDpEvent` with noise multiplier $1/\sqrt{2\rho}$. And truncated concentrated differential privacy, a variant built for this, does no better.

Several records of one user cost more: a group of $m$ records costs $m^2\rho$, so four records in a 0.1-zCDP release cost $\rho = 1.6$.

A budget never renews, because next year's releases about the same people compose with this year's, so fix the period and its total in advance (Release and governance). Diffprivlib's `BudgetAccountant` (Keeping a budget) adds $\varepsilon$ by plain composition, so keep a long ledger in $\rho$.

## 39.5. References

- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016): the definition.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020): the improved conversion.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 40. Amplification by shuffling and by iteration

## 40.1. What it is

In the shuffle model, each of $n$ users randomizes their own data, so each report satisfies local differential privacy (DP) at a level $\varepsilon_0$. A **shuffler** strips identities and mixes the order, so each report hides among the others. The bag of reports then satisfies central DP at a much smaller $\varepsilon$, which falls like $\sqrt{\ln(1/\delta)/n}$ for a slack $\delta$. Shuffling never lowers the pure-DP level $\varepsilon_0$ itself, so the smaller $\varepsilon$ always needs $\delta$ above 0. All results here are for replace-one neighbours.

For binary randomized response, the shuffled guarantee can be computed exactly, because the analyst learns only the count of 1-reports. For any other local randomizer, use the clone bound of Feldman, McMillan and Talwar (2021). With probability $e^{-\varepsilon_0}$, another user's report looks exactly like one you could have sent, and such **clones** hide yours. The **numeric clone bound** computes this exactly, while the paper's closed form is looser and needs enough users.

## 40.2. When you meet it

Shuffling appears in telemetry and analytics systems such as Prochlo (Bittau et al. 2017), where the decision is how large $\varepsilon_0$ can be. No maintained library accounts for it, so use the full edition's script, shuffling_iteration.py.

Two other things share the name and get none of this amplification. A data loader that shuffles the training set once per epoch is a sampler, accounted as in Match the sampler to the accountant. And federated training in which clients join rounds when they are available, rather than by Poisson sampling, has no shuffler either. Account it without amplification, or with matrix mechanisms, which need no sampling.

Amplification by iteration, where later training noise also protects earlier records, appears in papers that release only the final model. It holds for convex models, rarely for deep learning (Section 40.4).

## 40.3. How to use it

Work backwards from the target. Compute the numeric clone bound for a few values of $\varepsilon_0$ and keep the largest that meets it, which means the least noise per report.

Table 40.1.: Central $\varepsilon$ at $\delta = 10^{-6}$ after shuffling the reports of $n$ users. "n/a" marks too few users for the closed form.

| $n$ | $\varepsilon_0$ | Exact, randomized response | Clones, numeric | Clones, closed form |
|---|---|---|---|---|
| 1,000 | 4 | 1.6910 | 3.9900 | n/a |
| 10,000 | 2 | 0.0870 | 0.1550 | 0.5009 |
| 100,000 | 2 | 0.0247 | 0.0450 | 0.1862 |
| 100,000 | 4 | 0.0847 | 0.1698 | 0.5346 |

The choice of bound can decide feasibility. For 100,000 users and a target of 0.2 in Table 40.1, the closed form allows $\varepsilon_0$ up to 2 but the numeric clone bound allows 4. So deploy the numeric bound, and use the closed form only for rough sizing.

Before reporting a shuffled $\varepsilon$, check the bound's assumptions.

- **The order is uniformly random and hidden,** with no metadata such as arrival time or network address.
- **The shuffler does not collude with the analyst.**
- **Only honest users count in $n$,** since colluders reveal their reports to the analyst.
- **The count of users is the number of reports shuffled,** so with several batches it is the smallest batch.
- **Each user sends one report per batch,** or the reports compose like any other releases.

A leaked order or a colluding shuffler leaves only $\varepsilon_0$.

## 40.4. What goes wrong

Shortcuts fail where users are few or $\varepsilon_0$ is large. The closed form then says nothing, and a normal approximation of the count of 1-reports understates $\varepsilon$. For 1,000 users at $\varepsilon_0 = 4$, it gives 0.9667 where the exact value is 1.6910.

Amplification by iteration covers one pass of noisy stochastic gradient descent (SGD) that keeps the weights in a fixed convex set (Feldman et al. 2018). It needs two conditions. Only the final model may be released. And every step must be a contraction, never moving two sets of weights further apart, which takes a convex, smooth loss and a small enough step. Even then the gain depends on the record. Over 10,000 records with noise multiplier 1, the middle record gets $\varepsilon = 0.0707$ at $\delta = 10^{-5}$. But the last record gains nothing and gets 5.2985 through Rényi DP, as one Gaussian step does, so report the worst record's value. On a bounded set, the privacy loss of full-batch noisy gradient descent even stops growing (Altschuler and Talwar 2022).

Deep learning breaks both conditions. Checkpoints, logged models and a best checkpoint picked from several all release more than the final model, and non-convex losses are not contractions. Annamalai (2024) builds a non-convex loss whose final model leaks as much as all the intermediate models together. So never report an $\varepsilon$ from amplification by iteration for deep learning, only for convex models, such as logistic regression, that release just their final weights.

## 40.5. References

- Feldman, McMillan & Talwar, “Hiding Among the Clones” (FOCS 2021): the clone bound and how to compute it.

**Derivations.** Every result here is derived step by step in the full edition’s chapter.

# Part V.

# Differential privacy in machine learning

This part is for anyone who trains, ships or reviews a model under differential privacy (DP). It needs the Gaussian mechanism and composition from Part 3 and an accountant from Part 4. Its audits also reuse the membership-inference attacks of Part 2.

Every private training method has two halves. The privacy half is computed. The main algorithm is stochastic gradient descent (SGD) with clipped per-example gradients and Gaussian noise, called DP-SGD. Its accountant derives $\varepsilon$ from the noise multiplier, the sampling rate and the number of steps alone. The utility half, accuracy, is measured, not proved. So every page asks two questions: is the $\varepsilon$ correct, and is the accuracy at that $\varepsilon$ good enough?

Three pages are workflows, each a whole job done with real libraries on a public dataset. Train a model with DP-SGD trains a classifier on the Adult census data with Opacus, from the privacy unit to the run record. Fine-tune a language model privately with LoRA adapts a small public language model to a private text and checks whether it memorized planted secrets. Release and governance closes the part: it recomputes the guarantee, writes it down, maps it to the law and lists what to check before a model ships.

The other pages explain the decisions those jobs make, in the order a project meets them. DP-SGD comes first, after a short reference page on private empirical risk minimization for convex models. Then come hyperparameter tuning, which charges the search to the budget, the privacy unit in User-level differential privacy, and the budget itself in Choosing `ε`. Public data and private fine-tuning explains why starting from a public model helps so much. Disparate impact and Auditing private training are the checks before release.

Three more short reference pages cover methods you meet less often. They are private prediction with an ensemble of teacher models, correlated noise across training steps for when batches cannot be sampled at random, and adaptive data analysis, which keeps a reused validation set honest.

The code runs Opacus for PyTorch and Google's `dp_accounting`, and every number on these pages comes from a script you can run. By the end, you will be able to train a private model, defend its settings and its $\varepsilon$, audit it, and write the statement that goes out with it.

# 41. Private empirical risk minimization

## 41.1. What it is

Most of machine learning is **empirical risk minimization (ERM)**, which picks the weights $\theta$ with the least average loss over a training set. Write $D$ for that set and $n$ for its number of records. Usually a ridge penalty keeps the weights small. Its strength is a number $\lambda$, and training minimizes

$$\mathcal{L}(\theta; D) = \frac{1}{n}\sum_{i=1}^{n} \ell(\theta; z_i) + \frac{\lambda}{2}\|\theta\|^2, \tag{41.1}$$

where $\ell(\theta; z)$ is the loss of the weights on one record $z$. Differential privacy (DP) asks that replacing one record barely change the released weights. So noise must enter in the trained weights, in the objective or in each gradient step.

**Output perturbation** trains as usual and adds Gaussian noise to the final weights, which gives a guarantee with a small slack $\delta$. The noise is scaled to the most one record can move the minimizer $\theta^*$. Write $D$ and $D'$ for two datasets that differ in one record. For a convex loss, the ridge limits that move to

$$\|\theta^*(D) - \theta^*(D')\| \le \frac{2L}{\lambda n}. \tag{41.2}$$

Here the constant $L$ bounds the length of each record's loss gradient. It is 1 for logistic regression with labels $\pm 1$ and features of length at most 1.

**Objective perturbation** adds a random linear term to the objective and minimizes the result exactly. Part of the privacy budget $\varepsilon$ pays for the loss's curvature, which is at most a constant $c$, one quarter for the logistic loss. The noise is scaled to what is left,

$$\varepsilon' = \varepsilon - 2\ln\Big(1 + \frac{c}{n\lambda}\Big). \tag{41.3}$$

This must stay positive, so with little data the ridge $\lambda$ has to grow.

**Gradient perturbation** adds noise to every gradient step instead. With random batches and each record's gradient clipped to a fixed length, it becomes noisy stochastic gradient descent (SGD), DP-SGD, the only option for a model that is not convex.

## 41.2. When you meet it

You meet these methods for linear models, such as a logistic regression or a linear probe on features from a public pretrained network (Public data and private fine-tuning). IBM's diffprivlib offers `LogisticRegression(epsilon=..., data_norm=...)` for objective perturbation, with `data_norm` bounding each row's length.

The first decision is the method, and with a convex loss and a ridge, objective perturbation is usually the most accurate. Its noise is damped by all the objective's curvature, while output perturbation's is sized by the ridge alone. Use output perturbation when training is a black box, DP-SGD otherwise.

The second decision is whether you have enough data. On a ridge-penalized training objective, $n$ and $\varepsilon$ matter only through their product, so halving $\varepsilon$ costs as much as halving the data. On new data, with $d$ features and an $(\varepsilon, \delta)$ guarantee, privacy adds less error than sampling once $n$ is well above $d \ln(1/\delta)/\varepsilon^2$.

## 41.3. How to use it

diffprivlib 0.6.6's `LogisticRegression` fails with scikit-learn 1.8, so call the mechanism it wraps, `mechanisms.Vector`, directly. The bundled breast-cancer data has no public data dictionary, so the snippet sets 69 rows aside as a public stand-in and treats the other 500 as private. It takes the logarithm $\ln(1+x)$ of each feature value $x$, reads each feature's range from the stand-in rows only, clips the private rows into that range and maps it to $[-1, 1]$. A constant 1 replaces the intercept, and dividing by $\sqrt{31}$ keeps each row's length at most 1. In the call, `function_sensitivity` is the curvature bound $c$.

```
import numpy as np
from diffprivlib.mechanisms import Vector
from scipy.optimize import minimize
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
public, private = np.log1p(data.data[:69]), np.log1p(data.data[69:])  # 69 stand-in rows fix
lo, hi = public.min(axis=0), public.max(axis=0)
X = np.hstack([2 * (np.clip(private, lo, hi) - lo) / (hi - lo) - 1, np.ones((500, 1))]) / np
y = 2 * data.target[69:] - 1
Xt, yt, n = X[::2], y[::2], 250  # even rows train, odd rows test
lam = 1 / n  # diffprivlib's default ridge, fixed in advance

def objective(w):  # J(w) and its gradient
    m = yt * (Xt @ w)
    return np.logaddexp(0, -m).mean() + lam / 2 * w @ w, lam * w - Xt.T @ (yt / (1 + np.exp(

def accuracy(eps, seed):  # eps = inf adds no noise
    vector = Vector(epsilon=eps, function_sensitivity=0.25, dimension=31, alpha=n * lam, n=n
    w = minimize(vector.randomise(objective), np.zeros(31), jac=True, options={"gtol": 1e-9}
    return np.mean(np.sign(X[1::2] @ w) == y[1::2])

print(" ".join(f"{np.mean([accuracy(e, s) for s in range(20)]):.3f}" for e in [1, 4, np.inf]
```

Averaged over 20 noise draws, test accuracy at $\varepsilon = 1$ is 0.783, against 0.964 without noise. With only 250 training records, the budget has to rise to 4 before accuracy comes close, at 0.946. Nothing was tuned on the private rows, since all 30 features are used with diffprivlib's default

ridge. So the guarantee is pure DP with no slack, for the 250 private training rows and for neighbours that replace one record. The 69 stand-in rows get no guarantee, because their ranges are printed in the code.

## 41.4. What goes wrong

The most common failure is a bound taken from the data. For example, left unset, `data_norm` is computed from the training rows with a `PrivacyLeakWarning`, and the guarantee is gone. A table of ranges in a dataset's description is no better when it was computed from the same rows. Breast cancer's is, so each minimum and maximum is one record's value. So take every bound from public knowledge or public rows, and clip the private rows to it.

The second failure is the solver, because output and objective perturbation assume the exact minimizer. A solver that stops where the gradient still has length $g$ adds $2g/\lambda$ to the sensitivity, which must stay small beside the exact $2L/(\lambda n)$. So set the tolerance well below $L/n$, and check the final gradient rather than the number of steps.

The third failure is an unpenalized weight, because the sensitivity bound needs the ridge's curvature in every direction. Scikit-learn's loss, which diffprivlib reuses, leaves the intercept out, so the snippet appends a constant feature instead. Fix $\lambda$ by a rule or on public data, since tuning it on private data is a search that must be paid for (Hyperparameter tuning). Finally, these guarantees assume replaced records, unlike the add/remove accountants of later chapters, and the two do not convert freely, so state the relation next to every $\varepsilon$.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 42. DP-SGD

## 42.1. Why this matters

Nearly every deep-learning system that satisfies differential privacy (DP) runs one algorithm. It is stochastic gradient descent (SGD) with each record's gradient clipped to a fixed length and Gaussian noise added at every step, called DP-SGD (Abadi et al. 2016). Clipping limits how far one record moves each step, whatever the model, and the noise hides that move (Private empirical risk minimization). An **accountant** then adds up the cost of every step into one privacy level, the $\varepsilon$ of the guarantee. It comes with $\delta$, a small additive slack that the guarantee allows on top of $\varepsilon$.

The accountant sees only the noise, the sampling rate (the chance that a record joins a batch) and the number of steps, never the model. So those three decide $\varepsilon$, and the clipping norm and the learning rate are free to be tuned for accuracy. Most failures come from code that breaks an assumption of the privacy proof, so the checks matter as much as the settings. Train a model with DP-SGD does the whole job in Opacus.

## 42.2. The algorithm and its guarantee

A training set holds $n$ records. The gradient of one record's loss is its **per-example gradient**, written $g_i$ for record $i$. DP-SGD repeats five operations at every step.

1. Sample a batch. Each record enters independently with probability $q$, the **sampling rate**, which is called Poisson sampling.
2. Compute the per-example gradient of each record in the batch.
3. Clip each gradient longer than the **clipping norm** $C$ to length $C$.
4. Sum the clipped gradients, and add Gaussian noise with standard deviation $\sigma C$ to each coordinate of the sum, where the **noise multiplier** $\sigma$ measures the noise in units of $C$.
5. Divide the noisy sum by the expected batch size $B = qn$, a fixed public number, and take a step in the opposite direction, scaled by the **learning rate** $\eta$.

Write $\theta_t$ for the weights at step $t$ and $S_t$ for its batch. The Gaussian noise $\xi_t$ is drawn afresh, independently on each coordinate, and the update is

$$\theta_{t+1} = \theta_t - \frac{\eta}{B}\Big( \sum_{i \in S_t} g_i \min\Big(1, \frac{C}{\|g_i\|}\Big) + \xi_t \Big), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I). \tag{42.1}$$

Releasing every intermediate set of weights then satisfies $(\varepsilon, \delta)$-DP for **add/remove neighbours**, datasets that differ because one has a record the other lacks (Amplification by subsampling).

This part's running example trains a small network on 10,000 synthetic records (Section 42.7). It runs 10,000 steps with $\sigma = 1$, and each record joins a batch with probability 0.01. The Rényi differential privacy (RDP) accountant, a standard upper bound that is slightly loose, gives it $\varepsilon = 6.71$ at $\delta = 10^{-5}$. To pick $\sigma$ for a target $\varepsilon$ instead, let the accountant you will report solve for it. Train a model with DP-SGD does this in one Opacus call (Calibrate and account a DP-SGD run).

Choose $\delta$ by the book's default, $\delta \leq 1/n^{1.1}$ rounded down to a power of ten. That keeps it safely below $1/n$ (Choosing ε). Here $n$ counts people, and for the running example's 10,000 records, one per person, the rule gives its $10^{-5}$.

## 42.3. The implementation checklist

The guarantee rests on four properties of your code, and code that breaks one still trains and reports as before.

First, one record must move the clipped sum by at most $C$. Batch normalization breaks this, because its statistics let one record change every other record's gradient. So private models use group or layer normalization, which normalize each record on its own. With `strict=False`, Opacus's validator lists the layers to replace instead of raising an error.

```
from opacus.validators import ModuleValidator
from torchvision.models import resnet18

model = resnet18(weights=None)
print(len(ModuleValidator.validate(model, strict=False)))  # 20
model = ModuleValidator.fix(model)
print(len(ModuleValidator.validate(model, strict=False)))  # 0
```

The clipping norm must also be fixed in advance, from public data or from a private estimate. A $C$ read off the current batch makes the size of the noise depend on the data, and that dependence leaks information the accountant does not count.

Second, the accountant must assume the sampler the code actually runs (Amplification by subsampling). The Poisson accountant's saving comes from each record's chance of being left out. A loader that shuffles once per epoch and cuts fixed batches puts every record in exactly one batch per epoch. That chance is then gone, and the true $\varepsilon$ can be far above the Poisson number.

Batches of a fixed size, drawn afresh at every step, can still use a Poisson accountant. Give it the batch's share of the data as the rate and half your noise multiplier, because a record that enters a fixed-size batch pushes another out, so the sum can move by $2C$. That is expensive, since halving $\sigma$ raises the running example's $\varepsilon$ from 6.71 to 47.42. The rule holds only for add/remove neighbours, and Part 4 gives what to account for under the others.

Third, divide the noisy sum by the public number $B$, never by the actual batch size. The actual size is one larger when the protected record is drawn, so dividing by it reveals whether it was, and the accountant does not count that leak.

Fourth, the accountant must count every step that touched the data, including those of runs discarded during tuning.

Beyond the proof, state the neighbouring relation next to every $\varepsilon$, and make data-dependent preprocessing private too. The guarantee covers one record, so a person with many records needs user-level privacy. And the proof assumes exact Gaussian noise, which ordinary floating-point sampling does not produce, so turn on Opacus's `secure_mode=True` for a model you release (Discrete and floating-point-safe mechanisms).

## 42.4. Clipping and the learning rate

Clipping must happen per record. PyTorch's `torch.nn.utils.clip_grad_norm_` clips the gradient of a whole batch, which bounds nothing about any single record, so it gives no privacy at any threshold.

The clipping norm does not change $\varepsilon$, but it sets the two ways clipping costs accuracy. A small $C$ clips more gradients, so records with large gradients count for less, and they are often rare records (Disparate impact). A large $C$ instead means more noise, since the noise is $\sigma C$. When every gradient is clipped, each has length exactly $C$, and the noise is proportional to $C$ too. So only the product $\eta C$ of the learning rate and the clipping norm matters. The practical recipe is to clip hard and tune that one number. On the running example at $\sigma = 1$, holding $\eta C$ at 0.25 keeps test accuracy at 0.9599 or above for every $C$ up to 0.3. With the learning rate held at 0.5 instead, $C = 10$ drops it to 0.8347.

Noise also changes the best learning rate. On the logistic problem of Optimization, adding noise moves the best learning rate on a grid down from 0.178 to 0.056. Clipping pulls the other way, because clipped gradients are shorter, which is why private runs often start from a step several times the non-private one. The two effects do not cancel predictably. So search the step again whenever $\sigma$ or $C$ changes, and count the search's runs in $\varepsilon$.

Gradient norms also drift. The running example's median per-example norm falls from 3.592 to 0.006 in a thousand steps, so a $C$ that clips everything early clips only a minority later. Adaptive clipping is the private estimate that the checklist allows, since it moves $C$ toward a quantile of the norms measured by a small private count. Opacus offers it with `clipping="adaptive"`, but this is not a drop-in switch. It takes five more parameters, such as `target_unclipped_quantile`, the share of gradients to leave unclipped. And because part of the noise budget goes to the count, the gradient gets more noise at the same $\sigma$.

## 42.5. Batch size at a fixed budget

Averaging the noisy sum over $B$ records divides the added noise by $B$, so larger batches dilute it. At a fixed budget and number of epochs, a larger batch means fewer steps, and the noise multiplier must rise to keep $\varepsilon$, but more slowly than $B$ does. For a small sampling rate and a noise multiplier well above 1, the accountant's cost at a fixed number of epochs depends on $q/\sigma^2$, so

the noise multiplier only has to grow like $\sqrt{q}$. Since $q$ is $B/n$, the noise on the averaged gradient falls with the batch size,

$$\sigma \propto \sqrt{\frac{B}{n}}, \qquad \frac{\sigma C}{B} \propto \frac{C}{\sqrt{Bn}}. \tag{42.2}$$

Small batches are worse still, because they need a noise multiplier near or below 1, where subsampling saves much less.

So choose the largest batch your hardware allows, with a larger learning rate for the fewer steps. When the batch does not fit in memory, Opacus's `BatchMemoryManager` splits the logical batch, the one the sampler draws, into physical batches that fit. It still steps once per logical batch, so the sampling rate and the accounting stay the same.

## 42.6. Making it fast

Computed one record at a time, DP-SGD can run tens or hundreds of times slower than ordinary training. Of three fixes, the first two are free. Vectorized per-example gradients, from `torch.func.vmap` or Opacus's hooks, compute the same clipped sum in parallel. **Ghost clipping** also saves memory, because it gets each record's gradient norm from each layer's inputs and back-propagated errors without forming the gradient. Opacus turns it on with `grad_sample_mode="ghost"`, and `make_private` then also returns a loss function to train with.

Microbatching is the costly fix. It clips the average gradient of each fixed group of $m$ records instead of each record's gradient. One record can then change its group's clipped average by up to $2C$, so the noise must double. And since the update divides it by the $B/m$ groups rather than by $B$ records, the noise on each coordinate of the averaged update becomes

$$2m \cdot \frac{\sigma C}{B}. \tag{42.3}$$

On the running example at $\sigma = 1$, groups of 4 cut test accuracy from 0.9589 with per-example clipping to 0.9336, so use it only as a last resort.

## 42.7. What privacy costs in accuracy

The running example is a network with one hidden layer of 64 tanh units. Table 42.1 trains it at four budgets, with $C$ and $\eta$ both 0.5.

Table 42.1.: Test accuracy of the running example under DP-SGD at four budgets, with $\delta = 10^{-5}$ and the RDP accountant.

| | Noise multiplier | Test accuracy (mean ± standard deviation over 3 runs) |
|---|---|---|
| $\varepsilon = 0.5$ | 7.72 | $0.8209 \pm 0.0180$ |
| $\varepsilon = 1$ | 4.13 | $0.9175 \pm 0.0161$ |
| $\varepsilon = 4$ | 1.36 | $0.9548 \pm 0.0019$ |

| | Noise multiplier | Test accuracy (mean ± standard deviation over 3 runs) |
|---|---|---|
| $\varepsilon = 8$ | 0.917 | $0.9604 \pm 0.0019$ |
| Clipped, no noise | 0 | $0.9649 \pm 0.0009$ |
| Non-private SGD (1,000 steps) | — | $0.9610 \pm 0.0005$ |

Clipping without noise scores slightly above non-private SGD here, so the whole loss comes from the noise, and it grows steeply as $\varepsilon$ shrinks. Going from $\varepsilon = 8$ to 4 costs half a point, while halving it from 1 to 0.5 costs almost 10. The total length of the noise on a step also grows with the square root of the number of trained weights. The clipped signal cannot exceed $C$, so a smaller model or public pretraining often helps.

## 42.8. Cheat sheet

Table 42.2.: DP-SGD starting settings and checks.

| Setting | Start with | Check |
|---|---|---|
| $\delta$ | $1/n^{1.1}$ rounded down to a power of ten | $n$ counts people |
| Batch size | 256 to 4,096, as large as fits | Noise on the average is $\sigma C/B$ |
| Epochs | Fixed before calibrating; Adult uses 10 | $\sigma$ recomputed if they change |
| Noise multiplier | From the accountant you report | A second accountant gives the same $\varepsilon$ |
| Clipping norm | 1, or the median norm on public data | Per record, never `clip_grad_norm_` |
| Optimizer, learning rate | SGD with momentum 0.9; several times the non-private learning rate (ten on Adult), chosen on public data | Retuned when $\sigma$ or $C$ changes |
| Sampler | Poisson | Fixed-size batches: account at $\sigma/2$ (add/remove only) |
| Report | $\varepsilon$, $\delta$, neighbours, accountant | Every run counted |

**Rules of thumb**

- Fix $\varepsilon$, $\delta$ and the number of epochs first, since they are the contract. Then choose the batch, let the accountant set $\sigma$, and tune the learning rate last.
- Find the $\varepsilon$ below which accuracy collapses on your task before you promise a budget.

## 42.9. References

- Abadi et al., “Deep Learning with Differential Privacy” (CCS 2016): DP-SGD and the moments accountant.
- Ponomareva et al., “How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy” (JAIR 2023): what to clip, how to tune and what to report.

**Derivations.** Every result here is derived step by step in the full edition’s chapter.

# 43. Train a model with DP-SGD

## 43.1. Why this matters

You have a PyTorch training loop on sensitive data, and it must become private. The algorithm is stochastic gradient descent (SGD) with differential privacy (DP), called DP-SGD, which clips each record's gradient and adds Gaussian noise at every step (DP-SGD). The library is Opacus, and the hard part is not the call that wraps the model but the dozen decisions around it, made in the right order. Most of them are inputs to the privacy guarantee, so they must be settled before the first training step. A mistake in any of them leaves code that trains exactly as before but reports a false $\varepsilon$. That is the privacy level, which bounds what the model can reveal about any one person.

This chapter does the whole job on a real dataset, Adult from the UCI Machine Learning Repository. It holds 48,842 people from a census survey, and the task is to predict whether a person earns more than $50,000 a year. Each section is one step of the job. The script practitioner_dp_sgd.py runs every step except tuning, which practitioner_tuning.py does. Each runs in one to two minutes without a GPU. Its options `--epsilon` and `--epochs` change the main run; the defaults, 4 and 10, give the numbers below.

```
uv run python code/part05-dp-ml/practitioner_dp_sgd.py
uv run python code/part05-dp-ml/practitioner_tuning.py
```

## 43.2. Fix the privacy unit, n and δ

The first decision is what one record is, because the guarantee protects exactly that (User-level differential privacy). In Adult each row is one person, so protecting a row protects a person. Adult has no person identifier, so this rests on how the survey was collected. When your data has one, count distinct identifiers, not rows. If a person has many rows, such as visits or messages, together they move the model much further than one row, and you need user-level training.

The script splits the rows with seed 0. It keeps 30,000 rows as the private training set and 16,842 as the test set. The other 2,000 rows, the stand-in rows from here on, play the part of public data. The script uses them to choose the clipping norm, the length each record's gradient is cut to, and the learning rate, and it never trains the model on them. Treating them as public is an assumption made for the demonstration, since they come from the same survey, and in production this role belongs to genuinely public data. Without public data, the tuning runs on the private rows and its cost is counted in $\varepsilon$, as Section 43.10 shows.

Next comes $\delta$, the small additive slack that the guarantee allows on top of $\varepsilon$. It is fixed from the number of people $n$ in the private training set and nothing else. The book's default is $\delta \le 1/n^{1.1}$, rounded down to a power of ten, which keeps $\delta$ safely below $1/n$ (Choosing ε).

```
def delta_for(n: int) -> float:
    """The book's default: delta <= 1/n^1.1, rounded down to a power of ten."""
    return 10.0 ** math.floor(math.log10(n**-1.1))
```

Here $n$ is the 30,000 people of the private training set, not the 48,842 of the whole dataset, and the script prints $\delta = 10^{-5}$. The usual mistake is quoting a $\delta$ above $1/n$.

## 43.3. Preprocess from public facts

Every statistic computed from the private rows is itself a release, and the accountant, the code that adds up the privacy cost of training, does not count it. A scaler fitted on the training set leaks each column's mean and spread. An encoder fitted on it leaks which categories occur, even a country only one person comes from. So the script builds its features from facts that are public before any data is seen.

The category lists are copied from the dataset's documentation. The numeric columns are scaled by public bounds. Ages run from 17 to 90, the largest age the survey records, and hours per week run from 1 to 99. Capital gains and losses go on a log scale, capped at 99,999, the largest amount the survey records. A missing value becomes its own category, so no row is dropped.

```
def features(rows: pl.DataFrame) -> Array:
    """One-hot categories from the documented lists, numbers scaled by public bounds."""
    rows = rows.with_columns(pl.col("capital-gain", "capital-loss").log1p())
    numeric = [
        np.clip((rows[name].to_numpy() - low) / (high - low), 0.0, 1.0) for name, (low, high
    ]
    encoder = OneHotEncoder(categories=list(CATEGORIES.values()), sparse_output=False)
    text = rows.select(pl.col(list(CATEGORIES)).fill_null("?")).to_numpy()
    encoder.fit(text[:1])  # the categories are given, so fitting learns nothing from the ro
    return np.hstack([np.column_stack(numeric), encoder.transform(text)])
```

This gives 91 features. The model is a small network with one hidden layer of 64 units, which has 6,018 weights in all. To check this step in someone else's code, search for every `fit` call and ask what data it sees.

## 43.4. Train the baselines

A private accuracy means little on its own, so train three baselines with the same model, batch size, epochs and optimizer. The non-private model shows the best you can hope for. A model with clipping and no noise separates the cost of clipping from the cost of noise. And always predicting the majority class shows the floor, where a private model whose training collapsed lands.

```
non-private: test accuracy 0.847
clip only (C=1, no noise): test accuracy 0.845
always '<=50K': test accuracy 0.759
```

Clipping alone costs little here. Had clip-only accuracy been far below non-private, no privacy budget would fix it, because the problem would be the clipping norm or the learning rate.

## 43.5. Check the model

Opacus computes one gradient per record, and some layers make that impossible or break the guarantee. Batch normalization is the common case, because its statistics mix the records of a batch, so one record changes every other record's gradient. The validator lists such layers before you train.

```
errors = ModuleValidator.validate(model, strict=False)
print(f"validator errors: {len(errors)}")
```

Here it finds none. For a model with such layers, `ModuleValidator.fix` replaces batch normalization with group normalization (DP-SGD shows it on a ResNet).

## 43.6. Choose the clipping norm, batch and epochs

These settings are fixed before the private data is touched, because choosing them on it costs privacy. Start with the clipping norm $C$. The usual guide is the typical per-example gradient norm on public data or data like it. So the script measures it on the stand-in rows with Opacus's per-example gradients.

```
wrapped = GradSampleModule(model)  # adds hooks that keep one gradient per row
wrapped.train()
logits: torch.Tensor = wrapped(split.x)
nn.CrossEntropyLoss()(logits, split.y).backward()  # fills p.grad_sample for every weight p
norms = stacked_norms(wrapped)
```

```
at initialisation: median norm 1.444  90th percentile 1.538
fraction clipped at C=1: 1.000
```

The median rule would give a $C$ of about 1.4. The script keeps the usual default $C = 1$ instead, just below the median, so every gradient is clipped at the start, and that costs nothing. When every gradient is clipped, each has length exactly $C$ and the noise scales with $C$ too, so the learning rate times $C$ is the one number that matters (DP-SGD).

Next comes the expected batch size $B$, which should be as large as you can afford. It is an expected size because the sampler draws every batch at random (Section 43.8). The reason is that

the noise on the averaged gradient is $\sigma C/B$, where the noise multiplier $\sigma$ is the noise's standard deviation in units of $C$. The script takes a batch of 1,000. Opacus sets the sampling rate $q$, the chance that a row joins a batch, to one over the number of batches per epoch, $1/\lceil n/B \rceil$. That is $B/n = 0.0333$ here, but not when $B$ does not divide $n$, so plan with Opacus's value. An epoch is $1/q$ steps, so 10 epochs give 300 steps. At $\sigma = 1$ the noise on each coordinate of the averaged gradient is then only 0.0010. Fix the number of epochs now, since the noise is calibrated to it.

The optimizer is SGD with momentum 0.9, and its learning rate is chosen on the stand-in rows too. The script trains a non-private model on 1,500 of them at six rates, scores each on the other 500, and keeps the best.

```
  lr=0.05: 0.814  lr=0.1: 0.816  lr=0.2: 0.826  lr=0.5: 0.792  lr=1: 0.812  lr=2: 0.768
non-private learning rate 0.2; DP-SGD learning rate 2 (x10)
```

DP-SGD then uses ten times the winning rate. That factor is a rule of thumb rather than a measurement. Clipped gradients are shorter and noise changes the best step, so private runs usually start from a step several times the non-private one (DP-SGD). No private row has been touched, so this choice costs no privacy.

## 43.7. Calibrate the noise

Fix the target $\varepsilon$, $\delta$, $q$ and the number of epochs, and let the accountant solve for the smallest noise multiplier that meets them. Use the accountant you will report, here the privacy loss random variable (PRV) accountant, which computes $\varepsilon$ numerically and almost exactly. A looser accountant overestimates $\varepsilon$, and calibrating with it pays for the overestimate with extra noise (Calibrate and account a DP-SGD run).

```
sigma = get_noise_multiplier(
    target_epsilon=epsilon,
    target_delta=delta,
    sample_rate=sample_rate,
    epochs=epochs,
    accountant="prv",
)
```

For $\varepsilon = 4$ at $\delta = 10^{-5}$ it prints `sigma=0.973`. The result is only valid for the epochs and sampling rate you gave it. If you later train longer, or change the batch size, recompute $\sigma$ before training, or report the larger $\varepsilon$ the accountant gives for what actually ran.

## 43.8. Train

The privacy engine wraps the model, the optimizer and the data loader. The optimizer it returns clips each record's gradient and adds the noise, while the loader it returns draws Poisson batches, in which each row joins each batch independently with probability $q$. The accountant assumes exactly

this sampling. The loader the script passes in shuffles and cuts fixed batches, so `make_private` replaces it, and that replacement is what makes the assumption true.

```
engine = PrivacyEngine(accountant="prv")
optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)
model, optimizer, data_loader = engine.make_private(
    module=model,
    optimizer=optimizer,
    data_loader=data_loader,
    noise_multiplier=sigma,
    max_grad_norm=CLIP,
)  # Poisson sampling is the default: each row joins each batch with probability q
```

The training loop is an ordinary one inside `BatchMemoryManager`. It splits each logical batch of about 1,000 rows into physical batches of at most 500 that fit in memory, and the optimizer steps once per logical batch. So the sampling rate and the accounting do not change.

```
with BatchMemoryManager(
    data_loader=data_loader, max_physical_batch_size=PHYSICAL_BATCH, optimizer=optimizer
) as batches:
    for x, y in batches:  # Opacus splits each logical batch; it steps once per logical batc
        optimizer.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()
```

```
steps=300  q=0.0333
epsilon (Opacus PRV) = 3.99
test accuracy at epsilon=4: 0.846
```

The script leaves `secure_mode` off, so that its noise is seeded and its numbers repeat, and its record says so. For a model you release, pass `secure_mode=True` to `PrivacyEngine`. It draws the noise from a cryptographically secure generator, and it guards against the floating-point flaws of ordinary Gaussian samples (Discrete and floating-point-safe mechanisms).

## 43.9. Cross-check ε

An $\varepsilon$ from one library is a claim about that library's arithmetic. Recompute it independently with Google's `dp_accounting`, from the noise multiplier, sampling rate and step count of your plan, while Opacus counts the steps it actually took. Its privacy loss distribution (PLD) accountant, like the PRV accountant, computes $\varepsilon$ almost exactly, but by a different numerical method, so the two should agree to within rounding. Its Rényi differential privacy (RDP) accountant, an older bound that is slightly loose, runs alongside for comparison.

```
step = dp_accounting.PoissonSampledDpEvent(sample_rate, dp_accounting.GaussianDpEvent(sigma)
training = dp_accounting.SelfComposedDpEvent(step, steps)
epsilon_pld = dp_accounting.pld.PLDAccountant().compose(training).get_epsilon(delta)
epsilon_rdp = dp_accounting.rdp.RdpAccountant().compose(training).get_epsilon(delta)
```

```
epsilon PRV (Opacus) = 3.99
epsilon PLD (dp_accounting) = 3.98
epsilon RDP (dp_accounting) = 4.50
```

The PRV and PLD values agree. The RDP accountant gives 4.50 for the same run, and since RDP is a valid but looser bound, a statement must name its accountant. If the two tight accountants disagree by more than rounding, the run did not do what the plan says.

## 43.10. Tune honestly

The default is the one this run used. Choose every hyperparameter on public data, or on a public stand-in, and tuning costs nothing. When no public data exists, tuning must run on the private rows. Then every private run whose result influences what you release costs privacy, including the runs you discard (Hyperparameter tuning).

The tuning script measures that cost for six learning rates at a total $\varepsilon$ of 4. A single run with no tuning is the reference. A grid runs all six rates and pays for each by composition. A random search runs a number of models drawn from a Poisson distribution with mean 4, gives each a learning rate drawn from the six, and releases only the best. Only the RDP accountant of `dp_accounting` handles a random number of runs, so the script uses RDP throughout. Its single-run noise multiplier is therefore larger than the PRV value of earlier sections.

```
def random_search(sigma: float) -> dp_accounting.DpEvent:
    """A Poisson number of runs with mean 4 (shape=inf selects the Poisson case); only the b
    return dp_accounting.dp_event.RepeatAndSelectDpEvent(one_run(sigma), mean=MEAN_RUNS, sha


def sigma_for(strategy: Callable[[float], dp_accounting.DpEvent], epsilon: float, delta: flo
    """The smallest noise multiplier per run that keeps the whole strategy within (epsilon,

    The search starts at 0.1 and tries 1 first.
    """
    return dp_accounting.calibrate_dp_mechanism(
        dp_accounting.rdp.RdpAccountant, strategy, epsilon, delta, dp_accounting.LowerEndpoi
    )
```

```
one run, no tuning: sigma=1.029
grid of 6, all runs counted: sigma=1.816
random search, Poisson mean 4 runs: sigma=1.256
naive grid of 6 at sigma=1.029: reports epsilon=4.00
naive grid, true epsilon of all 6 runs = 9.84
```

The naive grid is the common mistake. It trains six models at the single-run noise, reports the $\varepsilon$ of one, and actually spends 9.84. The random search pays for its honesty with a modest rise in $\sigma$, from 1.029 to 1.256.

The search itself, with seed 0, drew only two runs, both at the learning rate 0.05. Its model reaches 0.828 on the test set, below the 0.846 of the run tuned on public data. A search with few runs can miss, and you may not redraw until you like the answer, because the guarantee depends on the randomness of the draw. So spend private runs on tuning only when no public data can stand in.

## 43.11. Measure utility and groups

Train at several budgets with everything else fixed, and compare against the baselines. Fix the budget you release before these runs, as the script does with $\varepsilon = 4$. The other runs train and are scored on private rows. Choosing $\varepsilon$ from them would be a use of private data that no $\varepsilon$ counts (Choosing ε).

Table 43.1.: Test accuracy on Adult against the privacy budget, with $\delta = 10^{-5}$ and one seed.

| Run | Noise multiplier | Test accuracy |
|---|---|---|
| Non-private | — | 0.847 |
| Clipped, no noise | 0 | 0.845 |
| $\varepsilon = 8$ | 0.722 | 0.846 |
| $\varepsilon = 4$ | 0.973 | 0.846 |
| $\varepsilon = 2$ | 1.431 | 0.845 |
| $\varepsilon = 1$ | 2.383 | 0.843 |
| $\varepsilon = 0.25$ | 8.125 | 0.793 |
| Always “≤50K” | — | 0.759 |

Privacy costs almost nothing on Adult down to $\varepsilon = 1$, and only at $\varepsilon = 0.25$ does accuracy drop, by about five points. At budgets of 1 and above, the differences are a thousandth or two on a single seed, too small to read as an effect.

Adult is easy for three reasons. The training set is large, so a batch of 1,000 still has a small sampling rate and divides the noise by 1,000. The model is small, and the noise’s total length grows with the square root of the number of trained weights. And the task suits a small network trained for ten epochs. Expect a steeper curve with a model of millions of weights trained from scratch, a few thousand records, a rare class or many more epochs. The running example of DP-SGD already loses several points at $\varepsilon = 1$. Public pretraining is the usual remedy (Public data and private fine-tuning).

An average can hide a group that pays, so check accuracy by group with exact (Clopper–Pearson) binomial intervals, which stay valid for small groups (Disparate impact).

```
in_group = sex == group
correct, total = int(right[in_group].sum()), int(in_group.sum())
interval = binomtest(correct, total).proportion_ci(confidence_level=0.95, method="exact")
```

```
non-private Female: 0.924 [0.917, 0.931]  (5,186/5,613)
non-private Male: 0.808 [0.800, 0.815]  (9,071/11,229)
DP Female: 0.922 [0.915, 0.929]  (5,177/5,613)
DP Male: 0.807 [0.800, 0.815]  (9,065/11,229)
```

At $\varepsilon = 4$ neither group pays a measurable price, since each private interval overlaps its non-private one almost entirely. The gap between the groups exists without privacy too, and it is the baseline a private model must be compared against.

## 43.12. When it goes wrong

Table 43.2 maps the common symptoms to their usual cause and fix.

Table 43.2.: Common DP-SGD failures and their fixes.

| Symptom | Likely cause | What to do |
|---|---|---|
| The validator reports errors | Batch normalization or an unsupported custom layer | `ModuleValidator.fix`, then build the optimizer on the fixed model's weights and retrain the baseline |
| "Model needs to be in training mode" | A model last used for evaluation | `model.train()` before `make_private` |
| Out of memory | Per-example gradients for a large physical batch | Lower `max_physical_batch_size`, or use ghost clipping: pass `grad_sample_mode="ghost"` and `criterion=` to `make_private`, which then returns the model, optimizer, loss function and loader, and train with that loss |
| Ten times slower than usual | Per-example gradients for every weight | Train fewer weights: freeze a public backbone, use adapters |
| Accuracy near the majority class | Noise too large for the signal | Check $\sigma C/B$; raise the batch, or train fewer epochs so that the accountant needs a smaller $\sigma$; start from a public model |
| $\varepsilon$ looks too good | A shuffling loader with `poisson_sampling=False`, uncounted tuning runs, or epochs changed after calibration | Check that the loader is Opacus's `DPDataLoader`, as the record's sampler does; `engine.get_epsilon` and the cross-check report the same $\varepsilon$ either way. Recount runs and epochs |

Accuracy and privacy fail independently. A run whose $\varepsilon$ is wrong usually trains normally, which is why the cross-check and the audit of Auditing private training exist.

## 43.13. Record the run

A reviewer can only check a claim whose inputs are written down, so the script writes a record of the run when it ends. It holds the unit, $n$ and $\delta$, and the accountant's three inputs, the noise multiplier, the sampling rate and the steps. It names the accountant and the library versions, and it gives the PRV, PLD and RDP values of $\varepsilon$. It reads the sampler from the loader Opacus returned and `secure_mode` from the engine, so it cannot claim Poisson sampling the code did not do. It also says how each setting was chosen, and whether that choice is counted in $\varepsilon$. Release and governance turns it into a statement.

## 43.14. Checklist

Table 43.3.: The steps of training with DP-SGD, their defaults and their checks.

| Step | Default | Check |
|---|---|---|
| Unit, $n$, $\delta$ | Person; $\delta \leq 1/n^{1.1}$ rounded down | $n$ counts people; $\delta < 1/n$ |
| Preprocessing | Public categories and bounds | No `fit` on private rows |
| Baselines | Non-private, clip-only, majority class | Clip-only close to non-private |
| Model | No batch normalization | `ModuleValidator.validate` returns nothing |
| Clipping, batch, epochs | $C$ near or below the median norm on public data; large $B$; epochs fixed | Chosen without private data |
| Noise | `get_noise_multiplier` with the reported accountant | Epochs and $q$ match the run |
| Training | Poisson sampling, `BatchMemoryManager`, `secure_mode=True` for release | The loader is Opacus's |
| Cross-check | PLD from `dp_accounting` | Agrees with PRV within rounding |
| Tuning | On public data; else random search with a Poisson number of runs | Every private run counted |
| Utility and groups | Several budgets; per-group exact intervals | Compared with the baselines |
| Record | Every parameter and version written | $\varepsilon$ recomputable from it |

## 43.15. References

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): the decisions of this chapter, with what to report.
- The Opacus documentation and tutorials (opacus.ai): the privacy engine, `BatchMemoryManager`, ghost clipping and the validator.

**Derivations.** This workflow draws on the full edition's chapters on DP-SGD in the full edition, hyperparameter tuning in the full edition, disparate impact in the full edition and choosing $\varepsilon$ in the full edition.

# 44. Hyperparameter tuning and private selection

## 44.1. Why this matters

Stochastic gradient descent (SGD) with clipped gradients and Gaussian noise satisfies differential privacy (DP) at the level its accountant reports for one run, and is called DP-SGD. But nobody trains once. You try several learning rates, score each model and ship the best, and every run and every score touched private data. So the search leaks too, and the privacy level $\varepsilon$ you report must pay for it.

With a fixed number of runs, releasing only the best can cost as much as releasing all of them. On the Adult census data, a grid of six runs that each meet $\varepsilon = 4$ really costs 9.84. That is less than the six costs added up (Section 44.3) but far above the 4 reported. If instead you draw the *number* of runs at random, from a distribution fixed in advance, the whole search costs little more than one run.

## 44.2. The procedure and the report

The cheapest tuning is the tuning you don't do on private data. Runs on public or proxy data cost nothing, and neither do published settings for similar tasks. Fix a large batch, the number of epochs and a small clipping norm in advance. With the clipping norm below the typical gradient norm, only its product with the learning rate matters, so that product is often the only thing left to search (DP-SGD).

If you must tune on private data, fix the search before you touch the data. Write down the candidate set, the distribution of the number of runs (Section 44.4), the score and the validation set. Trying "a few more", or restarting a disappointing search, makes the search depend on the data, and no accountant covers that. Make the score private as well, by scoring on records you treat as public or on noisy counts (Section 44.5).

Then account for the whole search. For two or three candidates, charge for every run (Section 44.3), which then costs little more than random search. For more candidates, use random search with a random number of runs (Section 44.4). Either way, count every run that touched the private data, including crashed and debugging runs and evaluations for early stopping.

Finally, log the candidate set, the drawn number of runs, and each run's hyperparameters, accounting settings and score. Then report two numbers. The $\varepsilon$ of the released run says what the model alone would leak if its hyperparameters had been public. The $\varepsilon$ of the whole procedure is what the release actually guarantees, and an audit cannot check it, since it tests only the released model. Set the slack $\delta$ by the book's default (Choosing $\varepsilon$), which gives $10^{-5}$ for Adult's 30,000 records. The Adult search of Section 44.6, whose noise was set so that the whole search costs $\varepsilon = 4$, reports as follows.

DP-SGD ran for 300 steps with Poisson sampling at rate 1/30. It clipped gradients to norm 1 and added noise with multiplier 1.256, so one run satisfies $(2.73, 10^{-5})$-DP. The learning rate was chosen by random search over $\{0.05, 0.1, 0.2, 0.5, 1, 2\}$. The number of runs was drawn from a Poisson distribution with mean 4, and each run was scored on 2,000 validation records treated as public. The whole procedure satisfies $(4.00, 10^{-5})$-DP for add/remove neighbours, datasets that differ by one record. It was computed with `dp_accounting` from the Rényi bound of Papernot & Steinke (2022).

## 44.3. Paying for every run

A common argument says that choosing among eight learning rates reveals at most three bits. It is wrong, because the release is a whole private run, picked because of how it came out. If each run says only "good" or "bad", the best is bad only when every run was bad, so a bad best reveals that all of them were. In the worst case that is as much as releasing every run. Tuning without privacy and then training once privately does not escape this. The winning regularization weight can change when a few outliers are added, so even that one number can reveal whether they were in the data (Membership inference).

The safe answer is to charge for every run, because the best run is computed from all the runs' outputs (Composition). For DP-SGD the charge is easy to compute, since a grid of $k$ runs, each with $T$ steps, costs exactly as much as one run of $kT$ steps. One run of the synthetic running example of DP-SGD costs $\varepsilon = 6.71$ at $\delta = 10^{-5}$. Ten such runs cost 27.23, about four times one run rather than ten, because the conversion to $\varepsilon$ pays its $\delta$-dependent cost once for the whole search.

## 44.4. A random number of runs

The way out is to draw the number of runs $K$ at random, independently of the data. Then a search whose runs all came out bad most likely drew few runs, which says little about the data. The whole search therefore costs about one run, plus a term that grows with the logarithm of the expected number of runs. For DP-SGD, use the version of Papernot & Steinke (2022). It works with the Rényi differential privacy (RDP) guarantee that DP-SGD's accountants already compute (Rényi DP).

To use it, fix the candidate set and the distribution of $K$, either Poisson or a truncated negative binomial with a shape parameter $\nu$ (Papernot and Steinke write it $\eta$). That family includes the geometric distribution, $\nu = 1$, which is the **random stopping** of Liu & Talwar, and the logarithmic one, $\nu = 0$. Then draw $K$, run that many candidates drawn at random from the set, and release the best one by a private score. Google's `dp_accounting` computes the cost when you wrap the event describing one run in a `RepeatAndSelectDpEvent`, with $\nu$ as its `shape` (infinite for Poisson), and compose it in an `RdpAccountant`. If each run's score is a noisy count on a validation set disjoint from its training set, the event for one run is the costlier of the two; otherwise compose both into one event. NumPy draws $K$ with `rng.poisson(mean)`, with `rng.geometric(1 / mean)` for the geometric case, and with `rng.logseries(p)` for the logarithmic one, whose mean is $-p/((1-p)\ln(1-p))$, so solve that for the $p$ that gives your mean.

Table 44.1.: The $\varepsilon$ at $\delta = 10^{-5}$ of a search over runs of the running example (one run: 6.71), from the full edition's chapter.

| Expected runs | 10 | 100 | 1,000 |
|---|---|---|---|
| Composition | 27.23 | 145.17 | 1035.93 |
| Poisson | 13.67 | 67.77 | 302.57 |
| Geometric ($\nu = 1$) | 11.31 | 14.32 | 16.76 |
| Logarithmic ($\nu = 0$) | 10.08 | 11.95 | 13.46 |

Table 44.1 shows how the choice plays out. The geometric and logarithmic searches barely grow from 10 to 1,000 runs, while composition grows almost linearly. For pure $\varepsilon$-DP runs, with $\delta = 0$, the search costs at most $(2 + \nu)\varepsilon$ however many runs it expects. So a smaller $\nu$ gives a cheaper bound, but the search also stops after one run more often. The Poisson bound needs runs that are already very private, and a run at $\varepsilon = 6.71$ is not, so here it grows almost like composition. A random number of runs may also miss the best candidate. For example, with eight candidates and a geometric number of runs with mean eight, it never tries the best one with probability 0.4667.

The theorems have three conditions, and each is easy to break. First, each run must draw its candidate independently from a fixed distribution, which is what random search does. Bayesian optimization picks each candidate from earlier scores, so it is not covered and needs adaptive composition. Second, the number of runs must be drawn independently of the data. And third, the score that ranks the runs must be private and counted in each run's cost (Section 44.5).

## 44.5. Selecting by a noisy validation score

Sometimes the candidates are fixed in advance, such as public pretrained checkpoints, and only the choice among them uses private data. That is a **selection** problem, which Selection solves with the exponential mechanism, a random choice weighted toward high scores. Score each candidate by its number of correct predictions, which one record changes by at most 1, rather than by an accuracy, which divides by a private set size. The mechanism's shortfall from the best count then grows only with the logarithm of the number of candidates.

For the random-number bounds, add Gaussian noise to each run's count of correct predictions, since an exact validation accuracy leaves the validation records unprotected. With noise of standard deviation 20, one noisy count costs $\varepsilon = 0.182$. If the validation set is disjoint from the training set, each record pays only one of the two costs. So a run costs the larger of its training cost and its count's cost, not their sum. On the running example the counts are the smaller, so they add nothing to the total.

## 44.6. Tuning on Adult

The script practitioner_tuning.py tunes the learning rate of a small network on the real Adult census data, with 30,000 private training records and 300 steps per run. It asks the accountant for the noise multiplier per run that each strategy can afford.

Table 44.2.: Tuning the learning rate on Adult at a total of $\varepsilon = 4$, $\delta = 10^{-5}$, with `dp_accounting`'s RDP accountant.

| Strategy | Noise multiplier per run | Total $\varepsilon$ |
|---|---|---|
| One run, no tuning | 1.029 | 4.00 |
| Grid of 6 at the noise of one run, one run's $\varepsilon$ reported | 1.029 | 9.84 (reported as 4.00) |
| Grid of 6, every run counted | 1.816 | 4.00 |
| Random search, Poisson number of runs with mean 4 | 1.256 | 4.00 |

The second row is the common reporting bug that Section 44.3 explains, and this snippet recomputes it together with the numbers of the report in Section 44.2.

```
import dp_accounting


def one_run(sigma: float) -> dp_accounting.DpEvent:
    step = dp_accounting.PoissonSampledDpEvent(1 / 30, dp_accounting.GaussianDpEvent(sigma))
    return dp_accounting.SelfComposedDpEvent(step, 300)


def epsilon(event: dp_accounting.DpEvent) -> float:
    return dp_accounting.rdp.RdpAccountant().compose(event).get_epsilon(1e-5)


search = dp_accounting.dp_event.RepeatAndSelectDpEvent(one_run(1.256), mean=4, shape=float("
naive_grid = dp_accounting.SelfComposedDpEvent(one_run(1.029), 6)
print(f"{epsilon(one_run(1.256)):.2f}")  # 2.73
print(f"{epsilon(search):.2f}")  # 4.00
print(f"{epsilon(naive_grid):.2f}")  # 9.84
```

The search itself, run with `uv run python code/part05-dp-ml/practitioner_tuning.py` and seed 0, was unlucky. It drew only two runs, both at the smallest learning rate, and its model reaches 0.828 on the test set. The run of Train a model with DP-SGD, whose learning rate was chosen for free on public stand-in rows, reaches 0.846 at the same target $\varepsilon = 4$. So a search with few runs can miss (Tune honestly).

## 44.7. Cheat sheet

Table 44.3.: Which tuning strategy for which search.

| Situation | Strategy | Cost |
|---|---|---|
| Two or three candidates | Fixed grid, composition | $k$ runs of $T$ steps = one run of $kT$ steps |
| Many candidates | Random search; compute Poisson, geometric and logarithmic for your per-run noise and mean, and take the cheapest | Poisson wins only for very private runs; at one run of 6.71, logarithmic is cheapest at every size in Table 44.1, but it stops after one run more often |
| Fixed candidates | Exponential mechanism on correct counts | Shortfall grows with the log of the candidate count |
| Trained candidates, disjoint validation set | Exponential mechanism or noisy counts | The larger of training and selection costs |

**Rules of thumb**

- Tune on public or proxy data when you can. Otherwise fix the search before touching the private data, and never extend it.
- Report the whole search's $\varepsilon$, never the selected run's alone.

## 44.8. References


- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022): the Poisson and truncated negative binomial bounds for DP-SGD.
- Liu & Talwar, "Private Selection from Private Candidates" (STOC 2019): random stopping and its bound of three times one run.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 45. User-level differential privacy

## 45.1. Why this matters

Every differential privacy (DP) guarantee in this part so far protects one record. Stochastic gradient descent (SGD) with clipping and noise, this part's DP-SGD, gives each record of its running example a privacy loss $\varepsilon$ of 6.71. But a messaging user contributes hundreds of messages, and together a person's records can reveal far more than any one of them.

The fix rests on one idea. A DP guarantee protects whatever differs between two neighbouring datasets, so **the neighbouring relation sets the privacy unit**. For the records, this chapter gives the running example's 10,000 synthetic records to 1,000 users, very unevenly. The median user holds 4 records and the largest 348.

## 45.2. The privacy unit and δ

Take a mechanism $M$, and call two datasets neighbours, $D$ and $D'$, when they differ by adding or removing all the records of one user. The mechanism is **user-level** $(\varepsilon, \delta)$-DP, with a small slack $\delta$, if every such pair and every set $S$ of outputs satisfy

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{45.1}$$

**Example-level** DP is the same inequality for datasets that differ in one record. So user-level DP is not a new notion, and every property and accountant of Part 4 holds for it unchanged. What changes is the sensitivity, because one user can now move the computation with all of their records at once.

The right unit, whether a record, a user, a household or a device, answers the question "whose participation do I want to hide?" Report the unit with every $\varepsilon$ (Report and review an ε claim). For a language model it is usually a document or a user, not a sentence (Fine-tune a language model privately). The unit also sets the slack, because the book's default $\delta$ (Choosing ε) now counts users. For 1,000 users it gives $10^{-4}$ (Section 45.8). This chapter's $10^{-5}$ is therefore stricter than it needs to be.

The textbook answer for a person with $k$ records is **group privacy** (Post-processing, group privacy and composition). It says that an example-level $(\varepsilon, \delta)$-DP mechanism is also user-level DP for users with at most $k$ records. The level becomes $k\varepsilon$, and the slack becomes

$$\delta_k = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1}. \tag{45.2}$$

The slack is worse, because the fraction grows like $e^{(k-1)\varepsilon}$, which is why a record-level $\delta$ must never be read as a user's. Holding a user's slack at $10^{-5}$ needs a much smaller per-record $\delta$, and so a

larger per-record $\varepsilon$. On the running example a user with two records therefore gets $\varepsilon = 18.61$, not twice 6.71. For ten records no per-record $\delta$ is small enough, and group privacy gives no guarantee at all.

## 45.3. What an example-level model gives a user

Group privacy treats DP-SGD as a black box, but you know exactly how a user's records enter each step. Each step includes every record independently with the sampling rate $q$, so a step sees some record of the user's with probability about $kq$. While $kq$ is small, two of their records rarely share a batch, and **a user with $k$ records looks like one record sampled at rate $kq$**. So you can screen a user's exposure by rerunning the record-level accountant at the rate $kq$, as the snippet in Section 45.8 does. An exact accountant for a user with $k$ records also exists (Charles et al. 2024).

Table 45.1.: The user-level $\varepsilon$ at $\delta = 10^{-5}$ of the running example's example-level DP-SGD ($q = 0.01$, $\sigma = 1$, 10,000 steps) for a user with $k$ records, from the full edition's Rényi accountant (6.72 at one record comes from its coarser grid of orders).

| Records of the user $k$ | 1 | 2 | 5 | 10 | 50 |
|---|---|---|---|---|---|
| Exact user accountant | 6.72 | 15.54 | 52.36 | 146.20 | 2161.43 |
| One record at rate $kq$ | 6.71 | 15.32 | 50.11 | 134.50 | 1673.08 |

At ten records the exact accountant gives 146.20, no protection to speak of. Still, it beats group privacy for every user with more than one record, so never use group privacy for DP-SGD. The rate-$kq$ screen stays within 8% of the exact accountant up to ten records but always comes out a little low. So report the exact value when you have an implementation of it. The libraries this book uses offer only the screen. Then report the screen's value, call it a screen that can be a few percent low, and keep $kq$ small. Treat it as the guarantee, not as a pessimistic bound, because a person's similar records do push the gradient partly the same way.

## 45.4. User-level DP-SGD

The better route makes the user the unit that DP-SGD samples, clips and accounts for (McMahan et al. 2018), as federated learning does with a device as the user. Starting from weights chosen without looking at the data, each step does five things.

1. **Sample users.** Put each user in the batch independently with probability $q$, now the user sampling rate.
2. **Compute one vector per user.** From that user's records alone, compute their average gradient, or the change in weights after a few training steps on their records.
3. **Clip per user.** Rescale that vector to length at most the clipping norm $C$.
4. **Add noise.** Sum the clipped vectors and add Gaussian noise of standard deviation $\sigma C$, for a noise multiplier $\sigma$.
5. **Step.** Divide by the expected number of sampled users and take a gradient step.

One user moves the sum by at most $C$, however many records they have. So the guarantee is user-level DP with exactly the accountant of example-level DP-SGD, run at the user sampling rate.

Changing the unit has two consequences. First, **you pay for having fewer units**. To see how much, take a shorter schedule on the running example, 1,000 steps with about 100 units per batch. Sampling records then needs $\sigma = 0.778$ for $\varepsilon = 4$. Sampling users instead needs a rate ten times higher, so the same budget needs $\sigma = 3.78$. Second, **records inside a user are free**, since they can be averaged, used for several training steps or subsampled without changing $\varepsilon$.

Start with the data plumbing, and carry a user identifier through the pipeline to sample by. Split training and test data by user too, so that no user's records sit on both sides. Opacus clips each row of a batch, so for user-level training you write a data loader that yields users and clip each user's vector yourself. Three things break the guarantee:

1. A user's vector depends on other users' data, for example through normalization statistics computed over all users.
2. One person is spread over several accounts, so the unit is the account and the person is again a group.
3. Users are iterated in a fixed shuffled order instead of Poisson-sampled, which is a different mechanism that this accountant does not cover (Matrix factorization).

## 45.5. Contribution bounding

A pipeline built around records can still protect users. **Contribution bounding** keeps at most $k$ records of each user, the cap, and runs example-level DP-SGD on what remains. The result is user-level DP with the $\varepsilon$ that the user accountant gives for $k$ records. The condition is that each user's kept records are chosen without looking at other users' data, for example at random. Otherwise adding one user could change which records other users keep, and neighbours would differ by more than one user's records. In Polars, shuffle the rows with a fixed seed and keep the first $k$ of each user. Releases of statistics bound customers the same way, on the Online Retail invoice data (Release statistics with a library). To calibrate the noise for a cap, solve for $\sigma$ with `dp_accounting.calibrate_dp_mechanism` at the rate $kq$, as the tuning step of Train a model with DP-SGD does at rate $q$.

```
import polars as pl

rows = pl.DataFrame({"user": ["a", "a", "a", "a", "b", "c", "c"], "x": [1, 2, 3, 4, 5, 6, 7]
k = 2  # the cap, fixed from public knowledge of the population
capped = rows.sample(fraction=1.0, shuffle=True, seed=0).group_by("user").head(k)
print(capped.height)  # 5
print(capped.group_by("user").len()["len"].max())  # 2
```

Choosing the cap trades noise against bias (Amin et al. 2019). A larger cap keeps more data but needs more noise, while a smaller cap discards records and reweights the data toward light users. On the running example at a user-level $\varepsilon = 4$, raising the cap from one record to fifty multiplies the noise by 5.8. That is far less than fifty, because the batch keeps its size and so the sampling

rate falls as the cap rises. Accuracy still falls from 0.8917 to 0.6401. So take the cap from public knowledge of the population. Or choose it privately and count the cost, for example from a private quantile of records per user (Beyond global sensitivity). Trying caps on test accuracy spends privacy that the accountant does not see.

## 45.6. Heterogeneous users

User-level training also changes the objective, because averaging per user weights users equally, while example-level training weights them by their number of records (Disparate impact). If heavy users should count for more, sum a user's gradients instead of averaging them before clipping. Their influence then rises with their records until clipping caps the sum at $C$, and $\varepsilon$ does not change.

## 45.7. Which strategy

Table 45.2 compares three routes on the running example, each with 1,000 steps and about 100 units per batch. Strategy (a) runs example-level DP-SGD on all records and accounts for the heaviest user.

Table 45.2.: Test accuracy at the same user-level budget, $\delta = 10^{-5}$, one run each. The non-private model reaches 0.9612.

| User-level $\varepsilon$ | (a) All records, $k = 348$ | (b) User-level DP-SGD | (c) At most 2 records per user |
|---|---|---|---|
| 1 | 0.5318 | 0.8065 | 0.7084 |
| 2 | 0.5582 | 0.9028 | 0.8374 |
| 4 | 0.5713 | 0.9347 | 0.8810 |
| 8 | 0.5478 | 0.9442 | 0.9333 |

Keeping the record as the unit fails. Strategy (a) stays near chance on this two-class task at every budget, because the heaviest user forces a noise multiplier between 70.3 and 449 on everyone. User-level DP-SGD is the best at every budget, and at $\varepsilon = 4$ it is 2.7 points below the non-private model. Its cost is compute, since each step processes all the records of about 100 users. So the cap is the fallback when the pipeline must stay record-based, and on this task it trails by 5 to 10 points at the smaller budgets. How far it trails depends on how records are spread over users, so measure it on your data.

## 45.8. Code

The rate-$kq$ screen needs only a record-level accountant, such as Google's `dp_accounting`. This snippet screens users with one and two records, and computes the default $\delta$ for 1,000 users.

```python
import math

import dp_accounting


def screen(k: int, q: float = 0.01, sigma: float = 1.0, steps: int = 10_000) -> float:
    """The epsilon of a user with k records, screened as one record sampled at rate k * q."""
    step = dp_accounting.PoissonSampledDpEvent(k * q, dp_accounting.GaussianDpEvent(sigma))
    accountant = dp_accounting.rdp.RdpAccountant()
    accountant.compose(dp_accounting.SelfComposedDpEvent(step, steps))
    return accountant.get_epsilon(1e-5)


print(f"{screen(1):.2f}")  # 6.71
print(f"{screen(2):.2f}")  # 15.35
print(10 ** math.floor(math.log10(1_000**-1.1)))  # 0.0001
```

Its 15.35 is a little above the table's 15.32, because the two accountants compute the Rényi guarantee at different orders, so quote the one you ran and name it. With $k = 1$ and $q$ the user sampling rate, the same code accounts for user-level DP-SGD.

## 45.9. Cheat sheet

Table 45.3.: Which route to a user-level guarantee.

| Situation | Use | What you get |
|---|---|---|
| Example-level model already trained | User accountant, or the rate-$kq$ screen | A user's $\varepsilon$ grows about linearly in $k$, then faster for large $kq$ |
| Records can be grouped by user | User-level DP-SGD | DP-SGD's accountant over users |
| Pipeline must stay record-based | Contribution bounding with a public cap | User accountant for $k$ records; loses data |
| Heavy users should count more | Sum before clipping instead of averaging | Same $\varepsilon$, different objective |

### Rules of thumb

- Name the privacy unit with every $\varepsilon$, and set $\delta$ from the number of users.
- Never use group privacy for DP-SGD; screen a user at rate $kq$ instead.

## 45.10. References

- Charles et al., "Fine-Tuning Large Language Models with User-Level Differential Privacy" (arXiv 2407.07737, 2024): the tight user accountant, and sampling records against sampling users.
- Chua et al., "Mind the Privacy Unit! User-Level Differential Privacy for Language Model Fine-Tuning" (Conference on Language Modeling 2024): contribution bounding against user-level DP-SGD.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 46. Choosing ε

## 46.1. Why this matters

Every model trained with differential privacy (DP) ships with a privacy budget $\varepsilon$. The chapter on private stochastic gradient descent (SGD), DP-SGD, showed what each budget costs in accuracy, which is half of the choice. The other half is what it means for the people in the data. Without it, $\varepsilon$ gets chosen by habit ("everyone uses 8"), and it does not survive a reviewer who asks what the number promises.

An attacker who wants to know whether a record was used for training runs a membership test. The test can catch real members, its true positives, and it can wrongly flag non-members, its false positives. The budget $\varepsilon$, together with its small slack $\delta$ (The definition of differential privacy), caps the rate of true positives in terms of the rate of false positives. But each training run also has its own curve of the best rates an attacker can reach, and that curve usually lies far below the cap. Choosing $\varepsilon$ well means computing the curve, reading it at the false positive rate your threat calls for, and setting it beside the accuracy that each budget buys.

## 46.2. Justifying a choice

A defensible choice says what the budget protects before it picks the number, in six steps.

1. **Unit and period.** Say whether one unit is a record, a user or a device, and whether neighbouring datasets differ by adding or removing one unit or by replacing one. Say which runs and searches the budget covers, since every run used to tune hyperparameters spends budget too (Hyperparameter tuning).
2. **Threat.** Say whether the concern is membership, reconstruction or attribute inference, and what the attacker knows and sees. This decides the false positive rate at which you read the curve, a low one for an attacker who must rarely accuse a non-member.
3. **Whole guarantee.** Choose $\delta$ by the book's default (Section 46.5), then compute the trade-off curve (Section 46.3) and read it there.
4. **Evidence.** The curve is a theorem about an algorithm, not about your code, so run an audit that could refute it (Auditing private training).
5. **Utility.** Put accuracy against $\varepsilon$ next to the risk, and pick the smallest budget past the bend, where accuracy stops improving. Draw it on public or proxy data, because private runs that choose $\varepsilon$ are a search whose cost the reported $\varepsilon$ leaves out.
6. **Comparison.** Only now compare with deployments (Section 46.6) and with the tiers in the rules of thumb at the end of this page.

Adult, a public census survey, shows step 5 on a real run. Train a model with DP-SGD trains one small network on 30,000 of its people at five budgets, each with $\delta = 10^{-5}$ and with the noise multiplier that meets it. Table 46.1 sets the test accuracies beside a measure of risk, the largest true positive rate (TPR) that any membership test can reach at a false positive rate (FPR) of $10^{-3}$. The snippet in Section 46.3 computes it.

Table 46.1.: Accuracy and worst-case membership risk on Adult, one seed.

| $\varepsilon$ | Noise multiplier | Test accuracy | Largest TPR at FPR $10^{-3}$ |
|---|---|---|---|
| 0.25 | 8.125 | 0.793 | 0.0013 |
| 1 | 2.383 | 0.843 | 0.0023 |
| 2 | 1.431 | 0.845 | 0.0046 |
| 4 | 0.973 | 0.846 | 0.0123 |
| 8 | 0.722 | 0.846 | 0.0445 |
| Non-private | — | 0.847 | — |

Accuracy rises by five points from $\varepsilon = 0.25$ to 1, then stays within a few thousandths of the non-private model, which on one seed is noise. The risk keeps growing, though, and faster with each doubling of the budget, from 0.0023 at $\varepsilon = 1$ to 0.0445 at $\varepsilon = 8$. So on Adult the bend sits at $\varepsilon = 1$, and a budget of 1 or 2 is the defensible choice. These runs trained and were scored on private rows, so here the table illustrates step 5; a real choice reads it from a public stand-in. Release and governance releases the $\varepsilon = 4$ run to show the steps, and a reviewer would be right to ask why it spends more than accuracy needs. Expect the bend further right on a harder task, a larger model or a smaller dataset.

## 46.3. The pair and the whole curve

A membership test says "member" or "non-member" about one target record. Its TPR is the chance that it says "member" when the target was in the training set, and its FPR the chance that it says so when the target was not. Every test against an $(\varepsilon, \delta)$-DP mechanism satisfies

$$\mathrm{TPR} \le \min\left\{1,\ e^{\varepsilon}\,\mathrm{FPR} + \delta,\ 1 - e^{-\varepsilon}(1 - \delta - \mathrm{FPR})\right\}. \tag{46.1}$$

The middle term is the one that binds at low FPR, and the last applies the same cap to the members a test misses. Attacks are judged at a low FPR, because an attack that rarely flags a non-member identifies people with confidence (Membership inference). Yet at $\varepsilon = 8$ and an FPR of $10^{-3}$, the pair caps the TPR at 0.9997, which constrains almost nothing.

A real training run satisfies DP for a whole range of pairs, one $\delta(\varepsilon)$ for each $\varepsilon$, because its accountant can trade a larger $\varepsilon$ for a smaller $\delta$. Each pair gives a line, the middle term of Equation 46.1, and every test must stay under all of these lines at once. The best test reaches the lowest line exactly, and that line is called the **trade-off curve**:

$$\mathrm{TPR}^{*}(\mathrm{FPR}) = \min_{\varepsilon}\left(e^{\varepsilon}\,\mathrm{FPR} + \delta(\varepsilon)\right). \tag{46.2}$$

For DP-SGD, a privacy loss distribution (PLD) accountant computes this curve. It can round the privacy loss up or down, and the two curves it gets bracket the true one (Numerical accounting).

Table 46.2 compares the pair with the curve for this part's running example from DP-SGD, a synthetic run sampled at rate 0.01 for 10,000 steps.

Table 46.2.: Largest TPR of a membership test against the running example at $\delta = 10^{-5}$, from the pair and from the curve.

| $\varepsilon$ | Noise multiplier | Pair, FPR $10^{-3}$ | Curve, FPR $10^{-3}$ | Curve, FPR $10^{-2}$ |
|---|---|---|---|---|
| 4 | 1.36 | 0.05461 | [0.01215, 0.01312] | [0.06739, 0.0724] |
| 8 | 0.917 | 0.9997 | [0.05555, 0.0589] | [0.1987, 0.2092] |

So a budget of 8 means little as a bare pair, while its curve caps the TPR at 0.0589 at an FPR of $10^{-3}$.

With Google's `dp_accounting`, build the run's PLD, ask it for $\delta$ on a grid of $\varepsilon$, and take the lowest line. The snippet's first print, for the running example at $\varepsilon = 4$, falls inside the bracket of Table 46.2. Its second uses Adult's sampling rate of 1/30 and 300 steps, and gives the last column of Table 46.1.

```
import numpy as np
from dp_accounting.pld import privacy_loss_distribution as pld

EPS_GRID = np.linspace(0.0, 20.0, 2001)


def tpr_at(sigma: float, q: float, steps: int, fpr: float) -> float:
    run = pld.from_gaussian_mechanism(sigma, sampling_prob=q).self_compose(steps)
    deltas = np.array(run.get_delta_for_epsilon(EPS_GRID))
    return float(np.min(np.exp(EPS_GRID) * fpr + deltas))


print(f"{tpr_at(1.36, 0.01, 10_000, 1e-3):.4f}")  # 0.0126
adult = [tpr_at(sigma, 1 / 30, 300, 1e-3) for sigma in (8.125, 2.383, 1.431, 0.973, 0.722)]
print(" ".join(f"{tpr:.4f}" for tpr in adult))  # 0.0013 0.0023 0.0046 0.0123 0.0445
```

The library's default PLD rounds the loss up, so its curve is a valid upper bound. A Rényi differential privacy (RDP) accountant gives a valid but looser curve. The Gaussian differential privacy (GDP) approximation is quick but not a bound, so use it to explain and the PLD to certify.

The curve is also a worst case. It describes a strong attacker who sees every intermediate model, and a typical record attacked through the final model alone is much safer. It also bounds reconstruction, since guessing which of $m$ equally likely records is the target is a membership test at an FPR of $1/m$ (Reconstruction attacks).

## 46.4. What an attacker can believe

Most people ask how sure an attacker can become that a given person was in the data. Write $p$ for the attacker's **prior**, its probability of "member" before it sees the model. Suppose no output is more than $e^{\varepsilon}$ times as likely with the person's record as without it, which is what a privacy loss of at most $\varepsilon$ means. Then the attacker's belief afterwards is at most

$$\Pr[\text{member} \mid \text{output}] \le \frac{p\, e^{\varepsilon}}{p\, e^{\varepsilon} + 1 - p}. \tag{46.3}$$

DP bounds the change of belief, not the belief itself. An attacker who starts at 1% can end at 35.55% when $\varepsilon = 4$. That is the worst output, though, and in DP-SGD the privacy loss of the model actually released is usually far below $\varepsilon$.

Base rates matter as much as the loss. An attacker who accuses every person its test flags as a member is right with probability

$$\Pr[\text{member} \mid \text{flag}] = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}}. \tag{46.4}$$

Take the running example's $\varepsilon = 8$ run and an attacker with a 1% prior who flags at an FPR of $10^{-2}$. The curve caps its TPR there at 0.2092 (Table 46.2), so its flags are right with probability at most 0.1744, and five out of six are false. An insider with a prior of 0.5 would be right far more often, so a statement about beliefs must name the attacker.

## 46.5. How small δ must be

A deliberately bad mechanism shows what $\delta$ permits. It publishes each of $n$ records in full, independently with probability $\delta$. It is $(0, \delta)$-DP, because adding or removing one record changes the output only when that record's own coin comes up. Yet it publishes at least one record with probability

$$1 - (1-\delta)^n \approx 1 - e^{-n\delta}. \tag{46.5}$$

For 10,000 records at $\delta = 1/n$, that is 0.6321. So $n\delta$ must be below 1. Here $n$ counts privacy units, the thing one guarantee protects, which means people when one person has many records (User-level differential privacy).

This book uses one default throughout. Take $\delta \le 1/n^{1.1}$ and round it down to a power of ten, which keeps $n\delta$ below 1 for every $n$. It gives $10^{-5}$ for both of this page's runs, and the `delta_for` function of Train a model with DP-SGD computes it. When $n$ is just above a power of ten, as with Adult's 30,000 people, the default is not far below $1/n$. That is one more reason to report a second, smaller $\delta$. NIST SP 800-226, the US guidelines for evaluating DP claims, reports a stricter common recommendation of $1/n^2$, and calls a $\delta$ above $10^{-5}$ suspicious unless justified.

For DP-SGD a smaller $\delta$ is cheap. The RDP accountant that set the running example's $\varepsilon = 4$ run puts it at only 5.209 when $\delta$ falls to $10^{-8}$. So report $\varepsilon$ at a second, smaller $\delta$ too, and never quote a $\delta$ as large as one over the number of units.

## 46.6. Comparing with deployed values

"Industry uses $\varepsilon$ between 1 and 10" hides what every $\varepsilon$ needs, its **privacy unit** and the **period** over which it is spent. The Census Bureau's 2020 redistricting data used zero-concentrated differential privacy (zCDP), which states the whole guarantee as one number, written $\rho$, that converts to $(\varepsilon, \delta)$ pairs. The unit was one person's record under the replace-one relation, and persons got $\rho = 2.56$, which the Bureau reported as $\varepsilon = 17.14$ at $\delta = 10^{-10}$. Google's Gboard keyboard models, trained in federated learning on users' phones, had per-device guarantees for a whole training run, from about 1 to 14 at the same $\delta$.

So convert both budgets to the same unit, neighbouring relation and $\delta$ before comparing. A zCDP parameter $\rho$ converts to a pair by

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{46.6}$$

For the Census figures this formula gives 17.92. The Bureau's 17.14 comes from a tighter conversion. Since a replacement is a removal followed by an addition, an add/remove guarantee $(\varepsilon, \delta)$ implies only the replace-one guarantee $(2\varepsilon, (1+e^{\varepsilon})\delta)$ (The definition of differential privacy). And a budget stated per day or per report adds up over time by composition, to 365 a year at $\varepsilon = 1$ per day.

So it would be wrong to call the running example's $\varepsilon = 8$ run "less than half the Census budget". At the Census's $\delta = 10^{-10}$ it is at 11.627, and under the Census's replace-one relation that can double.

## 46.7. Cheat sheet

Table 46.3.: Which tool answers which question when choosing $\varepsilon$.

| Question | Tool | Notes |
|---|---|---|
| What does DP-SGD allow? | The trade-off curve, Equation 46.2 | Certify with the PLD, not with GDP |
| How sure can an attacker become? | Equation 46.3 and Equation 46.4 | Name the prior; base rates dominate |
| How small a $\delta$? | $\delta \leq 1/n^{1.1}$, rounded down to a power of ten | $n$ counts privacy units |
| Comparing with a deployment? | Equation 46.6; replace-one doubles $\varepsilon$ | Same unit and $\delta$ |

**Rules of thumb**

- A budget of at most 1 is strong for any mechanism. A budget between 1 and 10 needs DP-SGD's curve as justification, and a budget above 10 needs the curve and an audit (Ponomareva et al. 2023).
- Never quote $\varepsilon$ without its unit, $\delta$ and period.

## 46.8. References


- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, 2025): privacy units, $\delta$, and the hazards of large budgets.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): tiers and reporting.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 47. Public data and private fine-tuning

## 47.1. Why this matters

The differential privacy (DP) version of stochastic gradient descent (SGD), DP-SGD, loses much accuracy when it trains a model from random weights. The loss is large at a privacy loss $\varepsilon$ of 1 or below.

The most accurate private models avoid that. They start from a model **pretrained** without privacy on a large public dataset, and only **fine-tune** it privately on the sensitive data. That makes them far more accurate than models trained privately from scratch (Tramèr & Boneh 2021).

Fine-tuning only a small part of the model often helps further. DP-SGD adds noise to every weight it trains, while the useful signal does not grow with their number, so fewer weights mean less noise for the same signal. This chapter says which weights to train, and what public data does *not* protect, and the workflow Fine-tune a language model privately with LoRA does the job end to end.

## 47.2. How many weights to train

Which weights you train leaves $\varepsilon$ unchanged, because the guarantee of DP-SGD needs only that one record changes the clipped gradient sum by at most the clipping norm $C$. The choice changes only how much noise lands on the model. Anything else computed from public data alone, such as the starting weights, also costs no privacy, provided nothing in it was chosen by looking at the private data. So a tokenizer or normalization statistics fitted on public data are free, while fitted on the private data they leak.

The noise grows with the number $d$ of trainable weights. Write $\sigma$ for the noise multiplier and $B$ for the expected batch size, expected because DP-SGD's Poisson sampling draws batches of random size. Then the noise on the averaged gradient has length about

$$\text{noise length} \approx \frac{\sigma C \sqrt{d}}{B}. \tag{47.1}$$

The signal, the average clipped gradient, is at most $C$ long whatever $d$ is. It can even get longer when fewer weights train, because clipping then cuts each record's gradient less.

The running example of DP-SGD, a small network with 3,329 weights, is pretrained on a public task close to the private one and then fine-tuned on the 10,000 private records. It trains for 10,000 steps with batches of 100 and a clipping norm of 0.5, and at $\varepsilon = 1$ the noise multiplier is 4.13. Compared with the signal at the pretrained weights, the noise is then 92.0 times longer when all weights train. It is only 6.7 times longer when the 65 weights of the last layer train. The gap

is wider than the formula predicts, because the signal is longer when only the last layer trains (0.0250 against 0.0130).

Counting weights overstates the cost of training them all, though. Noise does harm only along the directions in which the loss is curved, because a step along a flat direction barely changes the loss. Near a pretrained model few directions are curved (Li, Liu et al. 2022). So on the running example, full fine-tuning has 51 times the last layer's weights, but its noise is predicted to raise the training loss only about 7.5 times as much.

A larger learning rate lets the noise into more of the nearly flat directions within a run, so private fine-tuning wants smaller steps than training from scratch. Training fewer weights has its own cost, a **bias**, because no amount of private data can move the model along a frozen direction that the task needs.

## 47.3. Linear probing first

The simplest option freezes everything but the last layer and trains a linear classifier on the frozen features, which is **linear probing**. Because the features are fixed, the probe's loss is convex. Linear probing thus turns private deep learning back into the problem of Private empirical risk minimization, which makes it the safe first baseline.

The error of a private probe is the error that privacy adds to convex empirical risk minimization (ERM), which grows with the number $d$ of features, plus the **feature gap**. That gap is how far the best linear model on the features falls short of the best model of any kind. No number of records or size of $\varepsilon$ closes it. For example, the running example's probe fitted without privacy, its ceiling, has a test accuracy of 0.9536, while a network trained without privacy from scratch reaches 0.9606. The private probe stays within 0.006 of that ceiling, but never passes it.

Measure the probe's ceiling first, by fitting it without privacy on labelled public data that resemble the task. If it already meets your target, stop there. To train the probe privately, feed the frozen features to the `LogisticRegression` of diffprivlib, IBM's library of private scikit-learn models, or run DP-SGD on the one layer.

## 47.4. Low-rank adapters and full fine-tuning

When the features are not quite right, the probe stops at a ceiling below your target, and only changing the earlier layers can lift it. **Low-rank adaptation** (LoRA, Hu et al. 2022) freezes each pretrained matrix and learns a small correction to it. The correction is the product of two thin matrices that each have $r$ columns, where $r$ is the rank. A matrix with $d_{\text{in}}$ inputs and $d_{\text{out}}$ outputs has $d_{\text{in}}d_{\text{out}}$ weights, while its adapter has only

$$d = r(d_{\text{in}} + d_{\text{out}}). \tag{47.2}$$

The adapter comes from Hugging Face's `peft` library. This snippet puts a rank-4 adapter on the first layer of a network shaped like the running example, and trains its last layer in full.

```
import torch
from peft import LoraConfig, get_peft_model

model = torch.nn.Sequential(torch.nn.Linear(50, 64), torch.nn.Tanh(), torch.nn.Linear(64, 1)
print(sum(p.numel() for p in model.parameters()))  # 3329
config = LoraConfig(r=4, lora_alpha=8, target_modules=["0"], modules_to_save=["2"], lora_dro
adapted = get_peft_model(model, config)  # freezes every pretrained weight
print(sum(p.numel() for p in adapted.parameters() if p.requires_grad))  # 521
```

In the snippet, `target_modules` and `modules_to_save` name layers by their index in the `Sequential`, and `lora_alpha` scales the correction. It trains 521 weights, the adapter's $4 \times (50 + 64) = 456$ and the last layer's 65. The run in Section 47.5 treats the first layer's biases as a 51st input, so its adapter has $4 \times (51 + 64) = 460$ weights, and 525 with the last layer. Opacus's `PrivacyEngine`, used as in DP-SGD, then computes per-example gradients only for the parameters that require them, so the frozen weights cost nothing. Call `model.train()` before `make_private`, since Opacus rejects a model left in evaluation mode, as a pretrained model often is after you measure it.

The adapter's second factor starts at zero, so training begins exactly at the pretrained model. The first factor starts random, and that draw must not depend on the private data.

Yu et al. (2022) showed that private fine-tuning of language models with LoRA matches or beats private full fine-tuning, at a fraction of the compute and memory. Each adapter factor is an ordinary linear layer, so the ghost clipping of DP-SGD applies to it. It gets each record's gradient norm without ever forming the per-example gradient.

Full fine-tuning is not ruled out. Li, Tramèr et al. (2022) matched the adapters by fine-tuning all the weights of language models. They used large batches and a learning rate tuned for private training, so the evidence goes both ways. A third option adds noise only in the few directions in which public gradients mostly lie, and throws away the private signal outside them (Zhou, Wu & Banerjee 2021).

## 47.5. Which method at which budget

The numbers in this section come from the full edition's synthetic running example. No practitioner script reproduces them, so treat them as one task's evidence. On it, DP-SGD fine-tunes the last layer, a rank-4 adapter plus the last layer, or all weights, against a model trained privately from scratch. Each method's learning rate was chosen at $\varepsilon = 1$ on half of the private test set, and accuracy is reported on the other half. That choice looked at private data without paying for it, so the table compares methods at their best rather than what a real run could claim (Hyperparameter tuning).

Table 47.1.: Test accuracy (mean of two runs), $\delta = 10^{-5}$. The pretrained model without fine-tuning has 0.9035.

| | Scratch | Full | Last layer | Adapter, rank 4 |
|---|---|---|---|---|
| Trainable weights $d$ | 3,329 | 3,329 | 65 | 525 |
| Learning rate | 0.25 | 0.1 | 0.3 | 0.1 |
| $\varepsilon = 0.5$ | 0.8589 | 0.9395 | 0.9478 | 0.9464 |
| $\varepsilon = 1$ | 0.9246 | 0.9599 | 0.9497 | 0.9616 |
| $\varepsilon = 2$ | 0.9518 | 0.9663 | 0.9519 | 0.9653 |
| $\varepsilon = 8$ | 0.9611 | 0.9677 | 0.9514 | 0.9655 |

Here pretraining is worth about a factor of eight in $\varepsilon$, since the adapter at $\varepsilon = 1$ matches training from scratch at $\varepsilon = 8$.

At $\varepsilon = 0.5$ the last layer is best, because its noise is 7.2 times shorter than full fine-tuning's and its ceiling is not yet binding. From $\varepsilon = 1$ on, it is stuck near its ceiling while the others keep improving. The adapter is never more than 0.0022 below the best method at any budget, so it is the robust middle choice here. That ranking depends on the task. A public task further from the private one can reverse it even when the probe's ceiling looks fine. So rerun the comparison on a public stand-in for your task before you commit.

The learning rate matters as much as the method, since a step three times too large costs full fine-tuning 1.3 percentage points of accuracy. The adapter's rank, by contrast, is cheap to get roughly right, because at $\varepsilon = 1$ ranks 1, 4 and 16 differ by about the run-to-run spread.

## 47.6. What public data does not buy

Whatever the public data are, training on the private data is $(\varepsilon, \delta)$-DP, where $\delta$ is the small additive slack the guarantee allows. The public data get no guarantee of any kind. Non-private pretraining can memorize individual records (Memorization and training-data extraction), and the fine-tuned model inherits whatever the pretrained weights encode (Tramèr, Kamath & Carlini 2024).

A person in both datasets is protected only in the private copy. A membership attack against the pretrained model (Membership inference) has no bound at all. Web-scale "public" datasets contain posts, profiles and leaked documents, so private notes or emails may well overlap with them.

Text raises a second question, which unit the guarantee protects, called the privacy unit. A language model trains on fixed-length sequences, but one person's documents span many of them, so a guarantee per sequence protects far less than a person (User-level differential privacy).

Check three things before you train and when you report.

1. Check that the public data resemble the private task, and deduplicate the private set against them. Deduplication protects copies, not people, since exact matching misses paraphrases.
2. Choose the public model and the hyperparameters on public data. Otherwise every private run costs privacy (Hyperparameter tuning).

3. Name the public data and the overlap check, and state that the guarantee covers the private data given the pretrained model.

## 47.7. Cheat sheet

Table 47.2.: Where to start when fine-tuning privately; the ranking depends on the task.

| Situation | Train | Why |
|---|---|---|
| Tiny budget, or the probe meets the target | Last layer (linear probe) | Fewest weights; convex |
| Moderate budget, features not quite right | Low-rank adapter plus last layer | Never far from the best |
| Larger budget, large batches, tuned learning rate | All weights | Noise costs only along curved directions |

**Rule of thumb.** Measure the linear probe's ceiling before anything else. Then use large batches and a smaller learning rate than for training from scratch. Tune the learning rate by DP-SGD runs at the target noise on public data, since a non-private sweep does not show how the noise moves the best step.

## 47.8. References


- Tramèr & Boneh, "Differentially Private Learning Needs Better Features (or Much More Data)" (ICLR 2021): linear probing on good features.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022), and Yu et al., "Differentially Private Fine-tuning of Language Models" (ICLR 2022): full and adapter-based private fine-tuning.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 48. Fine-tune a language model privately with LoRA

## 48.1. Why this matters

You have a pretrained language model and private text, such as support tickets or clinical notes. You want the model to learn from that text without being able to repeat it, yet language models do repeat what they were trained on. A model fine-tuned without privacy can write out a phone number or a password that appeared often enough in its training text (Memorization and training-data extraction). The fix is to fine-tune with the differential privacy (DP) version of stochastic gradient descent (SGD), called DP-SGD. It costs least when it trains only a small adapter on top of the frozen model, which is often as accurate as training every weight (Public data and private fine-tuning). Here the adapter is low-rank adaptation (LoRA), from Hugging Face's `peft` library, and Opacus, the PyTorch library for DP-SGD, does the private training.

This chapter does the whole job on a model and a text small enough for a CPU. The model is Pythia-14m, a public language model of 14 million parameters from EleutherAI, and the private text is Tiny Shakespeare, a million characters of Shakespeare's plays. The script fine-tunes the model twice from the same starting point. The first run has no privacy. The second has a privacy loss $\varepsilon$ of 8 for each sequence of training text, which bounds what the model can reveal about any one sequence. Before training, the script also plants made-up secrets, called canaries, in the text, so that you can see what each model memorized. The script practitioner_lora.py runs every step in a few minutes without a GPU.

```
uv run python code/part05-dp-ml/practitioner_lora.py
```

The defaults give the numbers below. The option `--epsilon` changes the private run's target and `--rank` the adapters' rank, and `--save-adapter FOLDER` saves the private adapter (Section 48.11).

## 48.2. Fix the privacy unit, n and δ

The guarantee protects one unit, so decide first what the unit is. A language model trains on fixed-length sequences of tokens, the word pieces its tokenizer cuts text into. The script cuts the text into sequences of 32 tokens, about one or two lines of verse, and makes each sequence the unit. It keeps the first 1,024 sequences as the private training set and the last 128 as a held-out set, and leaves the rest unused so that the run stays short.

```
sequences of 32 tokens: 10,632  training n=1,024  held out 128
delta=1e-04 (privacy unit: one training sequence)
```

The slack $\delta$ is the small additive term the guarantee allows beside $\varepsilon$. For a training set of $n$ units, the book's default is $\delta \leq 1/n^{1.1}$, rounded down to a power of ten (Choosing ε). For 1,024 sequences that gives $10^{-4}$.

A sequence is the right unit for this demonstration, but rarely in production. One person's document spans many sequences, and one user writes many documents, so a guarantee per sequence protects a person much less than it seems. In production, make the unit a document or a user, and bound how many sequences each one contributes (User-level differential privacy). Then $n$ counts users, and $\delta$ is set from that count.

## 48.3. Plant canaries

To check memorization later, you need secrets whose presence you control. The script writes **canaries** into the training text. Each one is a sentence such as "the pin of Alice is 8 6 5 2 3", whose last five digits are the secret. There are four groups of three canaries. The first group is inserted once each, and the others 4, 16 and 64 times each. Three more, the controls, are drawn the same way but never inserted, so they show what a secret the model has never seen scores in the audit of Section 48.9.

```
def insert_canaries(
    train: torch.Tensor, canaries: list[Canary], tokenizer: PreTrainedTokenizerBase, rng: np
) -> torch.Tensor:
    """Write each copy of each canary over the start of a different training sequence."""
    train = train.clone()
    total = sum(c.copies for c in canaries)
    rows = iter(rng.choice(len(train), size=total, replace=False).tolist())
    for canary in canaries:
        tokens = torch.tensor(tokenizer(canary.text)["input_ids"])
        for _ in range(canary.copies):
            train[next(rows), : len(tokens)] = tokens
    return train
```

Each copy overwrites the start of a different training sequence, so the 255 copies take up a quarter of the training set. That is far denser than secrets in real data, and deliberately so, because it lets a model this small memorize within a run short enough for a CPU. The audit then has a positive control, a model that is known to leak.

```
canary sequences: 255 (3 canaries each at (1, 4, 16, 64) copies)
tokens per secret: [5]
candidates: 2,000  exposure cap 10.97 bits  (full space 16.61)
```

The digits are separated by spaces, so that every possible secret is exactly five tokens long, and the second line, the token counts found over 1,000 random secrets, confirms it. The third line, the most that the audit's score can show, belongs to Section 48.9.

## 48.4. Load a public model and tokenizer

Everything that is not trained privately must come from outside the private data. That includes the pretrained weights, and also the tokenizer, whose vocabulary would leak rare words if it were built from the private text. So both come from the same public checkpoint, Pythia-14m, pinned to one revision. The script loads the model in float32, because Hugging Face's `transformers` library loads some checkpoints in float16, which is far slower on a CPU.

The guarantee then covers the private text given the pretrained model, and nothing about the model's own training data. Here that caveat is real. Pythia was pretrained on the Pile, which includes books from Project Gutenberg, so Shakespeare's plays are very likely in it already. That is harmless here, but with real data, check for overlap between it and the public model's training data (Public data and private fine-tuning).

## 48.5. Choose what to train

DP-SGD adds noise to every weight it trains. So the noise on the averaged gradient grows with the square root of the number of trainable weights, while the clipped signal does not (Public data and private fine-tuning). That makes the choice of trainable weights one of the two main levers on accuracy, with the learning rate. LoRA freezes each pretrained matrix and learns a correction to it, the product of two thin factors whose width is the rank.

```
attention_and_mlp = ["query_key_value", "dense", "dense_h_to_4h", "dense_4h_to_h"]  # every
config = LoraConfig(r=rank, lora_alpha=2 * rank, target_modules=attention_and_mlp, lora_drop
lora_model = get_peft_model(model, config)  # freezes the pretrained weights
trainable_count = sum(p.numel() for p in lora_model.parameters() if p.requires_grad)
total_count = sum(p.numel() for p in lora_model.parameters())
print(f"trainable parameters: LoRA {trainable_count:,}  full fine-tuning {total_count - trai
```

```
full model parameters: 14,067,712
trainable parameters: LoRA 98,304  full fine-tuning 14,067,712
```

The script prints the model's size first, and then the excerpt's line. By default the adapters have rank 8 and sit on every linear layer, in attention and in the feed-forward blocks. They train 98,304 weights, while the frozen weights, which full fine-tuning would train, number about 14 million. `lora_alpha` scales the adapters' correction, and the script sets it to the usual twice the rank. Dropout is off.

Where the adapters go matters too. Adapters on the attention layers alone left this small model too little room to learn, or to memorize. For a real task, choose the rank and the layers on public data, since a choice made by looking at private runs costs privacy (Hyperparameter tuning).

## 48.6. Make the model Opacus-compatible

Opacus needs one gradient per sequence, and some layers make that impossible. Batch normalization is the usual culprit, because it mixes the records of a batch. Opacus's `ModuleValidator.validate` lists such layers before you train, and the script prints how many it found.

```
validator errors: 0
```

Pythia has none, and only the adapters train in any case. They are ordinary linear layers, which Opacus's default hooks handle, so the script passes `grad_sample_mode="hooks"`. Opacus computes per-sequence gradients only for the weights that require gradients, so the 14 million frozen weights add no cost. For a trainable layer with no hook, Opacus falls back by itself to PyTorch's slower functorch. Call `model.train()` before wrapping a model you have just evaluated, since Opacus rejects one in evaluation mode.

One step comes after training. The hooks stay attached to the model when the training loop ends, and they slow down everything the model does next. So the script calls `remove_hooks()` on the private model before it evaluates it.

## 48.7. Calibrate and train

Fix the target $\varepsilon$, $\delta$, the sampling rate and the number of epochs, and let the accountant find the noise. The script asks for $\varepsilon = 8$ over 3 epochs, with an expected batch of 32 sequences. The sampling rate $q$ is the share of the data in an average batch, here 32/1,024, so the 3 epochs take 96 steps. The accountant is the privacy loss random variable (PRV) accountant, which computes $\varepsilon$ numerically and almost exactly (Report and review an ε claim).

```
privacy_engine = PrivacyEngine(accountant="prv")
private_model, optimizer, loader = privacy_engine.make_private_with_epsilon(
    module=model,
    optimizer=optimizer,
    data_loader=loader,  # replaced by Poisson sampling with rate BATCH / n
    target_epsilon=epsilon,
    target_delta=delta,
    epochs=EPOCHS,
    max_grad_norm=MAX_GRAD_NORM,
    grad_sample_mode="hooks",
)
print(f"noise multiplier: {optimizer.noise_multiplier:.3f}")
```

The training loop is an ordinary one. Opacus's loader draws Poisson batches, in which each sequence joins each batch independently with probability $q$, so a batch holds a random number of sequences.

```
for (batch,) in loader:  # each batch holds a random number of sequences
    optimizer.zero_grad()
    loss = private_model(input_ids=batch, labels=batch).loss
    loss.backward()  # Opacus's hooks keep one gradient per sequence
    optimizer.step()  # clips each one to MAX_GRAD_NORM, sums, adds noise
    steps += 1
```

```
noise multiplier: 0.553
sample rate q=0.03125  steps=96  noise multiplier=0.553
epsilon (Opacus PRV)=7.99  epsilon (dp_accounting PLD)=7.98  at delta=1e-04
```

The noise multiplier is the noise's standard deviation in units of the clipping norm, `MAX_GRAD_NORM`, which the script sets to 1. The last line cross-checks the claimed $\varepsilon$ of 8. Google's `dp_accounting` recomputes $\varepsilon$ from the noise multiplier, the sampling rate and the steps with its privacy loss distribution (PLD) accountant, and the two values agree to within rounding.

Both runs use AdamW at a learning rate of $10^{-3}$, and the private run differs only by clipping, noise and Poisson sampling. That learning rate, the rank and the epochs were chosen in a few non-private trial runs on the same text. A real run must choose them on public text of the same kind, such as other public-domain plays here, or pay for the trials in $\varepsilon$ as Hyperparameter tuning shows.

Two more settings change for a real run. The batch of 32 is far smaller than DP-SGD likes, so with more data use batches in the thousands. Opacus's `BatchMemoryManager` splits them into physical batches that fit in memory (Train a model with DP-SGD). And a model you release needs `secure_mode=True` in `PrivacyEngine`, which draws the noise from a cryptographically secure generator. That generator cannot be seeded, so the script leaves it off to make its numbers repeat.

## 48.8. Evaluate

A language model is judged by how well it predicts held-out text. The script measures the mean cross-entropy per token on the 128 held-out sequences, and its exponential, the **perplexity**. A perplexity of 159.3, the non-private model's, means it is on average as unsure as if it chose among about 159 equally likely tokens, so lower is better.

```
held-out loss pretrained: 5.306 nats/token  perplexity 201.6
held-out loss LoRA non-private: 5.071 nats/token  perplexity 159.3
held-out loss LoRA DP eps=8: 5.044 nats/token  perplexity 155.1
```

Both fine-tuned models improve on the pretrained one, and the private model is no worse than the non-private one. From one seed, its slightly lower perplexity means only that DP costs nothing measurable here. Both gains are modest, because a quarter of the training sequences begin with canaries rather than Shakespeare. When the private task is classification, such as labelling tickets, measure held-out accuracy against the same two baselines instead.

Do not expect privacy to be this cheap on every task, because this run is short, trains few weights, and uses text that is very likely in the pretraining data already. With a harder task or a smaller $\varepsilon$, compare the private model with both baselines at several budgets, as Train a model with DP-SGD does.

## 48.9. Check memorization

The canaries now answer the question that perplexity cannot, which is whether the model will repeat a secret. The script measures each canary in two ways. The first is extraction. It gives the model the prefix "the pin of Alice is", lets it write five tokens, each time the likeliest next one, and checks whether they are the secret.

```
@torch.no_grad()
def greedy_completion(model: PeftModel, tokenizer: PreTrainedTokenizerBase, prefix: str) ->
    """The five tokens the model writes after the prefix when it always picks the likeliest
    ids = tokenizer(prefix, return_tensors="pt")["input_ids"]
    for _ in range(5):
        next_token = model(input_ids=ids).logits[:, -1].argmax(dim=-1, keepdim=True)
        ids = torch.cat([ids, next_token], dim=1)
    return tokenizer.batch_decode(ids[:, -5:])[0].strip()
```

The second is the **exposure** of Memorization and training-data extraction. It ranks the canary's secret among random secrets of the same format by how likely the model finds each, and turns the rank into bits. Each bit halves the number of guesses an attacker needs. The script scores each secret by the summed log-probability of its five digits.

```
@torch.no_grad()
def secret_scores(model: PeftModel, tokenizer: PreTrainedTokenizerBase, prefix: str, secrets
    """Summed log-probability of each secret's five digit tokens after the prefix."""
    prefix_ids = tokenizer(prefix)["input_ids"]
    rows = [prefix_ids + tokenizer(" " + s)["input_ids"] for s in secrets]
    ids = torch.tensor(rows)  # every secret is five tokens, so the rows have equal length
    logits = model(input_ids=ids[:, :-1], logits_to_keep=5).logits  # the predictions of the
    logprobs = logits.log_softmax(dim=-1).gather(-1, ids[:, -5:, None])
    return logprobs.sum(dim=(1, 2)).double().numpy()


def canary_exposure(
    model: PeftModel, tokenizer: PreTrainedTokenizerBase, canary: Canary, candidates: list[s
) -> tuple[float, float]:
    """The exposure of the canary's secret among the candidates, measured and extrapolated p
    candidate_scores = scores_batched(model, tokenizer, canary.prefix, candidates)
    score = float(scores_batched(model, tokenizer, canary.prefix, [canary.secret])[0])
    return exposure(score, candidate_scores), extrapolated_exposure(score, candidate_scores,
```

No library installed with this book computes exposure, so `exposure` and `extrapolated_exposure` come from the book's helper module practitioner_privacy_tests.py, after TensorFlow Privacy's `compute_exposure_interpolation` and `compute_exposure_extrapolation`. The script ranks each secret among 2,000 random candidates, so a canary that beats them all reaches a cap of 10.97 bits. A secret the model has never seen averages about 1.44 bits ($\log_2 e$), because its rank among the candidates is then random. Each line below gives a group's mean exposure, the same past the cap (explained below), and how many of its three secrets greedy decoding wrote out.

```
non-private 1 copies: mean exposure 2.0 bits  extrapolated 2.0  extracted 0/3
non-private 4 copies: mean exposure 5.9 bits  extrapolated 5.7  extracted 0/3
non-private 16 copies: mean exposure 10.0 bits  extrapolated 13.0  extracted 0/3
non-private 64 copies: mean exposure 11.0 bits  extrapolated 16.3  extracted 3/3
non-private controls: mean exposure 1.4 bits  extrapolated 1.5  extracted 0/3
DP 1 copies: mean exposure 0.9 bits  extrapolated 0.9  extracted 0/3
DP 4 copies: mean exposure 0.6 bits  extrapolated 0.6  extracted 0/3
DP 16 copies: mean exposure 4.2 bits  extrapolated 4.3  extracted 0/3
DP 64 copies: mean exposure 4.0 bits  extrapolated 3.9  extracted 0/3
DP controls: mean exposure 2.2 bits  extrapolated 2.2  extracted 0/3
```

The non-private model is the positive control, and it leaks. Exposure rises with the number of copies, from 2.0 bits for one copy to the cap for 64. The model writes out all three 64-copy secrets from their prefixes. The extrapolated column sees past the cap. It fits a skew-normal distribution, a normal with one heavier tail, to the candidates' scores, and reads off how far into that tail each secret lies. It puts the 64-copy secrets near the 16.61 bits of the whole space of secrets. The 4-copy canaries are the instructive case. Two of the three reach 6.9 and 8.2 bits, yet none is extracted, so exposure detects memorization that extraction alone would miss.

Nothing can be extracted from the private model, and its canaries with 1 or 4 copies score no higher than its controls. Those controls average 2.2 bits rather than 1.44, but a mean over three secrets scatters widely. Still, DP limits the effect without erasing it, since the canaries with 16 and 64 copies still average about 4 bits. The guarantee allows that, because $\varepsilon = 8$ protects one sequence, while a canary with many copies is many sequences. Group privacy bounds a group of $k$ units at about $k$ times $\varepsilon$. So a canary with 16 copies gets a level of $16 \times 8$, far too large to promise anything (User-level differential privacy). A secret that a user repeats across many documents is the same case, which is one more reason to make the unit the user.

Two cautions apply. Because the canaries are so dense, a real non-private model may leak less than this one, and a real audit should plant fewer copies. And three canaries per group is a small sample. For a fuller assessment before release, run the attacks of Pre-release privacy risk assessment. For a statistical lower bound on $\varepsilon$, use the one-run audit of Auditing private training.

## 48.10. Variants

Three variants matter for real models: ghost clipping, LoRA on GPT-2, and packages that wrap the whole loop. The block below shows them with pinned versions, and it was not run.

```
# opacus==1.6.0: ghost clipping, when per-sequence gradients do not fit in memory
model, optimizer, criterion, loader = privacy_engine.make_private_with_epsilon(
    module=model, optimizer=optimizer, data_loader=loader, criterion=torch.nn.CrossEntropyLo
    target_epsilon=8.0, target_delta=1e-4, epochs=3, max_grad_norm=1.0, grad_sample_mode="gh
)
logits = model(input_ids=batch).logits[:, :-1]                # predict each next token
loss = criterion(logits.flatten(0, 1), batch[:, 1:].flatten(), shape=logits.shape)
loss.backward()                                               # norms first, then the clipp

# peft==0.21.2: GPT-2 stores its attention as Conv1D, with the weight transposed
LoraConfig(r=8, lora_alpha=16, target_modules=["c_attn"], fan_in_fan_out=True)

# dp-transformers==1.0.0 (Microsoft): Opacus inside the Hugging Face Trainer
# private-transformers (Li et al.): install from its GitHub repository, pinned to a commit
```

Ghost clipping gets each sequence's gradient norm without storing the gradient, so memory no longer holds one gradient per sequence (DP-SGD). dp-transformers and private-transformers wrap the same steps for the Hugging Face trainer.

## 48.11. Report

The release statement names the unit, here one sequence of 32 tokens, along with $n$ and $\delta$. It names the accountant and the three inputs it needs, which are the noise multiplier, the sampling rate and the number of steps. It says that the guarantee covers the private text given the pretrained model, and names that model and its revision. It also says how the hyperparameters were chosen, here by non-private trials on the private text, which a real release could not claim for free. Attach the canary audit as evidence, with its caveats. Release and governance assembles all of this into a record and a statement.

What ships is the adapter. With `--save-adapter`, the script saves it after `remove_hooks()` and loads it back onto the same pinned checkpoint.

```
private.save_pretrained(path)  # only the adapter weights and config, after Opacus's hooks a
base_model = AutoModelForCausalLM.from_pretrained(folder, dtype=torch.float32)  # the same p
reloaded = PeftModel.from_pretrained(base_model, path)
```

The folder holds only the adapter, so the statement names the base checkpoint and revision it fits. The guarantee covers the adapter and anything computed from it.

## 48.12. Checklist

Table 48.1.: The steps of fine-tuning a language model privately, their defaults and their checks.

| Step | Default | Check |
|---|---|---|
| Unit, $n$, $\delta$ | Document or user in production; $\delta \leq 1/n^{1.1}$ rounded down | $n$ counts units, not tokens or lines |
| Canaries | A few per group, inserted 1 to many times, plus controls | Controls near 1.44 bits, allowing for the scatter of a few |
| Public model and tokenizer | One public checkpoint, pinned revision, float32 | Nothing fitted on private text; overlap checked |
| What to train | LoRA, rank 8, on every linear layer | Trainable count printed; chosen on public data |
| Opacus | Validator; hooks; `remove_hooks()` after training | No validator errors |
| Calibrate and train | PRV accountant; Poisson sampling; large batches | PLD agrees with PRV |
| Evaluate | Held-out perplexity against the pretrained and non-private models | Private model beats the pretrained one |
| Memorization | Exposure and greedy extraction, non-private and private | Nothing extracted; one-copy canaries no higher than the controls |
| Report and save | Unit, $\varepsilon$, $\delta$, accountant, public model, tuning; adapter saved after `remove_hooks()` | Statement names what is not covered and the base revision |

## 48.13. References

- Yu et al., "Differentially Private Fine-tuning of Language Models" (ICLR 2022): LoRA and other adapters with DP-SGD, and their accuracy.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022): full fine-tuning with ghost clipping, and the hyperparameters that make it work.
- Carlini et al., "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries and exposure.

**Derivations.** This workflow draws on the full edition's chapters on public data and private fine-tuning in the full edition, DP-SGD in the full edition and memorization and training-data extraction in the full edition.

# 49. Private prediction: PATE and in-context learning

## 49.1. What it is

Private prediction applies differential privacy (DP) to each *answer* instead of to the model. You split the private records into disjoint shards and train an ordinary model, a **teacher**, on each one. The teachers stay secret, and each query gets the winner of their vote after noise is added to the counts, private at a level $\varepsilon$ with a small slack $\delta$. A record lives in one shard, so it can move one teacher's vote from one label to another, and the counts by at most $\sqrt{2}$ in Euclidean length.

**GNMax** adds Gaussian noise with a standard deviation $\sigma$ to each count and releases only the label with the largest noisy count. In Rényi differential privacy (RDP), each answer costs a straight line of slope $1/\sigma^2$ in the order $\alpha$, and the slopes of successive answers add (Rényi DP). So write $\rho(\varepsilon, \delta)$ for the largest slope that still converts to your target. A fixed number $Q$ of answers, even adaptively chosen, then needs the noise

$$\sigma = \sqrt{\frac{Q}{\rho(\varepsilon, \delta)}}. \tag{49.1}$$

For example, at $\delta = 10^{-5}$ a budget of $\varepsilon = 1$ allows a slope of 0.0306. Then 1,000 answers need $\sigma = 181$. LNMax, with Laplace noise, wins only for one or two answers.

## 49.2. When you meet it

You meet private prediction when you cannot train a private model. The common case is a large language model (LLM) behind an interface, whose prompt holds private records and whose answers leak them. The fix splits the records into disjoint groups, prompts the model once per group, and releases the GNMax label of the answers (Duan et al. 2023). Wu et al. (2024) answered 10,000 sentiment queries this way. At $\varepsilon = 1$ they report 91.97% accuracy, as good as one non-private prompt.

The decision is between private answers and a private model, whose accuracy is fixed. The vote's accuracy falls as its noise grows with $\sqrt{Q}$, so answers win up to a **break-even** number of queries, and the model wins beyond it. On the full edition's 10,000-record synthetic task, the vote beats noisy stochastic gradient descent (SGD), DP-SGD, only on the first 18 queries at $\varepsilon = 1$. From $\varepsilon = 2$ on, DP-SGD beats even a noiseless vote of teachers trained on small shards. So compare the two at your budget and expected number of queries, with the DP-SGD baseline of Train a model with DP-SGD.

**PATE** (private aggregation of teacher ensembles) spends the budget once. It labels public, unlabelled inputs by GNMax and trains a new model, the **student**, on them, and only the student

is released, at exactly the cost of its labels. So PATE needs public inputs from the private data's distribution. Papernot et al.'s MNIST student comes within a percentage point of non-private accuracy, but at a data-dependent $\varepsilon$ (Section 49.4). On the synthetic task, though, DP-SGD beats the best student at every budget.

## 49.3. How to use it

`dp_accounting` accounts for the vote. One GNMax answer is a `GaussianDpEvent` whose noise multiplier, the noise over the sensitivity, is $\sigma/\sqrt{2}$.

```
import numpy as np
from dp_accounting import GaussianDpEvent, SelfComposedDpEvent
from dp_accounting.pld import PLDAccountant
from dp_accounting.rdp import RdpAccountant

sigma, queries = 181.0, 1000  # noise per count, answers to release
event = SelfComposedDpEvent(GaussianDpEvent(sigma / np.sqrt(2)), queries)  # sensitivity sqr
rdp = RdpAccountant().compose(event).get_epsilon(1e-5)
pld = PLDAccountant().compose(event).get_epsilon(1e-5)
print(f"{rdp:.3f} {pld:.3f}")  # 0.999 0.914
```

The RDP accountant confirms $\varepsilon = 0.999$. The privacy loss distribution (PLD) accountant, exact for Gaussian noise, gives 0.914, so solving for $\sigma$ with it gives less noise. To solve, pass a function from $\sigma$ to this event to `dp_accounting.calibrate_dp_mechanism`. For many answers the PLD is much slower than RDP, which is the fallback when it takes too long. When each query draws a fresh Poisson sample of records, as in Wu et al., wrap each event in a `PoissonSampledDpEvent`.

## 49.4. What goes wrong

The guarantee needs each record in exactly one shard. So assign shards by a hash of the record's or the user's identifier, never by position in a file, where one inserted record would shift every later shard. Release the label only and fix $Q$ in advance.

Noise this large wipes out close votes. With two labels, write $g$ for the gap between the two counts. The vote returns the plurality with probability $\Phi(g/(\sqrt{2}\sigma))$, where $\Phi$ is the standard normal distribution function. For example, the noise for 1,000 answers at $\varepsilon = 4$ is $\sigma = 51.8$, and a lopsided vote of 98 to 2 is then right with probability 0.9051. A close vote of 52 to 48, though, comes out right only with probability 0.5218. So measure the noiseless vote first, since no budget beats it. Adding teachers widens the gaps, since $\sigma$ does not depend on their number, until the shards get too small to train on.

Finally, beware of a data-dependent $\varepsilon$. When teachers agree, a sharper analysis charges less, but that smaller $\varepsilon$ is computed from the votes and leaks them. Until it is released with noise, it is a measurement, not a guarantee. **Confident-GNMax** answers only queries on which a noisy check finds agreement, the cheap and accurate ones.

## 49.5. References

- Papernot et al., "Scalable Private Learning with PATE" (ICLR 2018): GNMax, its data-dependent analysis and Confident-GNMax.
- Wu et al., "Privacy-Preserving In-Context Learning for Large Language Models" (ICLR 2024): subsampled voting over prompts.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 50. DP-FTRL and matrix factorization

## 50.1. What it is

Gradient descent without momentum adds up its steps, so its weights after each step are fixed by the sum of all clipped gradients so far, a **prefix sum**. Stochastic gradient descent (SGD) with differential privacy (DP), DP-SGD, adds fresh noise at every step, and the noise in the prefix sums then grows like a random walk. Matrix mechanisms correlate the noise across steps, so that each step's noise partly cancels earlier noise in the sums.

Stack the gradient sums of the steps as the rows of a matrix $x$. The prefix sums are then $Ax$, where $A$ has ones on and below the diagonal. The **matrix factorization mechanism** writes $A$ as a decoder $L$ times an encoder $R$. It adds a matrix of Gaussian noise, written $z$, to the encoded gradients and decodes the result,

$$\hat{S} = L(Rx + z) = Ax + Lz. \tag{50.1}$$

A record enters only through the column of $R$ for the step it joins, so privacy depends on the longest column of the encoder, called its sensitivity. Accuracy depends on the decoder, whose row $t$ carries the noise into the $t$-th prefix sum.

The first such method ran the follow the regularized leader (FTRL) optimizer on sums released by a binary tree, and is called DP-FTRL (Kairouz et al. 2021). The usual choice today is the **square-root factorization**, which uses the lower-triangular square root of $A$ for both factors. At equal privacy over one epoch of 100 steps, it leaves about a quarter of independent noise's standard deviation in the final sum.

## 50.2. When you meet it

You meet matrix mechanisms when your pipeline cannot honestly claim Poisson sampling, the random batches that DP-SGD's accountant assumes. Fixed batches cut from a shuffled order fail it, and so do data read in arrival order and, in federated learning, phones that check in when they can. Accounting them as Poisson understates $\varepsilon$ (Amplification by subsampling), and a matrix mechanism needs no sampling.

Giving up amplification is expensive with independent noise, and correlated noise wins most of it back. On the full edition's synthetic task, trained in a fixed order for ten epochs at $\varepsilon = 2$, independent noise reaches a test accuracy of 0.8641. The banded square root of Section 50.4 reaches 0.9491, against 0.9493 for DP-SGD with Poisson sampling. Google trained its Gboard keyboard models this way (Xu et al. 2023).

So if you can implement Poisson sampling, use DP-SGD with Opacus. If you cannot, use a banded matrix mechanism.

## 50.3. How to use it

Google's JAX Privacy implements banded matrix factorization. The block below was not run, since JAX is not in this book's environment. It cycles through 100 fixed groups of records, one per step, so each record takes part once every 100 steps.

```
# jax-privacy==2.0.0 (not run)
from jax_privacy.experimental.execution_plan import BandMFExecutionPlanConfig

config = BandMFExecutionPlanConfig.default(
    num_bands=100,       # at most the 100 steps between a record's turns
    iterations=1000,     # ten epochs of 100 steps
    epsilon=2.0,
    delta=1e-5,
    sampling_prob=1.0,   # all of the step's group joins: no amplification claimed
    l2_clip_norm=1.0,
)
plan = config.plan       # calibrates the noise multiplier with dp_accounting
```

The training loop draws each batch from `plan.batch_selection_strategy` and sums its clipped gradients with `plan.clipped_grad`. It passes the sum through `plan.noise_addition_transform` before the optimizer sees it. A `dp_accounting` accountant turns `plan.dp_event` into the $\varepsilon$ you report. The module is marked experimental, so pin the version.

## 50.4. What goes wrong

The pitfalls come from records that take part more than once. Over $E$ epochs a record enters through $E$ columns of the encoder. The square root's columns for nearby steps are nonzero in the same rows, so they add up and the sensitivity grows faster than $\sqrt{E}$. **Banding** keeps a number $b$ of the encoder's diagonals, the main one and those just below it. It needs every two participations of a record to be at least $b$ steps apart, a condition called $b$**-min-separation**. The record's columns then share no rows, so the sensitivity is at most $\sqrt{E}$ times the longest column, as for independent noise.

The data order is therefore part of the mechanism. Fix it before training, independently of the data, and keep the same permutation every epoch. Reshuffling each epoch can put a record at the end of one epoch and the start of the next, one step apart, which breaks the banded bound.

The neighbouring relation changes the numbers too. The analysis here leaves a neighbour's slot empty, so a step moves by at most the clipping norm. If a neighbour replaces the record with another instead, every sensitivity doubles. So state the relation your accountant assumes next to every $\varepsilon$.

The accounting library enforces this. In `dp_accounting` 0.6.0, the event for tree aggregation, `SingleEpochTreeAggregationDpEvent`, is accepted only by the Rényi differential privacy (RDP) accountant built with `NeighboringRelation.REPLACE_SPECIAL`, under which a neighbour replaces the record with an empty one. An accountant built for add/remove, the default for DP-SGD,

raises `UnsupportedEventError`, and the privacy loss distribution (PLD) accountant does not support the event at all. So DP-FTRL and DP-SGD runs cannot share one add/remove ledger.

## 50.5. References


- Choquette-Choo et al., "(Amplified) Banded Matrix Factorizations: A Unified Approach to Private Training" (NeurIPS 2023): banding, min-separation and amplification.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 51. Disparate impact

## 51.1. Why this matters

A private model's test accuracy is an average over everyone, so it cannot show which people lose the accuracy that privacy costs. Bagdasaryan, Poursaeed & Shmatikov (2019) trained networks with differential privacy (DP), using the private form of stochastic gradient descent (SGD) called DP-SGD. They found that the lost accuracy fell mostly on under-represented groups, and called this the **disparate impact** of DP.

So a model that is a point worse on average can be ten points worse for one group, often a small group that privacy is meant to protect. Before you ship a private model, measure its cost for each group against a non-private baseline, with a confidence interval on every number.

## 51.2. Evaluating a private model per group

Decide which groups matter before you look at any results, so that the results cannot steer the choice. Then train a non-private baseline with the same data and recipe. A gap between groups may exist without privacy, so what DP is responsible for is how far each group's own accuracy drops when training becomes private.

Write $\text{acc}_g$ for a model's accuracy on the test records of group $g$. Private training is random, so its accuracy is averaged over several training runs, written $\mathbb{E}$. The **privacy cost** $\pi_g$ is then the accuracy the group loses when the same data and hyperparameters are trained privately,

$$\pi_g = \text{acc}_g(\text{non-private}) - \mathbb{E}[\text{acc}_g(\text{private})]. \tag{51.1}$$

DP has disparate impact when groups have different privacy costs.

Every accuracy needs an interval. Each test record is either classified correctly or not, so a group's accuracy is a binomial proportion. Its exact Clopper–Pearson interval contains the true accuracy in at least 95% of test samples (Hypothesis testing). The gap between two groups needs its own interval. Write $\hat{p}_g$ for group $g$'s measured accuracy and $m_g$ for its number of test records. For two groups $a$ and $b$ measured on disjoint test records, the estimates are independent, so their variances add. For large counts, 1.96 standard deviations on each side give an approximate 95% interval for the gap,

$$(\hat{p}_a - \hat{p}_b) \pm 1.96\sqrt{\frac{\hat{p}_a(1-\hat{p}_a)}{m_a} + \frac{\hat{p}_b(1-\hat{p}_b)}{m_b}}. \tag{51.2}$$

These intervals are wider than most people expect. Suppose a test split holds only 100 records of a minority, and 88 of them are classified correctly. The majority gets 1,812 of its 1,900 records

right, an accuracy of 0.954. So the measured gap is 0.074. SciPy gives the minority's interval, and Equation 51.2 gives the gap's.

```
from math import sqrt

from scipy.stats import binomtest

minority = binomtest(88, 100).proportion_ci(confidence_level=0.95, method="exact")
print(f"{minority.low:.3f} {minority.high:.3f}")  # 0.800 0.936

gap, se = 0.954 - 0.880, sqrt(0.954 * 0.046 / 1_900 + 0.880 * 0.120 / 100)
print(f"{gap - 1.96 * se:.3f} {gap + 1.96 * se:.3f}")  # 0.010 0.138
```

So the gap is real, since its interval excludes zero, but it could be anywhere from one point to fourteen. An interval separates a 5-point gap from zero only with a few hundred test records from the group, and a 1-point gap only with thousands.

The privacy cost needs a different interval, because both models are scored on the same test records, so their accuracies are not independent and Equation 51.2 does not apply. Pair them instead. Among group $g$'s $m_g$ test records, write $u$ for the number that only the non-private model gets right, and $v$ for the number that only the private one gets right. One run's cost is then $(u - v)/m_g$, and for large counts an approximate 95% interval is

$$\frac{u - v}{m_g} \pm \frac{1.96}{m_g} \sqrt{u + v - \frac{(u - v)^2}{m_g}}. \tag{51.3}$$

The test records are one source of variation. Training is a second source, and a larger test set does nothing about it. A small group's accuracy varies far more from one training run to the next than a large group's. So a comparison of two recipes on one run each can easily reverse for it. Train each recipe several times with different seeds, average the per-run costs and report their spread. `fairlearn.metrics.MetricFrame` computes any metric per group, but without intervals.

The evaluation itself can also leak. Per-group accuracies on private test data are statistics of that data. To publish them, add noise to each group's count of correct predictions and count it in a budget of its own (Release statistics with a library). Otherwise keep them in the internal assessment.

## 51.3. Disparate impact, measured

The script practitioner_dp_sgd.py of Train a model with DP-SGD uses the Adult census data, which predicts whether a person's income exceeds $50K. It trains a small network once without privacy and once with DP-SGD at a privacy level $\varepsilon$ of 4 and a slack $\delta$ of $10^{-5}$. Then it scores both models on the test rows of each `sex`, the only attribute it measures. Other attributes, such as race, have much smaller groups and wider intervals. For each group it counts the correct predictions, marked in the boolean array `right`, and asks SciPy for the exact interval.

```
in_group = sex == group
correct, total = int(right[in_group].sum()), int(in_group.sum())
interval = binomtest(correct, total).proportion_ci(confidence_level=0.95, method="exact")
```

Its seed-0 run, `uv run python code/part05-dp-ml/practitioner_dp_sgd.py`, gives Table 51.1.

Table 51.1.: Accuracy on Adult by sex, with exact 95% intervals, for one training run of each model.

| Group | Test records | Non-private accuracy | Accuracy at $\varepsilon = 4$ | Privacy cost |
|---|---|---|---|---|
| Female | 5,613 | 0.924 [0.917, 0.931] | 0.922 [0.915, 0.929] | +0.002 |
| Male | 11,229 | 0.808 [0.800, 0.815] | 0.807 [0.800, 0.815] | +0.001 |

Here DP has no measurable disparate impact. Each group loses at most 0.002, well inside its intervals, which overlap almost entirely. It comes from one private run, a single draw of Equation 51.1 rather than its average. The large gap between the groups, 0.924 against 0.808, is there without privacy too, so DP is not responsible for it. Adult is easy, because both groups are large and the whole model loses almost nothing at this budget, 0.846 with DP against 0.847 without.

A small group tells a different story. The full edition's running example is a synthetic task with 10,000 training records, labelled by a fixed random network. Of these records, 500 form a minority whose inputs come from a different region and follow a different labelling rule (Table 51.2).

Table 51.2.: Test accuracy of each group of the full edition's running example with a 5% minority, averaged over 3 runs ($\delta = 10^{-5}$, noise multiplier $\sigma$). Costs are fractions of accuracy, and a negative cost is a gain.

| Training | Majority accuracy | Minority accuracy | Majority's privacy cost | Minority's privacy cost |
|---|---|---|---|---|
| Non-private SGD | 0.9529 | 0.9411 | — | — |
| Clipping only, $\sigma = 0$ | 0.9581 | 0.9007 | −0.0051 | 0.0404 |
| $\varepsilon = 4$, $\sigma = 1.36$ | 0.9470 | 0.8477 | 0.0059 | 0.0934 |

The minority pays almost all of the cost. At $\varepsilon = 4$ the majority loses 0.59 points, while the minority loses 9.34. The cost also depends steeply on the group's share of the data. At the same budget, a minority with 1% of the training records loses 34.88 points. With 20% of the records, it loses only 2.80. So check every small group, whatever the overall accuracy says.

## 51.4. Why small groups pay: clipping and noise

DP-SGD loses accuracy through two steps, clipping and noise, and both treat every record alike. The clipping row of Table 51.2 clips without adding noise, so it gives no privacy at all. Yet the minority already loses 4.04 points there, which means clipping by itself treats the groups unequally.

Clipping caps every record's pull on the update at the clipping norm $C$, so it cuts back the groups with long gradients hardest. A group's gradients are long where the model fits it badly, as it does a rare pattern that the rest of the data does not teach. So training gives such a group less weight than its share of the records.

The noise is the same for every group, while a group's signal grows with its size. So a small group's signal is the weakest against the noise, and its privacy cost grows roughly like one over its share of the training records. The two causes compound, because the noise acts on the weight that clipping has already shrunk. So a group whose gradients clipping halves pays like a group half its size.

To see which cause hurts your model, measure per-example gradient norms by group. Opacus's `GradSampleModule`, the wrapper DP-SGD training already uses, keeps one gradient per row in each parameter's `grad_sample`. So one backward pass gives the norms, which `stacked_norms` combines over all parameters.

```
wrapped = GradSampleModule(model)  # adds hooks that keep one gradient per row
wrapped.train()
logits: torch.Tensor = wrapped(split.x)
nn.CrossEntropyLoss()(logits, split.y).backward()  # fills p.grad_sample for every weight p
norms = stacked_norms(wrapped)
```

Group the norms by each row's group label, kept beside the batch but not fed to the model. To measure a model that Opacus has already wrapped, copy its weights into a fresh, unwrapped module first, because `GradSampleModule` refuses a model that already carries its hooks. Log each group's median norm and fraction clipped as training goes on, because the groups can separate late. At initialization on Adult, the median norm is 1.444, and $C = 1$ clips every row, whatever its group.

## 51.5. Mitigations and what they cost

At a fixed budget, a change to the training recipe only moves the cost between the groups or between clipping and noise (Table 51.3). Only the last row, which changes the data, the starting model or the budget, reduces it.

Table 51.3.: Mitigations for disparate impact, and when to use them.

| Fix | What happens | Use it? |
|---|---|---|
| Upweight the group's loss, or use a smaller clipping norm | Clipping cuts an upweighted gradient back to $C$, and a smaller norm clips more of the group | No |
| Larger clipping norm, with learning rate times $C$ held fixed | Less clipping, but more noise relative to short gradients | Rarely, since it did not help the running example |
| Clipping norm set to the largest gradient seen in the data | Breaks the privacy proof, which assumes $C$ is fixed in advance | Only if chosen privately and charged |
| Oversample the group | Its records' $\varepsilon$ is set by their higher sampling rate, so $\sigma$ must rise | Only with that accounting |
| Global scaling with an adaptive bound (Esipova et al. 2023), one shared scale for every gradient | Keeps the update's direction and needs no group labels | Worth a try |
| More data from the group, a public pretrained model (Public data and private fine-tuning), or a larger $\varepsilon$ | Raises the group's signal or lowers the noise | The fixes that work |

Oversampling costs the group's privacy, because a record's guarantee depends on its own chance of being sampled (Amplification by subsampling). So run the accountant at the oversampled group's rate. In the running example, the minority is sampled four times as often and its records are held at $\varepsilon = 4$, which raises the noise multiplier from 1.36 to 4.71. With it, the gap between the groups' accuracies narrows from 9.93 points to 3.99. But it narrows by **levelling down**, since the majority loses 8.01 points. Research methods that clip or penalize each group separately need lawful group labels and private per-group statistics.

Some of the cost cannot be avoided by any algorithm. Learning a small group's own rule means estimating it privately from that group's few records, and private estimation from few records has an error floor that no mechanism beats (Lower bounds).

## 51.6. Cheat sheet

Table 51.4.: The decisions of this chapter.

| Question | Answer | Conditions |
|---|---|---|
| Does DP hurt this group? | Its privacy cost, Equation 51.1, against the non-private baseline | Same data and recipe, averaged over runs |
| How big will the cost be? | Negligible for large groups at moderate $\varepsilon$, roughly one over the group's share for small ones | Grows as $\varepsilon$ falls |

**Rules of thumb**

- Report accuracy and privacy cost per group, with intervals, next to the non-private model's.
- Never report the gap alone, because levelling down closes it too.

## 51.7. References


- Bagdasaryan, Poursaeed & Shmatikov, "Differential Privacy Has Disparate Impact on Model Accuracy" (NeurIPS 2019): the observation.
- Esipova, Ghomi, Luo & Cresswell, "Disparate Impact in Differential Privacy from Gradient Misalignment" (ICLR 2023): global scaling.


**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 52. Auditing private training

## 52.1. Why this matters

When you train with differential privacy (DP), the accountant turns your configuration into a privacy level $\varepsilon$ at a slack $\delta$. That number is a proof about an algorithm, but what ships is code. A bug in the noise, the clipping or the sampling makes the model less private but barely changes its accuracy. Accuracy is flat across a wide range of noise levels (the training workflow), so the usual tests never notice the bug.

An **audit** checks the code by attacking it. It plants records called **canaries**, trains with and without them, and counts how often a membership test guesses right. DP caps how often any test can be right, so a test that is right too often proves that the code is less private than claimed. A test that is rarely right proves nothing, since a weak attack also fails against a leaky model. This chapter gives you two audits to run, a unit test for continuous integration (CI) and a one-run audit of a trained model. Table 52.2 compares them with two costlier ones.

## 52.2. Known bugs, and which audit catches them

These bugs break the private form of stochastic gradient descent (SGD), DP-SGD.

- **The variance passed where the standard deviation is expected,** or **noise scaled for the mean but added to the sum,** which adds the batch size times too little noise. The CI test catches both.
- **Clipping applied after averaging, or noise from a weak seed.** Nasr et al. (2023) found both in real code with per-step audits like the CI test below.
- **The accountant told the wrong numbers,** such as a record count that includes augmented copies, which makes $\varepsilon$ look smaller than it is. Only an end-to-end audit can catch it, and only if the error is large.
- **Batches shuffled but accounted as Poisson sampling,** which can make the true $\varepsilon$ several times the reported one (Amplification by subsampling). No audit here exercises the sampling, so check the loader itself.

Audits catch errors only in the unsafe direction, so code that adds too much noise passes them all.

## 52.3. A CI test for the noise and the clipping

The cheapest strong audit tests one step of the optimizer, many times. A **Dirac canary** is a per-example gradient that is zero except on one coordinate, where every other record's gradient is zero. Put it into every step of one run and leave it out of another. The reading $x_t$ is step $t$'s noisy gradient sum on that coordinate, divided by the clipping norm $C$. The canary is longer than $C$, so a correct optimizer clips it to length $C$ and it adds exactly 1 to the reading. The optimizer also adds noise of standard deviation $\sigma C$, where $\sigma$ is the noise multiplier, so each reading follows

$$x_t \sim \mathcal{N}(0, \sigma^2) \text{ without the canary}, \qquad x_t \sim \mathcal{N}(1, \sigma^2) \text{ with it.} \tag{52.1}$$

A threshold test says "canary in" whenever a reading exceeds a threshold. Its **true positive rate (TPR)** is how often it says so when the canary was there. Its **false positive rate (FPR)** is how often it says so when the canary was not. For two normal distributions a distance $\mu$ apart, every threshold gives $\Phi^{-1}(\text{TPR}) - \Phi^{-1}(\text{FPR}) = \mu$, where $\Phi$ is the standard normal distribution function. So measured rates bound the true separation $\mu$, which a correct optimizer keeps at $1/\sigma$:

$$\mu \geq \Phi^{-1}(\text{TPR}) - \Phi^{-1}(\text{FPR}). \tag{52.2}$$

The test plugs in the low end of the TPR's Clopper–Pearson interval and the high end of the FPR's. Each end is taken at level $\beta/2$, so the bound holds with probability $1 - \beta$ (Hypothesis testing). If this bound exceeds $1/\sigma$, the code adds less noise than it claims.

The script practitioner_audit.py runs the test on Opacus's own `DPOptimizer`. It writes per-example gradients into `grad_sample` by hand, as Opacus's hooks would, and reads the noisy sum that `optimizer.step()` leaves in `grad`.

```
def canary_coordinate(sigma: float, planted_bug: str, with_canary: bool, steps: int) -> Arra
    """Run the optimizer for `steps` steps; return the canary coordinate of each noisy sum,
    layer = torch.nn.Linear(DIM_CI, 1, bias=False)
    optimizer = make_optimizer(layer, sigma, planted_bug)
    observations = np.empty(steps)
    for t in range(steps):
        setattr(layer.weight, "grad_sample", batch_grad_samples(with_canary))  # as Opacus's
        optimizer.step()  # clip each row to C, sum, add N(0, σ²C²)
        assert layer.weight.grad is not None
        observations[t] = layer.weight.grad[0, CANARY_COORD].item() / CLIP_CI
        optimizer.zero_grad()
    return observations
```

The canary's gradient is ten times $C$, so the test also exercises clipping. The test chooses its threshold on the first half of the steps and counts on the second half, and says "canary in" only for a reading strictly above it. It also checks that the canary-free readings have a standard deviation within 10% of $\sigma$, because with no noise at all every reading is 0 and the rates mean nothing. Run `uv run python code/part05-dp-ml/practitioner_audit.py`, which tests the correct optimizer, one with the variance bug planted and one with the noise turned off, at $\sigma = 0.7$ and a batch of $B = 64$ records. The variance bug removes noise only because $\sigma$ is below 1, where $\sigma^2 < \sigma$; above 1 it would pass.

```
sigma=0.7  C=1.0  B=64  steps per run=2000  claim mu=1/sigma=1.429
correct: noise std 0.687 (claim 0.7)  FP 206/1000  TP 710/1000
correct: mu_lo=1.201  pass
variance for std: noise std 0.481 (claim 0.7)  FP 206/1000  TP 862/1000
variance for std: mu_lo=1.722  FAIL
no noise: noise std 0.000 (claim 0.7)  FP 0/1000  TP 1000/1000
no noise: mu_lo=5.360  FAIL
```

The correct optimizer's bound on $\mu$ is 1.201, below its claimed 1.429, so it passes. The variance bug's bound is 1.722 and the missing noise's is 5.360, so both fail. The test takes seconds, so run it on every commit.

Three things go wrong in practice. First, a correct optimizer still fails with probability $\beta$, here 5%, so an unseeded test fails one build in twenty. Fix the seed, or use a $\beta$ of $10^{-3}$ with more steps. Second, the script builds its own optimizer, so it tests Opacus, not your call to it. Point it at your code by having your setup function return the optimizer it makes, and drive that one. Third, the test never draws a batch or calls the accountant. For those, assert in the same suite that the loader is Opacus's `DPDataLoader` and that a second library's $\varepsilon$ matches yours (Train a model with DP-SGD).

## 52.4. From an attack to a lower bound

For a whole training run, the attack is a membership test on a planted canary. A **trial** is one training run, with or without the canary. Under $(\varepsilon, \delta)$-DP, every such test satisfies

$$\mathrm{TPR} \le e^{\varepsilon}\,\mathrm{FPR} + \delta, \qquad 1 - \mathrm{FPR} \le e^{\varepsilon}\,(1 - \mathrm{TPR}) + \delta. \tag{52.3}$$

Solving these two inequalities for $\varepsilon$ turns measured rates into a lower bound on it. Write $\overline{\mathrm{FPR}}$ for the one-sided Clopper–Pearson upper bound on the FPR at level $\beta/2$, and $\underline{\mathrm{TPR}}$ for the lower bound on the TPR at the same level. With probability at least $1 - \beta$, the code is then not $(\varepsilon, \delta)$-DP, at the claimed $\delta$, for any $\varepsilon$ below

$$\varepsilon_{\mathrm{lo}} = \max\left\{ \ln \frac{\underline{\mathrm{TPR}} - \delta}{\overline{\mathrm{FPR}}},\ \ln \frac{1 - \overline{\mathrm{FPR}} - \delta}{1 - \underline{\mathrm{TPR}}} \right\}. \tag{52.4}$$

The bound needs independent trials, and a threshold chosen on other trials than those it counts, since one tuned on the counted trials would inflate the bound.

SciPy's `binomtest(k, n).proportion_ci(method="exact")` gives the Clopper–Pearson intervals. No library installed with this book computes the bound itself, so the script imports `eps_lower_bound` from the book's helper module, practitioner_privacy_tests.py. The same module holds the bound on $\mu$ above and the one-run audit below. Suppose an attack catches the canary in 500 of 1,000 runs that include it, and raises one false alarm in 1,000 runs without it.

```
for false_pos, true_pos in ((1, 500), (0, 1_000)):  # out of 1,000 runs without and with the
    eps_lo = eps_lower_bound(false_pos, true_pos, 1_000, DELTA)
    print(f"FP {false_pos}/1000  TP {true_pos}/1000: certified eps lower bound {eps_lo:.2f}"
```

```
FP 1/1000  TP 500/1000: certified eps lower bound 4.43
FP 0/1000  TP 1000/1000: certified eps lower bound 5.60
```

The raw rates alone would suggest 6.21, but confidence is expensive, and the certified bound is only 4.43. Even a perfect attack certifies only 5.60 from 1,000 trials per hypothesis, so never expect an end-to-end audit to certify the whole of a large claim. Even a **white-box** attacker, who sees every intermediate model, certifies only 3.63 in the full edition against a claimed 6.71.

## 52.5. A one-run audit of the trained model

Thousands of runs are out of reach for a real model, so Steinke, Nasr & Jagielski (2023) get many trials from one run. Plant a number $m$ of canaries, include each one by its own fair coin, train once and score every canary. Then fix a list length $r$. Guess "in" for the $r/2$ highest scores and "out" for the $r/2$ lowest, and make no guess about the rest. Write $W$ for the number of correct guesses.

Under $\varepsilon$-DP, no attack guesses one canary's coin right with a probability above the limit $p_\varepsilon$ given below. So the count $W$ is no more likely to reach any value $v$ than a binomial count with that success rate is:

$$\Pr[W \geq v] \leq \Pr\left[\mathrm{Bin}(r, p_\varepsilon) \geq v\right], \qquad p_\varepsilon = \frac{e^\varepsilon}{1 + e^\varepsilon}. \tag{52.5}$$

An observed count that this binomial reaches with probability below $\beta$ rules out that $\varepsilon$. The certified bound is the largest $\varepsilon$ ruled out this way. Under $(\varepsilon, \delta)$-DP, the paper adds at most $2m\delta$ to that probability. The coins must be independent and fair. Trying several list lengths needs a Bonferroni correction, which runs each test at $\beta$ divided by their number.

The script runs it on the released pipeline of Train a model with DP-SGD, the Adult census data that predicts whether a person's income exceeds \$50K. It reuses that script's whole recipe and adds only the canaries, 1,000 held-out rows with flipped labels. Each is included by a coin, and 537 join the 30,000 training rows. DP-SGD runs at the release's noise multiplier, which gives $\varepsilon = 3.91$ for this slightly larger set. A non-private model uses the release's baseline recipe, and a memorizing control runs non-private Adam for 60 epochs with small batches. The slack is the book's default, $\delta = 10^{-5}$. Each canary is scored by minus its loss. The script tries six list lengths, from 20 to 1,000, and Table 52.1 gives each model's best.

Table 52.1.: One-run audit of the released pipeline on Adult with 1,000 mislabeled canaries, at 95% overall confidence and $\delta = 10^{-5}$.

| Training | Mean canary loss, in / out | Correct guesses | Certified $\varepsilon$ |
|---|---|---|---|
| DP-SGD, release recipe | 4.105 / 4.006 | 10 of 20 | 0.00 |
| Non-private, release recipe | 2.240 / 2.222 | 8 of 20 | 0.00 |
| Memorizing control | 1.994 / 2.992 | 75 of 100 | 0.53 |

Both release-recipe models certify nothing, and the reason is the recipe, not privacy. Ten epochs with large batches barely fit the flipped labels, so included canaries have the same loss as excluded ones, with or without noise. Adult's many near-duplicate rows with opposite labels make a flipped label look like ordinary noise. Only the memorizing control separates its canaries, and even it certifies 0.53 for a model with no finite $\varepsilon$, because each canary is a single trial and 1,000 trials cap any bound. Here a perfect list of all 1,000 guesses would certify at most 5.27.

So use the one-run audit as a test that exposes a bug when a DP model certifies more than its claim, never as evidence of privacy. Report it with its threat model, a black-box attacker who sees only the final model, and with a control that shows the attack can detect leakage on your data.

Stronger black-box attacks, such as the likelihood ratio attack (LiRA) of Pre-release privacy risk assessment, train **shadow models**, copies of the model trained with its exact recipe on similar data. Run them before release and keep their results with the release documents (Release and governance). LiRA's TPR at a low FPR shows what a realistic attacker learns. But its (model, record) pairs share models, so they are not independent trials, and its rates in Equation 52.4 give no certified bound. Certify only from independent trials, such as the one-run audit's coins or many separate training runs.

## 52.6. Cheat sheet

Table 52.2.: Which audit answers which question, and its rule.

| Question | Audit and cost | Rule | Blind to |
|---|---|---|---|
| Does the code clip and add the noise it claims? | CI test, seconds | Fail if the bound on $\mu$, Equation 52.2, exceeds $1/\sigma$ | Sampling and accounting bugs |
| Is the claimed $\varepsilon$ right, accounting included? | End-to-end audit, thousands of runs | Equation 52.4, Clopper–Pearson at $\beta/2$ | Bugs smaller than the bound's shortfall |
| Does a trained model leak planted records? | One-run audit, one extra run | Equation 52.5 plus $2m\delta$, Bonferroni | Most bugs, since its bounds are low |
| What can a realistic attacker learn? | Shadow-model attack (LiRA) | TPR at a low FPR; no certified bound | The most the code could leak |

## 52.7. References

- Nasr et al., "Tight Auditing of Differentially Private Machine Learning" (USENIX Security 2023): Dirac canaries, per-step audits and real bugs.
- Steinke, Nasr & Jagielski, "Privacy Auditing with One (1) Training Run" (NeurIPS 2023): the one-run audit.

**Derivations.** Every result here is derived step by step in the full edition's chapter.

# 53. Adaptive data analysis

## 53.1. What it is

When you tune on a held-out validation set, you read the scores and try new settings, so each question depends on earlier answers. Such **adaptive** reuse overfits the holdout set, whose scores then overstate how the model does on new data.

Each such question is a **statistical query**, which averages a per-record answer between 0 and 1 over the $n$ records you hold, such as the share of records a classifier gets right. Its population value is the same average over new data. Suppose the records are drawn independently and a number $k$ of queries are fixed before any answer is seen. Then the largest gap between the two values is on average at most

$$\sqrt{\frac{\ln(2k)}{2n}}. \tag{53.1}$$

The number of queries hardly matters, because it enters through a logarithm. Adaptive queries break this, because a later query can be built to fit the sample's own noise. Blum & Hardt's boosting attack does so on a leaderboard whose labels are coin flips, where nothing can beat 0.5 on new data. Yet after 1,000 submissions on as many holdout records, it reports 0.7182.

Differential privacy (DP) is the remedy, because answers that reveal little about any record cannot be used to overfit it. Let a mechanism's answers be on average within a distance $\alpha$ of the empirical values. Let the whole exchange be DP at a privacy level $\varepsilon$, with a small slack $\delta$, for samples that differ in one replaced record. The **transfer theorem** then bounds the expected largest error against the population by

$$\alpha + e^{\varepsilon} - 1 + \delta, \tag{53.2}$$

whatever the analyst does with the answers. For small $\varepsilon$ the middle term is about $\varepsilon$, so the bound is useful only well below $\varepsilon = 1$. It protects the validity of the answers, not the people in the holdout set.

## 53.2. When you meet it

You meet adaptivity whenever a validation set, a test set or a leaderboard answers more than one round of questions, as in hyperparameter search or early stopping. A round is a batch of questions fixed before any of its answers is seen, and the bounds below grow with the number of rounds. Table 53.1 gives the choice for each situation.

Table 53.1.: How to reuse a holdout set, by situation.

| Situation | What to do |
|---|---|
| A final evaluation | Keep a test set and use it once |
| Few adaptive rounds, plenty of data | Give each round its own slice of data |
| Many adaptive rounds, scarce data | Answer through noise, or use Thresholdout |
| A search that is private in the holdout set | Nothing more, since the transfer theorem covers the winner |
| A public leaderboard | Show a new score only when it beats the best by a margin (Blum & Hardt's Ladder) |

## 53.3. How to use it

The simplest fix is **sample splitting**, which answers each of $r$ rounds on its own $n/r$ records. The expected largest error is then at most $\sqrt{r \ln(2k)/(2n)}$ for $k$ queries in all, so the data you need grow in proportion to $r$. Take 10,000 validation records and a search over 1,000 configurations, each chosen from the earlier scores, so $r = k = 1{,}000$. Splitting then gives a bound of 0.6165, useless for scores between 0 and 1. A bound of 0.05 would need 1,520,181 records, where fixed queries would need 1,521.

Answering every query with Gaussian noise, under the transfer theorem, needs records that grow only like $\sqrt{k}$. Its constants are large, though, so splitting stays cheaper up to a few hundred adaptive queries.

**Thresholdout** (Dwork et al. 2015) answers from a training set while the training value agrees with the holdout value, which reveals nothing about the holdout set. When they differ by more than a noisy threshold, it records a **detection** and returns a noisy holdout value. Its privacy cost in the holdout records grows with the number of detections, however many queries were answered below the threshold. But its formal $\varepsilon$ in the full edition's experiment is about 29, and no mainstream library ships it, so treat it as a brake rather than a guarantee.

## 53.4. What goes wrong

The common failure is not an attack but a long search that picks each setting from earlier validation scores. A fixed grid scored once, as in scikit-learn's `GridSearchCV`, is a single round, so its winner's overstatement is bounded by Equation 53.1. A larger holdout set helps only slowly, since with splitting the records needed grow with every round.

So count rounds, and report how many rounds a validation number has been through. Practice is often milder than the worst case, but the risk is real when records are few and rounds many.

## 53.5. References


- Dwork et al., "The Reusable Holdout" (*Science* 2015): Thresholdout.

- Blum & Hardt, “The Ladder” (ICML 2015): the boosting attack and a leaderboard that resists it.

**Derivations.** Every result here is derived step by step in the full edition’s chapter.

# 54. Release and governance

## 54.1. Why this matters

A model trained with differential privacy (DP) is not finished when training stops. Someone else, a privacy reviewer or a data protection officer, decides whether it may ship. They will ask what the guarantee covers, whether its number is right and what it means under the law, and none of them can rerun your training. So the claim must travel with everything needed to check it.

This chapter packages a model trained with the private form of stochastic gradient descent (SGD), called DP-SGD, for release. It is the run of Train a model with DP-SGD, a small network trained on 30,000 people from Adult. Its target was a privacy level $\varepsilon$ of 4. The script practitioner_release.py runs every computed step in a few seconds. It starts from the run record, the file of settings that the training script writes. Pass that file with `--record PATH`. Without the option the script uses a built-in copy of the book's record, and a path it cannot read is an error, never a silent fall-back.

```
uv run python code/part05-dp-ml/practitioner_release.py
```

## 54.2. Assemble the run record

The record must hold what the accountant needs. An accountant, the code that turns a run's settings into $\varepsilon$ (Report and review an $\varepsilon$ claim), reads four of them. They are the noise multiplier $\sigma$ and the sampling rate $q$, then the number of steps and the slack $\delta$ that the guarantee allows. It also takes four things for granted, so the record states those too. The privacy unit is what one protected record is, here one person. The neighbouring relation says whether neighbouring datasets differ by adding or removing a unit or by replacing one. The others are the sampler, how the code drew each batch, and that no other use of the private data went uncounted. The training script reads the sampler from the loader Opacus returned rather than typing it, and records whether Opacus's `secure_mode` was on and the seed.

The record also gives $n$, the number of units, the library versions, how the hyperparameters were chosen and whether any preprocessing was fitted on private data. In the excerpt, `data` is the training script's record and `RELEASE_CONTEXT` holds the release owner's additions.

```
merged = RELEASE_CONTEXT | data  # the training record, plus what the release owner adds
rec = load_record(merged)
for key, value in asdict(rec).items():
    print(f"{key}: {value}")
problems = record_problems(merged)  # missing fields, sampler not Poisson, delta above 1/n^1
```

```
limitations = record_limitations(merged)  # stated, so not a problem, but the statement must
print(f"record problems: {len(problems)}")
for problem in problems:
    print(f"  - {problem}")
print(f"record limitations: {len(limitations)}")
for limitation in limitations:
    print(f"  - {limitation}")
```

```
unit: person (one row of Adult)
neighbours: add/remove
n: 30000
delta: 1e-05
sigma: 0.972900390625
sample_rate: 0.03333333333333333
sampler: poisson
secure_mode: False
seed: 0
steps: 300
...
record problems: 0
record limitations: 1
  - secure_mode=False: Opacus drew the noise from a seeded, non-cryptographic generator so t
```

The record's $\delta$ follows the book's default of $\delta \leq 1/n^{1.1}$ rounded down to a power of ten, which gives $10^{-5}$ for Adult (Choosing $\varepsilon$). The three values of $\varepsilon$ come from the training script's cross-check. Opacus's privacy loss random variable (PRV) accountant gave the first. Google's `dp_accounting` gave the others, by its privacy loss distribution (PLD) accountant and its looser Rényi differential privacy (RDP) accountant.

The tuning entry matters most to a reviewer, because every run that helps choose a hyperparameter looks at the data, and the final run's $\varepsilon$ leaves those runs out. Here the learning rate was chosen by non-private runs on 2,000 held-out Adult rows that play the part of public data, and $\varepsilon = 4$ was fixed before any private run. The runs at other budgets only drew a utility curve and chose nothing. Those 2,000 people come from the same survey and get no guarantee, so the statement names them (Train a model with DP-SGD). With that stated, the run's $\varepsilon$ is also the $\varepsilon$ of the whole procedure. If you tune on private data, report the run's $\varepsilon$ and the procedure's, as the template in Hyperparameter tuning does.

Each of these is a record problem that stops the release: a missing field, `secure_mode` included, a sampler that is not Poisson, a $\delta$ above $1/n^{1.1}$, or an unreported tuning cost. The one limitation, `secure_mode` left off so that the book's numbers repeat, is disclosed in the statement; a model you ship is retrained with it on.

## 54.3. Recompute the guarantee

Next, recompute $\varepsilon$ from $\sigma$, $q$ and the number of steps alone, with a different library from the one that trained. A wrong number passed to the accountant would be passed again by a recomputation

inside the training code. The default is the PLD accountant of `dp_accounting`, which rounds the privacy loss up so that its $\varepsilon$ is never too small. Read it at the record's $\delta$ and at one a hundred times smaller. Then read the trade-off curve at two false positive rates (FPR). The curve gives the largest true positive rate (TPR) that any membership test, an attack that guesses whether one person was in the training data, can reach at each FPR (Choosing ε).

```
pld = privacy_loss_distribution.from_gaussian_mechanism(sigma, sampling_prob=sample_rate).se
eps_pld = pld.get_epsilon_for_delta(delta)
eps_pld_small = pld.get_epsilon_for_delta(delta / 100)
eps_rdp = rdp_epsilon(sigma, sample_rate, steps, delta)
eps_rdp_small = rdp_epsilon(sigma, sample_rate, steps, delta / 100)
tpr = {fpr: tpr_bound(pld, fpr) for fpr in FPRS}  # largest TPR of any membership test
```

```
eps PLD at delta=1e-05: 3.981
eps RDP at delta=1e-05: 4.503
eps PLD at delta=1e-07: 5.191
eps RDP at delta=1e-07: 5.736
TPR bound at FPR 1e-03: 0.0123 (PLD curve)  0.0536 (pair eps, delta only)
TPR bound at FPR 1e-02: 0.0652 (PLD curve)  0.5355 (pair eps, delta only)
record's PRV eps: reported 3.99, recomputed 3.98: consistent
```

The recomputed PLD value, 3.981, agrees with the training run's PRV value, and the RDP value of 4.503 is higher because RDP is looser. Report the tight value with its accountant's name, and never set one accountant's $\varepsilon$ against a budget computed by another (Report and review an ε claim).

The curve gives the number a non-specialist can use. At an FPR of $10^{-3}$, no membership test can reach a TPR above 0.0123. The pair $(3.98, 10^{-5})$ on its own would allow 0.0536, more than four times as much.

This step goes wrong in two ways. The script flags a reported $\varepsilon$ more than 0.05 below the recomputation, a claim that is too good. The subtler failure is a recomputation that agrees on wrong inputs. For example, the code may shuffle the data into fixed batches while the accountant assumes Poisson sampling at rate $q$ (Amplification by subsampling). Neither the numbers nor Opacus's own `get_epsilon` show that, so the record's sampler, read from the loader, is what catches it.

## 54.4. Attach the evidence

An accountant proves a theorem about an algorithm, not about the code that runs it, so the release also needs evidence about the code and the model. Attach each piece in Table 54.1, and say which attacker each one models. The curve covers the strongest attacker, who sees every intermediate model, while the audit and the attack cover only the attacks they run.

Table 54.1.: The evidence attached to a DP model release, and its limits.

| Evidence | Source | On the Adult run | What it cannot show |
|---|---|---|---|
| Independent $\varepsilon$ | This chapter's recomputation | 3.981 against the reported 3.99 | That the sampler matches the code |
| Clipping and noise test | Continuous integration (CI) test of Auditing private training | Passes on Opacus's optimizer; fails with the variance passed as the standard deviation, or with no noise | Sampling or accounting bugs |
| One-run audit | Auditing private training: the released pipeline retrained with 1,000 planted canaries | A lower bound on $\varepsilon$ of 0.00 for DP-SGD and for the non-private baseline, against 0.53 for a memorizing control | That the model is private; a low bound proves nothing |
| Black-box attack | Likelihood ratio attack of Pre-release privacy risk assessment | Not run on this model; say so in the evidence | Whether a stronger attacker does better |
| Groups | Per-group accuracy with exact 95% intervals (Disparate impact) | Women 0.922 [0.915, 0.929], men 0.807 [0.800, 0.815] | Groups too small to measure |

The audits can only refute a claim. A one-run bound above the claimed 4 would prove a bug, while 0.00 shows only that this attack found nothing, as it also found nothing in the non-private baseline. The audit reruns the released training code with canaries added, so it tests that code, not the released weights. The group accuracies were computed on the 16,842 Adult test rows, which are private too. To publish them, add noise to each group's count of correct predictions and count it in a budget of its own (Release statistics with a library). Or measure them on a test set that is not private.

## 54.5. Write the statement

The statement goes in the model card's privacy section. The script's `format_statement` builds it from the record, so no number is typed by hand.

```
Guarantee: (3.98, 1e-05)-DP per person (one row of Adult), add/remove neighbours,
  for the whole training run on Adult (UCI 2), 30,000 training rows (n = 30,000).
  Also (5.19, 1e-07)-DP. Any membership test has TPR at most 0.0123 at FPR 1e-3.
Mechanism: DP-SGD, poisson sampling q = 0.0333, noise multiplier 0.9729,
```

```
  clipping norm 1, 300 steps (10 epochs).
Accounting: prv, Opacus 1.6.0; recomputed with dp_accounting PLD (RDP gives 4.50).
...
Noise generator: Opacus secure_mode=False, seed 0.
...
Not covered: data outside the training set, API outputs that use other private data, other r
```

The block leaves out the lines that repeat the record. Below the statement, add the evidence of Table 54.1, each item with the attacker it models.

The last line deserves the most care. The guarantee covers the weights and anything computed from them alone, such as the logits that the deployed application programming interface (API) returns, because post-processing cannot weaken DP. An output that also uses other private data, such as a retrieval index, is a new release, and so is a second model trained on the same people.

## 54.6. Map the claim to regulation

This section is not legal advice: every answer depends on facts beyond the model.

In the European Union, Recital 26 of the General Data Protection Regulation (GDPR) exempts anonymous information. It asks whether a person can be identified by the means "reasonably likely to be used". Pseudonymised data, which only information kept separately links to a person, remain personal data under Article 4(5). For AI models, the European Data Protection Board (EDPB) adopted Opinion 28/2024 in December 2024. Under it, a model is anonymous only if the likelihood of extracting personal data about its training set, directly or through queries, is insignificant. Each claim is assessed case by case. The opinion names differential privacy and attack tests, membership inference among them, as measures to weigh. It warns that passing a test shows resistance only to the attacks tested.

The trade-off curve speaks to extraction as well as membership. Guessing which of a number $m$ of equally likely values a person's record holds is a membership test at an FPR of $1/m$. So the curve caps reconstruction too (Choosing ε).

Table 54.2.: What DP evidence answers, and leaves open, for the legal questions a release meets.

| Legal question | What DP evidence supports | What DP does not answer |
|---|---|---|
| Is the model anonymous (GDPR Recital 26; EDPB Opinion 28/2024)? | A bound on what any attacker can learn about one person's presence or record: the TPR cap and the audits | Whether the bound is "insignificant"; the training data stay personal data |
| Is this only pseudonymisation (GDPR Art. 4(5))? | DP relies on no separately kept key that could re-link the data to people | Removing identifiers before training is pseudonymisation at most |
| What goes in the risk assessment (GDPR Art. 35)? | The statement, the curve, the audits and the group table | Necessity and proportionality (Art. 35(7)(b)), and the lawful basis (Art. 6) |

| Legal question | What DP evidence supports | What DP does not answer |
|---|---|---|
| Is the risk of identification very small (HIPAA §164.514(b)(1))? | A quantitative bound that an expert can cite, with documented methods | Whether it is "very small"; the expert's determination itself |
| Is the information deidentified (California Consumer Privacy Act, Cal. Civ. Code §1798.140(m))? | A bound on how much one person's record can change what is inferred | Whether that meets the standard, since DP does not stop inferences from population patterns; the business's own duties |
| Does the data governance meet the EU AI Act (Art. 10)? | Documented data handling and per-group evaluation | Data quality, representativeness and bias, which DP can worsen |

Training still processes personal data, so the GDPR applies to it, lawful basis included. Article 35 requires a data protection impact assessment (DPIA) before processing likely to result in a high risk, and that is the natural home for the statement. In the United States, the safe harbor route of the Health Insurance Portability and Accountability Act (HIPAA) removes listed identifiers under §164.514(b)(2). DP does not do that. Its expert determination route can cite DP evidence but not be replaced by it. California's law, as amended by the California Privacy Rights Act, adds duties against re-identification for the business.

None of these texts says that DP makes a model anonymous by itself. A DPIA entry built from this chapter's evidence can read as follows, with the bracketed parts filled in by the controller and counsel.

> The model was trained on personal data under [lawful basis, Art. 6]. Training used DP-SGD with a guarantee of $(3.98, 10^{-5})$-DP per person, add/remove neighbours, for the whole training run. No membership test against the released weights can exceed a TPR of 0.0123 at an FPR of $10^{-3}$, and the same curve caps reconstruction. The code passed a CI test of its noise and clipping, and a one-run audit of the released pipeline found nothing, while a memorizing control certified 0.53. [A shadow-model attack was / was not run.] These tests show resistance only to the attacks run. DP does not remove identifiers and is not pseudonymisation. Whether the bound makes the model anonymous is for [counsel], and we make no claim of automatic anonymity. Not covered: the 2,000 stand-in rows, the test rows, outputs that use other private data, and later model versions.

## 54.7. Review someone else's claim

The same steps run in reverse when someone else hands you a model. Ask for the record of Section 54.2, refuse a claim that cannot be recomputed, then recompute $\varepsilon$ from the claim's own parameters, and check its $\delta$ against the number of units. Report and review an ε claim gives the checks on the accountant and the sampler.

The script reviews a claim made up for this book, with every flaw a review should catch. It claims $\varepsilon = 1$ at $\delta = 10^{-3}$ for a model trained on 50,000 records, over 5,000 steps with noise multiplier

0.8. Its batches of 256 are accounted as Poisson sampling at rate 256/50,000, although they were in fact shuffled and of fixed size. And the claim states no unit, no clipping norm and no tuning.

```
def review(claim: dict[str, Any]) -> list[str]:
    """Recompute a claim from its own parameters and list what does not hold up."""
    findings = record_problems(claim)  # includes a sampler that is missing or not Poisson
    sigma, q, steps, delta = claim["sigma"], claim["sample_rate"], claim["steps"], claim["de
    pld = privacy_loss_distribution.from_gaussian_mechanism(sigma, sampling_prob=q).self_com
    recomputed = {"PLD": pld.get_epsilon_for_delta(delta)}
    if "rdp" in str(claim.get("accountant")).lower():  # also the accountant the claim names
        recomputed["RDP, the accountant named,"] = rdp_epsilon(sigma, q, steps, delta)
    for name, eps in recomputed.items():
        if eps > claim["epsilon"] + 0.05:
            findings.append(f"eps={claim['epsilon']} does not follow: {name} gives {eps:.2f}
    delta_ok = delta_for(claim["n"])
    findings.append(f"at delta={delta_ok:.0e} (1/n^1.1) PLD gives eps={pld.get_epsilon_for_d
    return findings
```

```
claim: Our model is trained with differential privacy, epsilon = 1.
findings: 10
  - missing field: unit
  - missing field: neighbours
  - missing field: max_grad_norm
  - missing field: secure_mode
  - sampler 'shuffled fixed-size batches' is not Poisson subsampling: Poisson-subsampled acc
  - delta=1e-03 is above 1/n^1.1 (use 1e-06)
  - tuning not reported: the cost of choosing hyperparameters is unknown
  - eps=1.0 does not follow: PLD gives 2.17 at its delta
  - eps=1.0 does not follow: RDP, the accountant named, gives 2.55 at its delta
  - at delta=1e-06 (1/n^1.1) PLD gives eps=3.75
```

By the vendor's own parameters the claimed 1 is 2.17, and the RDP accountant it names gives 2.55. At a $\delta$ the book accepts it is 3.75. Even that is wrong, because shuffled fixed-size batches get no amplification by Poisson subsampling, so the true $\varepsilon$ can be far above every one of these numbers (Amplification by subsampling). The script cannot see a wrong unit, since it takes $n$ as given. If the records are messages and a customer sends many, screen the per-customer $\varepsilon$ with User-level differential privacy yourself. Send the findings back with a request for a corrected record.

## 54.8. After release

The guarantee is fixed at release, but every retraining on the same people is a new release that composes with the old ones. For identical runs, compose the PLD of one run with itself once per version, `pld.self_compose(steps * versions)`, and state the budget per model version and in total over a fixed **period**. If the total over a schedule is too large, calibrate each version to a share of it.

Deletion requests need their own answer. DP deletes nothing. It says the model is about as likely as one trained without the person, but their row is still in the data. Honour a request by removing the row, from the stand-in and test rows too, and retraining on a fixed schedule. Whether that is fast enough for the GDPR's Article 17 is a question for counsel. Sharded, isolated, sliced and aggregated training trains separate models on disjoint shards so that a deletion retrains one shard. Combined with DP, each shard has its own smaller $n$, so its sampling rate and its $\delta$ follow from that $n$.

Give each model version its own record and statement, and name in advance the incidents that reopen a release. A failing CI test, an audit bound above the claim, or a bug in the sampler or the accountant calls for a corrected statement or a retrained model.

## 54.9. Checklist

Table 54.3.: The release checklist: each step, its default and its check.

| Step | Default | Check |
|---|---|---|
| Run record | Unit, $n$, $\delta$, neighbours, sampler read from the loader, $\sigma$, $q$, steps, clipping norm, accountant and version, `secure_mode`, seed, tuning, preprocessing, public data | `record problems: 0`; `secure_mode` on for the shipped model |
| $\delta$ | $\delta \leq 1/n^{1.1}$, rounded down to a power of ten | $n$ counts units, not rows |
| Tuning | Choices on public data, or every private run counted | Run and procedure $\varepsilon$ both reported |
| Recomputed $\varepsilon$ | PLD in `dp_accounting`, at $\delta$ and $\delta/100$, and TPR from the curve | Reported value not below the recomputation |
| Audits | CI test; one-run audit of the released pipeline with a memorizing control; shadow-model attack | CI passes on the released commit; no bound above the claim; the control detects leakage; any attack not run is named |
| Groups | Per-group accuracy with exact intervals | Test set not private, or its release accounted |
| Statement | Generated from the record | Every number recomputable; "not covered" filled in |
| Regulation | DPIA entry mapping the evidence to each legal question | Reviewed by counsel; no claim of automatic anonymity |
| Third-party claim | Recompute from its own parameters | Unit, sampler, $\delta$ and tuning all stated |

| Step | Default | Check |
|---|---|---|
| After release | Budget per period; retraining and deletion schedule; incident triggers | Versions composed; each has its own record |

## 54.10. References


- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, 2025): what to ask of any DP claim.
- European Data Protection Board, *Opinion 28/2024 on certain data protection aspects related to the processing of personal data in the context of AI models* (December 2024): when a model can count as anonymous.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): its reporting section lists what a privacy statement contains.


**Derivations.** This workflow has no counterpart in the full edition.

# A. Glossary

Acronyms used in the book and the pages that use them.

**AI** Artificial intelligence. Used in Release and governance.

**API** Application programming interface. Used in Threat models and privacy games, Pre-release privacy risk assessment, Release and governance.

**ART** Adversarial Robustness Toolbox. Used in Threat models and privacy games, Membership inference, Attribute inference and model inversion, Pre-release privacy risk assessment.

**AUC** Area under the curve. Used in Hypothesis testing, Membership inference, Pre-release privacy risk assessment.

**CDF** Cumulative distribution function. Used in The Laplace and Gaussian mechanisms.

**CI** Continuous integration. Used in Auditing private training, Release and governance.

**CLT** Central limit theorem. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy.

**CPU** Central processing unit. Used in Memorization and training-data extraction, Pre-release privacy risk assessment, Fine-tune a language model privately with LoRA.

**DLG** Deep Leakage from Gradients. Used in Gradient inversion.

**DP** Differential privacy. Used in Property inference, Gradient inversion, Pre-release privacy risk assessment, Differential privacy foundations, The definition of differential privacy, Post-processing, group privacy and composition, The Laplace and Gaussian mechanisms, Advanced and optimal composition, Selection: exponential mechanism, report-noisy-max, sparse vector, Query release and private multiplicative weights, Beyond global sensitivity, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, Private mean estimation, Release statistics with a library, Privacy accounting, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to ($\varepsilon$, $\delta$) and to tests, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Adaptive composition: filters, odometers and individual accounting, Concentrated differential privacy, Amplification by shuffling and by iteration, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, User-level differential privacy, Choosing $\varepsilon$, Public data and private fine-tuning, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Disparate impact, Auditing private training, Adaptive data analysis, Release and governance.

**DPIA** Data protection impact assessment. Used in Release and governance.

**EMD** Earth mover's distance. Used in Syntactic anonymity and how it breaks.

**ERM** Empirical risk minimization. Used in Optimization, Algorithmic stability and generalization, Private empirical risk minimization, Public data and private fine-tuning.

**FL** Federated learning. Used in Gradient inversion.

**FNR** False negative rate. Used in Hypothesis testing.

**FPR** False positive rate. Used in Hypothesis testing, Threat models and privacy games, Membership inference, Pre-release privacy risk assessment, The definition of differential privacy, Lower bounds: what privacy costs, f-DP and Gaussian differential privacy, Choosing $\varepsilon$, Auditing private training, Release and governance.

**FTRL** Follow the regularized leader. Used in Choosing an accountant, DP-FTRL and matrix factorization.

**GDP** Gaussian differential privacy. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Choosing $\varepsilon$.

**GDPR** General Data Protection Regulation. Used in Linkage and re-identification, Release and governance.

**GPU** Graphics processing unit. Used in Train a model with DP-SGD, Fine-tune a language model privately with LoRA.

**HIPAA** Health Insurance Portability and Accountability Act. Used in Syntactic anonymity and how it breaks, Release and governance.

**HIV** Human immunodeficiency virus. Used in Syntactic anonymity and how it breaks.

**HMAC** Hash-based message authentication code. Used in Cryptography basics, Linkage and re-identification.

**KL** Kullback–Leibler. Used in Divergences.

**LLM** Large language model. Used in Private prediction: PATE and in-context learning.

**LR** Likelihood ratio. Used in Hypothesis testing.

**MAP** Maximum a posteriori. Used in Attribute inference and model inversion.

**ML** Machine learning. Used in Membership inference, Choosing an accountant, Calibrate and account a DP-SGD run, DP-SGD, Train a model with DP-SGD, Choosing $\varepsilon$, Release and governance.

**MWEM** Multiplicative weights exponential mechanism. Used in Query release and private multiplicative weights.

**PATE** Private aggregation of teacher ensembles. Used in Beyond global sensitivity, Private prediction: PATE and in-context learning.

**PII** Personally identifiable information. Used in Memorization and training-data extraction.

**PIN** Personal identification number. Used in Information theory.

**PLD** Privacy loss distribution. Used in Advanced and optimal composition, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, From Rényi DP to $(\varepsilon, \delta)$ and to tests, Numerical accounting with privacy loss distributions, Train a model with DP-SGD, Choosing $\varepsilon$, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Release and governance.

**PRV** Privacy loss random variable. Used in Choosing an accountant, Calibrate and account a DP-SGD run, Numerical accounting with privacy loss distributions, Train a model with DP-SGD, Fine-tune a language model privately with LoRA, Release and governance.

**RDP** Rényi differential privacy. Used in Advanced and optimal composition, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to $(\varepsilon, \delta)$ and to tests, Numerical accounting with privacy loss distributions, Adaptive composition: filters, odometers and individual accounting, Concentrated differential privacy, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, Choosing $\varepsilon$, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Release and governance.

**RMSE** Root-mean-square error. Used in The Laplace and Gaussian mechanisms, Private mean estimation.

**ROC** Receiver operating characteristic. Used in Hypothesis testing, Threat models and privacy games, Membership inference.

**SGD** Stochastic gradient descent. Used in Optimization, Algorithmic stability and generalization, Pre-release privacy risk assessment, Privacy accounting, Choosing an accountant, Calibrate and account a DP-SGD run, Amplification by subsampling, Rényi differential privacy, From Rényi DP to ($\varepsilon$, $\delta$) and to tests, Numerical accounting with privacy loss distributions, f-DP and Gaussian differential privacy, Concentrated differential privacy, Amplification by shuffling and by iteration, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Train a model with DP-SGD, Hyperparameter tuning and private selection, User-level differential privacy, Choosing $\varepsilon$, Public data and private fine-tuning, Fine-tune a language model privately with LoRA, Private prediction: PATE and in-context learning, DP-FTRL and matrix factorization, Disparate impact, Auditing private training, Release and governance.

**SQL** Structured Query Language. Used in Release statistics with a library.

**TPR** True positive rate. Used in Hypothesis testing, Threat models and privacy games, Membership inference, Pre-release privacy risk assessment, The definition of differential privacy, Lower bounds: what privacy costs, f-DP and Gaussian differential privacy, Choosing $\varepsilon$, Auditing private training, Release and governance.

**TV** Total variation. Used in Divergences, Hypothesis testing, Syntactic anonymity and how it breaks.

**XOR** Exclusive or. Used in Cryptography basics.

**ZIP** Zone Improvement Plan code, the US postal code. Used in Linkage and re-identification.

# B. Index


AboveThreshold, 103, 108
adaptive composition, 143, 145, 181
advanced composition, 81, 100, 143–145
amplification by subsampling, 143, 164, 252
analytic Gaussian mechanism, 96
approximate differential privacy, 131, 132
approximate zCDP, 183
attribute inference, 31, 45, 78, 220
auxiliary knowledge, 33

basic composition, 92, 93, 100
binomial distribution, 8, 18, 213
black-box attack, 114, 263

canary, 33, 36, 52, 73, 78, 79, 239, 255, 256, 263
central limit theorem, 145, 150, 174
Chernoff bound, 8
clipping, 103, 114, 130, 132, 146, 193, 197, 206–208, 215, 218, 239, 248, 250, 252, 263, 265, 267
Clopper–Pearson interval, 8, 17, 18, 256
composition, 67, 68, 81, 87, 92, 93, 97, 100, 117, 143–145, 159, 164, 169, 175, 181, 189, 211, 213
composition attack, 68

deduplication, 229
Deep Leakage from Gradients, 57
differential privacy, 1, 31, 32, 52, 56, 81, 87, 93, 99, 108, 113, 124, 132, 141, 143, 145, 150, 158, 169, 172, 175, 178, 181, 184, 189, 198, 208, 213, 219, 225, 230, 239, 251, 256, 268
discrete Gaussian distribution, 118, 184
DP-SGD, 73, 78, 143–146, 150, 158, 161, 164, 169, 189, 196–198, 206, 207, 210, 213, 215, 217–219, 224, 239, 255, 263, 265

empirical risk minimization, 189
entropy, 21, 57, 67
exponential mechanism, 103, 107, 108, 112, 182, 213

f-DP, 145
frequency oracle, 124
fully adaptive composition, 181
Fundamental Law of Information Recovery, 31

Gaussian differential privacy, 143, 145, 158, 174, 176, 178, 224
Gaussian mechanism, 81, 96, 99, 117, 189
generalization gap, 42
global sensitivity, 81, 117
gradient inversion, 31, 56, 78
group privacy, 81, 92, 93, 97, 117, 169, 218

homomorphic encryption, 28
honest-but-curious, 32

imputation baseline, 45
individual privacy accounting, 181

k-anonymity, 67

l-diversity, 67
Laplace distribution, 3
Laplace mechanism, 99
likelihood ratio attack, 41, 42, 73, 78, 79, 256, 263
Lipschitz, 22
loss attack, 42, 78, 79

membership inference, 31, 36, 42, 79, 189
meta-classifier, 73
min-entropy, 21
model inversion, 45, 73
moments accountant, 169, 198

Narayanan–Shmatikov attack, 62

neighbouring datasets, 81, 92, 94, 97, 143, 161, 164, 220

optimal composition, 100

parallel composition, 92, 93
permutation invariance, 47
Poisson distribution, 145–147, 150, 158, 161, 164, 183, 193, 197, 206, 207, 210–213, 216, 239, 252
Poisson sampling, 145–147, 164, 193, 207, 210, 239, 252
post-processing, 81, 92, 93, 97, 117
privacy filter, 101
privacy game, 31, 36
privacy loss, 143–145
privacy loss distribution, 143–146, 148, 150, 158, 161, 164, 174, 207, 224, 239, 267
privacy loss random variable, 145
privacy profile, 99, 101
property inference, 31, 47, 78
PRV accountant, 175, 239
pure differential privacy, 87, 99, 145, 169

quantitative information flow, 21
quasi-identifier, 31, 62, 73, 78

randomized response, 123, 145, 166, 186
RDP curve, 166, 169, 172
re-identification, 31, 79, 87
reconstruction attack, 31, 71
report-noisy-max, 103, 108
ROC curve, 17, 18, 36
Rényi differential privacy, 143, 144, 148, 150, 158, 161, 164, 166, 169, 172, 174, 182, 196, 212
Rényi filter, 145, 181

sampling without replacement, 147, 150, 161, 164, 197
secure aggregation, 57, 73
sensitivity, 81, 97, 99, 114, 117, 118, 145, 166, 169, 182
shadow model, 31, 41, 73, 256, 265, 267
shuffle model, 123, 145, 186
sparse vector technique, 97, 107, 108
stochastic gradient descent, 73, 78, 143–146, 150, 158, 161, 164, 169, 189, 196–198, 206, 207, 210, 213, 215, 217–219, 224, 239, 248, 255, 263, 265
strong convexity, 22
subsample-and-aggregate, 114
subsampled Gaussian mechanism, 183

t-closeness, 67
threat model, 1, 31, 32, 78, 79
trade-off function, 3, 18, 220, 224

unary encoding, 123
unit of privacy, 81, 87, 97, 147, 215, 217–219

variance, 3, 5, 7, 41, 42, 130–132, 252, 263

zero-concentrated differential privacy, 113, 141, 145, 182, 183, 224